

Ciencia de Datos con R

2026-09-06

Contents

1	Ciencia de Datos con R	13
2	Instalación de paquetes	19
2.1	¿Por qué esta página?	19
2.2	Instalar todos los paquetes del libro	20
2.3	Verificar qué está instalado	21
2.4	Activar los paquetes principales	22
3	Flujo de trabajo	23
3.1	Temas:	23
3.2	Creación de un proyecto:	23
3.3	Información sobre el archivo	27
3.4	Ejemplos de Gráficos	32
4	Visualización de datos	35
4.1	Temas:	35
4.2	Leer datos de un archivo	50
4.3	Vea el siguiente gráfico	53
5	Calculadora sofisticada	57
5.1	Temas:	57
6	Transformación: Estructura de un archivo y seleccionar datos	65
6.1	Evaluar el archivo el vuelo	66
6.2	Temas: Reconocer y aplicar las diferentes funciones	69
6.3	Funciones	71
6.4	Agrupar por múltiples variables	73
7	Transformación: Operaciones booleanas y lógicas	75
7.1	Agrupar por múltiples variables	76
7.2	Operaciones lógicas boolean:	76
7.3	Ejercicios	78
7.4		79
7.5	2 Ejercicios:	82

8 Transformación: Valores Faltantes	83
8.1 Valores faltantes	83
8.2 Creamos un df para aclarar estas operaciones con NA en un data frame	84
8.3 Contabilizar los NA	86
8.4 Seleccionar solamente los vuelos donde el “horario de salida” es desconocido	87
8.5 Otras funciones: NA	87
9 Transformación: Seleccionar variables	91
9.1 Funciones en este módulo	91
9.2 rename()	94
9.3 Reorganizar el orden de las columnas, usando select() y everything() en la misma función	95
10 Transformación: Mutate, transmute, lag, lead, cumsum, cumean, cumvar	99
10.1 Funciones en este módulo	99
10.2 Crear un data frame más pequeño con las variables de interés . .	100
10.3 Crear nuevas variables	101
10.4 transmute()	102
10.5 lag()	103
10.6 Usa “Lag” con “IncCasosSaludNuevo” en COVID-19 PR	105
10.7 lead(),	107
10.8 cumsum	109
10.9 cmean() and cvar() del paquete TidyDensity	111
10.10 Uso de varianza acumulativa en investigación:	113
10.11 Que pasa con las funciones cmean , cvar si hay NA en el archivo de datos?	114
10.12 if_else()	115
11 Transformación: Rangos y otras funciones	119
11.1 Funciones en este módulo	120
11.2 Paquetes	120
11.3 Rangos y pruebas no paramétricas	120
11.4 min_rank() y min_rank(desc())	121
11.5 row_number()	122
11.6 dense_rank()	123
11.7 percent_rank()	124
11.8 cume_dist()	126
11.9 Ejemplo trabajado: los vuelos más atrasados	127
11.10 Resúmenes por grupo: group_by() y summarise()	128
11.11 Promedio móvil: rollmean()	133
12 Transformación: Índices paramétricos y no paramétricos	137
12.1 Temas	137

12.2 Paquetes	138
12.3 ¿Qué quiere decir paramétrico y no paramétrico?	138
12.4 Funciones estadísticas básicas	138
12.5 Tarea de grupo	140
12.6 Ejercicios del libro R4DS	140
12.7 Ejercicios del capítulo	140
13 Script	145
13.1 Temas	145
13.2 Paquetes	145
13.3 La consola y el script: dos lugares distintos	146
13.4 Ejecutando códigos desde el script	146
13.5 El directorio de trabajo y los proyectos de RStudio	147
13.6 Diagnóstico de RStudio	147
13.7 <code>source()</code> : correr un script completo	148
13.8 Empezar de cero: reiniciar R	148
13.9 Cómo se escribe un script que se entiende	148
13.10Un ejemplo completo	149
13.11Ejercicios del libro R4DS	150
13.12Ejercicios del capítulo	151
14 Errores comunes y cómo resolverlos	153
14.1 Temas	153
14.2 Error, advertencia y mensaje: no son lo mismo	154
14.3 <code>could not find function</code>	154
14.4 <code>object ... not found</code>	155
14.5 <code>non-numeric argument to binary operator</code>	155
14.6 <code>unused argument</code>	155
14.7 Confundir <code>=</code> con <code>==</code>	156
14.8 El caso especial de NA (no es un error, pero engaña)	156
14.9 <code>cannot open file ... No such file or directory</code>	156
14.10 <code>cannot open URL</code> (descargar datos de internet)	157
14.11 <code>there is no package called ...</code>	157
14.12 <code>object of type 'closure' is not subsettable</code>	158
14.13 <code>\$</code> operator is invalid for atomic vectors	158
14.14NAs introduced by coercion (una advertencia, no un error)	159
14.15 <code>longer object length is not a multiple ...</code> (reciclaje)	159
14.16Un método general para resolver errores	159
14.17Ejercicios del capítulo	160
15 Análisis Exploratorio	163
15.1 Temas	163
15.2 Paquetes	163
15.3 Términos importantes	164
15.4 ¿Qué es el análisis exploratorio?	164
15.5 Variación	165

15.6	Valores faltantes	171
15.7	Covariación: una variable categórica y una continua	173
15.8	Dos variables categóricas	175
15.9	Dos variables continuas	177
15.10	Patrones y modelos	180
15.11	Ejercicios del libro R4DS	181
15.12	Ejercicios del capítulo	181
16	Pipes	185
16.1	Temas: El paquete “magrittr” y sus funciones	185
16.2	¿Qué problema resuelve el pipe?	186
16.3	El ejemplo clásico de magrittr	189
16.4	Alternativas a los pipes	191
16.5	<code>\$\$</code> : el pipe de exposición	191
16.6	<code><></code> : el pipe de asignación	194
16.7	<code>in</code> : esto no es un pipe	195
16.8	<code>T</code> : el pipe en T	196
16.9	<code>!></code> : el pipe impaciente	197
16.10	Ejercicios del capítulo	197
17	Tibbles	201
17.1	Temas: El paquete Tibble	201
17.2	¿Por qué existe el tibble?	202
17.3	Crear un tibble	202
17.4	Crear un tribble	203
17.5	tibble vs. data.frame	204
17.6	Diferencias entre un tibble y un data frame	208
17.7	Selección de subconjuntos de datos	210
17.8	Cuándo hay que volver a un <code>data.frame</code>	210
17.9	Ejercicios del libro R4DS	211
17.10	Ejercicios del capítulo	211
18	Importar datos	215
18.1	Temas	215
18.2	De dónde salen los datos	215
18.3	Datos importados de la web.	216
18.4	Ejercicio	217
18.5	Datos importando de su computadora	217
18.6	Comparar con base R	222
18.7	Segmentar un vector	223
18.8	Números	225
18.9	Cadena de texto	225
18.10	Factores	226
18.11	Fechas, fechas-horas, horas	226
18.12	Segmentar un archivo	227
18.13	Escribir un Archivo	227

18.14Otros tipos de datos y paquetes	227
18.15Ejercicios del libro R4DS	228
18.16Ejercicios del capítulo	228
19 Datos Ordenados	233
19.1 Temas: Datos Ordenados	233
19.2 Paquetes	233
19.3 Las tres reglas	234
19.4 Las cinco tablas del ejemplo	234
19.5 El paquete tidyr	236
19.6 <code>separate()</code> y <code>unite()</code>	239
19.7 Valores faltantes implícitos y explícitos	240
19.8 Ejercicios del libro R4DS	241
19.9 Ejercicios del capítulo	241
20 Datos Relacionados	245
20.1 Temas: Datos Relacionados	245
20.2 Por qué hace falta más de una tabla	245
20.3 Claves	250
20.4 Uniones de transformaciones	255
20.5 Las uniones con los datos de verdad	261
20.6 Entendiendo las uniones	265
20.7 Definiendo las columnas clave	266
20.8 Otras implementaciones	267
20.9 Uniones de filtro	267
20.10Problemas con las uniones	268
20.11Ejercicios del libro R4DS	269
20.12Ejercicios del capítulo	269
21 Factores	273
21.1 Temas	273
21.2 ¿Qué es un factor?	273
21.3 El problema, en cuatro meses	274
21.4 ¿Qué ocurrió en el gráfico?	275
21.5 El paquete forcats	281
21.6 Haga un re-order por la cantidad de personas por religión	284
21.7 Modificar el orden de los factores	285
21.8 Ejercicios del libro R4DS	293
21.9 Ejercicios del capítulo	293
22 Cadena de Caracteres = STRINGS	297
22.1 Caracteres especiales	298
22.2 Para ver las opciones de caracteres y como trabajar cadenas de caracteres use <code>getOption()</code> para ver	299
22.3 Lista de caracteres especiales (unicode)	299
22.4 Temas: Cadenas de Caracteres	300

22.5	Dividir cadenas basado en la posición de los caracteres en la cadena	301
22.6	También se puede usar <code>str_sub()</code> para modificar una cadena . . .	302
22.7	Locales	303
22.8	Buscar patrones	303
22.9	Coincidencia básica	303
22.10	Anclas	315
22.11	Clases de caracteres y alternativas	316
22.12	Repetición	317
22.13	Agrupamiento y referencias previas	318
22.14	Herramientas: Objetivos	318
22.15	Extraer coincidencias	319
22.16	Coincidencias agrupadas:	320
22.17	Remplazar coincidencias	321
22.18	Divisiones	321
22.19	Otro tipos de patrones	322
22.20	stringi:	322
22.21	El texto de Quijote	323
22.22	Cuántas palabras tiene el primer capítulo de Quijote?	323
22.23	Cuántas veces aparece la palabra “caballero” en el primer capítulo de Quijote?	324
22.24	¿Cuántas palabras tienen más de 10 letras?	324
22.25	Cual es la palabra más larga	324
22.26	Cuántas veces aparece “Sancho Panza” en el primer capítulo de Quijote?	324
22.27	Cuántas veces aparece “Dios” en el primer capítulo de Quijote?	324
22.28	Cuántas palabras tienen ñ?	325
22.29	Remover del texto todas las siguientes palabras	325
22.30	Contabilizar las palabras más comunes en el primer capítulo de Quijote	327
22.31	Ejercicios del capítulo	331
23	Ecuaciones Matemáticas	335
23.1	Ecuaciones matemáticas	335
23.2	Temas	335
23.3	Lo mínimo que hay que saber	336
23.4	Símbolos de LATEX	338
23.5	Química 101: Símbolos	338
23.6	Símbolos comunes	338
23.7	Física	340
23.8	Biología	340
23.9	Ejercicios del capítulo	341
24	Opciones de Knitr	345
24.1	Temas	345
24.2	Qué es una opción de bloque	346
24.3	Para ver las opciones de knitr de los chunk	346

24.4	Tablas de opciones principales	348
24.5	Emojis y símbolos	354
24.6	<code>cache</code> : no repetir lo que ya se calculó	354
24.7	Ejercicios del capítulo	355
25	Fechas, horas y minutas	359
25.1	Temas:	359
25.2	Por qué las fechas son difíciles	360
25.3	Creando fechas/horas	360
25.4	El paquete <code>lubridate</code>	361
25.5	Intervalos y duraciones	364
25.6	El operador <code>%--%</code> simplifica el cálculo del intervalo	364
25.7	Ejercicio	365
25.8	Serie de funciones para calcular la duración de un intervalo de tiempo.	367
25.9	<code>as.duration</code>	367
25.10	Crear una fecha desde columnas individuales	369
25.11	<code>%/%</code> y <code>%%:</code> división entera y residuo	372
25.12	Extrayendo parte de las fecha-hora	376
25.13	Lapso de tiempo	377
25.14	duraciones	378
25.15	Puede agregar periodos de tiempo	379
25.16	períodos	379
25.17	Períodos	381
25.18	Intervalos	381
25.19	Resumen	382
25.20	Zonas horarias	382
25.21	Ejercicios del capítulo	383
26	Funciones	387
26.1	Temas:	387
26.2	¿Cuándo escribir una función?	388
26.3	La anatomía de una función	388
26.4	Ejecución condicional: <code>if</code> y <code>else</code>	390
26.5	Argumentos y valores por defecto	390
26.6	Comprobar los argumentos	391
26.7	Los puntos suspensivos	392
26.8	Valores de retorno	392
26.9	Lo que pasa dentro se queda dentro	393
26.10	Un vistazo a los vectores	393
26.11	¿Cuándo no escribir una función?	394
26.12	Ejercicios del libro R4DS	395
26.13	Ejercicios del capítulo	395
27	Iteración: <code>purrr</code>	399
27.1	Temas: El paquete <code>purrr</code>	399

27.2 ¿Qué es iterar?	400
27.3 El bucle <code>for</code> de base R	400
27.4 La familia <code>map()</code>	401
27.5 Fórmulas anónimas con <code>~ y .x</code>	402
27.6 Iterar sobre dos listas con <code>map2()</code>	402
27.7 <code>across()</code> : iterar sobre columnas dentro de <code>dplyr</code>	403
27.8 <code>walk()</code> : iterar para hacer algo	404
27.9 <code>safely()</code> : cuando alguna vuelta puede fallar	404
27.10 Un modelo por grupo	406
27.11 Cuando un <code>for</code> sigue siendo mejor	406
27.12 Ejercicios del libro R4DS	407
27.13 Ejercicios del capítulo	407
28 Modelos: <code>modelr</code>	411
28.1 Temás: Construir modelos con <code>modelr</code>	411
28.2 Qué es un modelo	412
28.3 Visualizar la relación	412
28.4 Ajustar un modelo lineal con <code>lm()</code>	414
28.5 Predicciones con <code>data_grid()</code> y <code>add_predictions()</code>	415
28.6 Residuales con <code>add_residuals()</code>	416
28.7 Predictoras categóricas y varias variables	418
28.8 Predecir no es explicar	419
28.9 Ejercicios del libro R4DS	419
28.10 Ejercicios del capítulo	419
29 Hex Stickers	423
29.1 Temás	423
29.2 Qué es un hex sticker	423
29.3 La función y los valores “default”	424
29.4 Ejemplo de Crear un Hex con un gatito	427
29.5 Ejercicios del capítulo	429
30 Nubes de palabra en R	433
30.1 Temás	433
30.2 Los cinco pasos	433
30.3 Ejemplo #1	434
30.4 Usando los datos en el paquete <code>wordcloud2</code> que se llama demoFreq	435
30.5 Paso 1	435
30.6 Subir el texto en formato Corpus	436
30.7 Mirar el documento, para evaluar su contenido	436
30.8 Transformar el texto para reemplazar algunos caracteres especiales, y reemplazarlos por espacio en blanco	436
30.9 el paquete tm es para text mining	436
30.10 Crear una matriz de las palabras de mi documento	438
30.11 Ejemplo 2	440
30.12 Cuando una nube de palabras no es la gráfica correcta	440

30.13	Como remover palabras de otra idioma	441
30.14	En español	441
30.15	Un documento para hacer un word cloud “La inteligencia artificial”	442
30.16	Convertir su documento en Corpus y identificar que es en español	442
30.17	Limpieza del documento	443
30.18	Transformar el texto en un documento de texto sencillo (Plain Text)	443
30.19	Transformar el texto para reemplazar algunos caracteres especiales	443
30.20	Crear una matriz de las palabras	443
30.21	Calcular la frecuencia de cada palabra	444
30.22	Ejercicios del capítulo	445
31	Temas Avanzados en Data Science con Tidyverse	449
31.1	Temas	449
31.2	Introducción	449
31.3	2. Modelado con tidymodels	453
31.4	3. Visualización avanzada con ggplot2	456
31.5	4. Datos anidados y listas con purrr	457
31.6	Ejercicios del capítulo	461
32	Leaflet: Mapas interactivos	465
32.1	Temas	465
32.2	Por qué un mapa interactivo	465
32.3	Bajar los datos de la web del USGS	466
32.4	Si bajo los datos a mi computadora	468
32.5	Importación de Paquetes y Datos	469
32.6	Mapa básico con Leaflet	470
32.7	Usando la paletas de color de viridis	471
32.8	Trabajando con datos de iNaturalist	472
32.9	Utilizando Leaflet	475
32.10	Trabajando Variables Categóricas con Leaflet:	478
32.11	Ejercicios del capítulo	480
Apéndice A:	COVID-19 acumulativo	483
.1	COVID 19 Cumulativo	483
##	[1] "2026-09-06"	

Chapter 1

Ciencia de Datos con R



¡Bienvenidos a BIOL4026! Una introducción práctica y visual a la ciencia de datos con R, RStudio, RMarkdown y el **tidyverse**. Aprenderás a importar, ordenar, transformar, **visualizar** y modelar datos reales —de Puerto Rico y de la biología— para contestar preguntas cuantitativas.

Instructor: Raymond L. Tremblay, PhD

Oficina: NL 104 **Teléfono:** (787) 850-0000 (dept de biología) **Correo electrónico:** raymond.tremblay_at_upr_dot_edu

1.0.1 Horario de clase

Presentación de temas y discusión: Lunes y miércoles 4:30-5:50pm (80 mins) NOTE: Necesita traer su laptop!

Hora de consulta con Estudiantes:

NL 104. - Lunes y miércoles de 9:00 a 12:30pm - martes y jueves de 1:00 a 2:00pm

1.0.2 Libros sugeridos

- Garrett Golemund, Hadley Wickham. R for Data Science (2.^a ed.) En Inglés.
 - Garrett Golemund, Hadley Wickham. R para Ciencia de Datos En Español.
 - Tremblay y Hernández-Serrano, 2018.
- Artículos revisados por pares serán asignados para fomentar el método de utilizar estas herramientas en ciencias.
-

1.0.3 Programados

Instala R y RStudio (ambos gratuitos) antes de la primera clase. Luego abre el capítulo “Instalación de paquetes”, que incluye un bloque para instalar de una sola vez todo lo necesario.

- R- free statistical programming language
 - RStudio
 - MSExcel, Numbers o Google Sheet
-

1.0.4 Prerequisitos

Ninguno formal. Se recomienda haber aprobado previamente el curso complementario de estadística, **BIOL3740 (Bioestadística)**, ya que este curso asume familiaridad con conceptos estadísticos básicos y se concentra en la organización, manipulación y visualización de datos.

1.0.5 Descripción del curso

Este curso introduce las técnicas modernas de ciencia de datos aplicadas a la biología usando R, RStudio y RMarkdown. El énfasis está en el flujo de trabajo completo con datos: importar, organizar, limpiar, transformar, visualizar y comunicar datos de manera reproducible con el ecosistema **tidyverse**. Se trabaja con datos reales para demostrar cómo producir análisis, tablas, gráficas y documentos profesionales para la ciencia. El curso complementa —y asume

familiaridad básica con— el curso de bioestadística (BIOL3740), en el cual se estudian a fondo las pruebas de hipótesis, sus supuestos y la selección del método estadístico según el diseño experimental.

1.0.6 Objetivos del curso

Tema general: desarrollar destrezas para organizar, transformar, visualizar y comunicar datos con el fin de contestar preguntas cuantitativas usando R, RStudio y RMarkdown.

Al finalizar el curso, el estudiante podrá:

- Importar y organizar datos de distintas fuentes.
 - Limpiar y transformar datos con `dplyr` y `tidyr`, incluyendo la combinación de datos relacionados.
 - Visualizar datos de forma efectiva con `ggplot2`.
 - Trabajar con cadenas de texto (`stringr`), factores (`forcats`) y fechas y horas (`lubridate`).
 - Escribir funciones e iterar procesos con `purrr`.
 - Construir y visualizar modelos básicos con `modelr`.
 - Producir documentos y sitios web reproducibles con RMarkdown.
 - Comunicar resultados cuantitativos de manera clara y profesional.
-

1.0.7 Puntuación:

Este curso será evaluado con los siguientes ítems:

Item	Valor
Ejercicios prácticos (10-15 total)	55%
Presentación final (sitio web en RMarkdown)	25%
Participación en clase	10%
Asistencia a clase	10%

NOTE: Escala de Notas:

- A (100 to 90)
 - B (89 to 80)
 - C (79 to 70)
 - D (69 to 60)
 - F (< 60)
-

1.0.8 Exámenes:

Ninguno.

1.0.9 Presentación final

El semestre culmina con una presentación final en la que cada estudiante construye y presenta su propio sitio web reproducible en RMarkdown (con su archivo `.yaml` y su hoja de estilo `.css`). El proyecto integra las destrezas del semestre —importar, organizar, transformar, visualizar y comunicar datos— y vale el 25% de la nota final.

1.0.10 Conferencias

En la clase las notas serán basadas primeramente en la participación y algunas pruebas cortas. Su participación es esencial para el aprendizaje (y para un ambiente positivo). Aprender *NO* es un proceso pasivo: los estudiantes deben participar haciendo preguntas y discutir el material con su conocimiento anterior (Su bagaje de conocimiento).

1.0.11 Ejercicios

Los ejercicios están enfocados en la aplicación de conceptos y métodos discutidos en la clase y solución de problemas. Se hará un esfuerzo de usar datos reales para demostrar cómo trabajar con los análisis, tablas, y gráficos en R, RStudio y RMarkdown. Típicamente, tendrán solamente una semana para hacer los ejercicios y entregarlos en formato `.html`.

Los ejercicios se entregan en formato `.html`, que se genera al hacer **Knit** de tu archivo `.Rmd` en RStudio.

1.0.12 Entregas tardías

Los ejercicios deben entregarse en la fecha y hora indicadas. **Por cada día de tardanza se descontará el 10% de la puntuación del trabajo** (por ejemplo, un ejercicio entregado con dos días de retraso recibe un descuento del 20%). A partir del décimo día calendario de tardanza, el trabajo recibe una puntuación de cero. Solo se considerarán excepciones con una excusa válida y documentada.

1.0.13 Faltar a clase:

Los trabajos cortos y pruebas cortas NO se reponen. Si falta a la clase es su responsabilidad hablar con los otros estudiantes para saber lo que se discutió en la clase. La presentación final se repone solamente por una excusa válida.

1.0.14 Derechos de Estudiantes con Impedimentos

La UPR-Humacao cumple con las leyes ADA (Americans with Disabilities Act) y 51 (Servicios Educativos Integrales para Personas con Impedimentos) para garantizar igualdad en el acceso a la educación y servicios. Estudiantes con impedimentos: informe al (la) profesor(a) de cada curso sobre sus necesidades especiales y/o de acomodo razonable para el curso, en la tarjeta de información de la primera semana y visite la Oficina de Servicios para la Población con Impedimentos (SERPI) a la brevedad posible. Se mantendrá la confidencialidad.

1.0.15 Integridad académica

La Universidad de Puerto Rico promueve los más altos estándares de integridad académica y científica. El Artículo 6.2 del Reglamento General de Estudiantes de la Universidad de Puerto Rico (Certificación Núm. 13, 2009-2010, de la Junta de Síndicos) establece que “la deshonestidad académica incluye, pero no se limita a: acciones fraudulentas, la obtención de notas o grados académicos valiéndose de falsas o fraudulentas simulaciones, copiar total o parcialmente la labor académica de otra persona, plagiar total o parcialmente el trabajo de otra persona, copiar total o parcialmente las respuestas de otra persona a las preguntas de un examen, haciendo o consiguiendo que otro tome en su nombre cualquier prueba o examen oral o escrito, así como la ayuda o facilitación para que otra persona incurra en la referida conducta”. Cualquiera de estas acciones estará sujeta a sanciones disciplinarias en conformidad con el procedimiento disciplinario establecido en el Reglamento General de Estudiantes de la UPR vigente.

1.0.16 Comentario sobre grabar videos y/o audio de las clases

Los estudiantes no PUEDEN grabar la clase por forma de video o audio sin el permiso del profesor. Algunos estudiantes con necesidades especiales pueden hablar con el profesor para pedir el permiso. La solicitud y aprobación del permiso tiene que ser por escrito (por ejemplo por email).

1.0.17 Espacio libre de acoso sexual

La Universidad de Puerto Rico prohíbe el discrimen por razón de sexo y género en todas sus modalidades, incluyendo el hostigamiento sexual. Según la Política Institucional contra el hostigamiento sexual, en la Universidad de Puerto Rico, Cert. Núm. 130 (2014-2015) de la Junta de Gobierno, si un(a) estudiante está siendo o fue afectado por conductas relacionadas a hostigamiento sexual, puede acudir ante la Oficina del Procurador Estudiantil, el Decanato de Estudiantes o el Coordinador de Cumplimiento con Título IX para una orientación o presentar una querrela.

1.0.18 Protocolo de la Clase

Los teléfonos móviles serán apagados durante la clase. Si necesita una calculadora traerla al salón. El teléfono no debería estar visible durante la clase a menos que pida permiso al instructor. Recuerda que se usarán computadoras portátiles en cada sesión.

Chapter 2

Instalación de paquetes

Fecha de la última revisión

[1] "2026-09-06"

2.1 ¿Por qué esta página?

A lo largo del libro usamos muchos paquetes de R. Un **paquete** es una colección de funciones, datos y documentación que amplía lo que R puede hacer. Antes de comenzar conviene **instalar de una sola vez** todos los paquetes que vamos a necesitar, para que ningún capítulo falle por un paquete que falta.

CRAN (*Comprehensive R Archive Network*), la red oficial de paquetes de R, funciona desde 1997 y hoy alberga más de 20,000 paquetes. El **tidyverse** — una colección de paquetes con una filosofía común para la ciencia de datos — fue creado por **Hadley Wickham** y su equipo en RStudio (hoy Posit); el nombre se popularizó alrededor de 2016.

Recuerda: solo hay que **instalar** un paquete una vez en la computadora (con `install.packages()`), pero hay que **activarlo** con `library()` en cada sesión en la que lo vayas a usar.

`install.packages("paquete")` y `library(paquete)` — instalar vs. activar.

- **`install.packages("x")`**: descarga e instala el paquete desde internet (CRAN). Se hace **una sola vez** por computadora.
- **`library(x)`**: carga el paquete en la sesión actual para poder usar sus funciones. Se hace **en cada sesión**.
- **Regla de oro**: instalar = una vez; activar con `library()` = cada vez que abres R.

Un error muy frecuente es instalar el paquete pero olvidar activarlo en la sesión. Si al usar una función aparece `could not find function`, casi siempre falta correr `library()`. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

2.2 Instalar todos los paquetes del libro

Copia y ejecuta el siguiente bloque **una sola vez** en la consola de RStudio. El script revisa cuáles paquetes ya están instalados e instala **solamente los que falten**. Instala cada paquete por separado, de modo que si alguno no se puede instalar, los demás sí se instalan y al final se te informa cuáles fallaron.

Instalar paquetes descarga archivos desde CRAN, así que necesitas conexión a internet. En una red lenta el proceso puede tardar varios minutos.

```
paquetes_curso <- c(
  # --- Núcleo: tidyverse y datos en español ---
  "tidyverse", "datos", "ggversa",
  "dplyr", "tidyr", "readr", "purrr", "stringr", "forcats",
  "ggplot2", "lubridate", "magrittr",

  # --- Datos de ejemplo ---
  "nycflights13", "babynames", "nasaweather", "fueleconomy", "Lahman",

  # --- Importar / limpiar / tablas ---
  "readxl", "haven", "data.table", "hms", "zoo",
  "gt", "flextable", "knitr", "bookdown", "gridExtra",

  # --- Modelos y estadística ---
  "modelr", "broom", "tidymodels", "MASS", "DescTools",
  "GMCM", "TidyDensity",

  # --- Mapas y visualización ---
  "leaflet", "maps", "RColorBrewer",

  # --- Hex stickers e imágenes ---
  "hexSticker", "showtext", "sysfonts", "magick",

  # --- Nubes de palabras / texto ---
  "wordcloud", "wordcloud2", "tm", "SnowballC", "stopwords",

  # --- Utilidades y datos externos ---
  "pacman", "devtools", "rinat", "cranlogs", "packageRank"
)
```

```
faltantes <- setdiff(paquetes_curso, rownames(installed.packages()))

fallidos <- character(0)
for (p in faltantes) {
  ok <- tryCatch({
    install.packages(p, dependencies = TRUE)
    TRUE
  }, error = function(e) FALSE, warning = function(w) FALSE)
  if (!ok || !(p %in% rownames(installed.packages())))) fallidos <- c(fallidos, p)
}

if (length(fallidos) == 0) {
  message("¡Listo! Todos los paquetes del libro están instalados.")
} else {
  message("No se pudieron instalar automáticamente: ", paste(fallidos, collapse = ", "),
    "\nInténtalos a mano con install.packages('nombre').")
}
```

`ggversa` (Gráficas Versátiles con `ggplot2`, de Tremblay & Hernández-Serrano) está disponible en CRAN, así que se instala igual que los demás con `install.packages("ggversa")`.

2.3 Verificar qué está instalado

Este bloque **no instala nada**; solamente te dice cuáles paquetes ya tienes y cuáles te faltan.

```
paquetes_curso <- c(
  "tidyverse", "datos", "ggversa", "nycflights13", "babynames", "nasaweather",
  "fueleconomy", "Lahman", "readxl", "haven", "data.table", "hms", "zoo",
  "gt", "flextable", "knitr", "bookdown", "gridExtra", "modelr", "broom",
  "tidymodels", "MASS", "DescTools", "GMCM", "TidyDensity",
  "leaflet", "maps", "RColorBrewer", "hexSticker", "showtext", "sysfonts",
  "magick", "wordcloud", "wordcloud2", "tm", "SnowballC", "stopwords",
  "pacman", "devtools", "rinat", "cranlogs", "packageRank"
)

instalados <- paquetes_curso %in% rownames(installed.packages())
data.frame(
  paquete = paquetes_curso,
  instalado = ifelse(instalados, "sí", "NO - falta")
)
```

2.4 Activar los paquetes principales

En cada capítulo verás al inicio un bloque con `library()`. El más común es:

```
library(tidyverse) # ggplot2, dplyr, tidyr, readr, purrr, tibble, stringr, forcats
library(datos)     # datos en español
```

Si al correr un capítulo aparece un error como `Error in library(xxx): there is no package called 'xxx'`, regresa a esta página y ejecuta el bloque de instalación de arriba.

Chapter 3

Flujo de trabajo

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/workflow-projects.html>

Español: <https://es.r4ds.hadley.nz/04-workflow-basics.html>

Fecha de la última revisión

[1] "2026-09-06"

3.1 Temas:

- Reduciendo Errores
- `getwd()`
- `setwd()`
- Su Proyecto
- Seleccionar el tab **Session** →→ **Set Working Directory** →→ **To Project Directory** ***

3.2 Creación de un proyecto:

Usa Proyectos de RStudio (menú **File** → **New Project**): mantienen juntos tus datos, scripts y resultados, y evitan depender de `setwd()`, que hace tu código difícil de compartir.

RStudio, el entorno de trabajo que usamos, fue creado por **JJ Allaire** y lanzado en 2011; la empresa se renombró **Posit** en 2022. Los *Proyectos* de RStudio nacieron para mantener cada análisis autocontenido y reproducible.

- crear un proyecto para cada curso.
- crear un proyecto para cada investigación.
- No se te olvida de añadir tus archivos de datos en el proyecto.
- Describe claramente todos tus análisis y donde conseguiste la información.
- Describe tu interpretación de los análisis o gráficos.
- Correr los “scripts” uno a la vez para asegurar que funcione.
- knit el archivo .rmd para asegurar que no falte nada.
- no mezclar proyectos de investigación en un mismo proyecto.

```
#install.packages("tidyverse")
library(tidyverse) # ggplot2, dplyr, tidyr, readr, purrr, tibble, stringr, forcats

# install.packages("datos")
library(datos)
```

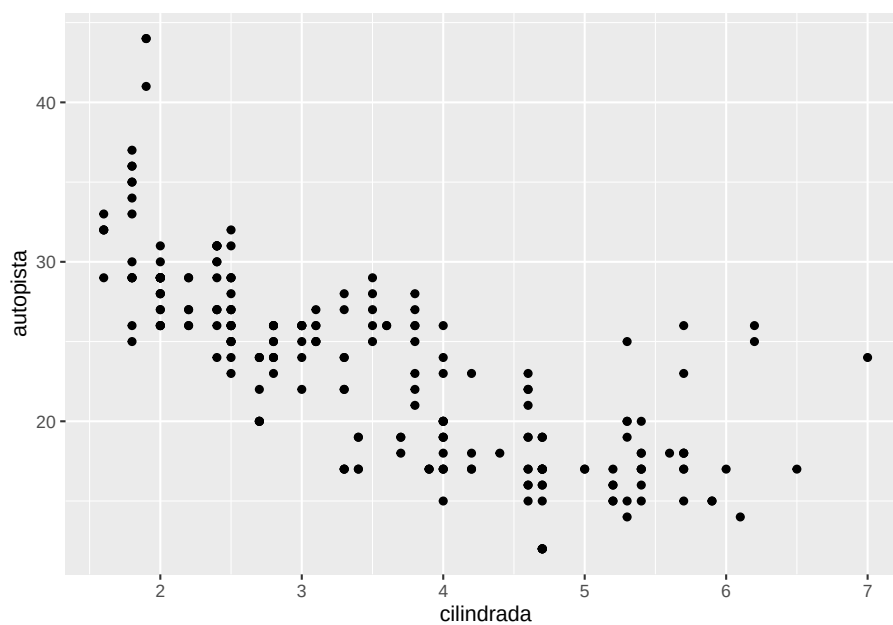
3.2.1 Un set de sobre carros en el archivo (paquete) datos

millas

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>      <chr>      <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi      a4          1.8  1999      4 auto(l5)    d        18
## 2 audi      a4          1.8  1999      4 manual(m5)  d        21
## 3 audi      a4          2    2008      4 manual(m6)  d        20
## 4 audi      a4          2    2008      4 auto(av)    d        21
## 5 audi      a4          2.8  1999      6 auto(l5)    d        16
## 6 audi      a4          2.8  1999      6 manual(m5)  d        18
## 7 audi      a4          3.1  2008      6 auto(av)    d        18
## 8 audi      a4 quattro    1.8  1999      4 manual(m5)  4        18
## 9 audi      a4 quattro    1.8  1999      4 auto(l5)    4        16
## 10 audi     a4 quattro    2    2008      4 manual(m6)  4        20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

3.2.2 Mi primer gráfico

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista))
```

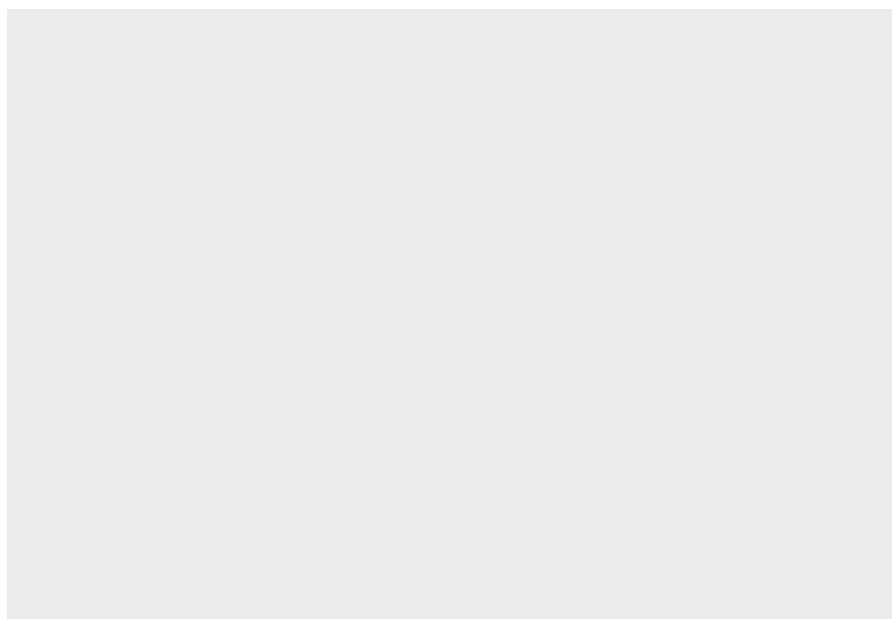



3.2.3 Las funciones head y tail

`head(x, n)` y `tail(x, n)` — primeras y últimas filas.

- **Qué hacen:** `head()` muestra las primeras `n` filas de un data frame (o elementos de un vector) y `tail()` las últimas. Por defecto `n = 6`.
- **Uso:** echar un vistazo rápido a los datos sin imprimirlos completos.

```
ggplot(data=millas)
```



```
millas
```

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>      <chr>      <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi      a4          1.8  1999      4 auto(l5)  d         18
## 2 audi      a4          1.8  1999      4 manual(m5) d         21
## 3 audi      a4          2    2008      4 manual(m6) d         20
## 4 audi      a4          2    2008      4 auto(av)  d         21
## 5 audi      a4          2.8  1999      6 auto(l5)  d         16
## 6 audi      a4          2.8  1999      6 manual(m5) d         18
## 7 audi      a4          3.1  2008      6 auto(av)  d         18
## 8 audi      a4 quattro    1.8  1999      4 manual(m5) 4         18
## 9 audi      a4 quattro    1.8  1999      4 auto(l5)  4         16
## 10 audi     a4 quattro    2    2008      4 manual(m6) 4         20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
head(millas, n=3) # las primeras 6 filas
```

```
## # A tibble: 3 x 11
##   fabricante modelo cilindrada  anio cilindros transmision traccion ciudad
##   <chr>      <chr>      <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi      a4          1.8  1999      4 auto(l5)  d         18
## 2 audi      a4          1.8  1999      4 manual(m5) d         21
## 3 audi      a4          2    2008      4 manual(m6) d         20
```

```
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
tail(millas, n=10) # las últimas 10 filas

## # A tibble: 10 x 11
##   fabricante modelo      cilindrada  año cilindros transmision traccion ciudad
##   <chr>          <chr>      <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 volkswagen new beetle      2    1999      4 auto(l4)    d        19
## 2 volkswagen new beetle    2.5    2008      5 manual(m5) d        20
## 3 volkswagen new beetle    2.5    2008      5 auto(s6)    d        20
## 4 volkswagen passat      1.8    1999      4 manual(m5) d        21
## 5 volkswagen passat      1.8    1999      4 auto(l5)    d        18
## 6 volkswagen passat      2    2008      4 auto(s6)    d        19
## 7 volkswagen passat      2    2008      4 manual(m6) d        21
## 8 volkswagen passat      2.8    1999      6 auto(l5)    d        16
## 9 volkswagen passat      2.8    1999      6 manual(m5) d        18
## 10 volkswagen passat      3.6    2008      6 auto(s6)    d        17
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

3.3 Información sobre el archivo

Usa el signo de interrogación ? antes del nombre de la función o archivo. Re-mueve el “#” para ver el resultado

```
## #?millas
```

```
## #?head
```

3.3.1 Las dimensiones del archivo

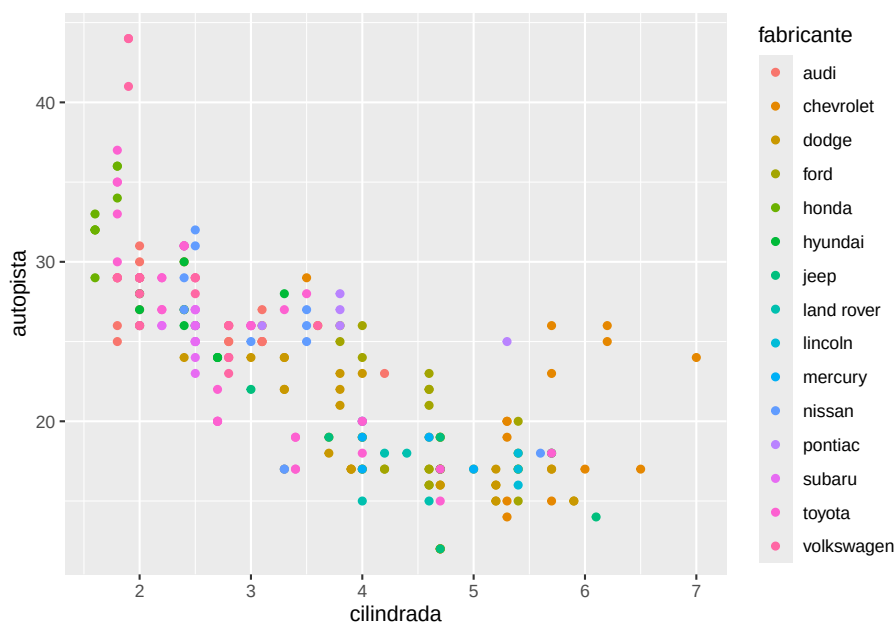
```
dim(millas) # dimension of data frame
```

```
## [1] 234 11
```

3.3.2 Construcción de mi primer gráfico

- el nombre del archivo es primero
- nombre de las variables (nombre de las columnas)
- si quiere tener un color para cada grupo

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista, colour=fabricante))
```

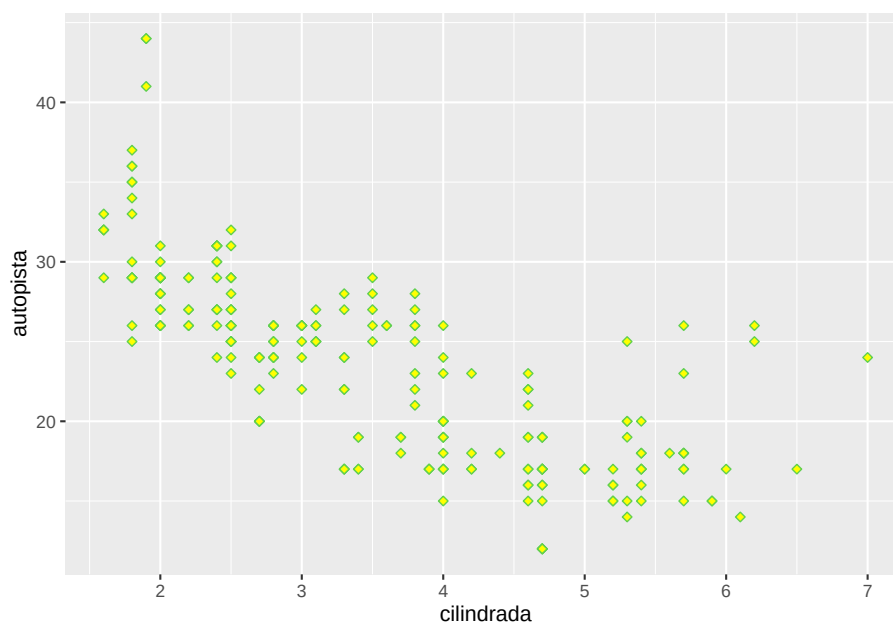


3.3.3 La función `shape` para cambiar la forma de los puntos

La función `shape` es para cambiar el estilo de los puntos

Los colores en hexadecimal deben empezar con #, por ejemplo `color = "#369787"`. Sin el #, R no reconoce el código y marcará error.

```
ggplot(data = millas) +  
  geom_point(mapping = aes(x = cilindrada, y = autopista), shape=23, color = "#369787",
```

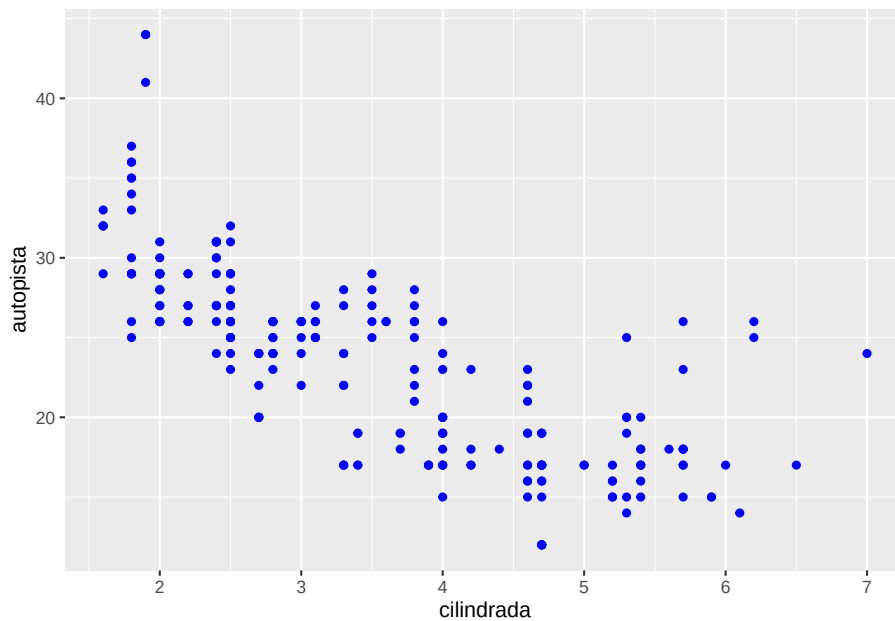


3.3.4 Salvar un gráfico en otro formato

Como salvar la figura en formato recuperable para subir en otros documentos o compartir

`ggsave()` guarda el último gráfico que creaste y lo escribe en tu directorio de trabajo. La extensión del nombre (`.png`, `.jpg`, `.tiff`) determina el formato del archivo.

```
ggplot(data = millas) +  
  geom_point(mapping = aes(x = cilindrada, y = autopista), color = "blue")
```



```
ggsave("img/cilindro_milla.jpg") #. png, .tiff
```

```
millas
```

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>         <chr>          <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi         a4              1.8  1999      4 auto(l5)  d        18
## 2 audi         a4              1.8  1999      4 manual(m5) d        21
## 3 audi         a4              2    2008      4 manual(m6) d        20
## 4 audi         a4              2    2008      4 auto(av)  d        21
## 5 audi         a4              2.8  1999      6 auto(l5)  d        16
## 6 audi         a4              2.8  1999      6 manual(m5) d        18
## 7 audi         a4              3.1  2008      6 auto(av)  d        18
## 8 audi         a4 quattro      1.8  1999      4 manual(m5) 4        18
## 9 audi         a4 quattro      1.8  1999      4 auto(l5)  4        16
## 10 audi        a4 quattro      2    2008      4 manual(m6) 4        20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

Ejercicio para someter:

1. baja el paquete “ggversa”
2. activar el paquete “ggversa”
3. mirar las variables del archivo en este paquete que se llama “Anolis”
4. haga un gráfico que incluye lo siguiente
 - a. en el eje de x = el SVL. que es el tamaño del lagarto del hocico a la

- cloaca y en la variable de “TAIL” en el eje de y.
- b. selecciona la variable “SEX_AGE” para color
- c. salva el gráfico en .png o .jpg
- d. subir el gráfico aquí

```
library(ggversa)
head(Anolis)
```

```
## # A tibble: 6 x 15
##   STUDY      Survey_Site      LOCATION TIME DATE SEASON SPECIES SEX_AGE HEIGHT
##   <chr>      <chr>      <chr>    <tim> <chr> <chr> <chr> <chr>    <dbl>
## 1 Mark/recap North Tower    El Verde 10:46 3/13~ dry   Anolis~ Female      0
## 2 Mark/recap Woods walkway t~ El Verde 10:15 2/20~ dry   Anolis~ Juvenil    0
## 3 Mark/recap Woods walkway t~ El Verde 11:15 2/21~ dry   Anolis~ Male       0
## 4 Mark/recap North Tower    El Verde 11:06 3/16~ dry   Anolis~ Juvenil    0.3
## 5 Mark/recap North Tower    El Verde 12:31 3/11~ dry   Anolis~ Male       0.3
## 6 Mark/recap North Tower    El Verde 01:00 3/9/~ dry   Anolis~ Female     0.4
## # i 6 more variables: DISTANCE_FROM_CENTERLINE <dbl>, PERCH_SUBSTRATE <chr>,
## #   PERCH_DIAMETER <int>, WEIGHT <dbl>, SVL <dbl>, TAIL <dbl>
```

3.3.5 Extracción de valores de un conjunto de datos

`dim()`, `nrow()`, `ncol()`, `unique()` — explorar la forma de los datos.

- `dim(x)`: filas y columnas; `nrow(x)` / `ncol(x)` dan cada una por separado.
- `unique(x$col)`: los valores distintos de una columna (útil para variables categóricas).
- `max()` / `min()`: valor máximo/mínimo de una variable numérica.

Aquí usamos el conjunto de datos de **diamantes** que se encuentra en el paquete **ggplot2**

3.3.5.1 Los primeros datos de un archivo

```
head(diamantes)
```

```
## # A tibble: 6 x 10
##   precio quilate corte      color claridad profundidad tabla      x      y      z
##   <int>    <dbl> <ord>    <ord> <ord>          <dbl> <dbl> <dbl> <dbl> <dbl>
## 1   326    0.23 Ideal      E     SI2          61.5   55   3.95   3.98   2.43
## 2   326    0.21 Premium    E     SI1          59.8   61   3.89   3.84   2.31
## 3   327    0.23 Bueno      E     VS1          56.9   65   4.05   4.07   2.31
## 4   334    0.29 Premium    I     VS2          62.4   58   4.2    4.23   2.63
## 5   335    0.31 Bueno      J     SI2          63.3   58   4.34   4.35   2.75
## 6   336    0.24 Muy bueno J     VVS2          62.8   57   3.94   3.96   2.48
```

3.3.5.2 La cantidad de fila

```
nrow(diamantes)
```

```
## [1] 53940
```

3.3.5.3 La cantidad de columnas (variables)

```
ncol(diamantes)
```

```
## [1] 10
```

3.3.5.4 Las dimensiones de un archivo

```
dim(diamantes)
```

```
## [1] 53940    10
```

3.3.5.5 El valor máximo de una variable

```
max(diamantes$precio)
```

```
## [1] 18823
```

3.3.5.6 El valor mínimo de una variable

```
min(diamantes$precio)
```

```
## [1] 326
```

3.3.5.7 Los valores discretos de una variable

```
unique(diamantes$corte)
```

```
## [1] Ideal    Premium  Bueno    Muy bueno Regular  
## Levels: Regular < Bueno < Muy bueno < Premium < Ideal
```

- `diamantes$corte`
- `diamantes$precio`

3.4 Ejemplos de Gráficos

```
library(readr)  
Vuelos_SJU_2018_Ene <- read_csv("Datos/Vuelos_SJU_2018_Ene.csv")
```



```
head(Vuelos_SJU_2018_Ene)
```

```
## # A tibble: 6 x 14
##   FL_DATE OP_UNIQUE_CARRIER ORIGIN ORIGIN_CITY_NAME ORIGIN_STATE_ABR DEST
##   <chr>   <chr>                <chr> <chr>                <chr>   <chr>
## 1 2/1/18  NK                      SJU   San Juan, PR        PR      MCO
## 2 2/1/18  AA                      MIA   Miami, FL           FL      SJU
## 3 2/1/18  AA                      SJU   San Juan, PR        PR      DFW
## 4 2/1/18  AA                      SJU   San Juan, PR        PR      MIA
## 5 2/1/18  AA                      SJU   San Juan, PR        PR      ORD
## 6 2/1/18  AA                      MIA   Miami, FL           FL      SJU
## # i 8 more variables: DEST_CITY_NAME <chr>, DEST_STATE_ABR <chr>,
## #   DEP_TIME <dbl>, DEP_DELAY <dbl>, CRS_ARR_TIME <dbl>, ARR_TIME <dbl>,
## #   ARR_DELAY <dbl>, CANCELLED <dbl>
```


Chapter 4

Visualización de datos

[1] "2026-09-06"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/data-visualisation.html>

Español: <https://es.r4ds.hadley.nz/03-visualize.html>

4.1 Temas:

- Introducción
 - paquete “tidyverse”
 - datos de “mgp”
 - ggplot2
 - El concepto de la gramática de gráficos
-

4.1.1 Los paquetes

```
library(tidyverse)
library(ggversa)
library(datos)
```

ggplot2 implementa la *gramática de gráficos* (*Grammar of Graphics*), un marco propuesto por **Leland Wilkinson** en 1999. **Hadley Wickham** lo llevó a R en 2007, con la idea de construir cualquier gráfico por capas. Hoy es uno de los paquetes más usados del mundo R.

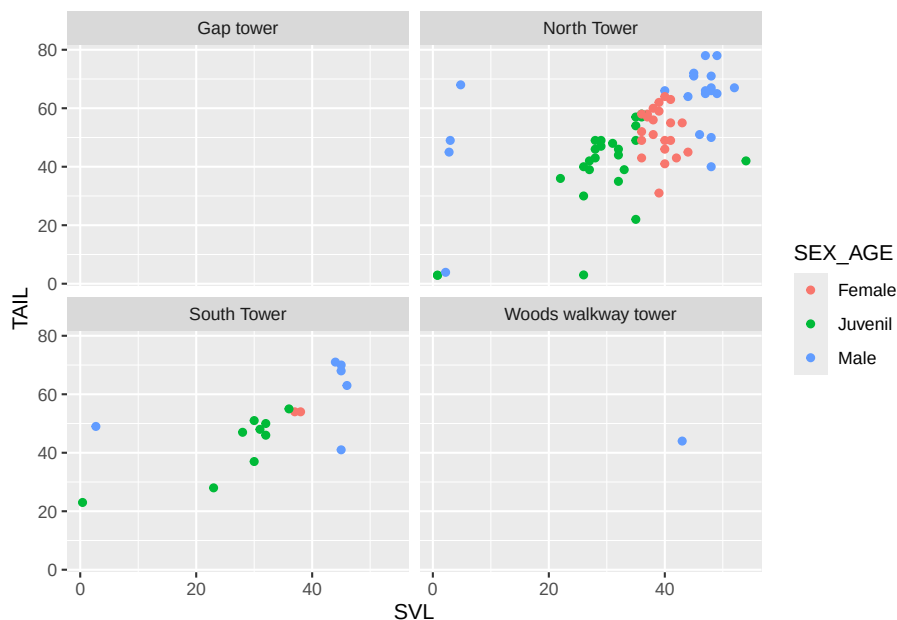
`ggplot(data) + geom_*(aes(...))` — construye gráficos por capas.

- `ggplot(data)`: define el conjunto de datos.
- `aes(x, y, color, ...)`: la *estética* — conecta variables con los ejes, el color, la forma, etc.
- `+ geom_*`: la geometría (`geom_point`, `geom_smooth`, `geom_bar`, `geom_boxplot`...). Las capas se **suman** con `+`.

4.1.2 Facet_wrap

`facet_wrap(~ variable)` divide el gráfico en un panel por cada categoría de la variable; es ideal para comparar grupos sin saturar una sola figura.

```
ggplot(data = Anolis) +
  geom_point(aes(x = SVL, y = TAIL, color=SEX_AGE)) +
  facet_wrap(~Survey_Site)
```

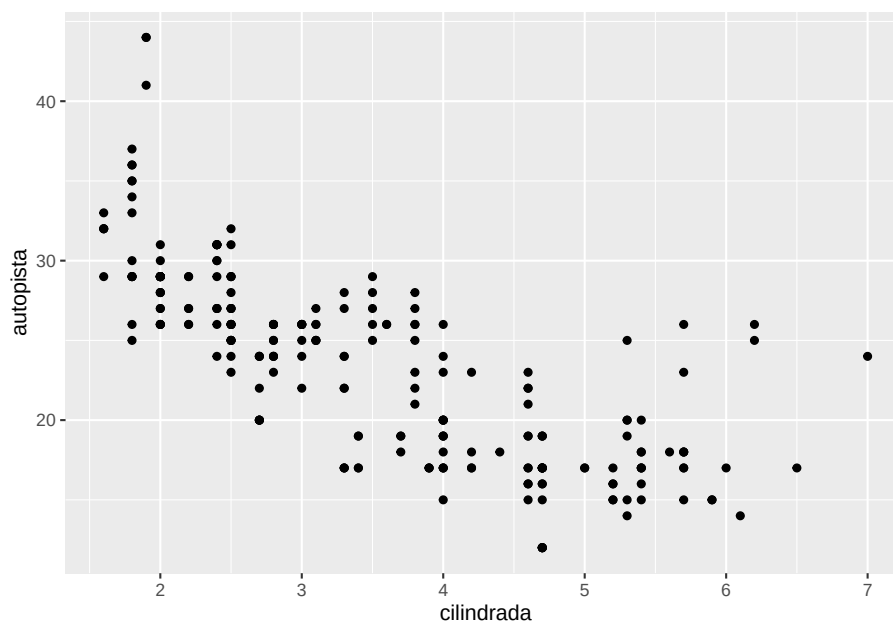


4.1.3 Regresión lineal

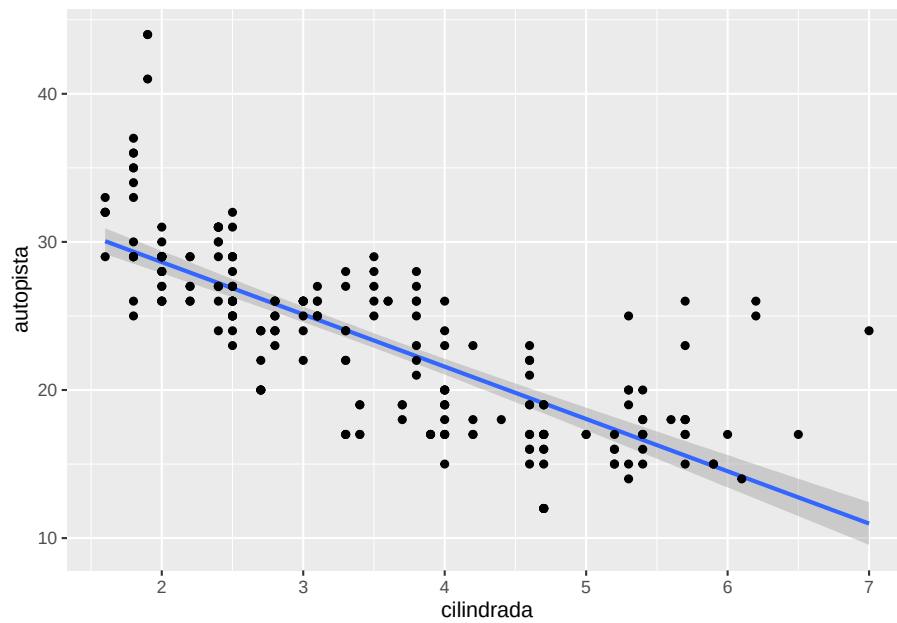
`geom_smooth(method = ...)` — añade una línea de tendencia.

- **Qué hace:** ajusta y dibuja una curva que resume la relación entre `x` e `y`, con su banda de confianza.
- `method = "lm"`: recta de regresión lineal. Sin especificar `method`, usa **LOESS** (una curva suave local) para muestras no muy grandes.
- `se = FALSE`: oculta la banda de confianza.

```
# izquierda  
ggplot(data = millas) +  
  geom_point(aes(x = cilindrada, y = autopista))
```



```
# derecha Linear Model  
ggplot(data = millas) +  
  geom_smooth(method=lm,mapping = aes(x = cilindrada, y = autopista))+ #  $y = mx+b$   
  geom_point(aes(x = cilindrada, y = autopista))
```

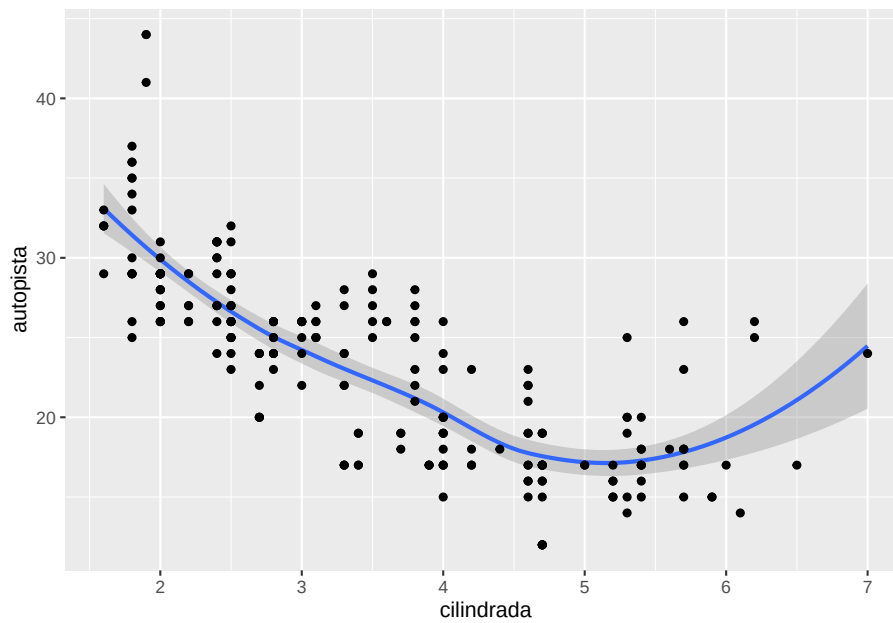


4.1.4 Regresión Loess

Vea este enlace para información sobre la regresión LOESS

<https://en.wikipedia.org/wiki/Local_regression>

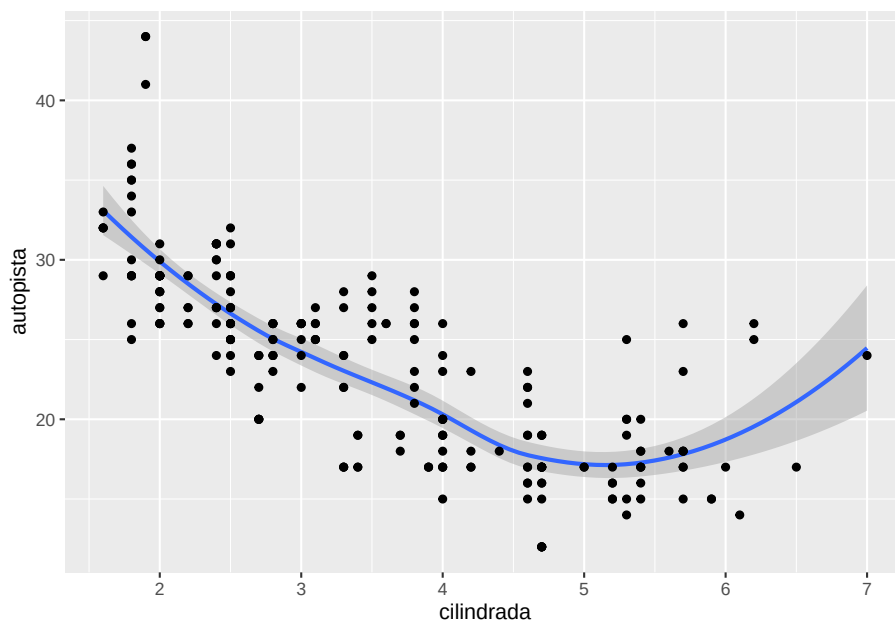
```
ggplot(data = millas) +  
  geom_smooth(mapping = aes(x = cilindrada, y = autopista)) + #  $y = mx + b$   
  geom_point(aes(x = cilindrada, y = autopista))
```



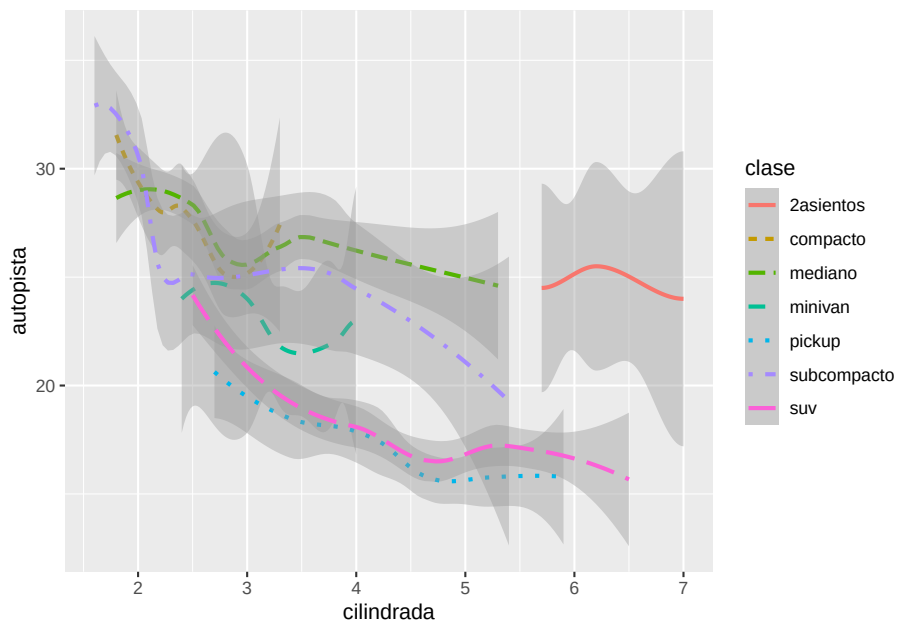
```
names(millas)
```

```
## [1] "fabricante" "modelo"      "cilindrada" "anio"      "cilindros"
## [6] "transmision" "traccion"    "ciudad"      "autopista" "combustible"
## [11] "clase"
```

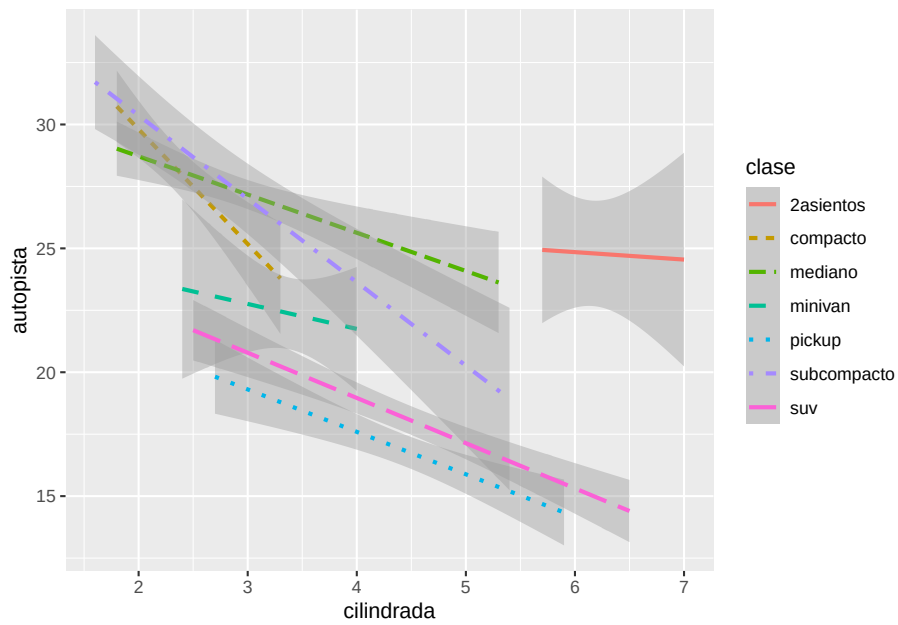
```
ggplot(data = millas) +
  geom_smooth(aes(x = cilindrada, y = autopista)) +
  geom_point(aes(x = cilindrada, y = autopista))
```



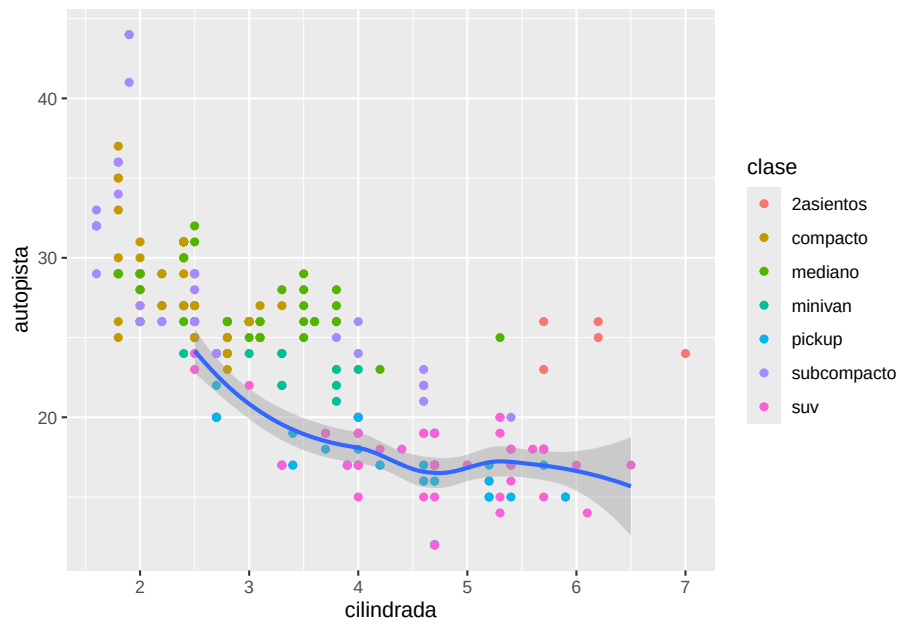
```
ggplot(data = millas) +  
  geom_smooth(aes(x = cilindrada, y = autopista, linetype = clase, colour=clase))
```



```
ggplot(data = millas) +  
  geom_smooth(method=lm, aes(x = cilindrada, y = autopista, linetype = clase, colour=clase))
```

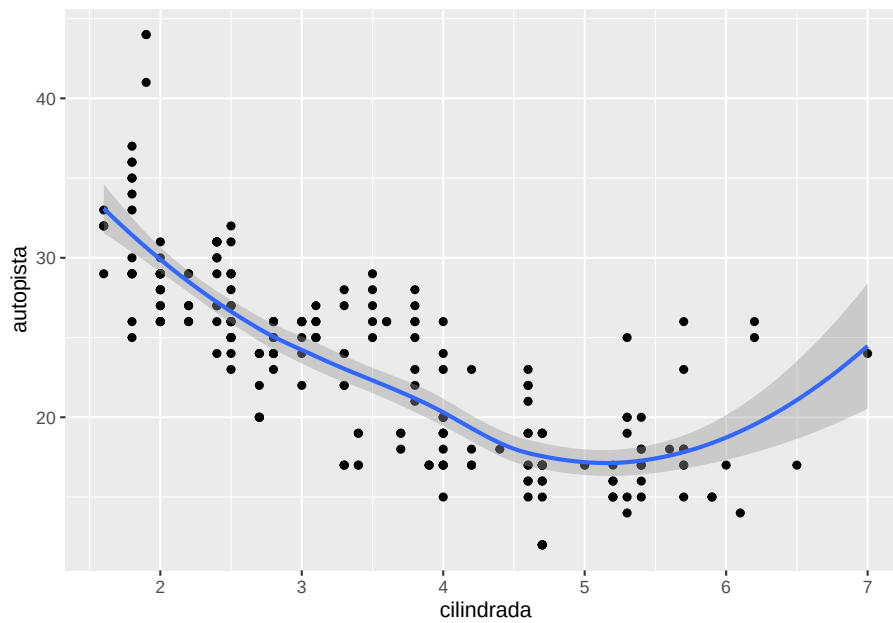



```
ggplot(millas,aes(x = cilindrada, y = autopista)) +  
  geom_point(aes(color = clase)) +  
  geom_smooth(data = filter(millas, clase == "suv"), se =TRUE)
```



```
ggplot(data = millas, mapping = aes(x = cilindrada, y = autopista)) +  
  geom_point() +  
  geom_smooth()
```

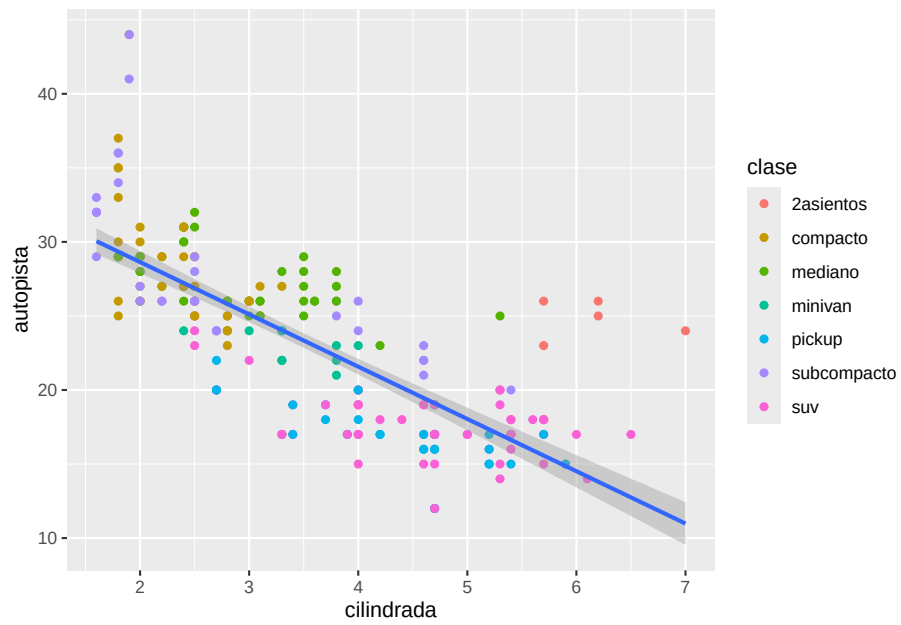
4.1.6 Regresión LOESS:



4.1.7 Regresión Lineal

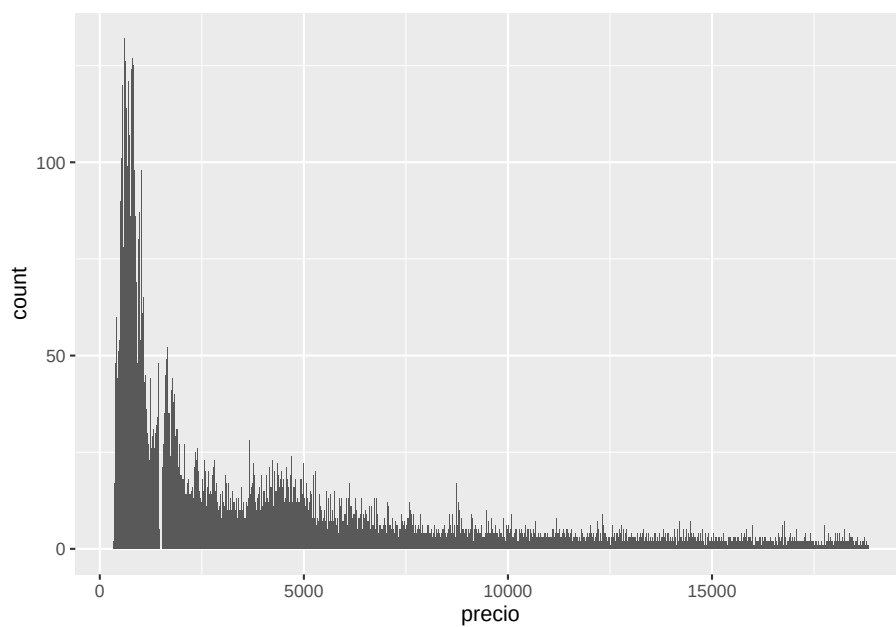
$$y = b + mx$$

```
ggplot(millas, mapping = aes(x = cilindrada, y = autopista)) +  
  geom_point(mapping = aes(color = clase)) +  
  geom_smooth(method=lm)
```

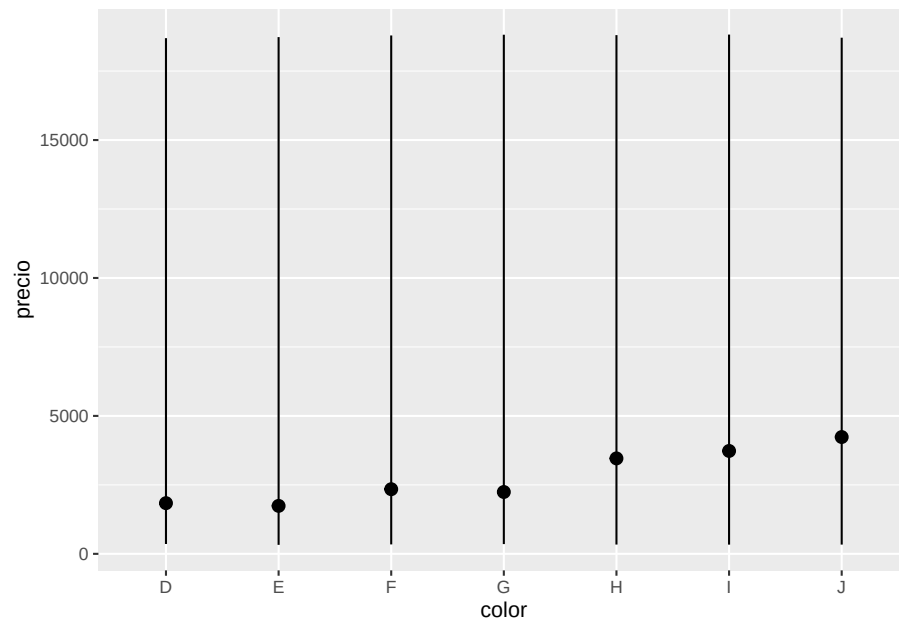


```
ggplot(diamantes) +  
  geom_bar(aes(x = precio))
```

4.1.8 geom_bar()

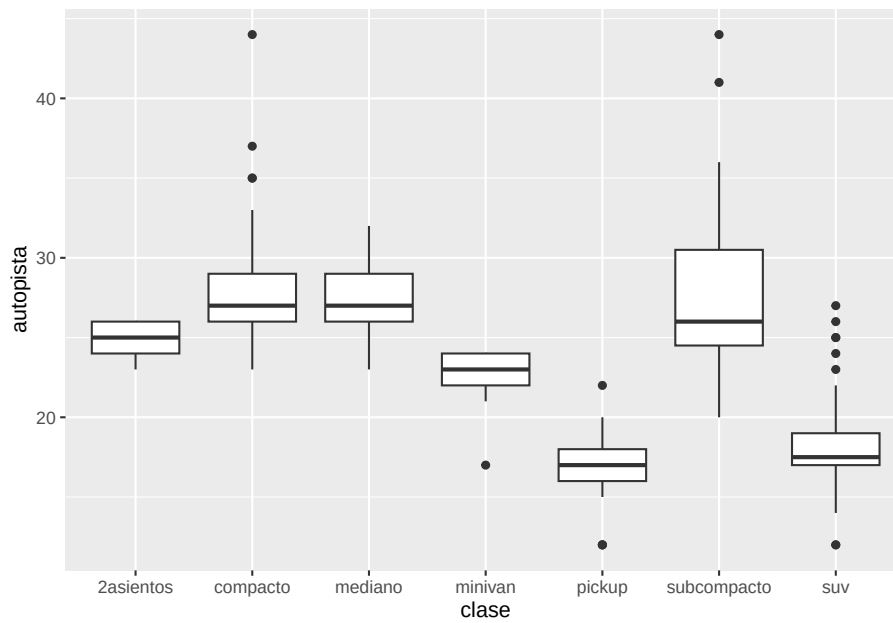


```
ggplot(diamantes) +  
  stat_summary(  
    mapping = aes(x = color, y = precio),  
    fun.min = min,  
    fun.max = max,  
    fun = median  
  )
```

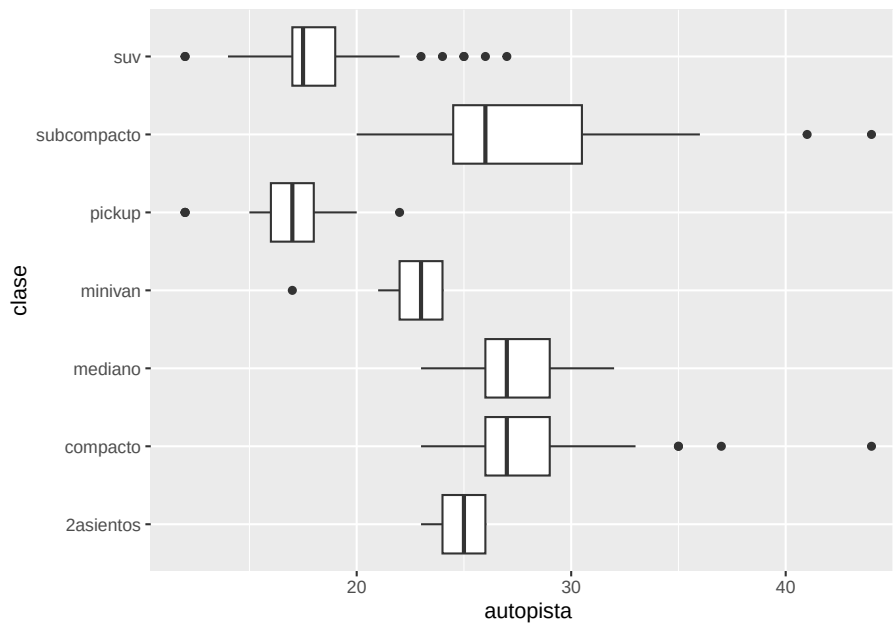


```
ggplot(millas, aes(x = clase, y = autopista)) +  
  geom_boxplot()
```

4.1.10 geom_boxplot

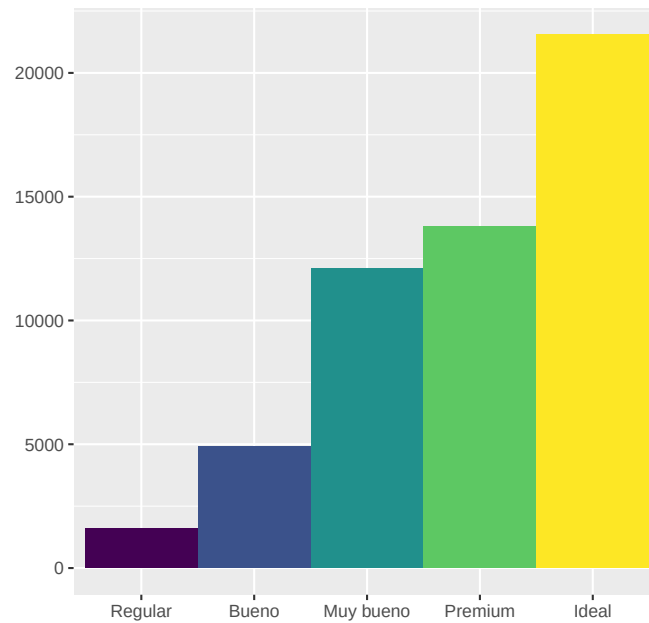


```
ggplot( millas, aes(x = clase, y = autopista)) +
  geom_boxplot() +
  coord_flip()
```

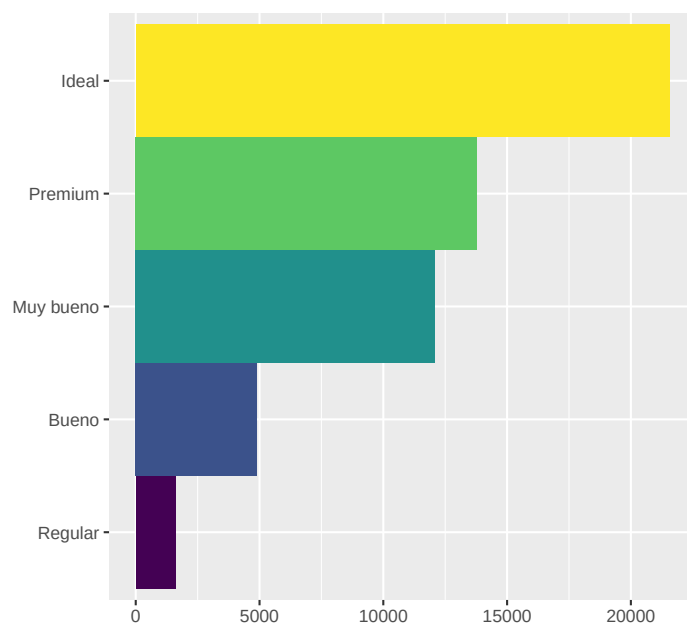


```
bar <- ggplot(data = diamantes) +  
  geom_bar(aes(x = corte, fill = corte),  
    show.legend = FALSE,  
    width = 1  
  ) +  
  theme(aspect.ratio = 1) +  
  labs(x = NULL, y = NULL)
```

bar

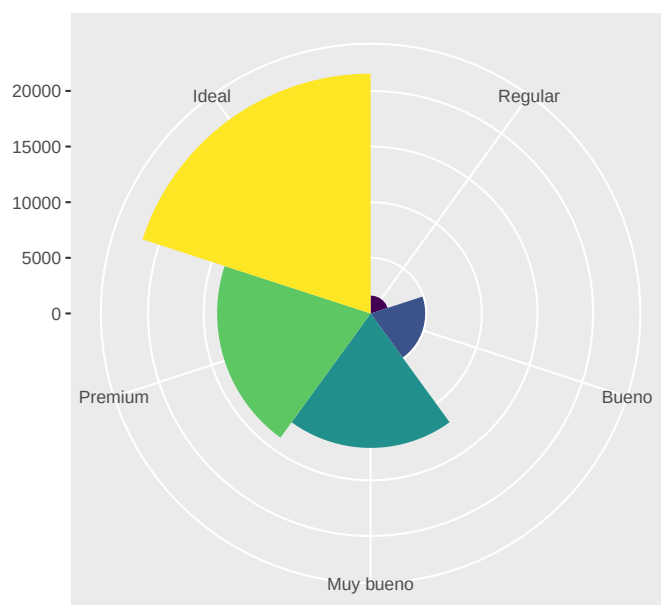


```
bar + coord_flip()
```

4.1.11.3 geom_bar & circular distribution

```
bar + coord_polar()
```



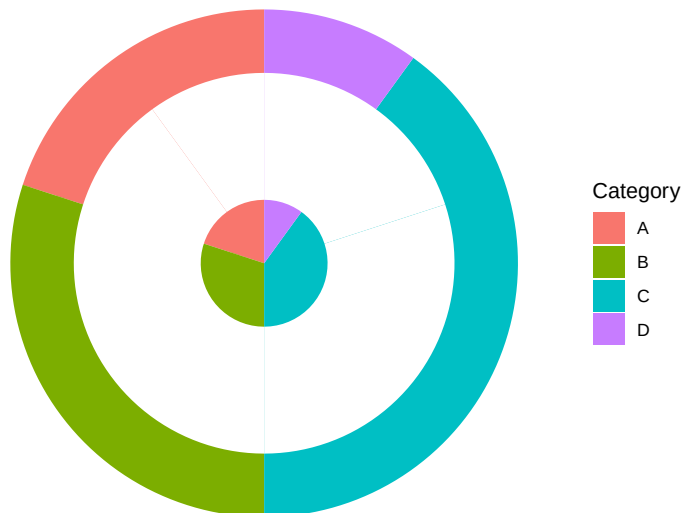
Donut Chart

```
library(ggplot2)
library(gridExtra)
data <- data.frame(
  Category = c("A", "B", "C", "D"),
  Value = c(20, 30, 40, 10)
)

pie_chart <- ggplot(data, aes(x = "", y = Value, fill = Category)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar(theta = "y") +
  theme_void() +
  labs(fill = "Category")

donut_chart <- pie_chart +
  geom_bar(data = data, aes(x = "", y = Value), stat = "identity",
    width = 0.5, fill = "white")+
  theme_void()

donut_chart
```



4.2 Leer datos de un archivo

1. Ejercicios:

Hacer los ejercicios en la sección 3.2.4 del libro en español

-
- Estética
-

2. Ejercicios:

Hacer los ejercicios en la sección 3.3.1 del libro en español

-
- Problemas comunes
 - Separar en facetas
-

3. Ejercicios:

Hacer los ejercicios en la sección 3.5.1 del libro en español

-
- Objetos geométricos ***

4. Ejercicios:

Hacer los ejercicios en la sección 3.6.1 del libro en español

-
- Transformación estadísticas
-

5. Ejercicios:

Hacer los ejercicios en la sección 3.7.1 del libro en español

-
- Ajuste de posición
-

6. Ejercicios:

Hacer los ejercicios en la sección 3.8.1 del libro en español

-
- Sistema de coordenadas
-

7. Ejercicios:

Hacer los ejercicios en la sección 3.9.1 del libro en español

Ejercicio para entregar (6 puntos)

1. Activa el paquete “ggversa”
2. Activa el paquete “tidyverse”
3. Utiliza los datos “PartosInfantes”. Leen la información sobre el archivo
Son tres graficas que tendrán que someter
4. Hacer un gráfico de puntos entre el número de muertes de infante y la cantidad de madres que mueren en el parto. (1 punto)
5. Añadir al gráfico anterior un modelo lineal (linear model). Y Demostrando todos los datos con un color por región geográfica, o sea añadir un color a los puntos por Grupo “region geográfica”. AM=America, EU= Union Europea, AF= Africa, O=Oceania, AS=Asia, Medio Oriente, (2 puntos)
6. Enseña el modelo de regresión lineal solamente para AFRICA y ASIA (en la misma gráfica) (3 puntos)

Someter las tres gráficas en formato .jpeg o .png en el portal.

```
library(ggplot2)
library(tidyverse)
library(ggversa)
```

```
head(PartosInfantes)
```

##	NMI	NMP	GSPC	Grupo	Pais
## 1	8723	400	605.1878	AM	Argentina
## 2	60	5	1720.1595	AM	Bahamas
## 3	42	1	1146.0417	AM	Barbados
## 4	121	2	278.5792	AM	Belize
## 5	7756	540	208.7842	AM	Bolivia
## 6	45682	1400	947.4277	AM	Brazil

```
#unique(PartosInfantes$Pais)
```

Hacer un gráfico de puntos entre el número de muertes por infante y la cantidad de madres que mueren en el parto

Añadir al gráfico anterior un modelo lineal (linear model)

Añadir un color a los puntos por grupo “region”. AM=America, EU= Union Europea, AF= Africa, O=Oceania, AS=Asia, Medio Oriente,

Demostrando todos los datos con un color por región geográfica, enseña el modelo de regresión lineal solamente para AFRICA

4.2.1 Ejercicios

Buscar el error en el código.

1. ¿Qué no va bien en este código? ¿Por qué hay puntos que no son azules en el siguiente código?

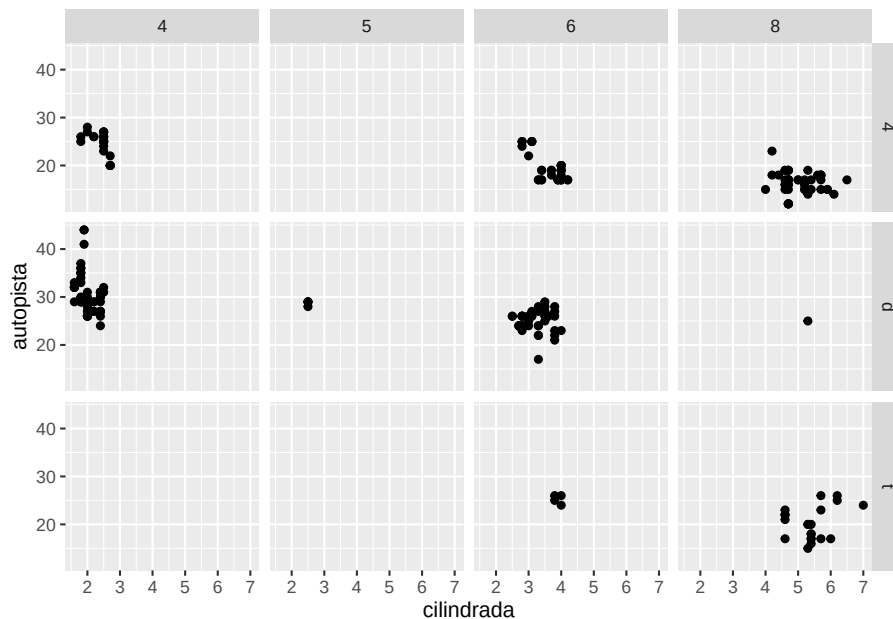
```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista, color = "blue"))
```

2. Por que la gráfica no se produce en el siguiente código

```
ggplot(data = millas)
+ geom_point(mapping = aes(x = cilindrada, y = autopista))
```

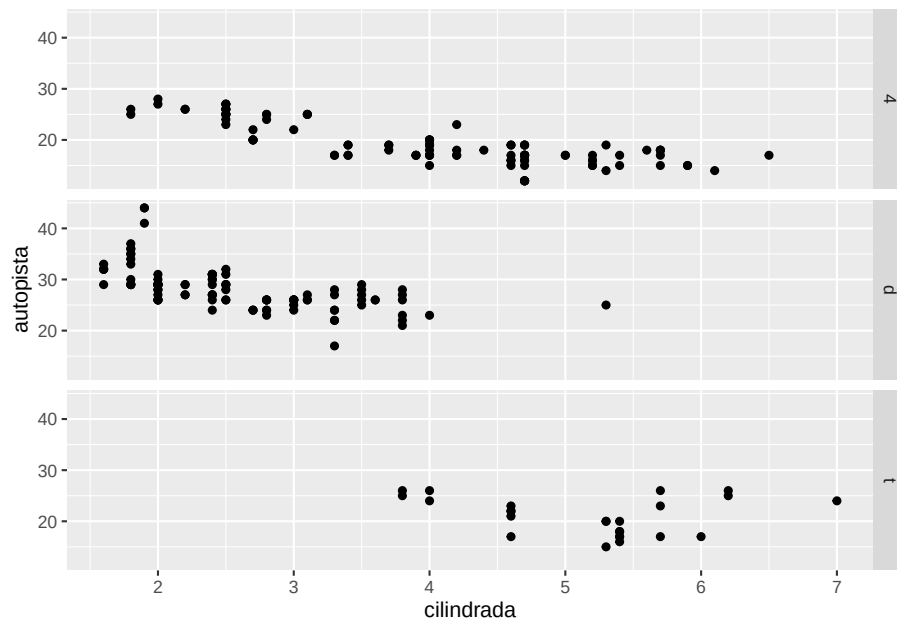
4.3 Vea el siguiente gráfico

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +
  facet_grid(traccion ~ cilindros)
```

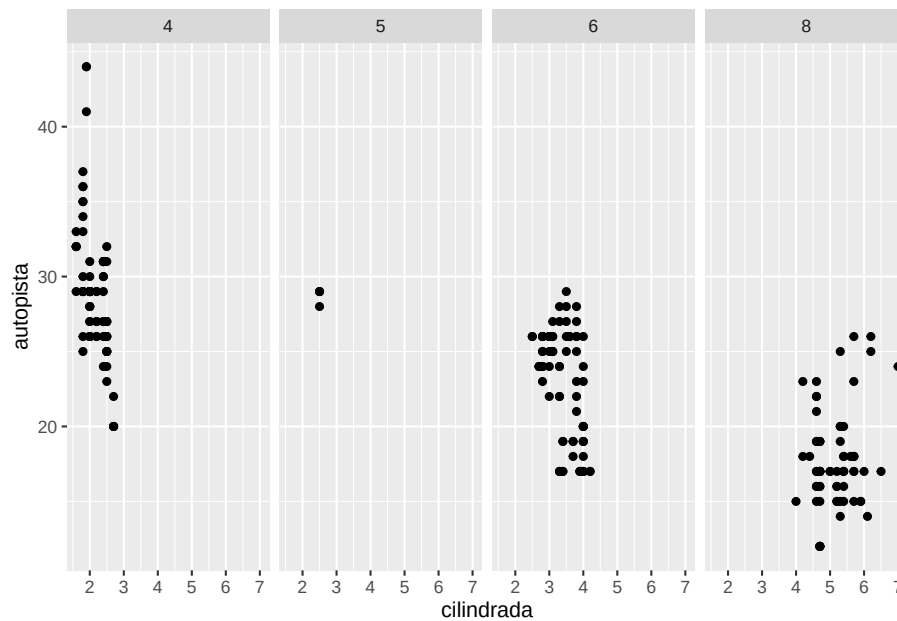


3. ¿Por qué hay facetas que son vacías?
4. Explica que hace "."? en estas funciones

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +
  facet_grid(traccion ~ .)
```

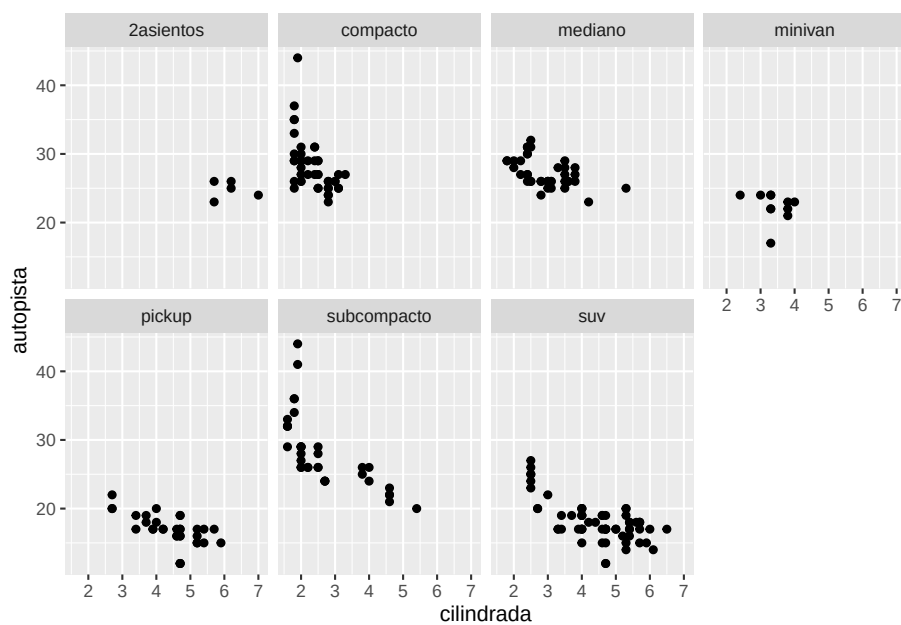


```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +
  facet_grid(. ~ cilindros)
```



5. Qué hace **nrow** en este código

```
ggplot(data = millas) +  
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +  
  facet_wrap(~ clase, nrow = 2)
```



Chapter 5

Calculadora sofisticada

Fecha de la última revisión

```
## [1] "2026-09-06"
```

Activar los siguientes paquetes

```
library(tidyverse)
library(datos)
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/workflow-basics.html>

Español: <https://es.r4ds.hadley.nz/04-workflow-basics.html>

5.1 Temas:

- Conocimientos básicos de programación
- La importancia de los nombres
- Usando funciones

5.1.1 Sumar

```
2+2
```

```
## [1] 4
```

5.1.2 Exactamente como una calculadora

```
(23+3+3)^2+2
```

```
## [1] 843
```

5.1.3 Trigonometría

```
sin(pi/3)
```

```
## [1] 0.8660254
```

5.1.4 Log natural y a base 10

```
log(100)
```

```
## [1] 4.60517
```

```
log10(100)
```

```
## [1] 2
```

5.1.5 Asignaciones de variables

Para asignar un valor puedes usar `<-` o `=`, pero para **comparar** dos cosas se usa `==` (doble igual). Confundir `=` con `==` es uno de los errores más comunes al empezar. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
x<- 4  
x
```

```
## [1] 4
```

```
y=3  
y
```

```
## [1] 3
```

```
x/y
```

```
## [1] 1.333333
```

5.1.6 Variables

- la función “seq” para secuencia

`seq()`, `c()`, `length()` — crear y medir vectores.

- `c(1, 2, 3)`: combina valores en un vector; `11:20` genera una secuencia consecutiva.
- `seq(101, 110)`: secuencia con más control (inicio, fin, paso).
- `length(x)`: cuántos elementos tiene un vector.

```
edad_nd=seq(101, 110)
edad_nd
```

```
## [1] 101 102 103 104 105 106 107 108 109 110
```

- variables asignada

```
edad_uprh=c(11:20)
edad_uprh
```

```
## [1] 11 12 13 14 15 16 17 18 19 20
```

5.1.7 Unir variables en un data frame

```
edad=data.frame(edad_nd, edad_uprh)
edad
```

```
##      edad_nd edad_uprh
## 1         101         11
## 2         102         12
## 3         103         13
## 4         104         14
## 5         105         15
## 6         106         16
## 7         107         17
## 8         108         18
## 9         109         19
## 10        110         20
```

```
edad$diff=edad$edad_nd-edad$edad_uprh
edad
```

```
##      edad_nd edad_uprh diff
## 1         101         11   90
## 2         102         12   90
## 3         103         13   90
## 4         104         14   90
## 5         105         15   90
## 6         106         16   90
## 7         107         17   90
## 8         108         18   90
## 9         109         19   90
## 10        110         20   90
```

```
edad$tres=edad$edad_uprh^3
edad
```

```
##      edad_nd edad_uprh diff tres
## 1         101         11   90 1331
## 2         102         12   90 1728
## 3         103         13   90 2197
## 4         104         14   90 2744
## 5         105         15   90 3375
## 6         106         16   90 4096
## 7         107         17   90 4913
## 8         108         18   90 5832
## 9         109         19   90 6859
## 10        110         20   90 8000
```

5.1.8 Errores cuando a variables no tienen la misma cantidad de entradas

```
edad_nd_1=base::seq(101, 111)
edad_nd_1
```

```
## [1] 101 102 103 104 105 106 107 108 109 110 111
```

- variables asignada

```
edad_uprh_1=c(11:20, NA)
edad_uprh_1
```

```
## [1] 11 12 13 14 15 16 17 18 19 20 NA
```

5.1.9 Unir variables en un data frame

```
edad_1=data.frame(edad_nd_1, edad_uprh_1)
edad_1
```

```
##      edad_nd_1 edad_uprh_1
## 1          101          11
## 2          102          12
## 3          103          13
## 4          104          14
## 5          105          15
## 6          106          16
## 7          107          17
## 8          108          18
## 9          109          19
## 10         110          20
```

```
## 11      111      NA
```

5.1.10 Cuantos valores hay en una variable

```
length(edad_uprh)
```

```
## [1] 10
```

```
length(edad$tres)
```

```
## [1] 10
```

```
length(edad)
```

```
## [1] 4
```

5.1.11 Seleccionar solamente parte de los datos de un archivo

- Primero visualizar los datos

```
library(datos)
```

```
millas
```

```
## # A tibble: 234 x 11
```

##	fabricante	modelo	cilindrada	anio	cilindros	transmision	traccion	ciudad
##	<chr>	<chr>	<dbl>	<int>	<int>	<chr>	<chr>	<int>
## 1	audi	a4	1.8	1999	4	auto(15)	d	18
## 2	audi	a4	1.8	1999	4	manual(m5)	d	21
## 3	audi	a4	2	2008	4	manual(m6)	d	20
## 4	audi	a4	2	2008	4	auto(av)	d	21
## 5	audi	a4	2.8	1999	6	auto(15)	d	16
## 6	audi	a4	2.8	1999	6	manual(m5)	d	18
## 7	audi	a4	3.1	2008	6	auto(av)	d	18
## 8	audi	a4 quattro	1.8	1999	4	manual(m5)	4	18
## 9	audi	a4 quattro	1.8	1999	4	auto(15)	4	16
## 10	audi	a4 quattro	2	2008	4	manual(m6)	4	20

```
## # i 224 more rows
```

```
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

5.1.11.1 Seleccionar los carros que tienen cilindros igual a 8 (solamente)

El pipe %>% lo creó Stefan Milton Bache en el paquete `magrittr` (2014); su nombre juega con el cuadro de Magritte “Ceci n’est pas une pipe”. Desde 2021, R incluye su propio pipe nativo `|>`.

La función de Pipe. %>% o |>

`select()` y `filter()` (paquete **dplyr**) — elegir columnas y filas.

- `select(columna)`: se queda con las **columnas** indicadas.
- `filter(condición)`: se queda con las **filas** que cumplen una condición (p. ej. `cilindros == 8`).
- El pipe `|>` / `%>%`: encadena pasos, pasando el resultado de uno como entrada del siguiente.

```
library(tidyverse)
millas
```

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>      <chr>          <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi      a4              1.8  1999      4 auto(15)    d        18
## 2 audi      a4              1.8  1999      4 manual(m5)  d        21
## 3 audi      a4              2    2008      4 manual(m6)  d        20
## 4 audi      a4              2    2008      4 auto(av)    d        21
## 5 audi      a4              2.8  1999      6 auto(15)    d        16
## 6 audi      a4              2.8  1999      6 manual(m5)  d        18
## 7 audi      a4              3.1  2008      6 auto(av)    d        18
## 8 audi      a4 quattro      1.8  1999      4 manual(m5)  4        18
## 9 audi      a4 quattro      1.8  1999      4 auto(15)    4        16
## 10 audi     a4 quattro      2    2008      4 manual(m6)  4        20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

```
names(millas)
```

```
## [1] "fabricante" "modelo"      "cilindrada"  "anio"        "cilindros"
## [6] "transmision" "traccion"    "ciudad"      "autopista"   "combustible"
## [11] "clase"
```

```
millas |> dplyr::select(cilindros) |>
dplyr::filter(cilindros == 8)
```

```
## # A tibble: 70 x 1
##   cilindros
##   <int>
## 1      8
## 2      8
## 3      8
## 4      8
## 5      8
## 6      8
## 7      8
## 8      8
## 9      8
## 10     8
```

```
## # i 60 more rows
```

5.1.11.2 Seleccionar los diamantes mayor de 3 quilate

```
diamantes |> dplyr::select(quilate) |>  
  filter(quilate > 3)
```

```
## # A tibble: 32 x 1  
##   quilate  
##   <dbl>  
## 1     3.01  
## 2     3.11  
## 3     3.01  
## 4     3.05  
## 5     3.02  
## 6     3.01  
## 7     3.65  
## 8     3.24  
## 9     3.22  
## 10    3.5  
## # i 22 more rows
```

5.1.11.3 Seleccionar los diamantes con igual o mayor de 4000 precio

```
## # A tibble: 6 x 1  
##   precio  
##   <int>  
## 1    4000  
## 2    4001  
## 3    4001  
## 4    4001  
## 5    4001  
## 6    4002
```

5.1.11.4 Contabiliza cuantos diamantes tienen un valor de \$4000 o más

```
nrow(dia_4000)
```

```
## [1] 19380
```


Chapter 6

Transformación: Estructura de un archivo y seleccionar datos

```
## [1] "2026-09-06"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/transform.html>

Español: <https://es.r4ds.hadley.nz/05-transform.html>

```
library(tidyverse)
library(datos)
head(vuelos)
```

```
## # A tibble: 6 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>         <dbl>
## 1  2013     1     1           517               515             2
## 2  2013     1     1           533               529             4
## 3  2013     1     1           542               540             2
## 4  2013     1     1           544               545            -1
## 5  2013     1     1           554               600            -6
## 6  2013     1     1           554               558            -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
```

```
## # hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

6.1 Evaluar el archivo el vuelo

Mirar el archivo y entender la información antes de hacer cualquier análisis/

- Cuáles son las variables
- Cuáles son los tipos de datos
 - **int** valores enteros,
 - **dbl** números reales, significa dobles
 - **chr** caracteres o vectores
 - **dtm** fecha y tiempo, minutos
 - **date** fecha
 - **S3: POSIXct** Fecha y tiempo, minutos
 - **lgl** valores lógicos “cierto” (TRUE) y “falso” (FALSE)
 - **factr** factores categóricos.

Evalua en este archivo los tipos de variables que se encuentra.

```
vuelos
```

```
## # A tibble: 336,776 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>          <dbl>
## 1 2013     1     1           517               515            2
## 2 2013     1     1           533               529            4
## 3 2013     1     1           542               540            2
## 4 2013     1     1           544               545           -1
## 5 2013     1     1           554               600           -6
## 6 2013     1     1           554               558           -4
## 7 2013     1     1           555               600           -5
## 8 2013     1     1           557               600           -3
## 9 2013     1     1           557               600           -3
## 10 2013     1     1           558               600           -2
## # i 336,766 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

6.1.1 Cual son todos los destinos de vuelos?

- cual función usará

```
sort(unique(vuelos$destino))
```

```
## [1] "ABQ" "ACK" "ALB" "ANC" "ATL" "AUS" "AVL" "BDL" "BGR" "BHM" "BNA" "BOS"
## [13] "BQN" "BTV" "BUF" "BUR" "BWI" "BZN" "CAE" "CAK" "CHO" "CHS" "CLE" "CLT"
```

```
## [25] "CMH" "CRW" "CVG" "DAY" "DCA" "DEN" "DFW" "DSM" "DTW" "EGE" "EYW" "FLL"
## [37] "GRR" "GSO" "GSP" "HDN" "HNL" "HOU" "IAD" "IAH" "ILM" "IND" "JAC" "JAX"
## [49] "LAS" "LAX" "LEX" "LGA" "LGB" "MCI" "MCO" "MDW" "MEM" "MHT" "MIA" "MKE"
## [61] "MSN" "MSP" "MSY" "MTJ" "MVY" "MYR" "OAK" "OKC" "OMA" "ORD" "ORF" "PBI"
## [73] "PDX" "PHL" "PHX" "PIT" "PSE" "PSP" "PVD" "PWM" "RDU" "RIC" "ROC" "RSW"
## [85] "SAN" "SAT" "SAV" "SBN" "SDF" "SEA" "SFO" "SJC" "SJU" "SLC" "SMF" "SNA"
## [97] "SRQ" "STL" "STT" "SYR" "TPA" "TUL" "TVC" "TYS" "XNA"
```

6.1.2 Selecciona los vuelos donde el destino es San Juan

```
vuelos %>%
  filter(destino=="SJU")
```

```
## # A tibble: 5,819 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>          <dbl>
## 1  2013     1     1           615               615            0
## 2  2013     1     1           628               630           -2
## 3  2013     1     1           701               700            1
## 4  2013     1     1           711               715           -4
## 5  2013     1     1           820               820            0
## 6  2013     1     1           820               820            0
## 7  2013     1     1           840               845           -5
## 8  2013     1     1           926               929           -3
## 9  2013     1     1          1202              1159            3
## 10 2013     1     1          1245              1249           -4
## # i 5,809 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

6.1.3 Selecciona los vuelos donde el destino es San Juan el día de navidad

Para comparar dentro de `filter()` se usa `==` (doble igual); un solo `=` es asignación y dará error. Para comparar contra varios valores, usa `%in% c("AA", "UA")` en vez de repetir `==`. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
vuelos %>%
  filter(destino=="SJU") %>%
  filter(mes==12, dia==25) %>%
  filter(aerolinea == "UA")
```

```
## # A tibble: 3 x 19
```

```
##      año   mes   día horario_salida salida_programada atraso_salida
##      <int> <int> <int>          <int>          <int>          <dbl>
## 1  2013    12    25             736             739           -3
## 2  2013    12    25            1202            1206           -4
## 3  2013    12    25            2015            2017           -2
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
vuelos %>%
  filter(destino=="SJU") %>%
  filter(mes==12, día==25) %>%
  filter(aerolinea %in% c("AA", "UA"))
```

```
## # A tibble: 6 x 19
##      año   mes   día horario_salida salida_programada atraso_salida
##      <int> <int> <int>          <int>          <int>          <dbl>
## 1  2013    12    25             736             739           -3
## 2  2013    12    25             759             805           -6
## 3  2013    12    25            1202            1206           -4
## 4  2013    12    25            1634            1630            4
## 5  2013    12    25            1908            1900            8
## 6  2013    12    25            2015            2017           -2
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
unique(vuelos$aerolinea)
```

```
## [1] "UA" "AA" "B6" "DL" "EV" "MQ" "US" "WN" "VX" "FL" "AS" "9E" "F9" "HA" "YV"
## [16] "OO"
```

```
# /> pipe Command shift M
```

6.1.4 Selecciona los vuelos que salen de San Juan en el día de tu cumpleaños y contabiliza cuanto hubo

```
df_día_cumpl=vuelos %>%
  filter(destino=="SJU") %>%
  filter(mes== 2, día %in% c(1,2)) |>
  nrow()

df_día_cumpl
```

```
## [1] 28
```

6.2. TEMAS: RECONOCER Y APLICAR LAS DIFERENTES FUNCIONES 69

6.1.4.1 Organizar los datos en orden de más grande a más pequeño

- Selecciona los mes, dia, destino, atraso_salida, atraso_llegada
- Selecciona solamente el mes de noviembre
- Selecciona solamente los vuelos a destino de a San Juan
- organiza los datos en orden de más atraso_salido de mayor a menor

```
names(vuelos)

## [1] "anio"          "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
## [19] "fecha_hora"

vuelos %>%
  dplyr::select(mes, dia, destino, atraso_salida, atraso_llegada) %>%
  filter(mes==11) %>%
  filter(destino == "SJU") %>%
  arrange(desc(atraso_salida))

## # A tibble: 429 x 5
##   mes dia destino atraso_salida atraso_llegada
##   <int> <int> <chr>          <dbl>          <dbl>
## 1  11 7 SJU          231            190
## 2  11 28 SJU         223            235
## 3  11 23 SJU         187            154
## 4  11 24 SJU         181            188
## 5  11 23 SJU         157            165
## 6  11 12 SJU         145            127
## 7  11 10 SJU          94             88
## 8  11 24 SJU          90            112
## 9  11 30 SJU          68             59
## 10 11 21 SJU          66             42
## # i 419 more rows
```

6.2 Temas: Reconocer y aplicar las diferentes funciones

- Tipos de variables:
 - int: valores enteros
 - dbl: números reales
 - chr: caracteres
 - dttm: fecha y tiempo

- lgl: valores logicos
- fctr: factores
- date: fecha

-
- Funciones de dplyr:
 - filter() : seleccionar filas
 - arrange(): ordenar filas
 - select(): seleccionar columnas
 - summarize(): resumir conjuntos de datos
 - group_by(): agrupar por variables

Los verbos de dplyr — cada uno hace una cosa y se encadenan con el pipe.

- **filter()**: selecciona **filas** por condición.
- **select()**: selecciona **columnas**.
- **arrange()**: **ordena** las filas (**desc()** para descendente).
- **group_by()** + **summarise()**: agrupa y calcula un **resumen** por grupo (media, mediana, conteo...).
- **mutate()**: crea columnas nuevas (ver capítulo de *mutate*).

dplyr fue creado por **Hadley Wickham** (2014) como una “gramática de la manipulación de datos”: un pequeño conjunto de verbos con nombres de acción. Sustituyó al paquete anterior **plyr** con una sintaxis más clara y rápida.

```
library(datos)
library(tidyverse)
por_dia <- vuelos %>% # pipe
  group_by(dia) %>% # group by day
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE))
por_dia
```

```
## # A tibble: 31 x 2
##       dia atraso_promedio
##   <int>         <dbl>
## 1     1           14.2
## 2     2           14.1
## 3     3           10.8
## 4     4            5.79
## 5     5            7.82
## 6     6            6.99
## 7     7           14.3
## 8     8           21.8
## 9     9           14.6
## 10    10           18.3
## # i 21 more rows
```

```
## The median of the data set by day of the month

por_dia_2 <- vuelos %>%
  group_by( dia) %>%
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE),
            atraso_mediana = median(atraso_salida, na.rm=TRUE))
por_dia_2
```

```
## # A tibble: 31 x 3
##       dia atraso_promedio atraso_mediana
##   <int>          <dbl>          <dbl>
## 1     1           14.2            -2
## 2     2           14.1            -1
## 3     3           10.8            -2
## 4     4            5.79           -3
## 5     5            7.82           -3
## 6     6            6.99           -2
## 7     7           14.3            -1
## 8     8           21.8            -1
## 9     9           14.6            -1
## 10    10           18.3            -1
## # i 21 more rows
```

6.3 Funciones

- summarise = resumir conjuntos de datos
- mean, # promedio
- median, # mediana
- mode, # moda
- operaciones boolean
-

```
# El valor más común en un conjunto de datos
# Crear una función: no existe en los paquetes de R

getmode <- function(v) {
  uniqv <- unique(v)
  uniqv[which.max(tabulate(match(v, uniqv)))]
}
```

```

por_dia_3=vuelos |>
  dplyr::select(dia, mes, atraso_salida) |>
  group_by(dia) %>%
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE),
            atraso_max=max(atraso_salida, na.rm=TRUE),
            atraso_mode = getmode(atraso_salida))
por_dia_3

```

```

## # A tibble: 31 x 4
##       dia atraso_promedio atraso_max atraso_mode
##   <int>          <dbl>      <dbl>      <dbl>
## 1     1           14.2        853         -5
## 2     2           14.1        696         -4
## 3     3           10.8        878         -4
## 4     4            5.79        545         -5
## 5     5            7.82        896         -5
## 6     6            6.99        589         -4
## 7     7           14.3        653         -5
## 8     8           21.8        520          NA
## 9     9           14.6       1301         -4
## 10    10           18.3       1126         -5
## # i 21 more rows

```

Añadir el promedio, mediana y moda al mismo data frame

```

## The mode of the value
library(DescTools)

por_dia_4 <- vuelos %>%
  group_by(dia) %>%
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE),
            atraso_median = median(atraso_salida, na.rm=TRUE),
            atraso_mode = getmode(atraso_salida))
por_dia_4

```

```

## # A tibble: 31 x 4
##       dia atraso_promedio atraso_median atraso_mode
##   <int>          <dbl>      <dbl>      <dbl>
## 1     1           14.2        -2         -5
## 2     2           14.1        -1         -4
## 3     3           10.8        -2         -4
## 4     4            5.79        -3         -5
## 5     5            7.82        -3         -5
## 6     6            6.99        -2         -4
## 7     7           14.3        -1         -5
## 8     8           21.8        -1          NA
## 9     9           14.6        -1         -4

```



```
## 10      10      18.3      -1      -5
## # i 21 more rows
```

6.4 Agrupar por múltiples variables

Aquí se agrupar por día y mes

```
library(tidyverse)
library(datos)
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_llegada, destino, aerolinea) %>%
  filter(destino == "SJU") %>%
  filter(dia == 25) %>%
  # filter(mes == 12) %>%
  group_by(dia, aerolinea) %>%
  summarise(atraso = mean(atraso_llegada, na.rm = TRUE))

## # A tibble: 4 x 3
## # Groups:   dia [1]
##   dia aerolinea atraso
##   <int> <chr>      <dbl>
## 1    25 AA        -5.41
## 2    25 B6         4.21
## 3    25 DL        -4.19
## 4    25 UA        -4.33
```


Chapter 7

Transformación: Operaciones booleanas y lógicas

Fecha de la última revisión

```
## [1] "2026-09-06"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/transform.html>

Español: <https://es.r4ds.hadley.nz/05-transform.html>

```
library(tidyverse)
library(datos)
head(vuelos)
```

```
## # A tibble: 6 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>         <dbl>
## 1  2013     1     1           517               515             2
## 2  2013     1     1           533               529             4
## 3  2013     1     1           542               540             2
## 4  2013     1     1           544               545            -1
## 5  2013     1     1           554               600            -6
## 6  2013     1     1           554               558            -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
```

```
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

Tema de este módulo:

- Filtrar filas con `filter()`
- Seleccionar columnas con `select()`
- Agrupar con `group_by()`
- Resumir con `summarise()`
- Ordenar con `arrange()`
- funciones Booleanas y lógicas

7.1 Agrupar por múltiples variables

Aquí se agrupa por día y mes

7.2 Operaciones lógicas boolean:

- `&` “ampersand” == “conjunción”
- `|` or = “o”
- `!`, no incluye, excluir...
- `%in%`, filtrar para múltiples valores
- `==`, es igual a... “exactamente”
- `<` “menor que”
- `>` “mayor que”
- `<=` “es menor o igual”
- `>=` “es mayor o igual”

`filter()` con operadores lógicos — combinar condiciones.

- `&` (y), `|` (o), `!` (no / excluir).
- `%in% c(...)`: ¿el valor está en esta lista? (evita repetir `==`).
- `between(x, a, b)`: equivale a `x >= a & x <= b`.
- **Comparaciones:** `==`, `!=`, `<`, `>`, `<=`, `>=`.

Boolean, que incluye múltiples opciones `%in%`

No confundas `=` con `==`. Un solo `=` asigna un valor, mientras que `==` compara si dos valores son iguales; dentro de `filter()` siempre se usa `==`. Para comparar contra varios valores usa `%in%` en lugar de encadenar muchos `==`. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_llegada, destino) %>%
  filter(destino == "SJU") %>%
  filter(mes %in% c(6:12) & dia == 25) %>%
  group_by(dia, mes) %>%
  summarise(atraso = mean(atraso_llegada, na.rm = TRUE))
```

```
## # A tibble: 7 x 3
## # Groups:   dia [1]
##   dia   mes atraso
##   <int> <int> <dbl>
## 1    25     6  34.3
## 2    25     7   3.58
## 3    25     8  11.1
## 4    25     9 -15.2
## 5    25    10  -3.77
## 6    25    11 -18.9
## 7    25    12   3.57
```

```
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_salida, aerolinea) %>%
  group_by(mes, aerolinea) %>%
  filter(mes %in% c(1:6)) %>% # mes entre 1 y 6
  filter(aerolinea %in% c("AA", "UA")) %>%
  summarise(atraso = mean(atraso_salida, na.rm = TRUE))
```

```
## # A tibble: 12 x 3
## # Groups:   mes [6]
##   mes aerolinea atraso
##   <int> <chr>      <dbl>
## 1     1 AA         6.93
## 2     1 UA         8.33
## 3     2 AA         8.28
## 4     2 UA         7.71
## 5     3 AA         8.70
## 6     3 UA        11.7
## 7     4 AA        11.7
## 8     4 UA        13.7
## 9     5 AA         9.66
## 10    5 UA        12.3
## 11    6 AA        14.6
## 12    6 UA        20.3
```

7.2.1 Otra alternativa para filtrar

7.2.1.1 “|” or

```
vuelos %>%
  group_by(mes) %>%
  dplyr::select(anio, mes, dia, atraso_salida) %>%
  filter(mes == 1 | mes == 12) |> # mes 1 o 12
  summarise(atraso = mean(atraso_salida, na.rm = TRUE))
```

```
## # A tibble: 2 x 2
##   mes atraso
##   <int> <dbl>
## 1     1  10.0
## 2    12  16.6
```

7.2.1.2 & el “y”

```
names(vuelos)
```

```
## [1] "anio"           "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigo_cola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
## [19] "fecha_hora"
```

```
vuelos %>%
```

```
  dplyr::select(anio, mes, dia, atraso_salida, aerolinea) %>%
  filter(mes == 12 & dia == 25 & aerolinea == "AA")
```

```
## # A tibble: 78 x 5
##   anio  mes  dia atraso_salida aerolinea
##   <int> <int> <int>         <dbl> <chr>
## 1  2013   12   25             2 AA
## 2  2013   12   25            -4 AA
## 3  2013   12   25             1 AA
## 4  2013   12   25             0 AA
## 5  2013   12   25            -3 AA
## 6  2013   12   25            -2 AA
## 7  2013   12   25            26 AA
## 8  2013   12   25            -7 AA
## 9  2013   12   25            -6 AA
## 10 2013   12   25            21 AA
## # i 68 more rows
```

7.3 Ejercicios

Como se seleccionaría los meses de diciembre pero desde la del 25 o más

7.4

!, **excluye algo**, En este caso estamos excluyendo las aerolíneas “AA” y “DL”

```
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_salida, aerolinea) %>%
  filter(mes == 11 & !aerolinea %in% c("AA", "DL")) %>%
  group_by(aerolinea) %>%
  summarise(atraso = mean(atraso_salida, na.rm = TRUE))
```

```
## # A tibble: 14 x 2
##   aerolinea atraso
##   <chr>      <dbl>
## 1 9E         7.56
## 2 AS         3.08
## 3 B6         3.52
## 4 EV         9.83
## 5 F9        13.5
## 6 FL        16.9
## 7 HA        -5.44
## 8 MQ         3.28
## 9 OO         0.8
## 10 UA        6.37
## 11 US        0.576
## 12 VX         7.80
## 13 WN        11.0
## 14 YV        10.5
```

Ejercicios:

Hacer los ejercicios en la sección 5.2.4 del libro en español

5.2.4 Ejercicios

1. Encuentra todos los vuelos que:
 - a. Tuvieron un retraso de llegada de dos o más horas
 - b. Volaron a Houston (IAH o HOU)

Cuáles son estos aeropuertos, Hou == Houston, IAH == George Bush, Texas

2. Selecciona solamente los vuelos que fueron operados por United, American o Delta
3. Selecciona solamente los vuelos que partieron en invierno del hemisferio sur (julio, agosto y septiembre)
4. Selecciona solamente los vuelos excluyendo los días del 2 al 29 de cualquier mes

```
vuelos %>%
  filter(atraso_salida <=0 & atraso_llegada >=120)
```

```
## # A tibble: 29 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>           <int>         <dbl>
## 1  2013     1   27          1419             1420          -1
## 2  2013    10    7          1350             1350           0
## 3  2013    10    7          1357             1359          -2
## 4  2013    10   16           657              700          -3
## 5  2013    11    1           658              700          -2
## 6  2013     3   18          1844             1847          -3
## 7  2013     4   17          1635             1640          -5
## 8  2013     4   18           558              600          -2
## 9  2013     4   18           655              700          -5
## 10 2013     5   22          1827             1830          -3
## # i 19 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

5. Selecciona solamente los vuelos que llegaron más de dos horas tarde, pero no salieron tarde
6. Selecciona solamente los vuelos que se retrasaron por lo menos una hora, pero repusieron más de 30 minutos en vuelo
7. Selecciona solamente los vuelos que partieron entre la medianoche y las 6a.m. (incluyente)
8. Otra función de dplyr que es útil para usar filtros es `between()`. ¿Qué hace? ¿Puedes usarla para simplificar el código necesario para responder a los desafíos anteriores?

`between(x, izq, der)` es un atajo equivalente a `x >= izq & x <= der` e incluye ambos extremos. Hace tus filtros más cortos y legibles.

9. Determinar cual “aerolinea” tiene más vuelos

7.4.1 La función `arrange()`

`arrange()`

`arrange()` — ordenar las filas.

- **Qué hace:** reordena el data frame según una o más columnas (ascendente por defecto).

- `desc(col)`: orden descendente (de mayor a menor).

```
vuelos %>%
  dplyr::select(horario_salida) %>%
  arrange() |>
  head(n=10)
```

```
## # A tibble: 10 x 1
##   horario_salida
##   <int>
## 1         517
## 2         533
## 3         542
## 4         544
## 5         554
## 6         554
## 7         555
## 8         557
## 9         557
## 10        558
```

```
vuelos %>%
  arrange(desc(horario_salida)) |>
  head(n=10)
```

```
## # A tibble: 10 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>         <int>         <dbl>
## 1  2013   10   30         2400         2359             1
## 2  2013   11   27         2400         2359             1
## 3  2013   12    5         2400         2359             1
## 4  2013   12    9         2400         2359             1
## 5  2013   12    9         2400         2250            70
## 6  2013   12   13         2400         2359             1
## 7  2013   12   19         2400         2359             1
## 8  2013   12   29         2400         1700          420
## 9  2013    2    7         2400         2359             1
## 10 2013    2    7         2400         2359             1
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

5.3.1 Ejercicios

Ordena vuelos para encontrar los vuelos más retrasados en salida.

Encuentra los vuelos que salieron más temprano.

Ordena vuelos para encontrar los vuelos más rápidos (que viajaron a mayor velocidad).

¿Cuáles vuelos viajaron más lejos?

¿Cuál viajó más cerca?

- Otra función
 - desc()
-

7.5 2 Ejercicios:

Hacer los ejercicios en la sección 5.3.1 del libro en español

Chapter 8

Transformación: Valores Faltantes

```
## [1] "2026-09-06"
```

Fecha de la última revisión

```
library(tidyverse)
library(datos)
```

8.1 Valores faltantes

- NA: Not Available

8.1.1 ¿Qué pasa con los valores faltantes y resultados de análisis sin sentido?

```
2^2
```

```
## [1] 4
```

```
1^2
```

```
## [1] 1
```

```
1^0
```

```
## [1] 1
```

```
0^1
```

```
## [1] 0
```

```
NA^0
## [1] 1
NA^1
## [1] NA
NA == 1
## [1] NA
NA | TRUE
## [1] TRUE
NA | FALSE
## [1] NA
FALSE & NA
## [1] FALSE
TRUE & NA
## [1] NA
FALSE * NA
## [1] NA
TRUE * NA
## [1] NA
NA * 0
## [1] NA
NA * 1
## [1] NA
12 * 3
## [1] 36
NA - 3
## [1] NA
```

8.2 Creamos un df para aclarar estas operaciones con NA en un data frame

- valores numéricos

8.2. CREAMOS UN DF PARA ACLARAR ESTAS OPERACIONES CON NA EN UN DATA FRAME85

- variable lógica TRUE/FALSE
- variable con NA y 0/1

```
df=tibble(~values, ~T_F, ~NA_not,  
          1, TRUE, NA,  
          0, TRUE, 1,  
          1, FALSE, NA,  
          0, FALSE, 1,  
          0, TRUE, 1,  
          0, FALSE, 0,  
          NA, NA, NA)  
  
df
```

```
## # A tibble: 7 x 3  
##   values T_F  NA_not  
##   <dbl> <lgl> <dbl>  
## 1     1 TRUE     NA  
## 2     0 TRUE     1  
## 3     1 FALSE    NA  
## 4     0 FALSE     1  
## 5     0 TRUE     1  
## 6     0 FALSE     0  
## 7    NA NA      NA
```

Ahora multiplicamos valores con la variable lógica.

```
df |>  
  mutate(values_TF = values * T_F)
```

```
## # A tibble: 7 x 4  
##   values T_F  NA_not values_TF  
##   <dbl> <lgl> <dbl>     <dbl>  
## 1     1 TRUE     NA         1  
## 2     0 TRUE     1         0  
## 3     1 FALSE    NA         0  
## 4     0 FALSE     1         0  
## 5     0 TRUE     1         0  
## 6     0 FALSE     0         0  
## 7    NA NA      NA         NA
```

Multiplicamos la variable lógica con NA/1/0

```
df |>  
  mutate(values_TF = T_F * NA_not)
```

```
## # A tibble: 7 x 4  
##   values T_F  NA_not values_TF  
##   <dbl> <lgl> <dbl>     <dbl>
```

```
## 1      1 TRUE      NA      NA
## 2      0 TRUE      1       1
## 3      1 FALSE     NA      NA
## 4      0 FALSE     1       0
## 5      0 TRUE      1       1
## 6      0 FALSE     0       0
## 7      NA NA      NA      NA
```

Multiplicamos valores con la variable NA/1/0

```
df |>
  mutate(valuesNA = values * NA_not)
```

```
## # A tibble: 7 x 4
##   values T_F  NA_not valuesNA
##   <dbl> <lgl>  <dbl>   <dbl>
## 1      1 TRUE     NA       NA
## 2      0 TRUE     1        0
## 3      1 FALSE   NA       NA
## 4      0 FALSE   1        0
## 5      0 TRUE     1        0
## 6      0 FALSE   0        0
## 7      NA NA      NA       NA
```

8.3 Contabilizar los NA

`is.na()` e `if_else()` — detectar y reemplazar valores faltantes.

- `is.na(x)`: devuelve TRUE donde hay NA; con `!is.na(x)` te quedas con lo que NO falta.
- `if_else(condición, sí, no)`: reemplaza según una condición (p. ej. `if_else(is.na(x), 0, x)`).
- **Ojo:** cualquier operación con NA da NA; por eso muchas funciones necesitan `na.rm = TRUE`.
- ¿Cuántos vuelos tienen datos faltantes en `horario_salida`?
- ¿Qué otras variables tienen valores faltantes?
- ¿Qué representan estas filas?

```
vuelos %>%
  dplyr::select(horario_salida, aerolinea) %>%
  is.na() %>%
  table()
```

```
## .
## FALSE TRUE
## 665297 8255
```

8.4 Seleccionar solamente los vuelos donde el “horario de salida” es desconocido

```
vuelos %>%
  dplyr::select(anio, mes, dia, horario_salida, aerolinea) |>
  filter(is.na(horario_salida)) %>%
  group_by(dia) %>%
  summarise(n = n())
```

```
## # A tibble: 31 x 2
##   dia      n
##   <int> <int>
## 1     1   246
## 2     2   250
## 3     3   109
## 4     4    82
## 5     5   226
## 6     6   296
## 7     7   318
## 8     8   921
## 9     9   593
## 10    10  535
## # i 21 more rows
```

8.5 Otras funciones: NA

- na.rm=TRUE
- !is.na() = is not NA, pq al frente tiene “!”

Para ignorar los NA en un cálculo usa el argumento `na.rm = TRUE` (por ejemplo `mean(x, na.rm = TRUE)`). Para eliminar filas con NA puedes usar `drop_na()`.

```
no_cancelados <- vuelos %>%
  filter(!is.na(atraso_salida), !is.na(atraso_llegada))
no_cancelados
```

```
## # A tibble: 327,346 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>          <int>          <int>          <dbl>
## 1  2013     1     1           517            515            2
## 2  2013     1     1           533            529            4
## 3  2013     1     1           542            540            2
## 4  2013     1     1           544            545           -1
## 5  2013     1     1           554            600           -6
## 6  2013     1     1           554            558           -4
## 7  2013     1     1           555            600           -5
```

```
## 8 2013      1      1          557          600          -3
## 9 2013      1      1          557          600          -3
## 10 2013     1      1          558          600          -2
## # i 327,336 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
#vuelos %>%
# drop_na(atraso_salida, atraso_llegada)
```

```
no_cancelados %>%
  group_by(anio, mes, dia) %>%
  summarise(mean = mean(atraso_salida))
```

```
## # A tibble: 365 x 4
## # Groups:   anio, mes [12]
##   anio  mes  dia mean
##   <int> <int> <int> <dbl>
## 1 2013     1     1 11.4
## 2 2013     1     2 13.7
## 3 2013     1     3 10.9
## 4 2013     1     4  8.97
## 5 2013     1     5  5.73
## 6 2013     1     6  7.15
## 7 2013     1     7  5.42
## 8 2013     1     8  2.56
## 9 2013     1     9  2.30
## 10 2013     1    10  2.84
## # i 355 more rows
```

```
vuelos %>%
  group_by(anio, mes, dia) %>%
  summarise(
    mean = mean(atraso_salida, na.rm = TRUE),
    max = max(atraso_salida, na.rm = TRUE),
    n = n()
  )
```

```
## # A tibble: 365 x 6
## # Groups:   anio, mes [12]
##   anio  mes  dia mean  max    n
##   <int> <int> <int> <dbl> <dbl> <int>
## 1 2013     1     1 11.5   853   842
## 2 2013     1     2 13.9   379   943
## 3 2013     1     3 11.0   291   914
```



```
## 4 2013      1      4 8.95   288   915
## 5 2013      1      5 5.73   327   720
## 6 2013      1      6 7.15   202   832
## 7 2013      1      7 5.42   366   933
## 8 2013      1      8 2.55   188   899
## 9 2013      1      9 2.28  1301   902
## 10 2013     1     10 2.84  1126   932
## # i 355 more rows
```

Reemplazar los NA por “0” en el archivo de datos

Cuál es el problema de reemplazar los NA por 0?

Reemplazar NA por 0 cambia el significado de los datos: un valor desconocido no es lo mismo que un cero real. Esto distorsiona promedios, sumas y conteos, así que piénsalo bien antes de hacerlo. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
datos_NA = c(1, NA, 3:8, NA, 10:12)
```

```
datos_NA_df = as.data.frame(datos_NA)
datos_NA_df
```

```
##      datos_NA
## 1           1
## 2          NA
## 3           3
## 4           4
## 5           5
## 6           6
## 7           7
## 8           8
## 9          NA
## 10          10
## 11          11
## 12          12
```

```
# reemplazar NA con 0
```

```
datos_NA_df = datos_NA_df %>%
  mutate(datos_0 = if_else(is.na(datos_NA), 0, datos_NA))
```

```
datos_NA_df
```

```
##      datos_NA datos_0
## 1           1       1
## 2          NA       0
```

```
## 3      3      3
## 4      4      4
## 5      5      5
## 6      6      6
## 7      7      7
## 8      8      8
## 9      NA     0
## 10     10     10
## 11     11     11
## 12     12     12
```

AHORA

Alguien le dio un set de datos en Excel con 0 cuando no tenían datos
reemplace los valores de “zero” con NA con la función `if_else` de `dplyr`

```
library(dplyr)
Datos_Excel = c(1, 0, 3:8, 0, 10:12)

Datos_df = as.data.frame(Datos_Excel)

Datos_df = Datos_df %>%
  mutate(Datos_NA = dplyr::if_else(Datos_Excel == 0, NA, Datos_Excel))

Datos_df
```

```
##   Datos_Excel Datos_NA
## 1          1        1
## 2          0       NA
## 3          3        3
## 4          4        4
## 5          5        5
## 6          6        6
## 7          7        7
## 8          8        8
## 9          0       NA
## 10         10       10
## 11         11       11
## 12         12       12
```

Chapter 9

Transformación: Seleccionar variables

Fecha de la última revisión

[1] "2026-09-06"

9.1 Funciones en este módulo

- `starts_with()` # selecciona variables que empiezan con una palabra o letra
- `ends_with()` # selecciona variables que terminan con una palabra o letra
- `rename()` # cambiar el nombre de una variable
- `contains()` # selecciona variables que contienen una palabra o letra
- `everything()` # selecciona todas las variables
- `if_else()` # reemplaza valores en una columna basado en una condición

Es una función para seleccionar basado en una características del nombre de la columna

Los ayudantes como `starts_with()`, `ends_with()` y `contains()` te evitan escribir cada nombre de columna a mano, sobre todo en tablas con muchas variables.

```
library(datos)
names(vuelos)
```

```
## [1] "anio"           "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
```

```
## [19] "fecha_hora"
```

```
library(tidyverse)
vuelos %>%
  dplyr::select(horario_salida)
```

```
## # A tibble: 336,776 x 1
##   horario_salida
##           <int>
## 1             517
## 2             533
## 3             542
## 4             544
## 5             554
## 6             554
## 7             555
## 8             557
## 9             557
## 10            558
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(starts_with("horario"))
```

```
## # A tibble: 336,776 x 2
##   horario_salida horario_llegada
##           <int>           <int>
## 1             517             830
## 2             533             850
## 3             542             923
## 4             544            1004
## 5             554             812
## 6             554             740
## 7             555             913
## 8             557             709
## 9             557             838
## 10            558             753
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(ends_with("salida"))
```

```
## # A tibble: 336,776 x 2
##   horario_salida atraso_salida
##           <int>         <dbl>
## 1             517             2
## 2             533             4
## 3             542             2
```

```
## 4          544          -1
## 5          554          -6
## 6          554          -4
## 7          555          -5
## 8          557          -3
## 9          557          -3
## 10         558          -2
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(contains("salida"))
```

```
## # A tibble: 336,776 x 3
##   horario_salida salida_programada atraso_salida
##   <int>          <int>          <dbl>
## 1         517         515            2
## 2         533         529            4
## 3         542         540            2
## 4         544         545           -1
## 5         554         600           -6
## 6         554         558           -4
## 7         555         600           -5
## 8         557         600           -3
## 9         557         600           -3
## 10        558         600           -2
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(contains("ue"))
```

```
## # A tibble: 336,776 x 2
##   vuelo tiempo_vuelo
##   <int>      <dbl>
## 1  1545        227
## 2  1714        227
## 3  1141        160
## 4   725        183
## 5   461        116
## 6  1696        150
## 7   507        158
## 8  5708         53
## 9    79        140
## 10  301        138
## # i 336,766 more rows
```

```
#vuelos %>%
# dplyr::filter(contains("AA")) # no funciona con filter
```

```
vuelos %>%
  dplyr::filter(aerolinea %in% c("AA", "DL"))
```

```
## # A tibble: 80,839 x 19
##   anio   mes   dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>         <int>         <dbl>
## 1  2013     1     1           542             540             2
## 2  2013     1     1           554             600            -6
## 3  2013     1     1           558             600            -2
## 4  2013     1     1           559             600            -1
## 5  2013     1     1           602             610            -8
## 6  2013     1     1           606             610            -4
## 7  2013     1     1           606             610            -4
## 8  2013     1     1           615             615             0
## 9  2013     1     1           623             610             13
## 10 2013     1     1           628             630            -2
## # i 80,829 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

9.2 rename()

rename(nuevo = viejo) — cambiar el nombre de columnas.

- **Qué hace:** renombra columnas; el **nombre nuevo va primero**, el original después.
- **Relacionadas:** **select(nuevo = viejo, ...)** renombra y selecciona a la vez; **everything()** reordena poniendo unas columnas al frente y el resto después.

Cambiar el nombre de la columna

```
vuelos %>%
  rename(aeropuerto_origen=origen) %>%
  rename(aeropuerto_destino=destino) %>%
  dplyr::select(contains("aeropuerto")) # nombre nuevo= nombre original
```

```
## # A tibble: 336,776 x 2
##   aeropuerto_origen aeropuerto_destino
##   <chr>             <chr>
## 1 EWR              IAH
## 2 LGA              IAH
## 3 JFK              MIA
## 4 JFK              BQN
```

9.3. REORGANIZAR EL ORDEN DE LAS COLUMNAS, USANDO SELECT() Y EVERYTHING() EN LA MISMA

```
## 5 LGA ATL
## 6 EWR ORD
## 7 EWR FLL
## 8 LGA IAD
## 9 JFK MCO
## 10 LGA ORD
## # i 336,766 more rows
```

9.3 Reorganizar el orden de las columnas, usando select() y everything() en la misma función

```
head(vuelos)
```

```
## # A tibble: 6 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>          <dbl>
## 1  2013     1     1           517               515            2
## 2  2013     1     1           533               529            4
## 3  2013     1     1           542               540            2
## 4  2013     1     1           544               545           -1
## 5  2013     1     1           554               600           -6
## 6  2013     1     1           554               558           -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
vuelos %>%
  dplyr::select(distancia, aerolinea, everything())
```

```
## # A tibble: 336,776 x 19
##   distancia aerolinea anio  mes  dia horario_salida salida_programada
##   <dbl> <chr>    <int> <int> <int>         <int>             <int>
## 1    1400 UA      2013     1     1           517               515
## 2    1416 UA      2013     1     1           533               529
## 3    1089 AA      2013     1     1           542               540
## 4    1576 B6      2013     1     1           544               545
## 5     762 DL      2013     1     1           554               600
## 6     719 UA      2013     1     1           554               558
## 7    1065 B6      2013     1     1           555               600
## 8     229 EV      2013     1     1           557               600
## 9     944 B6      2013     1     1           557               600
## 10     733 AA      2013     1     1           558               600
## # i 336,766 more rows
```

```
## # i 12 more variables: atraso_salida <dbl>, horario_llegada <int>,
## #   llegada_programada <int>, atraso_llegada <dbl>, vuelo <int>,
## #   codigoCola <chr>, origen <chr>, destino <chr>, tiempo_vuelo <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

Seleccionar de un conjunto de variables en orden que aparece en el data.frame

```
vuelos |> dplyr::select(mes:salida_programada)
```

```
## # A tibble: 336,776 x 4
##   mes dia horario_salida salida_programada
##   <int> <int>      <int>      <int>
## 1     1     1         517          515
## 2     1     1         533          529
## 3     1     1         542          540
## 4     1     1         544          545
## 5     1     1         554          600
## 6     1     1         554          558
## 7     1     1         555          600
## 8     1     1         557          600
## 9     1     1         557          600
## 10    1     1         558          600
```

```
## # i 336,766 more rows
```

```
x=c(10:15)
```

```
x
```

```
## [1] 10 11 12 13 14 15
```

```
vuelos[,c(4:7)]
```

```
## # A tibble: 336,776 x 4
##   horario_salida salida_programada atraso_salida horario_llegada
##           <int>           <int>         <dbl>         <int>
## 1           517           515             2           830
## 2           533           529             4           850
## 3           542           540             2           923
## 4           544           545            -1          1004
## 5           554           600            -6           812
## 6           554           558            -4           740
## 7           555           600            -5           913
## 8           557           600            -3           709
## 9           557           600            -3           838
## 10          558           600            -2           753
## # i 336,766 more rows
```


9.3. REORGANIZAR EL ORDEN DE LAS COLUMNAS, USANDO SELECT() Y EVERYTHING() EN LA MISMA

3. Ejercicios:

Hacer los ejercicios en la sección 5.4.1 del libro en español

<https://es.r4ds.hadley.nz/05-transform.html#ejercicios-2>

```
?mean
```

```
library(readr)
url_COVID_PR <- read_csv("Datos/url_COVID_PR.csv")

url_COVID_PR

## # A tibble: 587 x 18
##   ...1 Fecha      Muertes IncrementoMuertes CasosPCR_Salud IncCasosPCR_Salud
##   <dbl> <chr>      <dbl>          <dbl>          <dbl>          <dbl>
## 1     1 1 3/12/20      0              0              NA              NA
## 2     2 2 3/13/20      0              0              NA              NA
## 3     3 3 3/14/20      0              0              NA              NA
## 4     4 4 3/15/20      0              0              NA              NA
## 5     5 5 3/16/20      0              0              NA              NA
## 6     6 6 3/17/20      0              0              NA              NA
## 7     7 7 3/18/20      0              0              NA              NA
## 8     8 8 3/19/20      0              0              NA              NA
## 9     9 9 3/20/20      0              0              NA              NA
## 10    10 10 3/21/20      0              0              NA              NA
## # i 577 more rows
## # i 12 more variables: CasosSaludNuevo <dbl>, IncCasosSaludNuevo <dbl>,
## #   HospitCOV19 <dbl>, CamasICU <dbl>, CamasICU_disp <dbl>, Ventiladores <dbl>,
## #   MuertesSalud <dbl>, IncMueSalud <dbl>, VacDoses <dbl>, VacAdm <dbl>,
## #   N1MoreDoses <dbl>, N2Doses <dbl>
```

```
library(tidyverse)
df2=url_COVID_PR %>%
  dplyr::select(IncCasosSaludNuevo) %>%
  mutate(Cambios_Casos=IncCasosSaludNuevo-lag(IncCasosSaludNuevo,7))
```

```
df2
```

```
## # A tibble: 587 x 2
##   IncCasosSaludNuevo Cambios_Casos
##   <dbl>          <dbl>
## 1     2              NA
```

```
## 2          3      NA
## 3          3      NA
## 4          0      NA
## 5          9      NA
## 6          7      NA
## 7          6      NA
## 8          5       3
## 9         14      11
## 10        12       9
## # i 577 more rows
```

Chapter 10

Transformación: Mutate, transmute, lag, lead, cumsum, cummean, cumvar

10.1 Funciones en este módulo

- `mutate()` # crea nuevas variables
- `transmute()` # crea nuevas variables y elimina las antiguas
- `lag()` # calcula la diferencia entre valores en la misma columna
- `lead()` # calcula el siguiente valor en un vector
- `cumsum()` # suma acumulativa
- `cummean()` # promedio acumulativo
- `cumvar()` # varianza acumulativa

`mutate()` y `transmute()` — crear columnas nuevas.

- **`mutate(nueva = expresión)`**: añade columnas calculadas y **conserva** las demás.
- **`transmute()`**: igual, pero **solo** deja las columnas nuevas.
- **Uso**: la expresión puede combinar otras columnas (p. ej. `velocidad = distancia / tiempo`).

Ejercicios de este capítulo. Al final de cada idea hay un ejercicio con un conjunto de datos **distinto** al de los ejemplos (aquí usamos `millas` y `clima` del paquete `datos`). El código de los ejemplos te sirve de modelo, pero el script del ejercicio lo escribes tú: copiar y pegar no enseña.

Para recibir los puntos hay que entregar, en un documento `.Rmd` bien organizado: (1) el script, (2) el resultado y (3) una explicación **en palabras** de lo que encontraste.

```
library(tidyverse)
library(datos)

names(vuelos)

## [1] "anio"          "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
## [19] "fecha_hora"

# /> es igual %>%
new_df=vuelos |> dplyr::select(distancia, tiempo_vuelo, atraso_salida, atraso_llegada,
  mutate( ganancia = atraso_salida - atraso_llegada,
          velocidad = distancia / tiempo_vuelo * 60))

new_df

## # A tibble: 336,776 x 6
##   distancia tiempo_vuelo atraso_salida atraso_llegada ganancia velocidad
##   <dbl>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
## 1      1400          227             2             11            -9          370.
## 2      1416          227             4             20           -16          374.
## 3      1089          160             2             33           -31          408.
## 4      1576          183            -1            -18           17          517.
## 5       762          116            -6            -25           19          394.
## 6       719          150            -4             12           -16          288.
## 7      1065          158            -5             19           -24          404.
## 8       229           53            -3            -14            11          259.
## 9       944          140            -3             -8             5          405.
## 10      733          138            -2              8          -10          319.
## # i 336,766 more rows
```

10.2 Crear un data frame más pequeño con las variables de interés

```
library(tidyverse)
library(nycflights13)
head(flights)

## # A tibble: 6 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
```

```
## 1 2013      1      1      517      515      2      830      819
## 2 2013      1      1      533      529      4      850      830
## 3 2013      1      1      542      540      2      923      850
## 4 2013      1      1      544      545     -1     1004     1022
## 5 2013      1      1      554      600     -6      812      837
## 6 2013      1      1      554      558     -4      740      728
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
names(vuelos)
```

```
## [1] "anio"      "mes"      "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea" "vuelo"      "codigoCola"
## [13] "origen"    "destino"    "tiempo_vuelo"
## [16] "distancia" "hora"       "minuto"
## [19] "fecha_hora"
```

```
vuelos_sml <- dplyr::select(vuelos,
  anio:dia,
  starts_with("atraso"),
  distancia,
  tiempo_vuelo
)
head(vuelos_sml)
```

```
## # A tibble: 6 x 7
##   anio  mes  dia atraso_salida atraso_llegada distancia tiempo_vuelo
##   <int> <int> <int>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 2013     1     1         2         11      1400        227
## 2 2013     1     1         4         20      1416        227
## 3 2013     1     1         2         33      1089        160
## 4 2013     1     1        -1        -18      1576        183
## 5 2013     1     1        -6        -25       762        116
## 6 2013     1     1        -4         12       719        150
```

10.3 Crear nuevas variables

```
mutate(vuelos_sml,
  ganado = atraso_salida - atraso_llegada,
  velocidad = distancia / tiempo_vuelo * 60
)
```

```
## # A tibble: 336,776 x 9
```

```
##      año   mes   día atraso_salida atraso_llegada distancia tiempo_vuelo ganado
##      <int> <int> <int>          <dbl>          <dbl>      <dbl>      <dbl> <dbl>
## 1  2013     1     1              2              11       1400       227    -9
## 2  2013     1     1              4              20       1416       227   -16
## 3  2013     1     1              2              33       1089       160  -31
## 4  2013     1     1             -1             -18       1576       183   17
## 5  2013     1     1             -6             -25        762       116   19
## 6  2013     1     1             -4              12        719       150  -16
## 7  2013     1     1             -5              19       1065       158  -24
## 8  2013     1     1             -3             -14        229        53   11
## 9  2013     1     1             -3              -8        944       140    5
## 10 2013     1     1             -2              8         733       138  -10
## # i 336,766 more rows
## # i 1 more variable: velocidad <dbl>
```

Ejercicio 1: crear variables nuevas con `mutate()`

Usa el conjunto de datos `millas` del paquete `datos` (NO vuelos). Empieza con `names(millas)` y `head(millas)` para ver qué variables tienes.

1. Crea dos variables nuevas: `promedio_millas`, el promedio del rendimiento en ciudad y en autopista, y `diferencia`, cuánto más rinde el vehículo en autopista que en ciudad.
2. ¿Cuál clase de vehículo tiene la mayor `diferencia` promedio? Contesta en palabras, no solamente con la tabla.

Verifica: tu tabla debe conservar todas las columnas originales de `millas` más las dos nuevas.

```
# Escribe tu código aquí
```

Ejercicio 2: cuidado con las unidades

En `millas` el rendimiento está en millas por galón. Crea la variable `km_por_litro` multiplicando `autopista` por 0.425144.

1. ¿Cuál fabricante tiene el promedio más alto de `km_por_litro`?
2. ¿Cambia la respuesta si haces lo mismo con `ciudad` en vez de `autopista`?
¿Por qué crees que pasa eso?

```
# Escribe tu código aquí
```

10.4 `transmute()`

Para guardar solamente la nueva variable usa `transmute`

Usa `mutate()` cuando quieras conservar las columnas originales y añadir nuevas; usa `transmute()` cuando solo te interesen las columnas nuevas.

```
flights |> transmute(
  gain = dep_delay - arr_delay,
  hours = air_time / 60,
  gain_per_hour = gain / hours
)
```

```
## # A tibble: 336,776 x 3
##   gain hours gain_per_hour
##   <dbl> <dbl>         <dbl>
## 1    -9  3.78          -2.38
## 2   -16  3.78          -4.23
## 3   -31  2.67         -11.6
## 4    17  3.05           5.57
## 5    19  1.93           9.83
## 6   -16  2.5           -6.4
## 7   -24  2.63          -9.11
## 8    11  0.883          12.5
## 9     5  2.33           2.14
## 10  -10  2.3           -4.35
## # i 336,766 more rows
```

Ejercicio 3: `transmute()` no es lo mismo que `mutate()`

Con millas, crea la variable `razon = autopista / ciudad` de dos maneras: primero con `transmute()` y después con `mutate()`.

1. ¿Cuántas columnas tiene cada resultado?
2. Explica en una oración cuándo te conviene usar cada una.
3. ¿Qué significa biológicamente (o mecánicamente) que `razon` sea mayor que 1?

Escribe tu código aquí

10.5 lag()

`lag()` y `lead()` — valor anterior / siguiente.

- **`lag(x, n)`:** trae el valor de `n` filas **atrás** (por defecto 1); útil para calcular cambios (`x - lag(x)`).
- **`lead(x, n)`:** trae el valor de `n` filas **adelante**.
- **Requisito:** ordena las filas primero (`arrange()`), porque operan según el orden actual.

Para calcular diferencias entre variables en la misma columna

`lag()` y `lead()` trabajan según el orden actual de las filas. Ordena tus datos (por ejemplo por fecha con `arrange()`) antes de usarlas, o los desplazamientos

no tendrán sentido.

```
set.seed(12345) # que los datos sean al azar, siempre sean los mismo, se usa el "set.s
#rnorm() DATOS CON DISTRIBUCION NORMAL
```

```
data=rpois(14, 10)
df=as_tibble(data)
df
```

```
## # A tibble: 14 x 1
##   value
##   <int>
## 1     11
## 2     12
## 3      9
## 4      8
## 5     11
## 6      4
## 7     11
## 8      7
## 9      8
## 10    11
## 11     9
## 12    10
## 13     9
## 14    11
```

```
df %>%
  dplyr::select(value) %>%
  mutate(lag1=lag(value)) %>%
  mutate(lag3=lag(value, 3)) %>%
  mutate(lag7=lag(value,5))
```

```
## # A tibble: 14 x 4
##   value lag1 lag3 lag7
##   <int> <int> <int> <int>
## 1     11  NA   NA   NA
## 2     12  11   NA   NA
## 3      9  12   NA   NA
## 4      8   9   11   NA
## 5     11   8   12   NA
## 6      4  11    9   11
## 7     11   4    8   12
## 8      7  11   11    9
## 9      8   7    4    8
## 10    11   8   11   11
```


10.6. USA “LAG” CON “INCCASOSSALUDNUEVO” EN COVID-19 PR105

```
## 11      9      11      7      4
## 12     10      9      8     11
## 13      9     10     11      7
## 14     11      9      9      8

## Calcular la diferencia usando lag

df %>%
  dplyr::select(value) %>%
  mutate(lag1=lag(value)) %>%
  mutate(Changes=value-lag(value, 1)) # El cambio en posición de los valores entre las filas

## # A tibble: 14 x 3
##   value lag1 Changes
##   <int> <int>   <int>
## 1     11    NA      NA
## 2     12    11       1
## 3      9    12     -3
## 4      8     9     -1
## 5     11     8       3
## 6      4    11     -7
## 7     11     4       7
## 8      7    11     -4
## 9      8     7       1
## 10     11     8       3
## 11      9    11     -2
## 12     10     9       1
## 13      9    10     -1
## 14     11     9       2
```

10.6 Usa “Lag” con “IncCasosSaludNuevo” en COVID-19 PR

Evalúa la diferencia en números de casos entre 7 días de la semana en números de casos nuevos de COVID, “IncCasosSaludNuevo”

```
library(readr)
library(dplyr)
#names(url_COVID_PR)
url_COVID_PR <- read_csv("Datos/url_COVID_PR.csv")
head(url_COVID_PR, 100)

## # A tibble: 100 x 18
##   ...1 Fecha Muertes IncrementoMuertes CasosPCR_Salud IncCasosPCR_Salud
##   <dbl> <chr>      <dbl>          <dbl>          <dbl>          <dbl>
## 1      1 1 3/12/20      0              0              NA              NA
```

```
## 2      2 3/13/20      0      0      NA      NA
## 3      3 3/14/20      0      0      NA      NA
## 4      4 3/15/20      0      0      NA      NA
## 5      5 3/16/20      0      0      NA      NA
## 6      6 3/17/20      0      0      NA      NA
## 7      7 3/18/20      0      0      NA      NA
## 8      8 3/19/20      0      0      NA      NA
## 9      9 3/20/20      0      0      NA      NA
## 10     10 3/21/20      0      0      NA      NA
## # i 90 more rows
## # i 12 more variables: CasosSaludNuevo <dbl>, IncCasosSaludNuevo <dbl>,
## #   HospitCOV19 <dbl>, CamasICU <dbl>, CamasICU_disp <dbl>, Ventiladores <dbl>,
## #   MuertesSalud <dbl>, IncMueSalud <dbl>, VacDoses <dbl>, VacAdm <dbl>,
## #   N1MoreDoses <dbl>, N2Doses <dbl>
```

```
#names(url_COVID_PR)
names(url_COVID_PR)
```

```
## [1] "...1"      "Fecha"      "Muertes"
## [4] "IncrementoMuertes" "CasosPCR_Salud" "IncCasosPCR_Salud"
## [7] "CasosSaludNuevo"   "IncCasosSaludNuevo" "HospitCOV19"
## [10] "CamasICU"          "CamasICU_disp"   "Ventiladores"
## [13] "MuertesSalud"      "IncMueSalud"      "VacDoses"
## [16] "VacAdm"            "N1MoreDoses"      "N2Doses"
```

```
df2=url_COVID_PR %>%
  dplyr::select(IncCasosSaludNuevo) %>%
  mutate(Cambios_Casos=IncCasosSaludNuevo-lag(IncCasosSaludNuevo,7))
```

```
df2
```

```
## # A tibble: 587 x 2
##   IncCasosSaludNuevo Cambios_Casos
##   <dbl>             <dbl>
## 1         2          NA
## 2         3          NA
## 3         3          NA
## 4         0          NA
## 5         9          NA
## 6         7          NA
## 7         6          NA
## 8         5           3
## 9        14         11
```

```
## 10                12                9
## # i 577 more rows
df2 %>%
  dplyr::select(IncCasosSaludNuevo, Cambios_Casos) %>%
  colMeans(na.rm=TRUE)

## IncCasosSaludNuevo      Cambios_Casos
##           257.8310580           0.4507772

#df2 %>%
#  select()
#  slice(na.rm=TRUE) # lets you index rows by their (integer) locations. It allows you to select,
```

Ejercicio 4: lag() con datos de clima

Usa el conjunto de datos `clima` del paquete `datos`: son mediciones por hora de tres aeropuertos de Nueva York. Mira primero `names(clima)`.

1. Filtra las filas de `origen == "EWR"` del mes de enero (`mes == 1`) y **ordena por fecha_hora** (¿por qué es indispensable ordenar antes?).
2. Crea `cambio_temp`: cuánto cambió la `temperatura` respecto a la hora anterior.
3. ¿Por qué la primera fila tiene NA?
4. ¿En qué hora ocurrió el mayor aumento de temperatura de una hora a la otra?

Verifica: `cambio_temp` debe tener exactamente un NA, y debe estar en la primera fila.

```
# Escribe tu código aquí
```

10.7 lead(),

is the “next” (`lead()`) values in a vector/column

```
set.seed(12345)
data=rpois(15, 10)
df=as_tibble(data)
df

## # A tibble: 15 x 1
##   value
##   <int>
## 1     11
## 2     12
## 3      9
## 4      8
```

```
## 5    11
## 6     4
## 7    11
## 8     7
## 9     8
## 10   11
## 11    9
## 12   10
## 13    9
## 14   11
## 15   13
```

```
df %>%
  dplyr::select(value) %>%
  mutate(lead1=lead(value)) %>%
  mutate(lead3=lead(value, 3))
```

```
## # A tibble: 15 x 3
##   value lead1 lead3
##   <int> <int> <int>
## 1    11    12     8
## 2    12     9    11
## 3     9     8     4
## 4     8    11    11
## 5    11     4     7
## 6     4    11     8
## 7    11     7    11
## 8     7     8     9
## 9     8    11    10
## 10    11     9     9
## 11     9    10    11
## 12    10     9    13
## 13     9    11    NA
## 14    11    13    NA
## 15    13    NA    NA
```

Calculate the change in value from one (1) time period and four (4) time periods

```
df %>%
  dplyr::select(value) %>%
  mutate(lead1=value-lead(value)) %>%
  mutate(lead7=value-lead(value, 7))
```

```
## # A tibble: 15 x 3
##   value lead1 lead7
##   <int> <int> <int>
## 1    11    -1     4
## 2    12     3     4
```

```
## 3      9      1     -2
## 4      8     -3     -1
## 5     11      7      1
## 6      4     -7     -5
## 7     11      4      0
## 8      7     -1     -6
## 9      8     -3     NA
## 10     11      2     NA
## 11      9     -1     NA
## 12     10      1     NA
## 13      9     -2     NA
## 14     11     -2     NA
## 15     13     NA     NA
```

Ejercicio 5: `lead()`, mirar hacia adelante

Con el mismo subconjunto de `clima` (EWR, enero, ordenado por `fecha_hora`):

1. Crea `humedad_en_3h`, el valor de `humedad` tres horas más adelante.
2. Crea `cambio_humedad`, la diferencia entre ese valor futuro y el actual.
3. ¿Cuántos NA aparecen y en cuál extremo de la tabla están? Explica por qué, comparando con lo que pasó en el ejercicio anterior con `lag()`.

Escribe tu código aquí

10.8 cumsum

`cumsum()`, `cmean()`, `cvar()` — acumulados.

- **`cumsum(x)`:** suma acumulada a lo largo de la columna.
- **`cmean()` / `cvar()`** (paquete **TidyDensity**): promedio y varianza acumulados.
- **Uso típico:** ver cómo evoluciona un total o un promedio a medida que se añaden datos.

Suma acumulativa: los valores se suman a lo largo del vector o columna

```
set.seed(12345) # set.seed() es para que los datos NO sean al azar y se puede replicar los resultados
```

```
x <- sample(1:15, 10, replace=TRUE) # sample es para seleccionar valores al azar de un vector
```

```
x
```

```
## [1] 14  3 10 12  8 10 13 11  8  2
```

```
df=as_tibble(x)
```

```
df
```

```
## # A tibble: 10 x 1
##   value
##   <int>
## 1     14
## 2      3
## 3     10
## 4     12
## 5      8
## 6     10
## 7     13
## 8     11
## 9      8
## 10     2
```

```
df %>%
  dplyr::select(value) %>%
  mutate(suma=cumsum(value))
```

```
## # A tibble: 10 x 2
##   value suma
##   <int> <int>
## 1     14    14
## 2      3    17
## 3     10    27
## 4     12    39
## 5      8    47
## 6     10    57
## 7     13    70
## 8     11    81
## 9      8    89
## 10     2    91
```

```
url_COVID_PR <- read_csv("Datos/url_COVID_PR.csv")
names(url_COVID_PR)
```

```
## [1] "...1" "Fecha" "Muertes"
## [4] "IncrementoMuertes" "CasosPCR_Salud" "IncCasosPCR_Salud"
## [7] "CasosSaludNuevo" "IncCasosSaludNuevo" "HospitCOV19"
## [10] "CamasICU" "CamasICU_disp" "Ventiladores"
## [13] "MuertesSalud" "IncMueSalud" "VacDoses"
## [16] "VacAdm" "N1MoreDoses" "N2Doses"
```

```
head(url_COVID_PR)
```

```
## # A tibble: 6 x 18
##   ...1 Fecha Muertes IncrementoMuertes CasosPCR_Salud IncCasosPCR_Salud
##   <dbl> <chr>   <dbl>         <dbl>         <dbl>         <dbl>
## 1     1 1 3/12/20     0             0             NA             NA
```

```
## 2      2 3/13/20      0      0      NA      NA
## 3      3 3/14/20      0      0      NA      NA
## 4      4 3/15/20      0      0      NA      NA
## 5      5 3/16/20      0      0      NA      NA
## 6      6 3/17/20      0      0      NA      NA
## # i 12 more variables: CasosSaludNuevo <dbl>, IncCasosSaludNuevo <dbl>,
## #   HospitCOV19 <dbl>, CamasICU <dbl>, CamasICU_disp <dbl>, Ventiladores <dbl>,
## #   MuertesSalud <dbl>, IncMueSalud <dbl>, VacDoses <dbl>, VacAdm <dbl>,
## #   N1MoreDoses <dbl>, N2Doses <dbl>

url_COVID_PR %>%
dplyr::select(IncCasosSaludNuevo, CasosSaludNuevo) %>%
mutate(suma=cumsum(IncCasosSaludNuevo))

## # A tibble: 587 x 3
##   IncCasosSaludNuevo CasosSaludNuevo suma
##   <dbl>                <dbl> <dbl>
## 1             2             2      2
## 2             3             5      5
## 3             3             8      8
## 4             0             8      8
## 5             9            17     17
## 6             7            24     24
## 7             6            30     30
## 8             5            35     35
## 9            14            49     49
## 10            12            61     61
## # i 577 more rows
```

Ejercicio 6: cumsum() con la lluvia

Sigue con `clima`, origen EWR, mes de enero, ordenado por `fecha_hora`.

1. Calcula la lluvia acumulada del mes con `cumsum(precipitacion)`.
2. ¿Qué le pasa a la suma acumulada si hay un NA en `precipitacion`? Arreglalo con `replace_na()` y explica qué supuesto estás haciendo al sustituir por 0.
3. Grafica la lluvia acumulada contra `fecha_hora` y describe en palabras la forma de la curva: ¿llovió parejo todo el mes o en pocos eventos?

```
# Escribe tu código aquí
```

10.9 cmean() and cvar() del paquete TidyDensity

Para calcular el promedio cumulativo y la varianza

- las funciones son
- `cmean()` # del paquete TidyDensity
- `cvar()` # del paquete TidyDensity

Para calcular el promedio en periodos cumulativos

- `rollmean()` # del paquete zoo
- `rollapply()` # del paquete zoo para calcular el rolling varianza

Vea las otras funciones cumulativas en el paquete TidyDensity (`cmean`, `cvar`, `csd`, `cmedian`, `cskewness`, `ckurtosis`): ¿hay otra que podría usar en sus estudios?

```
set.seed(678)
x <- rnorm(1000, 5, .001)
head(x)
```

```
## [1] 4.999227 5.000933 5.000466 4.998915 4.997844 4.999281
```

```
df=as_tibble(x)
head(df)
```

```
## # A tibble: 6 x 1
##   value
##   <dbl>
## 1  5.00
## 2  5.00
## 3  5.00
## 4  5.00
## 5  5.00
## 6  5.00
```

```
mean(df$value)
```

```
## [1] 4.999966
```

```
# library(MASS) # innecesario aquí y enmascara dplyr::select(); cmean()/cvar() vienen
library(TidyDensity)
library(zoo)
```

```
df3=df %>%
  dplyr::select(value) %>%
  mutate(Prom_cum=cmean(value)) %>%
  mutate(Var_cum=TidyDensity::cvar(value)) |>
  mutate(promedio_corriendo= zoo::rollmean(value, k=7, fill=NA, align="left")) |>
  mutate(varianza_corriendo= zoo::rollapply(value, width=7, FUN=var, fill=NA, align="l
```

```
df3
```

```
## # A tibble: 1,000 x 5
```



```
##      value Prom_cum      Var_cum promedio_corriendo varianza_corriendo
##      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1  5.00      5.00 NaN          5.00      0.00000139
## 2  5.00      5.00 0.00000146    5.00      0.00000145
## 3  5.00      5.00 0.000000778    5.00      0.00000123
## 4  5.00      5.00 0.000000937    5.00      0.00000133
## 5  5.00      5.00 0.00000154    5.00      0.00000135
## 6  5.00      5.00 0.00000124    5.00      0.000000532
## 7  5.00      5.00 0.00000139    5.00      0.000000813
## 8  5.00      5.00 0.00000129    5.00      0.00000139
## 9  5.00      5.00 0.00000115    5.00      0.00000131
## 10 5.00      5.00 0.00000113    5.00      0.00000132
## # i 990 more rows
```

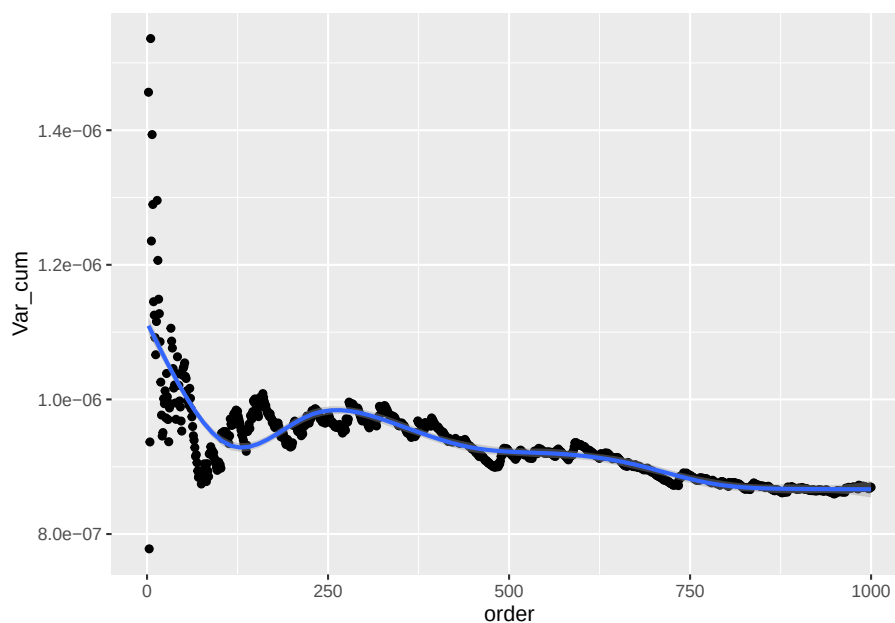
10.10 Uso de varianza cumulativa en investigación:

Un método para determinar si la cantidad de muestras recolectada es suficiente, es evaluar si la varianza cumulativa sigue cambiando cuando se añade nuevas recolección de datos.

```
df3$order=c(1:1000)
df3
```

```
## # A tibble: 1,000 x 6
##      value Prom_cum      Var_cum promedio_corriendo varianza_corriendo order
##      <dbl>      <dbl>      <dbl>      <dbl>      <dbl> <int>
## 1  5.00      5.00 NaN          5.00      0.00000139     1
## 2  5.00      5.00 0.00000146    5.00      0.00000145     2
## 3  5.00      5.00 0.000000778    5.00      0.00000123     3
## 4  5.00      5.00 0.000000937    5.00      0.00000133     4
## 5  5.00      5.00 0.00000154    5.00      0.00000135     5
## 6  5.00      5.00 0.00000124    5.00      0.000000532     6
## 7  5.00      5.00 0.00000139    5.00      0.000000813     7
## 8  5.00      5.00 0.00000129    5.00      0.00000139     8
## 9  5.00      5.00 0.00000115    5.00      0.00000131     9
## 10 5.00      5.00 0.00000113    5.00      0.00000132    10
## # i 990 more rows
```

```
ggplot(df3, aes(order, Var_cum))+
  geom_point()+
  geom_smooth()
```



10.11 Que pasa con las funciones cmean, cvar si hay NA en el archivo de datos?

10.11.1 Reemplazar los NA con 0, usando la función `replace_na()` con un valor específico `replace_na("variable", 0)`

```
datos_NA=c(1:8, NA, 10:20)
datos_NA=as.data.frame(datos_NA)

datos_NA %>%
  dplyr::select(datos_NA) %>%
  mutate(Prom_cum=cmean(datos_NA)) %>%
  mutate(Var_cum=cvar(datos_NA))
```

##	datos_NA	Prom_cum	Var_cum
## 1	1	1.0	NaN
## 2	2	1.5	0.500000
## 3	3	2.0	1.000000
## 4	4	2.5	1.666667
## 5	5	3.0	2.500000
## 6	6	3.5	3.500000

```
## 7      7      4.0 4.666667
## 8      8      4.5 6.000000
## 9     NA      NA      NA
## 10     10     NA      NA
## 11     11     NA      NA
## 12     12     NA      NA
## 13     13     NA      NA
## 14     14     NA      NA
## 15     15     NA      NA
## 16     16     NA      NA
## 17     17     NA      NA
## 18     18     NA      NA
## 19     19     NA      NA
## 20     20     NA      NA
```

Solución para resolver los NA. Cual problema hay con usar NA?

Ejercicio 7: ¿cuántas observaciones son suficientes?

Este es el uso del ejemplo anterior, pero con datos reales. Toma las primeras 300 filas de `temperatura` de EWR en enero (ordenadas por `fecha_hora`).

1. Añade una columna `orden` con el número de fila (`row_number()`).
2. Calcula el promedio acumulado y la varianza acumulada de `temperatura`.
3. Grafica cada uno contra `orden`.
4. Según tus gráficas, ¿a partir de cuántas observaciones se estabiliza el promedio? ¿Y la varianza? ¿Cuál de las dos necesita más datos?

Escribe tu código aquí

10.12 if_else()

La función `if_else()` es una función de reemplazo de valores en una columna basado en una condición

```
library(tidyverse)
library(dplyr)

some_data = tibble(day = 1:10)
some_data
```

```
## # A tibble: 10 x 1
##   day
##   <int>
## 1     1
## 2     2
```

```
## 3      3
## 4      4
## 5      5
## 6      6
## 7      7
## 8      8
## 9      9
## 10     10
```

```
some_data |>
  mutate(day_p = if_else(day %in% c(0:3), "pre", "normal"))
```

```
## # A tibble: 10 x 2
##       day day_p
##   <int> <chr>
## 1     1 pre
## 2     2 pre
## 3     3 pre
## 4     4 normal
## 5     5 normal
## 6     6 normal
## 7     7 normal
## 8     8 normal
## 9     9 normal
## 10    10 normal
```

```
plant_repr=tibble(num_flowers=c(0,0,0,0,1,2,0,1,2,0,2, 100, 1000))
```

```
plant_repr
```

```
## # A tibble: 13 x 1
##   num_flowers
##   <dbl>
## 1         0
## 2         0
## 3         0
## 4         0
## 5         1
## 6         2
## 7         0
## 8         1
## 9         2
## 10        0
## 11        2
## 12       100
## 13      1000
```

```
plant_repr |>
  mutate(flower_status = if_else(num_flowers == 0, "not reproductive",
                                "reproductive"))
```

```
## # A tibble: 13 x 2
##   num_flowers flower_status
##   <dbl> <chr>
## 1         0 not reproductive
## 2         0 not reproductive
## 3         0 not reproductive
## 4         0 not reproductive
## 5         1 reproductive
## 6         2 reproductive
## 7         0 not reproductive
## 8         1 reproductive
## 9         2 reproductive
## 10        0 not reproductive
## 11        2 reproductive
## 12       100 reproductive
## 13      1000 reproductive
```

```
plant_repr |>
  mutate(flower_status = if_else(num_flowers %in% c(0), "not reproductive",
                                "reproductive"))
```

```
## # A tibble: 13 x 2
##   num_flowers flower_status
##   <dbl> <chr>
## 1         0 not reproductive
## 2         0 not reproductive
## 3         0 not reproductive
## 4         0 not reproductive
## 5         1 reproductive
## 6         2 reproductive
## 7         0 not reproductive
## 8         1 reproductive
## 9         2 reproductive
## 10        0 not reproductive
## 11        2 reproductive
## 12       100 reproductive
## 13      1000 reproductive
```

```
plant_repr |>
  mutate(flower_status =
    if_else(num_flowers %in% c(0),
            "not reproductive",
```

```

      if_else(num_flowers %in% c(1:2), "reproductive",
              "super reproductive"))

## # A tibble: 13 x 2
##   num_flowers flower_status
##   <dbl> <chr>
## 1         0 not reproductive
## 2         0 not reproductive
## 3         0 not reproductive
## 4         0 not reproductive
## 5         1 reproductive
## 6         2 reproductive
## 7         0 not reproductive
## 8         1 reproductive
## 9         2 reproductive
## 10        0 not reproductive
## 11        2 reproductive
## 12       100 super reproductive
## 13      1000 super reproductive

```

Ejercicio 8: `if_else()` para clasificar

Regresa a millas.

1. Crea **eficiencia** con dos categorías: “eficiente” si **autopista** es 30 o más, y “no eficiente” en los demás casos.
2. Ahora hazlo con tres categorías usando `if_else()` anidado: “bajo” (menos de 20), “medio” (20 a 29) y “alto” (30 o más).
3. Cuenta cuántos vehículos hay en cada categoría por **clase**. ¿Alguna clase es toda de una sola categoría?

Escribe tu código aquí

Ejercicio 9 (reto integrador): combinar `group_by()` con `lag()`

Con **clima completo** (los tres aeropuertos, todo el año):

1. Calcula la temperatura promedio **por origen y por día**.
2. Para cada origen por separado, calcula el cambio respecto al día anterior. Ojo: tienes que agrupar por **origen antes** de usar `lag()`, o el último día de un aeropuerto se va a restar con el primer día del siguiente.
3. ¿Cuántos días de cada aeropuerto fueron más calientes que el día anterior?
4. Enseña con una gráfica o con una tabla por qué el paso 2 estaría mal si no agrupas primero.

Escribe tu código aquí

Chapter 11

Transformación: Rangos y otras funciones

Fecha de la última revisión

```
## [1] "2026-09-06"
```

```
##  
## platform      aarch64-apple-darwin23  
## arch          aarch64  
## os            darwin23  
## system        aarch64, darwin23  
## status  
## major         4  
## minor         6.1  
## year          2026  
## month         06  
## day           24  
## svn rev       90187  
## language      R  
## version.string R version 4.6.1 (2026-06-24)  
## nickname      Happy Hop
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/transform.html>

Español: <https://es.r4ds.hadley.nz/05-transform.html>

11.1 Funciones en este módulo

Rangos:

- `min_rank()`: asigna rangos a los valores, dejando huecos en los empates
- `dense_rank()`: igual, pero sin dejar huecos
- `row_number()`: un número distinto para cada fila, aunque haya empates
- `rank()`: la versión de R base, promedia los empates
- `desc()`: invierte el orden, de mayor a menor

Rangos en escala de 0 a 1:

- `percent_rank()`: la proporción de valores más pequeños
- `cume_dist()`: la proporción de valores menores o iguales

Resúmenes y series:

- `group_by()` con `summarise()`: resumir por grupo
- `first()`: el primer valor en el orden de las filas
- `rollmean()`: promedio móvil (paquete `zoo`)

11.2 Paquetes

```
library(tidyverse)
library(datos)
```

11.3 Rangos y pruebas no paramétricas

Las pruebas no paramétricas no son basadas en distribución normal y los índices como promedio, desviación estándar no se usan.

El rango es la pieza básica de casi todas ellas: en vez de trabajar con el valor observado, se trabaja con la **posición** que ocupa ese valor entre los demás.

Trabajar con rangos en vez de valores es una idea vieja y muy productiva. **Charles Spearman** publicó en 1904 su coeficiente de correlación de rangos, y en 1945 **Frank Wilcoxon** propuso las pruebas de rangos con signo y de suma de rangos, que hoy son el sustituto no paramétrico estándar de la prueba *t*.

11.4 min_rank() y min_rank(desc())

Asignar valores de rangos a los valores originales o de más grande a más pequeño o viceversa.

`min_rank()`, `dense_rank()`, `row_number()` — asignar rangos.

- **Qué hacen:** convierten valores en su **posición** (rango) de menor a mayor; `desc()` invierte el orden.
- **Diferencia en empates:** `min_rank()` deja huecos, `dense_rank()` no, `row_number()` da un número único a cada fila.
- **Relacionadas:** `percent_rank()` y `cume_dist()` dan rangos en escala 0-1.

```
set.seed(45678)
x <- sample(1:50, 10)
head(x)
```

```
## [1] 22  1 36 46  7 39
```

```
df <- as_tibble(x)
df
```

```
## # A tibble: 10 x 1
##   value
##   <int>
## 1     22
## 2      1
## 3     36
## 4     46
## 5      7
## 6     39
## 7     34
## 8     10
## 9     24
## 10    30
```

```
df %>%
  dplyr::select(value) %>%
  mutate(rango_minimo=min_rank(value)) %>%
  mutate(rango_min_desc=min_rank(desc(value)) )
```

```
## # A tibble: 10 x 3
##   value rango_minimo rango_min_desc
##   <int>         <int>         <int>
## 1     22             4             7
## 2      1             1            10
## 3     36             8             3
## 4     46            10             1
```

```
## 5      7      2      9
## 6     39      9      2
## 7     34      7      4
## 8     10      3      8
## 9     24      5      6
## 10    30      6      5
```

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan **diamantes** y **países** del paquete **datos**, es decir, conjuntos distintos a los de los ejemplos de arriba: no se resuelven cambiando el nombre de una columna en el script copiado. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_11_rangos.Rmd`.

Ejercicio 1: `min_rank()` y `desc()`. Toma una muestra de 500 diamantes con `set.seed(2026)` y `slice_sample(diamantes, n = 500)`. Añade el rango del precio de menor a mayor y otro de mayor a menor con `desc()`, y enseña los 10 diamantes más caros de la muestra.

Verifica: la fila con el precio más alto debe tener rango descendente igual a 1.

En palabras: ¿cuánto suman los dos rangos en una misma fila? ¿Es siempre el mismo número? ¿Qué te dice eso sobre los empates?

Escribe tu código aquí

11.5 `row_number()`

Qué hace la función `row_number`?

```
set.seed(45678)
y <- c(10,21,22,NA,5,4)
head(y)
```

```
## [1] 10 21 22 NA 5 4
```

```
df <- as_tibble(y)
df
```

```
## # A tibble: 6 x 1
##   value
##   <dbl>
## 1     10
## 2     21
## 3     22
## 4     NA
```

```
## 5      5
## 6      4

df %>%
  dplyr::select(value) %>%
  mutate(row=row_number(value)) # equivalente a rank,
```

```
## # A tibble: 6 x 2
##   value   row
##   <dbl> <int>
## 1     10     3
## 2     21     4
## 3     22     5
## 4     NA     NA
## 5      5     2
## 6      4     1
```

11.6 dense_rank()

`min_rank()`, `dense_rank()` y `row_number()` se diferencian en cómo tratan los empates: `min_rank()` deja huecos tras un empate, `dense_rank()` no deja huecos y `row_number()` da un valor distinto a cada fila.

```
set.seed(45678)
z <- c(10,12,12,NA,51,4)
df <- as_tibble(z)
df
```

```
## # A tibble: 6 x 1
##   value
##   <dbl>
## 1     10
## 2     12
## 3     12
## 4     NA
## 5     51
## 6      4
```

```
df %>%
  dplyr::select(value) %>%
  mutate(dense=dense_rank(value)) # equivalente a rank, NOTA que los NA no son asignado un valor
```

```
## # A tibble: 6 x 2
##   value dense
##   <dbl> <int>
## 1     10     2
```

```
## 2    12    3
## 3    12    3
## 4    NA    NA
## 5    51    4
## 6     4    1
```

Ejercicio 2: los empates. Con la misma muestra de diamantes, crea de una vez tres columnas sobre `quilate`: `min_rank()`, `dense_rank()` y `row_number()`. Ordena por `quilate` y enseña las primeras 20 filas.

Verifica: el valor máximo de la columna de `dense_rank()` tiene que ser menor o igual que el de `min_rank()`.

En palabras: ¿cuál de las tres deja huecos después de un empate, cuál no deja huecos, y cuál le da un número distinto a cada fila aunque estén empatadas?

Escribe tu código aquí

11.7 percent_rank()

Un número entre 0 y 1: es `min_rank()` re-escalado, así que el valor más pequeño recibe 0 y el más grande recibe 1.

```
set.seed(45678)
w <- c(1,2,2,NA,5,4)
w
```

```
## [1] 1 2 2 NA 5 4
```

```
df <- as_tibble(w)
df
```

```
## # A tibble: 6 x 1
##   value
##   <dbl>
## 1     1
## 2     2
## 3     2
## 4    NA
## 5     5
## 6     4
```

```
df %>%
  dplyr::select(value) %>%
  mutate(porcentaje_rank=percent_rank(value)) # equivalente a rank, Un número entre a
```

```
## # A tibble: 6 x 2
##   value porcentaje_rank
```

```
##      <dbl>          <dbl>
## 1      1            0
## 2      2          0.25
## 3      2          0.25
## 4     NA          NA
## 5      5            1
## 6      4          0.75
```

11.7.1 percent_rank() sin NA

```
set.seed(45678)
x2 <- sample(1:50, 7)
df2 <- as_tibble(x2)
df2
```

```
## # A tibble: 7 x 1
##   value
##   <int>
## 1    22
## 2     1
## 3    36
## 4    46
## 5     7
## 6    39
## 7    34
```

```
df2 %>%
  dplyr::select(value) %>%
  mutate(porc2=percent_rank(value))
```

```
## # A tibble: 7 x 2
##   value porc2
##   <int> <dbl>
## 1    22 0.333
## 2     1  0
## 3    36 0.667
## 4    46  1
## 5     7 0.167
## 6    39 0.833
## 7    34 0.5
```

Ejercicio 3: los rangos y los NA. Los diamantes no traen valores faltantes, así que crea unos: con `if_else()`, haz una columna igual a `precio` pero con NA cuando el precio sea mayor de 10000. Cuenta cuántos NA quedaron y calcula `min_rank()` y `row_number()` sobre esa columna nueva.

En palabras: ¿qué rango recibe una fila con NA? ¿Se lo saltan, quedan de

últimas, o quedan como NA? ¿Por qué esto importa cuando quieres “los 10 más grandes” de una variable con datos incompletos?

Escribe tu código aquí

11.8 cume_dist()

Es la proporción de valores menores o iguales al valor de la fila, es decir, la distribución acumulada empírica.

```
set.seed(45678)
x <- c(1,2,3,NA,5,4, 10, 8)
x

## [1] 1 2 3 NA 5 4 10 8
df <- as_tibble(x)
df

## # A tibble: 8 x 1
##   value
##   <dbl>
## 1     1
## 2     2
## 3     3
## 4    NA
## 5     5
## 6     4
## 7    10
## 8     8

df %>%
  dplyr::select(value) %>%
  mutate(rangos_cumulativo=cume_dist(value))

## # A tibble: 8 x 2
##   value rangos_cumulativo
##   <dbl>                <dbl>
## 1     1                0.143
## 2     2                0.286
## 3     3                0.429
## 4    NA                NA
## 5     5                0.714
## 6     4                0.571
## 7    10                1
## 8     8                0.857
```

Ejercicio 4: percent_rank() y cume_dist(). Con la muestra de diamantes, calcula las dos sobre quilate y enseña la fila del diamante más pequeño y la del más grande.

Verifica: uno de los dos índices vale exactamente 0 en el mínimo y el otro no.

En palabras: una de las dos contesta “¿qué proporción de los diamantes es más pequeña que este?” y la otra “¿qué proporción es menor o igual que este?”. Di cuál es cuál y por qué el mínimo vale 0 en una y no en la otra.

Escribe tu código aquí

11.9 Ejemplo trabajado: los vuelos más atrasados

Este es el ejercicio de la sección 5.5.2 del libro en español, resuelto en clase: encontrar los 10 vuelos más retrasados con una función de ordenamiento, y decidir qué hacer con los empates. Lee la documentación de `min_rank()` mientras lo sigues.

```
names(vuelos)

## [1] "anio"          "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
## [19] "fecha_hora"

vuelos%>%
  dplyr::select(atraso_salida, aerolinea)%>%
  arrange(desc(atraso_salida))%>%
  mutate(mas_atrados=min_rank(desc(atraso_salida))) %>%
  head(n=10)

## # A tibble: 10 x 3
##   atraso_salida aerolinea mas_atrados
##   <dbl> <chr>         <int>
## 1      1301 HA             1
## 2      1137 MQ             2
## 3      1126 MQ             3
## 4      1014 AA             4
## 5      1005 MQ             5
## 6       960 DL             6
## 7       911 DL             7
```

```
## 8      899 DL      8
## 9      898 DL      9
## 10     896 AA     10

names(vuelos)

## [1] "anio"      "mes"      "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea" "vuelo"      "codigoCola"
## [13] "origen"    "destino"    "tiempo_vuelo"
## [16] "distancia" "hora"       "minuto"
## [19] "fecha_hora"

vuelos %>%
  arrange(atraso_salida) %>%
  mutate(rango_minimo=min_rank(atraso_salida)) %>%
  head(n=10)

## # A tibble: 10 x 20
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>          <int>          <dbl>
## 1  2013   12    7          2040            2123          -43
## 2  2013    2    3          2022            2055          -33
## 3  2013   11   10          1408            1440          -32
## 4  2013    1   11          1900            1930          -30
## 5  2013    1   29          1703            1730          -27
## 6  2013    8    9           729             755          -26
## 7  2013   10   23          1907            1932          -25
## 8  2013    3   30          2030            2055          -25
## 9  2013    3    2          1431            1455          -24
## 10 2013    5    5           934             958          -24
## # i 14 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>, rango_minimo <int>
```

11.10 Resúmenes por grupo: `group_by()` y `summarise()`

`group_by()` y `summarise()` — resumir por grupo.

- `group_by(variable)`: no cambia los datos, solo marca cómo hay que dividirlos.

- `summarise()`: colapsa cada grupo a **una** fila, con las columnas que le pidas.
- **Ojo:** `mutate()` también respeta la agrupación, y conviene cerrar con `ungroup()`.

Esta sección usa `flights` del paquete `nycflights13`, que tiene los nombres de las columnas en inglés (`dep_delay`, `arr_delay`, `dest`). En el resto del libro usamos vuelos del paquete `datos`, que es la misma tabla con los nombres en español (`atraso_salida`, `atraso_llegada`, `destino`).

```
library(datos)
library(nycflights13)
summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

```
flights %>%
  summarise(delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

```
by_day <- group_by(flights, year, month, day)
```

```
summarise(by_day, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##   year month   day delay
##   <int> <int> <int> <dbl>
## 1  2013     1     1  11.5
## 2  2013     1     2  13.9
## 3  2013     1     3  11.0
## 4  2013     1     4   8.95
## 5  2013     1     5   5.73
## 6  2013     1     6   7.15
## 7  2013     1     7   5.42
## 8  2013     1     8   2.55
## 9  2013     1     9   2.28
## 10 2013     1    10   2.84
## # i 355 more rows
```

```
flights %>%
  group_by(year, month, day) %>%
```

```
summarise(delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##   year month   day delay
##   <int> <int> <int> <dbl>
## 1 2013     1     1 11.5
## 2 2013     1     2 13.9
## 3 2013     1     3 11.0
## 4 2013     1     4  8.95
## 5 2013     1     5  5.73
## 6 2013     1     6  7.15
## 7 2013     1     7  5.42
## 8 2013     1     8  2.55
## 9 2013     1     9  2.28
## 10 2013     1    10  2.84
## # i 355 more rows
```

11.10.1 La aerolínea peor en atraso de salidas

```
flights %>%
  group_by(carrier) %>%
  summarise(delay = mean(dep_delay, na.rm = TRUE)) %>%
  arrange(desc(delay))
```

```
## # A tibble: 16 x 2
##   carrier delay
##   <chr>   <dbl>
## 1 F9      20.2
## 2 EV      20.0
## 3 YV      19.0
## 4 FL      18.7
## 5 WN      17.7
## 6 9E      16.7
## 7 B6      13.0
## 8 VX      12.9
## 9 OO      12.6
## 10 UA      12.1
## 11 MQ      10.6
## 12 DL       9.26
## 13 AA       8.59
## 14 AS       5.80
## 15 HA       4.90
## 16 US       3.78
```

11.10.2 Atraso promedio y distancia por destino

```
by_dest <- group_by(flights, dest)
by_dest
```

```
## # A tibble: 336,776 x 19
## # Groups:   dest [105]
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>      <dbl>   <int>         <int>
## 1  2013     1     1     517           515         2     830           819
## 2  2013     1     1     533           529         4     850           830
## 3  2013     1     1     542           540         2     923           850
## 4  2013     1     1     544           545        -1    1004          1022
## 5  2013     1     1     554           600        -6     812           837
## 6  2013     1     1     554           558        -4     740           728
## 7  2013     1     1     555           600        -5     913           854
## 8  2013     1     1     557           600        -3     709           723
## 9  2013     1     1     557           600        -3     838           846
## 10 2013     1     1     558           600        -2     753           745
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
delay <- summarise(by_dest,
  count = n(),
  dist = mean(distance, na.rm = TRUE),
  delay = mean(arr_delay, na.rm = TRUE))
```

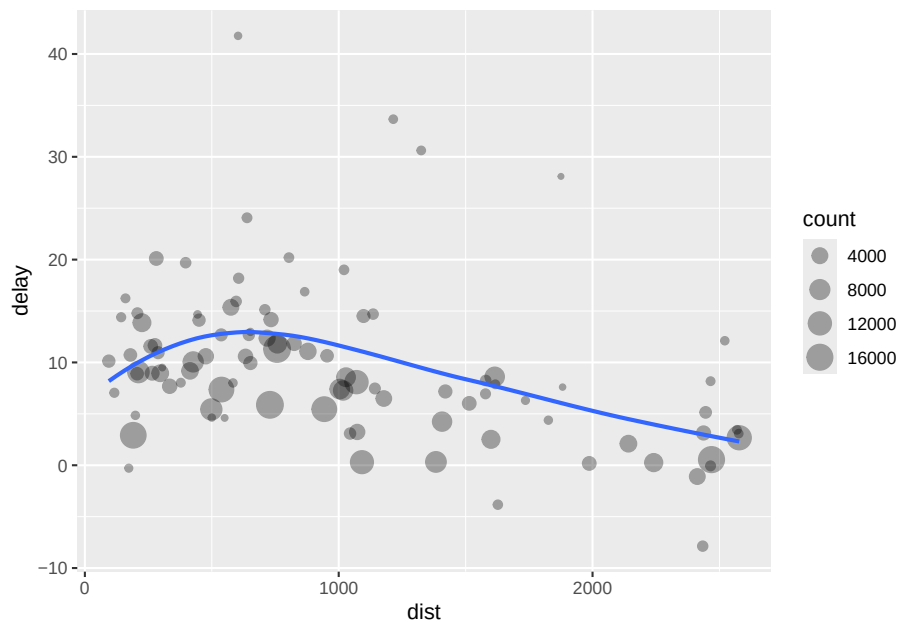
```
delay
```

```
## # A tibble: 105 x 4
##   dest  count  dist delay
##   <chr> <int> <dbl> <dbl>
## 1 ABQ    254 1826  4.38
## 2 ACK    265  199  4.85
## 3 ALB    439  143 14.4
## 4 ANC      8 3370 -2.5
## 5 ATL  17215  757. 11.3
## 6 AUS   2439 1514.  6.02
## 7 AVL    275  584.  8.00
## 8 BDL    443  116  7.05
## 9 BGR    375  378  8.03
## 10 BHM    297  866. 16.9
## # i 95 more rows
```

```
#> `summarise()` ungrouping output (override with `.groups` argument)
delay <- filter(delay, count > 20, dest != "HNL")
```

```
# It looks like delays increase with distance up to ~750 miles
# and then decrease. Maybe as flights get longer there's more
# ability to make up delays in the air?
```

```
ggplot(data = delay, aes(x = dist, y = delay)) +
  geom_point(aes(size = count), alpha = 1/3) +
  geom_smooth(se = FALSE)
```



```
#> `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```

```
names(delay)
```

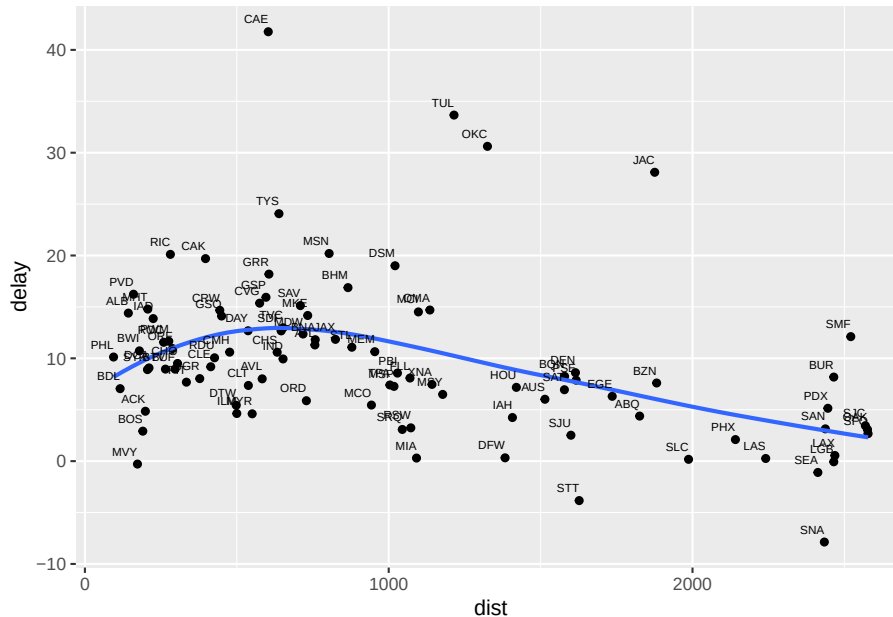
```
## [1] "dest" "count" "dist" "delay"
```

```
head(delay)
```

```
## # A tibble: 6 x 4
##   dest  count  dist delay
##   <chr> <int> <dbl> <dbl>
## 1 ABQ    254 1826   4.38
## 2 ACK    265  199   4.85
## 3 ALB    439  143  14.4
## 4 ATL  17215  757.  11.3
## 5 AUS   2439 1514.   6.02
```

```
## 6 AVL      275  584.  8.00
```

```
ggplot(data = delay, aes(x = dist, y = delay, label=dest)) +
  geom_point() +
  geom_smooth(se = FALSE)+
  geom_text(size=2,aes(label=dest), hjust=1, vjust=-1)
```



Ejercicio 5: group_by() con summarise(). Con `diamantes` completo, calcula por cada categoría de `corte` el número de diamantes, el precio promedio y el quilate promedio, y ordena de mayor a menor precio promedio.

Verifica: la suma de la columna de conteos debe dar el total de filas de `diamantes`.

En palabras: el corte "Ideal" es el mejor corte. ¿Sale con el precio promedio más alto? Mira la columna del quilate promedio y propón una explicación.

```
# Escribe tu código aquí
```

11.11 Promedio móvil: rollmean()

`rollmean()` (paquete `zoo`) — promedio móvil.

- **Qué hace:** sustituye cada valor por el promedio de una ventana de `k` observaciones vecinas, lo que suaviza la serie.
- **align:** "center" (por defecto), "right" (usa solo el pasado) o "left".

- `fill = NA`: rellena con NA las posiciones donde la ventana no cabe.

Fuente: <https://stackoverflow.com/questions/743812/calculating-moving-average>

```
library(tidyverse)
library(zoo)
```

```
some_data = tibble(day = 1:10)
some_data
```

```
## # A tibble: 10 x 1
##       day
##   <int>
## 1     1
## 2     2
## 3     3
## 4     4
## 5     5
## 6     6
## 7     7
## 8     8
## 9     9
## 10    10
```

```
# cma = centered moving average
# tma = trailing moving average
some_data %>%
  mutate(cma = rollmean(day, k = 3, fill = NA)) %>%
  mutate(tma = rollmean(day, k = 3, fill = NA, align = "right")) %>%
  mutate(lma = rollmean(day, k = 3, fill = NA, align = "left")) %>%
  mutate(cmax=rollmax(day, k=30, fill=NA))
```

```
## # A tibble: 10 x 5
##       day  cma  tma  lma cmax
##   <int> <dbl> <dbl> <dbl> <lgl>
## 1     1   NA   NA    2  NA
## 2     2    2   NA    3  NA
## 3     3    3    2    4  NA
## 4     4    4    3    5  NA
## 5     5    5    4    6  NA
## 6     6    6    5    7  NA
## 7     7    7    6    8  NA
## 8     8    8    7    9  NA
## 9     9    9    8   NA  NA
## 10    10   NA    9   NA  NA
```

```
some_data
```

```
## # A tibble: 10 x 1
##   day
##   <int>
## 1     1
## 2     2
## 3     3
## 4     4
## 5     5
## 6     6
## 7     7
## 8     8
## 9     9
## 10    10
```

Ejercicio 6: `rollmean()`. Con `países`, filtra un país (por ejemplo "Costa Rica") y **ordena por año**. Calcula tres promedios móviles de `esperanza_de_vida` con ventana de 3: centrado, `align = "right"` y `align = "left"`, con `fill = NA`. Grafica la serie original junto al promedio móvil centrado.

En palabras: ¿en qué posiciones aparece NA en cada versión y por qué cambian de lugar? ¿Qué le hace el promedio móvil a las subidas y bajadas de la serie?

```
# Escribe tu código aquí
```

Ejercicio 7 (reto integrador). Combina dos funciones del capítulo. Con `diamantes` completo, agrupa por `color`, calcula `cume_dist(precio)` dentro de cada grupo y quédate con el 5 % más caro **de su propio color**. Cuenta cuántos son por color. Después repite el ejercicio sin agrupar, con el 5 % más caro de toda la tabla, y compara las dos listas.

En palabras: ¿cuál de las dos versiones contesta la pregunta “¿cuáles son los diamantes caros para su color?” y por qué la otra no?

```
# Escribe tu código aquí
```


Chapter 12

Transformación: Índices paramétricos y no paramétricos

Fecha de la última revisión

[1] "2026-09-06"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/transform.html>

Español: <https://es.r4ds.hadley.nz/05-transform.html>

12.1 Temas

- Qué quiere decir paramétrico y no paramétrico
 - Medidas de centro: `mean()`, `median()`
 - Medidas de dispersión: `sd()`, `IQR()`, `mad()`
 - Medidas de posición y extremos: `min()`, `max()`, `quantile()`, `first()`, `last()`
 - Cómo se reporta un número: `signif()`, `round()`
 - Valores faltantes y `na.rm`
-

12.2 Paquetes

```
library(tidyverse)
library(datos)
```

12.3 ¿Qué quiere decir paramétrico y no paramétrico?

Un índice **paramétrico** describe los datos suponiendo que vienen de una distribución con cierta forma, casi siempre la normal, y usa **todas** las observaciones: el promedio y la desviación estándar son los dos ejemplos clásicos. Un índice **no paramétrico** no supone ninguna forma y se apoya en el orden de los datos, no en su magnitud: la mediana, el rango intercuartil y la desviación absoluta mediana.

La consecuencia práctica es la que hay que recordar: un solo valor extremo mueve mucho al promedio y a la desviación estándar, y casi no mueve a la mediana ni al rango intercuartil. Por eso, cuando la distribución tiene una cola larga, los índices no paramétricos describen mejor lo que es típico.

12.4 Funciones estadísticas básicas

Casi todas estas funciones (`mean()`, `sd()`, `min()`, `max()`, `IQR()`...) devuelven `NA` si hay valores faltantes. Añade `na.rm = TRUE` para calcularlas ignorando los `NA`.

Funciones estadísticas de resumen, cada una devuelve un solo número.

- **Centro:** `mean()` (promedio), `median()` (mediana).
- **Dispersión:** `sd()` (desviación estándar), `IQR()` (rango intercuartil), `mad()` (desviación absoluta mediana).
- **Extremos y posición:** `min()`, `max()`, `quantile()`, `first()`, `last()`.
- **Recuerda:** añade `na.rm = TRUE` si hay valores faltantes.

12.4.1 Un ejemplo mínimo de cada una

```
x <- c(4, 8, 15, 16, 23, 42, NA)

mean(x, na.rm = TRUE)      # centro, paramétrico
```

```
## [1] 18
```

```

median(x, na.rm = TRUE)      # centro, no paramétrico

## [1] 15.5
sd(x, na.rm = TRUE)          # dispersión, paramétrica

## [1] 13.49074
IQR(x, na.rm = TRUE)         # dispersión, no paramétrica

## [1] 11.5
mad(x, na.rm = TRUE)         # dispersión, no paramétrica y robusta

## [1] 11.1195
min(x, na.rm = TRUE)

## [1] 4
max(x, na.rm = TRUE)

## [1] 42
quantile(x, c(0.25, 0.50, 0.75), na.rm = TRUE)

##   25%   50%   75%
##  9.75 15.50 21.25
first(x)                      # el primero en el orden de las filas

## [1] 4
last(x)                       # el último en el orden de las filas

## [1] NA
signif(mean(x, na.rm = TRUE), 3) # 3 cifras significativas

## [1] 18
round(mean(x, na.rm = TRUE), 2)  # 2 decimales

## [1] 18

```

`first()` y `last()` **no** son `min()` y `max()`. Leen la posición de la fila, no el tamaño del valor, así que dependen del orden de la tabla: si no ordenaste con `arrange()`, el resultado puede cambiar sin que te des cuenta.

La mediana y el rango intercuartil se volvieron herramientas de uso diario con **John W. Tukey**, que en *Exploratory Data Analysis* (1977) propuso resumir cualquier variable con cinco números resistentes a los valores extremos. La desviación absoluta mediana es todavía más vieja, aparece en trabajos de **Gauss**,

y fue rescatada en los años setenta por la estadística robusta de **Peter J. Huber** y **Frank Hampel**.

12.5 Tarea de grupo

Preparar un `.rmd` con las explicaciones de cómo utilizar la función asignada. Debe incluir

1. La definición de la función en palabra y matemática.
2. un script sencillo (con pocos datos) (uno o más ejemplos) para explicar la función
3. un script con los datos de “vuelos” o de “Covid-19 de PR”.
4. a las 2:20pm cada grupo presentará su trabajo.
5. Después de la clase cada grupo, mejorará su `.rmd` con los comentarios recibidos.
6. Domingo se subirá el `.rmd` y el `.html` en MSteam (cada estudiante lo subirá: tendrá el nombre de cada estudiante en el trabajo)
7. El profesor revisará los trabajos y subsiguiente se distribuirá los `.rmd` y `html` a los estudiantes.

Grupo	Función	Tipo de índice
G1	<code>IQR()</code>	dispersión, no paramétrico
G2	<code>mad()</code>	dispersión, no paramétrico
G3	<code>first()</code> y <code>last()</code>	posición
G4	<code>quantile()</code>	posición, no paramétrico
G5	<code>signif()</code>	presentación del resultado
sin asignar	<code>min()</code> , <code>max()</code>	extremos
sin asignar	<code>mean()</code> , <code>sd()</code>	centro y dispersión, paramétricos

12.6 Ejercicios del libro R4DS

Hacer los ejercicios en la sección 5.7.1 del libro en español.

12.7 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan `países` y `diamantes` del paquete `datos`, no `vuelos`

ni los datos de COVID de Puerto Rico. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_12_indices.Rmd`.

La idea de fondo es siempre la misma: para cada pregunta hay un índice **paramétrico** (promedio, desviación estándar), que supone una distribución bien portada, y uno **no paramétrico** (mediana, rango intercuartil, `mad()`), que no lo supone. Casi todos los ejercicios piden calcular los dos y decidir cuál describe mejor los datos.

Empieza creando el subconjunto de un solo año:

```
library(tidyverse)
library(datos)
p2007 <- paises |> filter(anio == 2007)
```

Ejercicio 1: `mean()` y `median()`. Con `p2007`, calcula por continente el promedio, la mediana y el número de países, y añade la diferencia entre promedio y mediana.

Verifica: la suma de los conteos debe dar el número de países de `p2007`.

En palabras: ¿qué te dice el **signo** de esa diferencia sobre la forma de la distribución? Si el promedio es menor que la mediana, ¿hacia dónde hay valores extremos?

Escribe tu código aquí

Ejercicio 2: `sd()` e `IQR()`. Con `p2007`, calcula por continente la desviación estándar y el rango intercuartil de `esperanza_de_vida`. Ordena por cada una de las dos y compara los dos ordenamientos.

En palabras: uno de los dos índices es la distancia típica al promedio y el otro es el ancho de la mitad central de los datos. Di cuál es cuál, y explica qué significa que los dos ordenamientos no coincidan.

Escribe tu código aquí

Ejercicio 3: `mad()`, la dispersión robusta. Usa `pib_per_capita` en `p2007`. Calcula promedio, mediana, desviación estándar y `mad()`, y haz un histograma con el promedio y la mediana marcados. Después elimina los cinco países con el `pib_per_capita` más alto y repite los cuatro cálculos.

En palabras: ¿cuáles medidas cambiaron mucho y cuáles casi no se movieron? Esa es la definición práctica de **robusto**: explícala con tus propios números.

Escribe tu código aquí

Ejercicio 4: `quantile()`. Con `p2007` y `pib_per_capita`, calcula el primer cuartil, la mediana, el tercer cuartil y los percentiles 10 y 90. Dentro de `summarise()` pide **una** probabilidad a la vez.

Verifica: el rango intercuartil del ejercicio anterior tiene que ser igual a la diferencia entre el tercer cuartil y el primero.

En palabras: interpreta el percentil 90 en una oración completa, del tipo “el 90 % de los países tiene un producto interno bruto por persona menor de ...”.

Escribe tu código aquí

Ejercicio 5: `min()`, `max()` y el rango. Con `p2007`, saca por continente el mínimo, el máximo y la diferencia entre los dos, y usa `slice_max()` y `slice_min()` para identificar cuál país está en cada extremo.

En palabras: el rango se calcula con exactamente dos observaciones de todo el continente. ¿Por qué eso lo hace una medida frágil, y cuál de las medidas de los ejercicios 2 y 3 preferirías para comparar la variabilidad entre continentes?

Escribe tu código aquí

Ejercicio 6: `first()`, `last()` y el orden. Con `países` completo, agrupa por país, ordena por año y calcula la esperanza de vida del primer año, la del último y el cambio. Enseña los cinco países que más subieron y los que bajaron. Repite el cálculo sin ordenar y compara.

En palabras: ¿por qué `first()` y `last()` no son lo mismo que `min()` y `max()`, aunque en esta variable muchas veces coincidan?

Escribe tu código aquí

Ejercicio 7: los NA y `na.rm`. A partir de `p2007`, crea una columna igual a `pib_per_capita` pero con NA en los países cuyo valor sea mayor de 40000. Calcula el promedio sin `na.rm = TRUE` y con `na.rm = TRUE`, y compáralo con el promedio original.

En palabras: ¿qué tendría que ser cierto sobre los datos que faltan para que el promedio con `na.rm = TRUE` siga siendo honesto? En este caso, ¿se cumple?

Escribe tu código aquí

Ejercicio 8 (reto integrador): paramétrico frente a no paramétrico. Con `diamantes`, calcula para `precio` el promedio, la mediana, la desviación estándar, el rango intercuartil y `mad()`, y haz un histograma. Repite todo con `log10(precio)`.

En palabras: si tuvieras que describir el precio de un diamante típico con dos números, ¿cuáles escogerías y por qué? ¿Se parecen más el promedio y la mediana después de la transformación? ¿Qué le pasa a las unidades de tu respuesta cuando trabajas en escala logarítmica?

Escribe tu código aquí



Chapter 13

Script

Fecha de la última revisión

[1] "2026-09-06"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/workflow-scripts.html>

Español: <https://es.r4ds.hadley.nz/06-workflow-scripts.html>

13.1 Temas

- La consola y el script: para qué sirve cada uno
 - Ejecutando códigos desde el script
 - El proyecto de RStudio y el directorio de trabajo
 - Diagnóstico de RStudio
 - `source()`: correr un script completo
 - Empezar de cero: reiniciar R
 - Cómo se escribe un script que se entiende meses después
-

13.2 Paquetes

```
library(tidyverse)
library(datos)
```

13.3 La consola y el script: dos lugares distintos

Todo lo que hacemos en R se puede escribir en dos sitios. La **consola** es el lugar donde R contesta de inmediato: uno escribe, presiona *Enter*, y la respuesta aparece abajo. Es cómoda para probar una idea suelta, para ver cuántas filas tiene una tabla o para recordar cómo se llamaba un argumento. Lo que se escribe allí, sin embargo, se pierde: al cerrar RStudio no queda ningún registro ordenado de lo que se hizo ni en qué orden se hizo.

El **script** es un archivo de texto con extensión `.R` que guarda las órdenes en el orden en que hay que ejecutarlas. Ese archivo es el verdadero producto del trabajo. Un análisis no es el número que salió en la pantalla; es el script que permite volver a producir ese número mañana, en otra computadora, o cuando un revisor pregunte cómo se calculó un promedio. La regla práctica es corta: **lo que se prueba va en la consola, lo que funciona va en el script.**

Por eso el script se guarda temprano y se guarda seguido. Un script sin guardar es exactamente igual de frágil que la consola.

No copies y pegues en la consola: coloca el cursor en la línea (o selecciona varias) y ejecútala con `Cmd/Ctrl+Enter` directamente desde el script.

Copiar y pegar parece más rápido, pero rompe justamente lo que hace útil al script: la línea que ejecutaste y la línea que quedó guardada dejan de ser la misma. Al ejecutar desde el script siempre corre lo que está escrito, y lo que está escrito es lo que otra persona (o tú, en dos meses) va a leer.

13.4 Ejecutando códigos desde el script

Vale la pena aprender de memoria estos atajos: son los que más se usan en todo el curso.

Atajo	Qué hace
<code>Cmd/Ctrl + Enter</code>	Ejecuta la línea donde está el cursor, o la selección, y pasa a la siguiente
<code>Cmd/Ctrl + Shift + Enter</code>	Ejecuta el script completo
<code>Cmd/Ctrl + S</code>	Guarda el script
<code>Cmd/Ctrl + Shift + S</code>	Ejecuta el archivo con <code>source()</code>
<code>Alt/Option + -</code>	Escribe el operador de asignación <code><-</code>
<code>Cmd/Ctrl + Shift + M</code>	Escribe el pipe <code>%>%</code>
<code>Tab</code>	Completa nombres de funciones, de objetos y de columnas

La tecla *Tab* no es solo para escribir menos: es la forma más rápida de evitar erratas. Si el nombre no aparece al presionar *Tab*, casi siempre significa que el objeto todavía no existe o que el paquete no está activado.

13.5 El directorio de trabajo y los proyectos de RStudio

R siempre está “parado” en una carpeta, que se llama el **directorio de trabajo**. Cuando escribes `read_csv("datos.csv")` sin más, R busca ese archivo ahí y en ningún otro sitio. La mayoría de los errores de “no encuentro el archivo” no son errores de código: son errores de ubicación.

`getwd()`, `setwd()` y `list.files()` (base R): saber y cambiar dónde está parado R.

- `getwd()`: devuelve la carpeta en la que R está trabajando ahora mismo.
- `list.files()`: enseña los archivos que R ve desde esa carpeta.
- `setwd("ruta")`: cambia la carpeta de trabajo. Úsala lo menos posible, porque una ruta como `"/Users/tu_nombre/Escritorio"` solo existe en tu computadora.

La solución no es escribir `setwd()` al principio de cada script, sino trabajar dentro de un **Proyecto de RStudio** (*File > New Project*). Un proyecto crea un archivo `.Rproj` en una carpeta y, cada vez que lo abres, R queda parado ahí automáticamente. A partir de ese momento las rutas se escriben **relativas** al proyecto, por ejemplo `"Datos/mi_archivo.csv"`, y el script funciona igual en tu computadora, en la de un compañero y en la de un revisor.

Un script que empieza con `setwd("/Users/raymond/Documentos/clase")` está roto para todo el mundo menos para quien lo escribió. Es la razón más común por la que un análisis “no corre” al enviárselo a otra persona.

13.6 Diagnóstico de RStudio

Mientras escribes, RStudio revisa el código y coloca un símbolo pequeño en el margen izquierdo del script:

- una **x roja** avisa de un error de sintaxis: falta un paréntesis, falta una coma, sobra un signo;
- un **triángulo amarillo** avisa de algo sospechoso pero legal, por ejemplo una variable que creaste y nunca usaste;
- una **i azul** ofrece una sugerencia de estilo.

Si pasas el cursor por encima del símbolo, RStudio explica qué encontró. Vale la pena atender el margen **antes** de ejecutar: es mucho más barato arreglar un paréntesis que perseguir un error en la consola.

Otra ayuda del mismo tipo: si en la consola aparece un **+** en lugar del **>**, R no está trabado, está esperando que termines algo (falta cerrar un paréntesis o una comilla). Presiona *Esc* y vuelve a empezar la línea.

13.7 `source()`: correr un script completo

`source()` (base R): ejecuta un archivo `.R` completo, de arriba hacia abajo, como si hubieras corrido todas sus líneas una por una.

- **Uso típico:** `source("funciones_del_curso.R")` al principio de un análisis, para cargar funciones que escribiste en otro archivo.
 - **`echo = TRUE`:** además de ejecutar, muestra en la consola cada línea que va corriendo.
 - **Para qué sirve:** separar el código que *hace* algo del código que lo *usa*, y no repetir la misma función en cinco scripts distintos.
-

13.8 Empezar de cero: reiniciar R

La prueba de fuego de un script es esta: **reiniciar R y correrlo entero de arriba hacia abajo**. En RStudio se hace con *Session > Restart R*. Si después de reiniciar el script se cae, es que dependía de algún objeto que estaba en la memoria pero que nadie creó dentro del archivo, y por lo tanto el análisis no es reproducible.

Entra a *Tools > Global Options > General* y deja “Restore .RData into workspace at startup” **sin marcar**, y “Save workspace to .RData on exit” en **Never**. Así R arranca siempre limpio y los problemas aparecen enseguida, no el día antes de entregar.

13.9 Cómo se escribe un script que se entiende

Un script se lee muchas más veces de las que se escribe, casi siempre por la misma persona que lo escribió y que ya no se acuerda de nada. Cuatro costumbres resuelven casi todo:

1. **Todos los `library()` al principio.** Quien abra el archivo sabe en la primera pantalla qué paquetes necesita instalar.

2. **Nombres que se entienden solos.** promedio_autopista dice lo que contiene; x2 no dice nada. Sin espacios, sin acentos y sin eñes en los nombres de objetos, para que no fallen en otras computadoras.
3. **Comentarios que explican el porqué, no el qué.** # se eliminan los vuelos cancelados porque no tienen atraso medido sirve; # filtrar no aporta nada que el código no diga ya.
4. **Secciones con títulos.** Si terminas un comentario con cuatro guiones o más, RStudio lo convierte en una sección navegable:

```
# Paquetes y datos -----
# Limpieza -----
# Análisis -----
```

El menú desplegable al pie del editor (o Cmd/Ctrl + Shift + O) muestra entonces el esquema completo del script y permite saltar de una sección a otra.

La idea de guardar las órdenes en un archivo aparte, en lugar de escribirlas una por una, viene del lenguaje **S**, desarrollado en los Laboratorios Bell a partir de 1976 por **John M. Chambers** y sus colegas; de ahí hereda R la función `source()`. El entorno **RStudio**, que puso el script y la consola en la misma pantalla, fue publicado en 2011 por la compañía fundada por **J. J. Allaire**, que desde 2022 se llama **Posit**.

13.10 Un ejemplo completo

El bloque siguiente es un script en miniatura: activa los paquetes, mira los nombres de las columnas, quita los vuelos cancelados y calcula un promedio. Cada paso deja un objeto con nombre, así que se puede revisar por partes.

```
library(dplyr)
library(datos)
library(nycflights13)

names(vuelos)

## [1] "anio"           "mes"            "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"          "codigoCola"
## [13] "origen"         "destino"        "tiempo_vuelo"
## [16] "distancia"      "hora"           "minuto"
## [19] "fecha_hora"
```

```

no_cancelado <- vuelos %>%
  filter(!is.na(atraso_salida), !is.na(atraso_llegada))
head(no_cancelado)

## # A tibble: 6 x 19
##   anio   mes   dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>           <int>           <dbl>
## 1  2013     1     1           517             515             2
## 2  2013     1     1           533             529             4
## 3  2013     1     1           542             540             2
## 4  2013     1     1           544             545            -1
## 5  2013     1     1           554             600            -6
## 6  2013     1     1           554             558            -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
no_cancelado %>%
  group_by(anio, mes, dia) %>%
  summarise(media = mean(atraso_salida))

## # A tibble: 365 x 4
## # Groups:   anio, mes [12]
##   anio   mes   dia media
##   <int> <int> <int> <dbl>
## 1  2013     1     1  11.4
## 2  2013     1     2  13.7
## 3  2013     1     3  10.9
## 4  2013     1     4   8.97
## 5  2013     1     5   5.73
## 6  2013     1     6   7.15
## 7  2013     1     7   5.42
## 8  2013     1     8   2.56
## 9  2013     1     9   2.30
## 10 2013     1    10   2.84
## # i 355 more rows
x <- 10

```

13.11 Ejercicios del libro R4DS

Hacer los ejercicios en la sección 6.3 del libro en español.

13.12 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan **millas** del paquete **datos**, no **vuelos**. Este capítulo trata de la forma de trabajar, así que hay menos ejercicios que en otros, pero cada uno hay que hacerlo de verdad en RStudio, no solo leerlo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_13_scripts.Rmd`.

13.12.1 Ejercicio 1. ¿Dónde está parada tu sesión?

Corre `getwd()` y `list.files()` en tu documento.

Verificación: entre los nombres que devuelve `list.files()` debe aparecer el de tu propio archivo `.Rmd`.

En palabras: si el archivo `datos.csv` estuviera en tu Escritorio y escribieras `read_csv("datos.csv")`, ¿lo encontraría R? Explica por qué sí o por qué no, usando lo que te devolvió `getwd()`.

Escribe tu código aquí

13.12.2 Ejercicio 2. De la consola al script

Datos: **millas**. Construye una tabla con el consumo promedio en **autopista** para cada **clase**, ordenada de mayor a menor. Prueba en la consola hasta que salga, y pega **solo** las líneas que funcionaron.

Verificación: tu tabla debe tener **7 filas**, una por clase, y **pickup** debe quedar en el **último** lugar.

En palabras: ¿por qué el resultado de **pickup** tiene sentido?

Escribe tu código aquí

13.12.3 Ejercicio 3. Tu propio archivo de funciones

Crea, en la misma carpeta de tu documento, un archivo llamado `funciones_millas.R` que contenga **una sola función** llamada `resumen_clase()`, que reciba una tabla y devuelva el promedio de **autopista** por **clase**. Luego, en el bloque de abajo, cárgala con `source()` y úsala con `millas`.

Verificación: después del `source()`, `exists("resumen_clase")` debe devolver `TRUE`, y el resultado debe ser el mismo del Ejercicio 2.

En palabras: ¿qué ganas al tener la función en su propio archivo en lugar de copiarla en cada análisis?

Escribe tu código aquí

13.12.4 Ejercicio 4. Un script que se entiende

Este código funciona, pero está escrito de manera que nadie lo entiende:

```
x=millas
y=x[x$anio==2008,]
z=mean(y$ciudad)

z
```

Reescribelo con nombres claros, con el operador `<-`, en estilo tidyverse (`filter()` y `summarise()`), y añade dos comentarios que expliquen **por qué** se hace cada paso.

Verificación: tu versión debe dar exactamente el mismo número que la versión fea.

En palabras: de todos los cambios que hiciste, ¿cuál ayuda más a alguien que lea el script dentro de seis meses?

Escribe tu código aquí

13.12.5 Ejercicio 5. La prueba de la sesión limpia

Reinicia R (*Session > Restart R*) y ejecuta tu documento completo con *Knit*, desde la primera línea.

Verificación: el documento tiene que compilar sin errores desde una sesión recién reiniciada.

En palabras: si se cayó, ¿en qué línea fue y qué faltaba? Si compiló a la primera, explica qué costumbre tuya lo hizo posible.

Escribe tu código aquí

13.12.6 Ejercicio 6 (BONO). Secciones y navegación

Organiza tu documento en al menos tres secciones usando comentarios terminados en cuatro guiones o más (**# Paquetes ----**), y ábrelo con el esquema (Cmd/Ctrl + Shift + O).

Verificación: el esquema debe mostrar tus tres secciones.

En palabras: ¿en qué se parece esto a los encabezados **##** de R Markdown?

Escribe tu código aquí

Chapter 14

Errores comunes y cómo resolverlos

Fecha de la última revisión

[1] "2026-09-06"

Equivocarse **es parte normal de programar**. Hasta quienes llevan años usando R ven errores todos los días. La destreza que de verdad importa no es *evitar* los errores, sino **aprender a leerlos y resolverlos**, porque cuando hagas tu propio análisis nadie va a haber “arreglado” el código de antemano.

Un mensaje de error casi siempre te dice **dos cosas**: *qué* salió mal y, muchas veces, *dónde* (la función o la línea). Léelo despacio de arriba hacia abajo; la primera línea suele ser la más importante.

En este capítulo provocamos **a propósito** los errores más frecuentes para que los reconozcas cuando aparezcan. En cada ejemplo verás el mensaje real de R y luego cómo resolverlo.

14.1 Temas

- Cómo se lee un mensaje de error
- La diferencia entre un error, una advertencia y un mensaje
- `could not find function` y `there is no package called`
- `object ... not found`
- Operaciones con texto: `non-numeric argument` y NAs introduced by coercion

- `unused argument, =` frente a `==`
- `object of type 'closure' is not subsettable` y `$` operator is invalid for atomic vectors
- `NA` y `na.rm`
- Archivos que no aparecen y direcciones de internet que no responden
- Un método general para resolver errores

```
library(dplyr)
```

14.2 Error, advertencia y mensaje: no son lo mismo

Los tres se imprimen en rojo en la consola, y por eso al principio asustan por igual. Pero solo uno de los tres detiene el trabajo.

- Un **error** (`Error in ...`) significa que R **no pudo hacer** lo que le pediste. La línea no se ejecutó y no hay resultado. Hay que arreglarlo.
- Una **advertencia** (`Warning message:`) significa que R **sí hizo** algo, pero sospecha que no es lo que querías. Hay resultado, y por eso las advertencias son más peligrosas que los errores: es fácil seguir de largo con un número equivocado.
- Un **mensaje** (`Attaching package:`, `Joining with by = ...`) es solo información. R te cuenta lo que está haciendo y no hay nada que arreglar.

El error te para; la advertencia te deja seguir. Si tu análisis produce una advertencia y tú la ignoras, el número que reportas puede estar mal y nadie te va a avisar. Léelas todas.

14.3 could not find function

```
promedio(c(2, 4, 6))
```

```
## Error in `promedio()`:  
## ! could not find function "promedio"
```

`could not find function "promedio"` significa que R no conoce esa función. Las tres causas más comunes son: (1) escribiste mal el nombre, (2) **olvidaste activar el paquete** con `library()`, o (3) la función es de otro paquete que no está instalado. Solución: revisa la ortografía y corre el `library()` correspondiente.

R distingue mayúsculas de minúsculas, así que `Sqrt` no es lo mismo que `sqrt`:

```
Sqrt(9)
```

```
## Error in `Sqrt()`:  
## ! could not find function "Sqrt"
```

Si una función “no existe” pero estás seguro de que sí, revisa las mayúsculas: `sqrt()`, no `Sqrt()`; `TRUE`, no `True`.

14.4 object ... not found

```
resultado + 1
```

```
## Error:  
## ! object 'resultado' not found
```

object 'resultado' not found quiere decir que usaste una variable que **todavía no existe**. Suele pasar cuando hay una errata en el nombre o cuando no corriste antes el bloque que crea esa variable. Solución: crea la variable primero (`resultado <- ...`) y ejecuta los bloques **en orden**.

14.5 non-numeric argument to binary operator

```
"3" * 2
```

```
## Error in `"3" * 2`:  
## ! non-numeric argument to binary operator
```

R no puede multiplicar **texto** por un número: `"3"` (con comillas) es una cadena de caracteres, no el número 3. Conviértelo con `as.numeric("3")` antes de operar.

14.6 unused argument

```
mean(c(1, 2, 3), n = 2)
```

```
## [1] 2
```

unused argument (`n = 2`) significa que le pasaste a la función un argumento que **no reconoce**. Casi siempre es un nombre de argumento equivocado. Revisa

la ayuda con `?mean` para ver los nombres correctos (aquí no existe `n`; quizás querías `trim`).

14.7 Confundir `=` con `==`

```
filter(mtcars, cyl = 6)
```

```
## Error in `filter()`:  
## ! We detected a named input.  
## i This usually means that you've used `=` instead of `==`.  
## i Did you mean `cyl == 6`?
```

Dentro de `filter()` (y para comparar en general) se usa `==`, no `=`. Un solo `=` **asigna**; el doble `==` **compara**. Por suerte, `dplyr` hasta te sugiere la corrección: `cyl == 6`.

14.8 El caso especial de `NA` (no es un error, pero engaña)

Esto **no** produce un error, pero el resultado sorprende a muchos:

```
mean(c(1, 2, NA))
```

```
## [1] NA
```

Casi cualquier cálculo con un valor faltante (`NA`) devuelve `NA`. No es un error: R te avisa que no puede calcular la media sin saber ese dato. La solución es pedir que ignore los faltantes con `na.rm = TRUE`:

```
mean(c(1, 2, NA), na.rm = TRUE)
```

```
## [1] 1.5
```

14.9 `cannot open file ... No such file or directory`

```
read.csv("mis_datos_que_no_existen.csv")
```

```
## Error in `file()`:  
## ! cannot open the connection
```

Este error aparece cuando R **no encuentra el archivo**. Las causas típicas son: el nombre está mal escrito, el archivo está en otra carpeta, o tu **directorio de trabajo** no es el que crees. La mejor prevención es trabajar siempre dentro de un **Proyecto de RStudio** y usar rutas relativas como "Datos/mi_archivo.csv".

14.10 cannot open URL (descargar datos de internet)

Cuando un capítulo baja datos en vivo, por ejemplo con `fread("https://...")` o `download.file()`, y no hay conexión (o el servidor no responde), verás algo como esto:

```
Error in download.file(...) :
  cannot open URL 'https://ejemplo.com/datos.csv'
```

(No ejecutamos ese código aquí a propósito, para no depender de la red.)

`cannot open URL` **no es un error de tu código**: significa que no hay internet o que el servidor está caído o lento. Verifica tu conexión y vuelve a intentar.

Una técnica profesional para que tu análisis no se caiga por completo cuando esto pasa es `tryCatch()`: intenta la descarga en vivo y, si falla, usa una copia local que guardaste antes. Este es exactamente el patrón que usamos en el capítulo de mapas:

```
datos <- tryCatch(
  fread("https://ejemplo.com/datos.csv"),           # intenta en vivo
  error = function(e) readr::read_csv("Datos/copia_local.csv") # si falla, usa la copia
)
```

Buena práctica: descarga los datos **una vez**, guárdalos con `write_csv()` en la carpeta `Datos/`, y en adelante trabaja con la copia local. Así tu análisis es reproducible aunque no tengas internet.

14.11 there is no package called ...

```
library(paqueteQueNoExiste)
```

```
## Error in `library()`:
## ! there is no package called 'paqueteQueNoExiste'
```

Este mensaje **no** es el mismo que `could not find function`. Aquí el paquete no está **instalado** en la computadora. Instalar y activar son dos pasos distintos: `install.packages("nombre")` se hace **una sola vez** por computadora y baja

el paquete de internet; `library(nombre)` se hace **en cada sesión** y lo activa. Si escribes `library()` de algo que nunca instalaste, sale este error.

`install.packages()` va **entre comillas** y `library()` normalmente **sin comillas**. Y nunca dejes `install.packages()` dentro de un script que vas a compartir: obliga a los demás a reinstalar el paquete cada vez que corren tu análisis.

14.12 object of type 'closure' is not subsettable

Este mensaje tiene fama de incomprensible, y en realidad dice algo sencillo. “Closure” es la palabra que usa R para decir **función**. El error aparece cuando tratas a una función como si fuera una tabla:

```
data$edad
```

```
## Error in `data$edad`:
## ! $ operator is invalid for atomic vectors
```

`data` es una **función** de R, no tus datos. Cuando le pides `data$edad`, R contesta que a una función no se le puede sacar una columna. Pasa siempre por lo mismo: le pusiste a tu tabla un nombre que ya usa R (`data`, `df`, `mean`, `sum`, `c`, `t`) y luego olvidaste crearla, o el bloque que la creaba no corrió. Solución: dale a tus tablas nombres propios (`millas_2008`, `datos_covid`) y ejecuta los bloques en orden.

14.13 \$ operator is invalid for atomic vectors

```
x <- c(a = 1, b = 2, c = 3)
x$a
```

```
## Error in `x$a`:
## ! $ operator is invalid for atomic vectors
```

El `$` solo funciona en tablas y listas, no en **vectores**. Un vector no tiene columnas, tiene posiciones y a veces nombres. Para sacarle un elemento se usan corchetes: `x["a"]` conserva el nombre y `x[["a"]]` devuelve el valor pelado.

```
x["a"]
```

```
## a
## 1
```

```
x[["a"]]
```

```
## [1] 1
```

14.14 NAs introduced by coercion (una advertencia, no un error)

```
as.numeric(c("3", "4", "cinco"))
```

```
## Warning: NAs introduced by coercion
```

```
## [1] 3 4 NA
```

R sí hizo la conversión, pero lo que no pudo convertir lo dejó en NA. No hay error, hay resultado, y por eso este es un caso donde ignorar la advertencia sale caro: si esa columna venía de un Excel con una celda que decía “n/a” o “sin dato”, acabas de convertirla en un NA sin darte cuenta. Antes de `as.numeric()`, mira siempre qué valores raros trae la columna.

14.15 longer object length is not a multiple ... (reciclaje)

```
c(1, 2, 3, 4) + c(10, 20, 30)
```

```
## Warning in c(1, 2, 3, 4) + c(10, 20, 30): longer object length is not a
## multiple of shorter object length
```

```
## [1] 11 22 33 14
```

Cuando dos vectores tienen largos distintos, R **repite** el corto hasta completar el largo, en vez de negarse. Eso se llama *reciclaje*. Si los largos encajan (4 y 2) lo hace en silencio; si no encajan (4 y 3) avisa con esta advertencia. En cualquiera de los dos casos, si tus dos columnas debían tener el mismo largo, el resultado está mal. Revisa con `length()` o `nrow()`.

14.16 Un método general para resolver errores

1. **Lee el mensaje completo**, sobre todo la primera línea.
2. Identifica la **función** o la **línea** que menciona.
3. Pregúntate: ¿escribí bien el nombre?, ¿activé el paquete?, ¿existe la variable?, ¿el archivo está donde digo?
4. Si te atorras, **copia el mensaje de error y búscalo en internet**: casi siempre alguien ya tuvo el mismo problema.
5. Reduce el problema a un **ejemplo mínimo** que lo reproduzca; muchas veces al simplificarlo descubres la causa.

Guarda la calma: un error no significa que “no sirves para esto”. Significa que R necesita que le aclares algo. Cada error que resuelves te hace mejor analista.

14.17 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Aquí la explicación pesa más que en otros capítulos: lo que se evalúa es que sepas **leer** el mensaje, no solo que el código acabe corriendo. Los ejercicios usan **millas** y **diamantes** del paquete **datos**, no **mtcars** ni **iris**, que son los del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_14_errores.Rmd`.

Truco para que tu documento compile aunque muestres un error a propósito: añade `error=TRUE` a las opciones del bloque, como se hace en este capítulo.

14.17.1 Ejercicio 1. Leer el mensaje

Estas cinco líneas están rotas, cada una por una razón distinta:

```
Filter(millas, clase == "compact")

millas$autopsta

filter(millas, anio = 2008)

"8" + millas$cilindros[1]

mean(millas$clase)
```

Para cada una: córrela con `error=TRUE`, **copia lo que salió**, explica en una oración qué significa, y escribe debajo la versión corregida.

Verificación: las cinco versiones corregidas deben correr limpias, sin error y sin advertencia.

En palabras: solo tres de las cinco producen un **error** que detiene la línea. ¿Cuáles son las otras dos, qué devuelven, y por qué son más peligrosas que las tres que sí fallan de frente?

```
# Escribe tu código aquí
```

14.17.2 Ejercicio 2. Instalar no es activar

Sin activar ningún paquete nuevo, corre `rollmean(1:10, 3)` en un bloque con `error=TRUE`. Después arréglalo de dos maneras distintas: una con `library()` y otra escribiendo `paquete::funcion()`.

Verificación: las dos versiones corregidas deben devolver el mismo vector de 8 valores.

En palabras: ¿qué mensaje distinto habrías visto si el paquete `zoo` no estuviera instalado en tu computadora? ¿Por qué no conviene dejar `install.packages("zoo")` dentro del script que entregas?

Escribe tu código aquí

14.17.3 Ejercicio 3. El NA no es un error

Datos: `millas`. Haz una copia y métele valores faltantes a propósito:

```
millas_na <- millas
```

```
millas_na$autopista[c(3, 17, 100)] <- NA
```

Calcula el promedio de `autopista` en `millas_na` de tres maneras: sin `na.rm`, con `na.rm = TRUE`, y quitando antes las filas con NA usando `filter(!is.na(autopista))`.

Verificación: la primera devuelve NA; la segunda y la tercera devuelven el mismo número, calculado sobre 231 filas.

En palabras: las tres son distintas maneras de tratar el mismo problema. ¿Cuándo sería un **error de análisis** usar `na.rm = TRUE` sin pensarlo?

Escribe tu código aquí

14.17.4 Ejercicio 4. =, == y <-

Datos: `diamantes`. Escribe tres líneas, cada una en su propio bloque con `error=TRUE` si hace falta:

1. `filter(diamantes, corte = "Ideal")`
2. `filter(diamantes, corte == "Ideal")`
3. `diamantes_ideal = filter(diamantes, corte == "Ideal")`

Verificación: la segunda y la tercera producen la misma tabla; la primera no produce ninguna.

En palabras: el `=` de la línea 1 y el `=` de la línea 3 son el mismo signo, pero solo uno de los dos está mal. Explica qué hace `=` en cada sitio y por qué en el curso preferimos `<-` para asignar.

Escribe tu código aquí

14.17.5 Ejercicio 5. El ejemplo mínimo

Este bloque se cae, pero el mensaje no dice en cuál de los cuatro pasos:

```
resumen <- diamantes |>
```

```
filter(quilate > 0.5) |>
group_by(corte) |>
summarise(precio_medio = mean(Precio))
```

Encuentra el paso que falla **reduciendo el problema**: corre el primer paso solo, luego los dos primeros, y así hasta que reaparezca el error.

Verificación: cuando lo arregles, la tabla debe tener 5 filas, una por cada nivel de corte.

En palabras: ¿en qué paso estaba el problema y cómo lo supiste? Explica por qué reducir el código a un ejemplo mínimo es más rápido que releer las cuatro líneas enteras.

Escribe tu código aquí

14.17.6 Ejercicio 6 (BONO). Que no se caiga por culpa del internet

Guarda una tabla pequeña de millas en Datos/millas_copia.csv con `write_csv()`. Después escribe un `tryCatch()` que intente leer un archivo desde una dirección de internet que no existe y que, al fallar, lea tu copia local.

Verificación: el bloque debe correr sin error y devolver la tabla de tu copia local.

En palabras: ¿qué diferencia hay entre este `tryCatch()` y simplemente poner `error=TRUE` en el bloque?

Escribe tu código aquí

Chapter 15

Análisis Exploratorio

Fecha de la última revisión

[1] "2026-09-06"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/exploratory-data-analysis.html>

Español: <https://es.r4ds.hadley.nz/07-eda.html>

15.1 Temas

- Variación
 - Valores típicos y valores inusuales
 - Valores faltantes
 - Covariación
 - Patrones y modelos
 - Argumentos en ggplot2
-

15.2 Paquetes

```
library(tidyverse)
library(datos)
```

15.3 Términos importantes

Antes de empezar conviene tener claros cuatro términos, porque todo el capítulo los usa:

- una **variable** es una cantidad, cualidad o propiedad que se puede medir (el quilate de un diamante, su color);
 - un **valor** es el estado de esa variable cuando se mide (0.23 quilates, el color E);
 - una **observación** es el conjunto de valores medidos en la misma unidad y en las mismas condiciones (un diamante, con todas sus medidas);
 - en **datos tabulados** cada valor ocupa su propia celda, cada variable su propia columna y cada observación su propia fila.
-

15.4 ¿Qué es el análisis exploratorio?

El análisis exploratorio de datos, o EDA por sus siglas en inglés, no es una receta con pasos fijos: es un **ciclo de preguntas**. Uno se hace una pregunta sobre los datos, la contesta con un resumen o con una gráfica rápida, y lo que encuentra sugiere la próxima pregunta. Al principio las preguntas son malas, porque todavía no se conocen los datos; después de veinte preguntas malas aparecen las dos o tres buenas, que son las que valen.

Casi todas las preguntas del EDA se reducen a dos:

1. ¿Qué tipo de **variación** hay dentro de cada una de mis variables?
2. ¿Qué tipo de **covariación** hay entre mis variables?

Las gráficas de esta etapa son rápidas y feas a propósito. No son para publicar, son para mirar. La gráfica bonita, con títulos y colores escogidos, viene después, cuando ya se sabe qué se quiere enseñar.

El **análisis exploratorio de datos** (EDA) fue impulsado por **John W. Tukey**, quien publicó el libro *Exploratory Data Analysis* en 1977. Su idea: antes de probar hipótesis, hay que **mirar** los datos con gráficos sencillos para descubrir patrones, valores típicos y atípicos.

count() y los **geom_* de exploración**: resumir y visualizar.

- **count(variable)**: cuenta cuántas observaciones hay por categoría.
 - **Una variable**: `geom_histogram()` / `geom_bar()` para ver su distribución.
 - **Dos variables**: `geom_point()`, `geom_boxplot()`, `geom_count()`, `geom_tile()`, `geom_hex()` según los tipos.
-

15.5 Variación

La **variación** es la tendencia de una variable a cambiar de medición en medición. Todo varía: si mides el mismo diamante dos veces con la misma balanza obtendrás números ligeramente distintos, y si mides diamantes distintos las diferencias serán mucho mayores. La forma de esa variación, es decir, cómo se reparten los valores, es lo primero que hay que mirar.

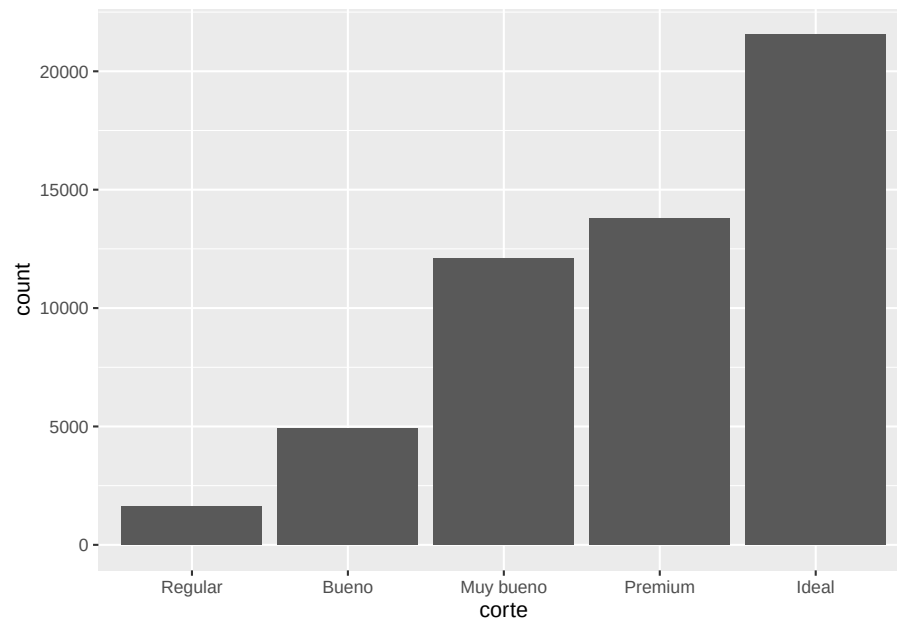
15.5.1 Una variable categórica

Con una variable categórica la pregunta es sencilla: ¿cuántas observaciones hay en cada categoría? Se contesta de dos maneras, con una tabla y con una gráfica, y conviene hacer las dos porque dicen cosas distintas.

```
diamantes |> count(corte)
```

```
## # A tibble: 5 x 2
##   corte      n
##   <ord>    <int>
## 1 Regular  1610
## 2 Bueno   4906
## 3 Muy bueno 12082
## 4 Premium 13791
## 5 Ideal   21551
```

```
ggplot(diamantes, aes(x = corte)) +
  geom_bar()
```

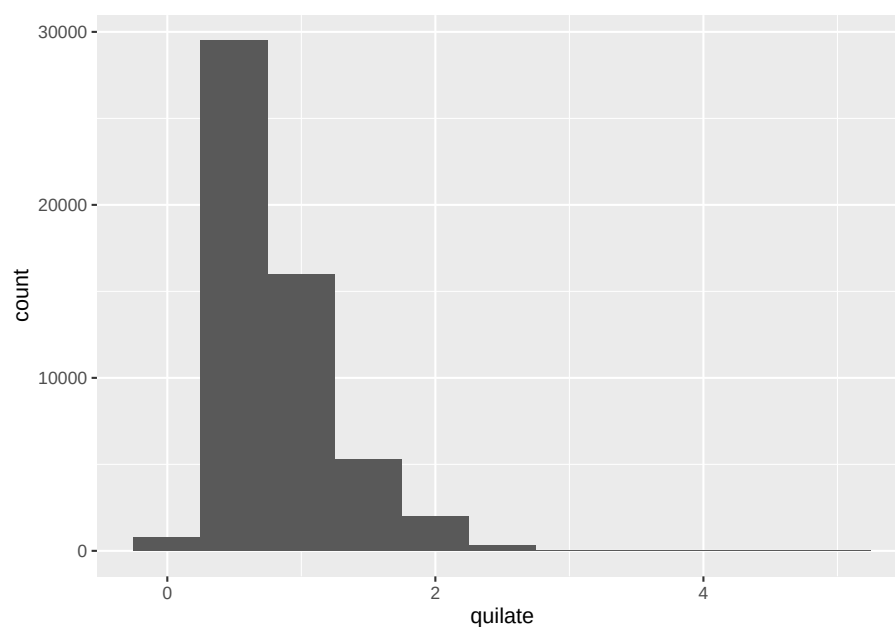


Ya aparece algo que no era obvio: el corte **Ideal**, que es el mejor, es también el **más común**. Esa es exactamente la clase de observación que abre una pregunta nueva.

15.5.2 Una variable continua

Con una variable continua hay que agrupar los valores en intervalos antes de contarlos, y el ancho del intervalo (`binwidth`) cambia por completo lo que se ve. No hay un ancho correcto: hay que probar varios.

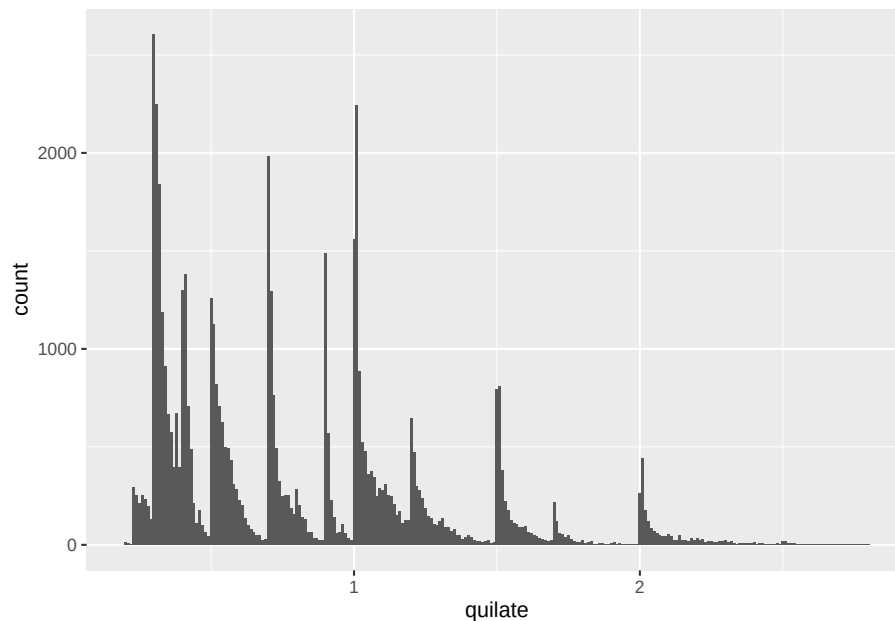
```
ggplot(diamantes, aes(x = quilate)) +  
  geom_histogram(binwidth = 0.5)
```



Con intervalos de medio quilate se ve una distribución sesgada hacia la derecha y poco más. Vale la pena mirar de cerca los diamantes pequeños, que son la inmensa mayoría, con un intervalo mucho más fino:

```
pequenos <- diamantes |> filter(quilate < 3)

ggplot(pequenos, aes(x = quilate)) +
  geom_histogram(binwidth = 0.01)
```



Ahora la gráfica cuenta otra historia: hay **picos** en los valores redondos (0.3, 0.5, 0.7, 1.0 quilates) y muy pocos diamantes justo por debajo de cada uno. Eso no es biología ni geología, es mercado: conviene cortar la piedra para que llegue al número redondo. Con `binwidth = 0.5` ese patrón era invisible.

Nunca te quedes con el primer histograma. Cambia el `binwidth` al menos tres veces. Un intervalo muy ancho esconde los patrones; uno muy fino los ahoga en ruido.

`count()` y `cut_width()`: contar por categoría y por intervalo.

- **count(x)**: tabla de frecuencias de una variable categórica; `count(x, y)` la hace para las combinaciones de dos.
- **cut_width(x, ancho)**: (ggplot2) corta una variable continua en intervalos del mismo ancho y devuelve una variable categórica.
- **Juntas**: `count(cut_width(quilate, 0.5))` es la versión en tabla de lo que el histograma enseña en gráfica.

```
diamantes |> count(cut_width(quilate, 0.5))
```

```
## # A tibble: 11 x 2
##   `cut_width(quilate, 0.5)`     n
##   <fct>                     <int>
## 1 [-0.25,0.25]               785
## 2 (0.25,0.75]             29498
## 3 (0.75,1.25]             15977
## 4 (1.25,1.75]              5313
```


##	5	(1.75,2.25]	2002
##	6	(2.25,2.75]	322
##	7	(2.75,3.25]	32
##	8	(3.25,3.75]	5
##	9	(3.75,4.25]	4
##	10	(4.25,4.75]	1
##	11	(4.75,5.25]	1

15.5.3 Valores típicos

Frente a un histograma, las preguntas útiles son siempre las mismas:

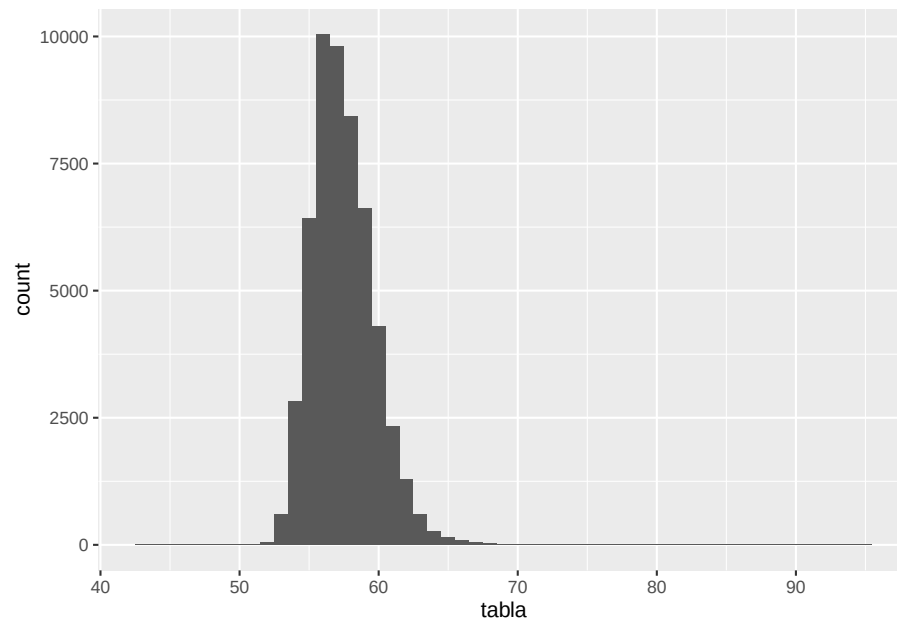
- ¿cuáles valores son los más comunes, y por qué?
- ¿cuáles son raros, y eso encaja con lo que esperabas?
- ¿hay patrones raros, como picos regulares o huecos?
- si hay **grupos separados**, ¿en qué se distinguen las observaciones de cada grupo?

Un hueco en un histograma casi siempre significa una de dos cosas: o hay subpoblaciones distintas mezcladas en la misma tabla, o el instrumento de medición no puede producir esos valores.

15.5.4 Valores inusuales

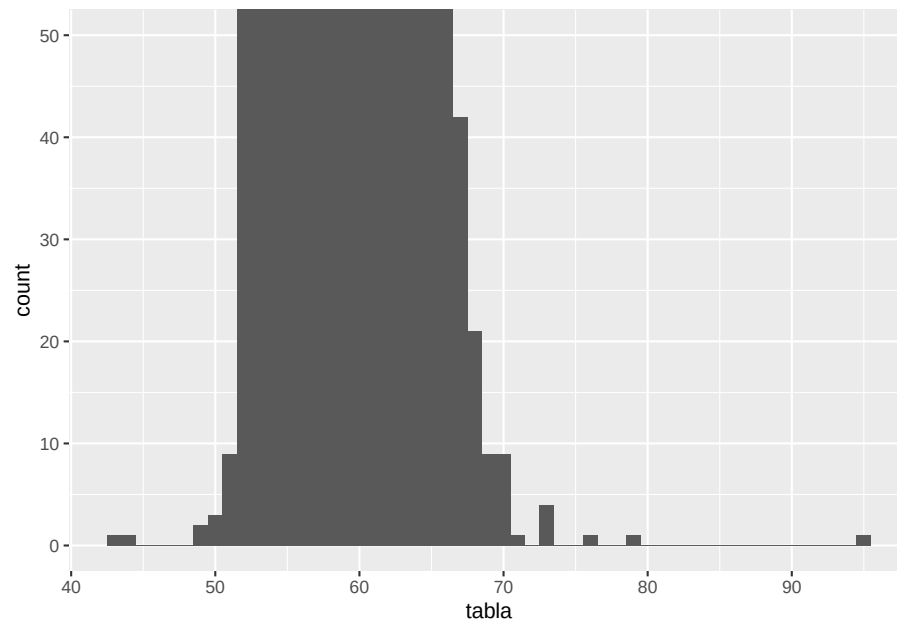
Los valores extremos a veces son errores de tecleo y a veces son el hallazgo del estudio. En un histograma con muchas observaciones son difíciles de ver, porque las barras que los contienen tienen altura casi cero:

```
ggplot(diamantes, aes(x = tabla)) +  
  geom_histogram(binwidth = 1)
```



La solución no es cambiar los datos sino **acercar la vista** al eje vertical:

```
ggplot(diamantes, aes(x = tabla)) +  
  geom_histogram(binwidth = 1) +  
  coord_cartesian(ylim = c(0, 50))
```



Ahora sí se ven las barras diminutas de los extremos. El paso siguiente es siempre mirar esas observaciones una por una:

```
inusuales <- diamantes |>
  filter(tabla < 50 | tabla > 70) |>
  select(precio, quilate, corte, tabla) |>
  arrange(tabla)
```

```
inusuales
```

```
## # A tibble: 12 x 4
##   precio quilate corte   tabla
##   <int>   <dbl> <ord>   <dbl>
## 1  4997    1.04 Ideal     43
## 2   474    0.29 Muy bueno  44
## 3  3951     1   Regular   49
## 4   630    0.3   Regular   49
## 5  1633    0.68 Regular   71
## 6  2100    0.7   Regular   73
## 7  2368    0.71 Regular   73
## 8  2579    0.5   Regular   73
## 9  2579    0.5   Regular   73
## 10 2362    0.79 Regular   76
## 11 2301    0.81 Regular   79
## 12 13387   2.01 Regular   95
```

`coord_cartesian()`: acercar la vista sin tocar los datos.

- `ylim = c(0, 50)`: recorta lo que se **muestra** en el eje vertical.
- **Diferencia con `ylim()` o `scale_y_continuous(limits = ...)`**: esas dos **eliminan** las observaciones que quedan fuera antes de calcular la gráfica, y por eso pueden cambiar el resultado; `coord_cartesian()` solo mueve la ventana.

No borres un valor inusual porque estorba. Primero averigua **por qué** es inusual. Si es un error de captura documentado, se sustituye por NA; si no sabes qué es, se reportan los dos análisis, con y sin él, y se dice en el texto lo que cambia.

15.6 Valores faltantes

Cuando decides que un valor no es de fiar, tienes dos caminos. El primero es descartar la fila entera con `filter()`, y casi nunca es buena idea: por un valor sospechoso pierdes todas las demás medidas de esa observación. El segundo, mejor, es sustituir **solo ese valor** por NA y quedarte con el resto de la fila.

`between()` e `if_else()`: marcar valores fuera de rango.

- `between(x, min, max)`: devuelve TRUE si el valor está dentro del intervalo, límites incluidos. Es más corto y más legible que `x >= min & x <= max`.
- `if_else(condicion, valor_si, valor_no)`: escoge entre dos valores. A diferencia de `ifelse()` de base R, respeta el tipo de dato, y por eso hay que escribir `NA_real_` (no NA) cuando la columna es numérica.

```
diamantes2 <- diamantes |>
  mutate(tabla = if_else(between(tabla, 50, 70), tabla, NA_real_))

sum(is.na(diamantes2$tabla))
```

```
## [1] 12
```

En R los valores faltantes se representan con NA. Funciones como `mean()` o `sum()` los propagan y devuelven NA, por eso conviene usar el argumento `na.rm = TRUE`.

```
mean(diamantes2$tabla)
```

```
## [1] NA
```

```
mean(diamantes2$tabla, na.rm = TRUE)
```

```
## [1] 57.45517
```

Hay una pregunta que casi nadie se hace y que suele ser la más interesante: **¿en qué se diferencian las observaciones que tienen el dato de las que no lo tienen?** En vuelos, por ejemplo, un NA en el atraso de salida no es ruido: significa que el vuelo se canceló.

```
vuelos |>
  mutate(cancelado = is.na(atraso_salida)) |>
  count(cancelado)
```

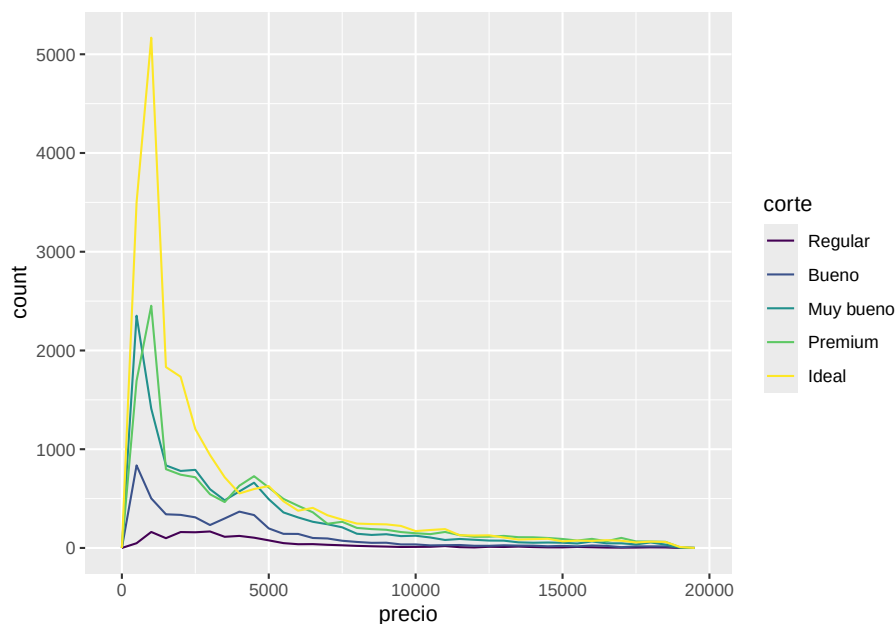
```
## # A tibble: 2 x 2
##   cancelado     n
##   <lgl>       <int>
## 1 FALSE     328521
## 2 TRUE       8255
```

La idea de que **el patrón de los datos que faltan es en sí mismo un dato** se formalizó con **Donald B. Rubin** en 1976, en el artículo *Inference and missing data*. De ahí vienen los términos que todavía se usan: faltantes completamente al azar, faltantes al azar y faltantes no al azar. La diferencia importa: solo en el primer caso quitar los NA deja el análisis intacto.

15.7 Covariación: una variable categórica y una continua

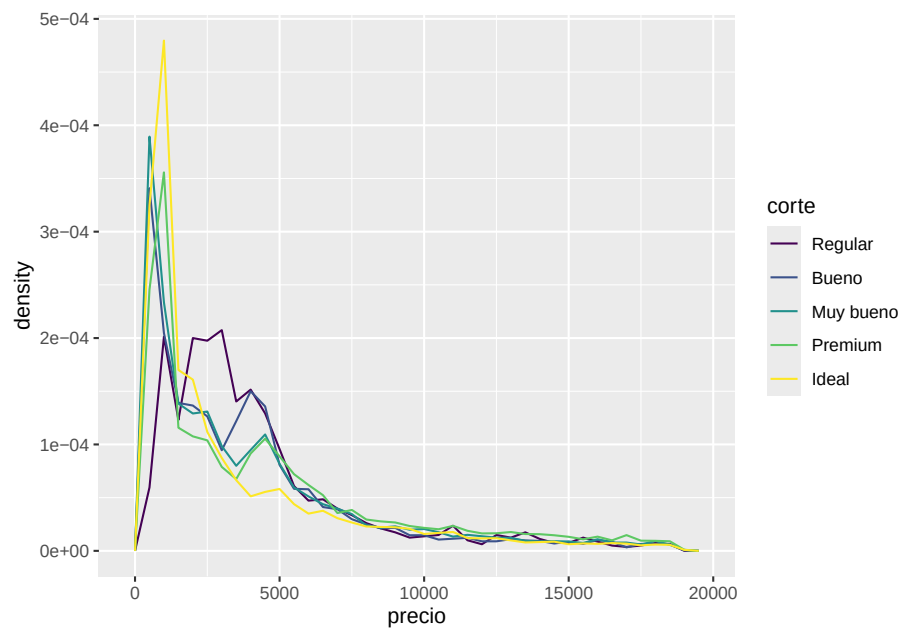
La **covariación** es la tendencia de dos o más variables a cambiar juntas. Con una variable categórica y una continua, la primera idea es superponer las distribuciones:

```
ggplot(diamantes, aes(x = precio)) +  
  geom_freqpoly(aes(color = corte), binwidth = 500)
```



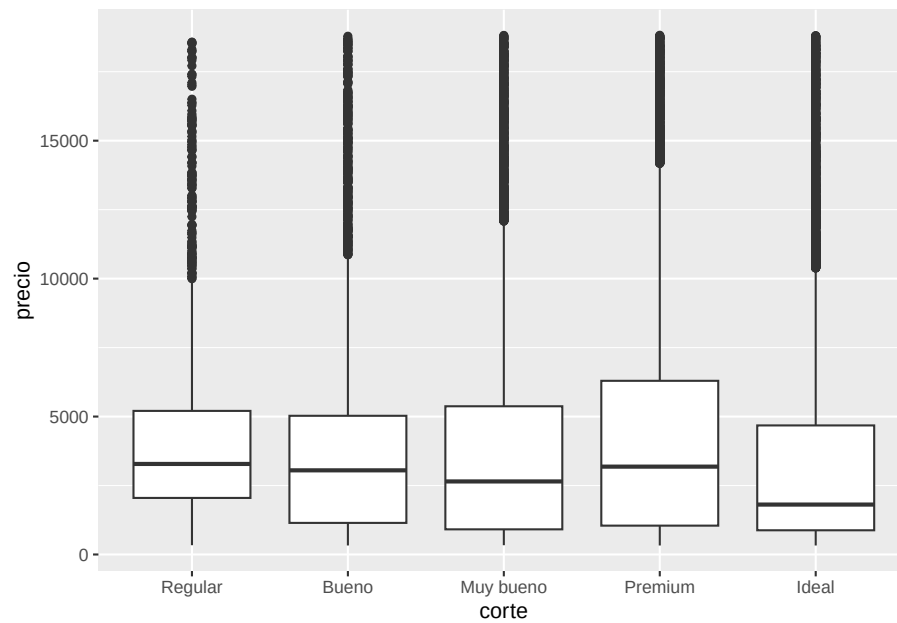
Esta gráfica no sirve de mucho, y vale la pena entender por qué: las categorías tienen tamaños muy distintos, así que las curvas altas son simplemente las categorías con más diamantes. Para comparar **formas** hay que poner las curvas en la misma escala, con la densidad en lugar del conteo:

```
ggplot(diamantes, aes(x = precio, y = after_stat(density))) +  
  geom_freqpoly(aes(color = corte), binwidth = 500)
```



Aun así, cinco curvas encimadas se leen mal. El diagrama de caja dice lo mismo de forma mucho más compacta:

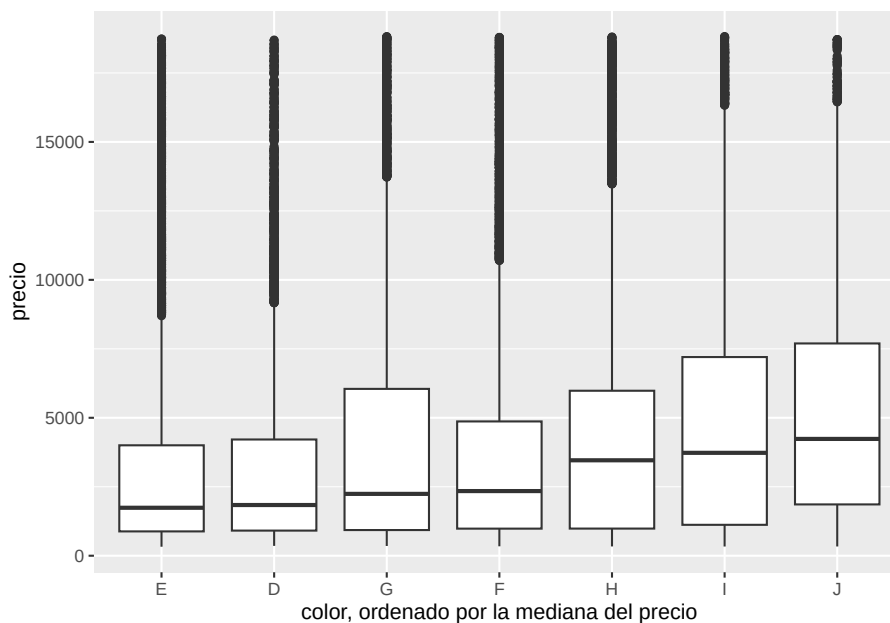
```
ggplot(diamantes, aes(x = corte, y = precio)) +  
  geom_boxplot()
```



Y aquí sale el resultado que sorprende a todo el mundo: **los diamantes de mejor corte son, en promedio, los más baratos**. No es un error de los datos. Es una pregunta nueva, y la respuesta está en el quilate, que es la variable que de verdad manda en el precio.

Cuando la variable categórica no tiene un orden natural, conviene ordenarla por la mediana antes de graficar, porque así la gráfica se lee de un vistazo:

```
ggplot(diamantes, aes(x = reorder(color, precio, FUN = median), y = precio)) +
  geom_boxplot() +
  labs(x = "color, ordenado por la mediana del precio")
```



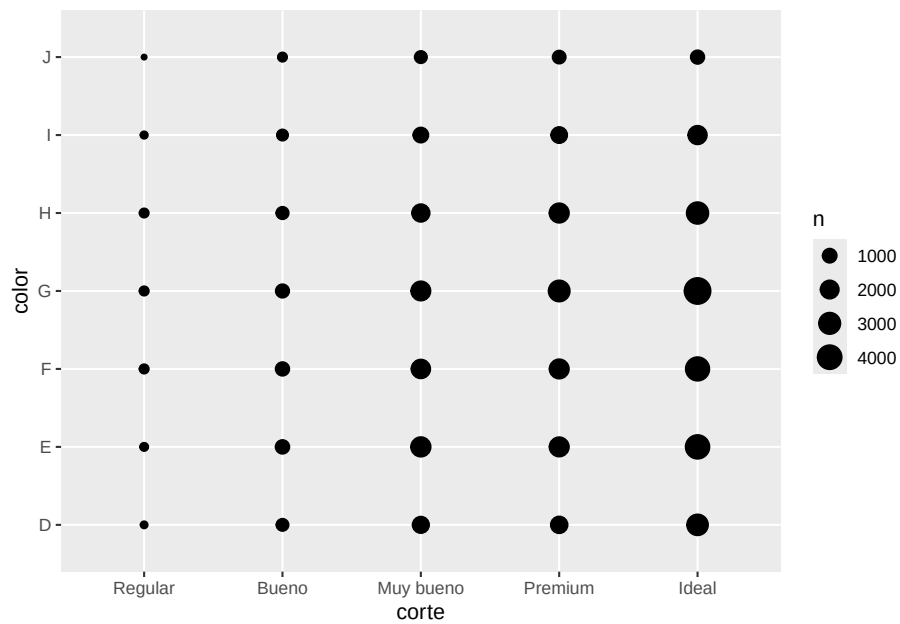
`geom_boxplot()` y `reorder()`: comparar distribuciones entre grupos.

- **La caja** va del primer al tercer cuartil, y la línea de adentro es la **mediana**.
- **Los bigotes** llegan hasta la observación más lejana que esté a menos de 1.5 rangos intercuartiles de la caja; los puntos sueltos que quedan fuera son los valores extremos.
- `reorder(categoria, valor, FUN = median)`: reordena los niveles de la variable categórica según un resumen del valor, casi siempre la mediana.

15.8 Dos variables categóricas

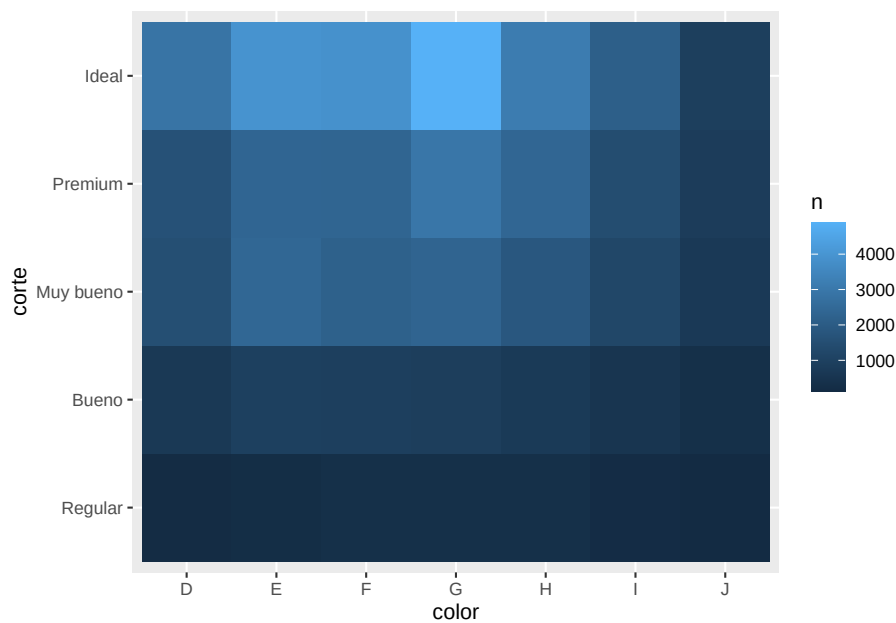
Con dos categóricas hay que contar las combinaciones. `geom_count()` dibuja un punto cuyo tamaño es el conteo:

```
ggplot(diamantes, aes(x = corte, y = color)) +  
  geom_count()
```



La alternativa es contar primero y colorear después. Esta versión suele leerse mejor cuando hay muchas combinaciones:

```
diamantes |>  
  count(color, corte) |>  
  ggplot(aes(x = color, y = corte)) +  
  geom_tile(aes(fill = n))
```

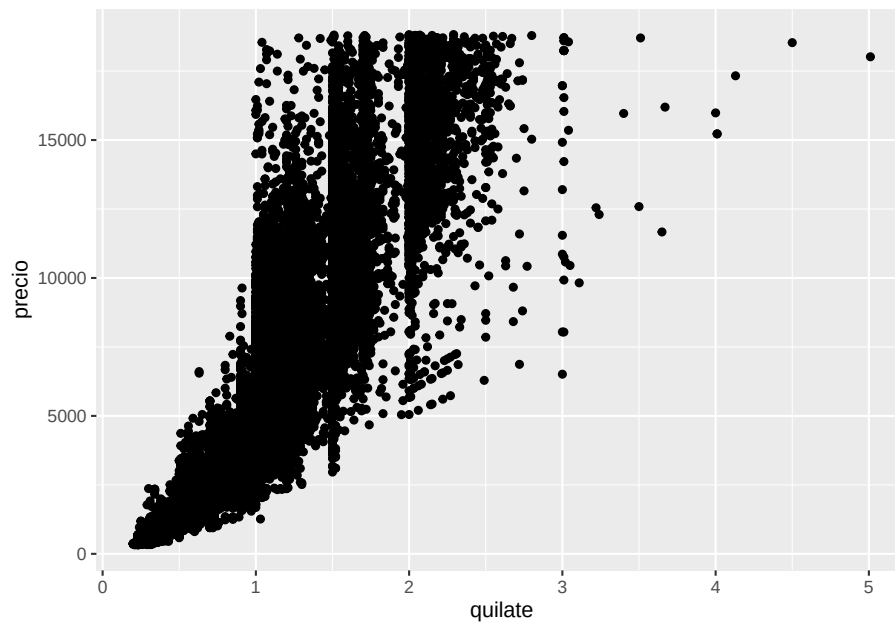



El ojo humano compara **tamaños** mejor que **colores**. Si la gráfica es para decidir algo, `geom_count()` o una tabla de `count()` suelen ser más honestas que un mapa de calor bonito.

15.9 Dos variables continuas

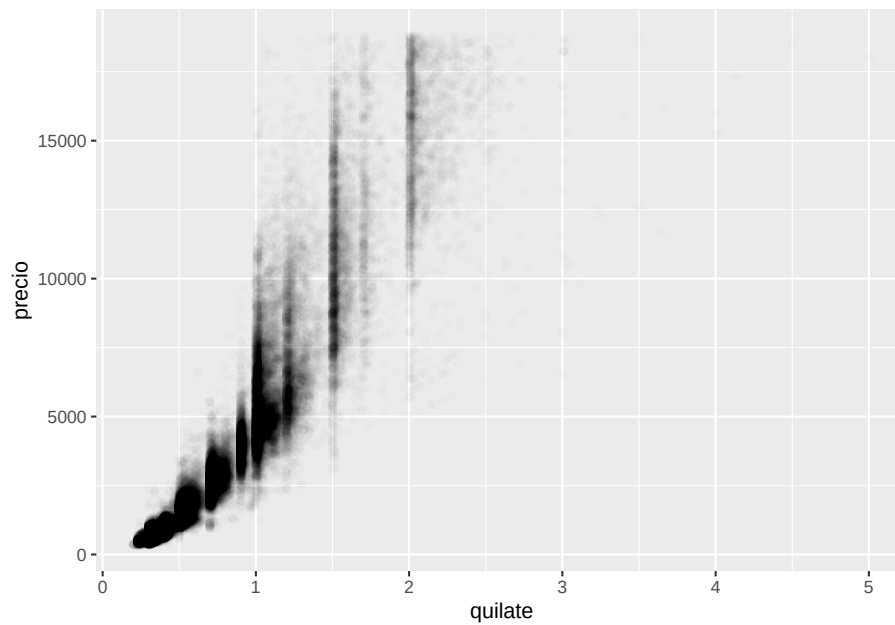
El diagrama de dispersión es la herramienta obvia, y con muchas observaciones es también la que más engaña, porque los puntos se apilan unos encima de otros:

```
ggplot(diamantes, aes(x = quilate, y = precio)) +  
  geom_point()
```



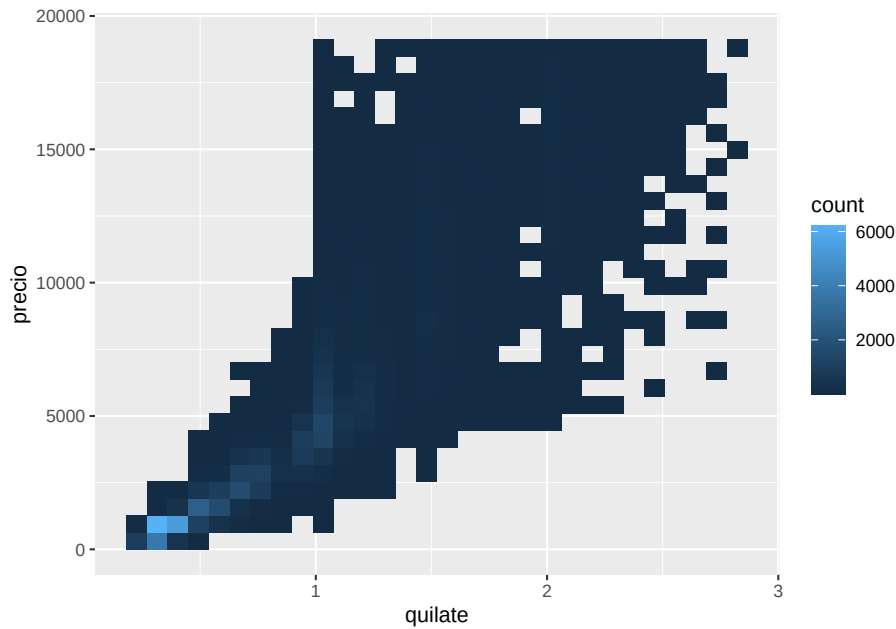
Hay tres maneras de arreglarlo. La primera es hacer los puntos transparentes, para que la densidad se vea como una mancha más oscura:

```
ggplot(diamantes, aes(x = quilate, y = precio)) +  
  geom_point(alpha = 1 / 100)
```



La segunda es dividir el plano en celdas y contar cuántos puntos cae en cada una, igual que un histograma pero en dos dimensiones:

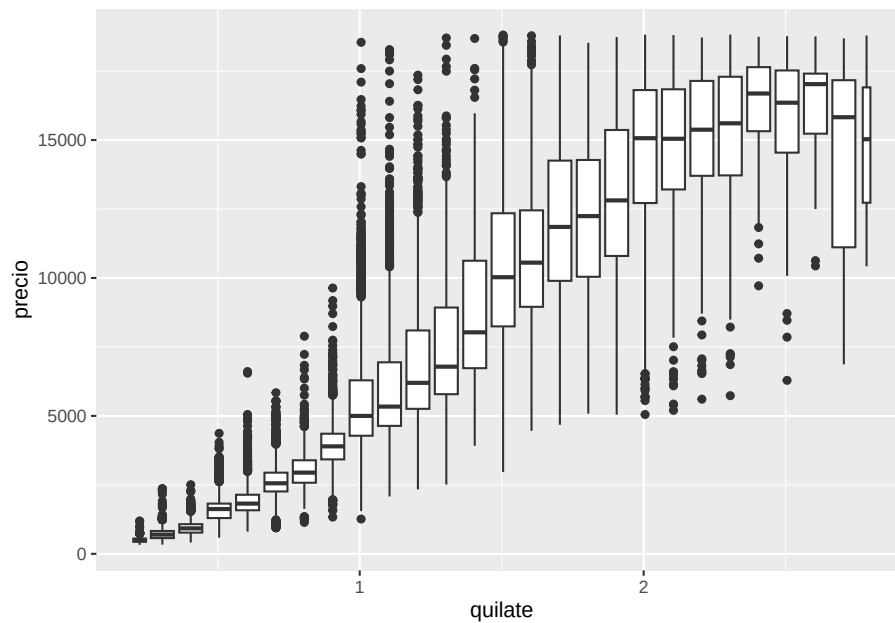
```
ggplot(pequenos, aes(x = quilate, y = precio)) +  
  geom_bin2d()
```



`geom_hex()` hace lo mismo con celdas hexagonales y muchos lo prefieren, pero necesita el paquete `hexbin` instalado. `geom_bin2d()` viene con `ggplot2` y no requiere nada más.

La tercera es convertir una de las dos continuas en categórica con `cut_width()`, y usar diagramas de caja:

```
ggplot(pequenos, aes(x = quilate, y = precio)) +  
  geom_boxplot(aes(group = cut_width(quilate, 0.1)))
```



Esta última tiene una ventaja sobre las otras dos: además de la tendencia, enseña cómo cambia la **dispersión** del precio a medida que crece el quilate.

15.10 Patrones y modelos

Un patrón es una pista de que dos variables están relacionadas. Cuando encuentres uno, hazte estas preguntas antes de creértelo:

- ¿podría ser coincidencia, dado el número de observaciones?
- ¿qué tan fuerte es la relación?
- ¿qué otras variables podrían estar detrás?
- ¿cambia el patrón si miras subgrupos por separado?

La covariación es útil porque **reduce la incertidumbre**: si sabes el quilate, sabes bastante sobre el precio. Un **modelo** es la herramienta que separa el patrón del resto. Si ajustas el precio en función del quilate y luego miras lo que sobra (los residuales), estás mirando el precio ya limpio del efecto del tamaño, y ahí es donde por fin se ve que el corte **Ideal** sí vale más.

Ese es el tema del capítulo de modelos con `modelr`, más adelante en el libro. Por ahora basta con quedarse con la idea: **el EDA hace las preguntas, el modelo contesta la que quedó en pie**.

15.11 Ejercicios del libro R4DS

Hacer los ejercicios de las siguientes secciones del libro en español:

- sección 7.3.4 (variación),
 - sección 7.4.1 (valores faltantes),
 - sección 7.5.1.1 (covariación: categórica y continua),
 - sección 7.5.2.1 (dos variables categóricas),
 - sección 7.5.3.1 (dos variables continuas).
-

15.12 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que la gráfica o la tabla se vean) y una **explicación en palabras**. En un capítulo de exploración la explicación es lo más importante: una gráfica sin lectura no contesta nada. Los ejercicios usan `millas` y `países` del paquete `datos`, no `diamantes`, que es con lo que se enseña aquí. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_15_exploratorio.Rmd`.

15.12.1 Ejercicio 1. Variación de una variable categórica

Datos: `millas`. Cuenta los vehículos por `clase` con `count()` y haz la gráfica equivalente con `geom_bar()`.

Verificación: la tabla debe tener 7 filas, la suma de los conteos debe dar 234, y `suv` debe ser la clase más común.

En palabras: ¿por qué la tabla y la gráfica no son redundantes? Di algo que se vea mejor en una y algo que se vea mejor en la otra.

```
# Escribe tu código aquí
```

15.12.2 Ejercicio 2. El `binwidth` cambia la historia

Datos: `países`, solo el año 2007. Haz tres histogramas de `esperanza_de_vida` con `binwidth` de 10, de 2 y de 0.5.

Verificación: los tres histogramas tienen que sumar 142 países.

En palabras: ¿cuál de los tres enseña que la distribución tiene **dos grupos**? ¿Qué esconde el de intervalos anchos y qué inventa el de intervalos muy finos?

```
# Escribe tu código aquí
```

15.12.3 Ejercicio 3. Valores inusuales sin borrarlos

Datos: `países` de 2007, variable `pib_per_capita`. Haz un histograma, acércate al eje vertical con `coord_cartesian()` hasta ver las barras de la cola derecha, y después saca en una tabla los países que están en esa cola.

Verificación: tu tabla de países extremos debe tener menos de diez filas y todas deben ser del año 2007.

En palabras: ¿son errores o son datos verdaderos? Justifica mirando cuáles países son. ¿Qué le pasaría a la media si los borraras, y por qué la mediana casi no se movería?

```
# Escribe tu código aquí
```

15.12.4 Ejercicio 4. Sustituir por NA, no borrar la fila

Datos: `millas`. Con `if_else()` y `between()`, crea `millas2` donde el valor de `autopista` se sustituya por `NA_real_` cuando esté fuera del intervalo de 12 a 40 millas por galón.

Verificación: `millas2` debe seguir teniendo 234 filas, y `sum(is.na(...))` debe decirte cuántos valores marcaste.

En palabras: compara el promedio de `autopista` con `na.rm = TRUE` y sin él. ¿Por qué es mejor esto que haber usado `filter()` para botar las filas?

```
# Escribe tu código aquí
```

15.12.5 Ejercicio 5. Covariación: categórica y continua

Datos: `millas`. Haz un diagrama de caja de `autopista` según `clase`, con las clases ordenadas por la mediana usando `reorder()`.

Verificación: `pickup` debe quedar en el extremo de las medianas más bajas.

En palabras: además de la mediana, ¿qué clase tiene la caja más **ancha**, y qué significa eso? Una mediana alta con mucha dispersión, ¿es mejor o peor noticia que una mediana un poco más baja pero consistente?

```
# Escribe tu código aquí
```

15.12.6 Ejercicio 6. Dos variables categóricas

Datos: `millas`, variables `clase` y `cilindros` (trátala como categórica). Cuenta las combinaciones con `count()` y dibújalas de las dos maneras del capítulo: con `geom_count()` y con `geom_tile()`.

Verificación: `cilindros` tiene 4 valores distintos, y no todas las combinaciones de `clase` y `cilindros` existen.

En palabras: nombra dos combinaciones que **no aparecen** en los datos y explica por qué tiene sentido que no existan.

Escribe tu código aquí

15.12.7 Ejercicio 7. Dos variables continuas

Datos: millas, cilindrada contra autopista. Haz las tres versiones del capítulo: puntos con `alpha`, `geom_bin2d()`, y diagramas de caja agrupando con `cut_width(cilindrada, 1)`.

Verificación: las tres tienen que enseñar la misma tendencia general.

En palabras: ¿cuál de las tres enseña mejor la **dispersión** de autopista para un mismo tamaño de motor? ¿Y cuál usarías para una presentación de cinco minutos?

Escribe tu código aquí

15.12.8 Ejercicio 8 (BONO). El patrón que se esconde en los subgrupos

Sigue con cilindrada y autopista en millas, pero ahora colorea los puntos por clase o sepáralos con `facet_wrap(~ clase)`.

Verificación: debe verse que la relación no es igual en todas las clases.

En palabras: ¿hay alguna clase que se salga de la tendencia general? Esta es la misma situación del corte **Ideal** en **diamantes**: una tercera variable estaba escondiendo el patrón. Explica cuál es aquí esa tercera variable.

Escribe tu código aquí

Chapter 16

Pipes

Fecha de la última revisión

[1] "2026-09-06"

```
library(tidyverse)
library(magrittr) # El paquete de Pipe, se activa usando tidyverse
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/pipes.html>

Español: <https://es.r4ds.hadley.nz/>

16.1 Temas: El paquete “magrittr” y sus funciones

El operador pipe `%>%` fue creado por **Stefan Milton Bache** en el paquete **magrittr** (2014); el nombre juega con el cuadro de Magritte «*Ceci n'est pas une pipe*». Fue tan popular que en 2021 R incorporó su propio pipe nativo `|>`. La idea: leer el código de izquierda a derecha, como una receta de pasos.

- Qué problema resuelve el pipe
- Cómo funciona `%>%`: el resultado de la izquierda entra por la derecha
- El marcador de posición `.`
- El pipe nativo `|>` y en qué se diferencia de `%>%`
- Los pipes especiales de magrittr: `%%`, `%<>%`, `%T>%`

- `%in%`, que no es un pipe
 - Cuándo **no** conviene usar un pipe
-

16.2 ¿Qué problema resuelve el pipe?

Supón que quieres hacer cuatro cosas seguidas con una tabla: filtrarla, agruparla, resumirla y ordenar el resultado. Sin pipe hay dos maneras de escribirlo, y las dos son incómodas.

La primera es **anidar** las funciones, una dentro de otra: `arrange(summarise(group_by(filter(tabla, ...), ...), ...), ...)`. Se lee de adentro hacia afuera, al revés del orden en que ocurren las cosas, y basta un paréntesis mal puesto para perder media hora.

La segunda es guardar **pasos intermedios**: `tabla1 <- filter(...)`, `tabla2 <- group_by(tabla1, ...)`, y así. Se lee mejor, pero acaba llenando la memoria de objetos que no interesan a nadie, con nombres como `tabla2` que en dos semanas no significan nada.

El pipe resuelve las dos cosas a la vez: **pone los pasos en el orden en que ocurren y no deja basura en el camino**. El código empieza a leerse como una receta: toma la tabla, luego filtra, luego agrupa, luego resume.

Los operadores que vamos a evaluar son los siguientes. Los cinco primeros sí son pipes; el último no lo es, aunque se escriba igual:

- `%>%` “Pipe”: pasa el resultado de la izquierda como primer argumento de la derecha. Es el que se usa el 95 % de las veces.
- `|>` “Pipe nativo”: lo mismo, pero viene con R y no necesita ningún paquete.
- `%!>%` “Eager pipe”, pipe impaciente: igual que `%>%`, pero calcula cada paso en el momento en vez de esperar. Solo importa cuando los pasos tienen efectos secundarios.
- `$$` “Exposition pipe”, pipe de exposición: expone las columnas de la tabla para poder nombrarlas sueltas, como en `cor(x, y)`.
- `%<>%` “Assignment pipe”, pipe de asignación: pasa el objeto **y guarda el resultado encima del original**. Peligroso, ver más abajo.
- `%T>%` “Tee pipe”, pipe en T: hace un paso de lado (dibujar, imprimir, guardar) y sigue pasando el objeto que entró, no el resultado.

Y el que **no** es un pipe:

- `%in%` **no es un pipe**. Es el operador de **pertenencia**: contesta “¿este valor está en esta lista?” y devuelve `TRUE` o `FALSE`. Está aquí porque se escribe con signos de porcentaje a los lados, igual que los pipes, y por eso se confunde. Tiene su propia sección más abajo.

En R, **cualquier** operador escrito entre signos de porcentaje (`%algo%`) es un operador definido por alguien, no necesariamente un pipe. Ya conoces otros dos que no tienen nada que ver con encadenar: `%%` (el residuo de una división) y `%%/` (la división entera), del capítulo de fechas y horas. Los signos de porcentaje son solo la manera que tiene R de escribir operadores propios.

Los pipes de magrittr: encadenar operaciones.

- `%>%` / `|>`: pasa el resultado de la izquierda como primer argumento de la derecha.
- `$$`: expone las columnas para usarlas por nombre (p. ej. `cor(x, y)`).
- `%<>%`: aplica y **reasigna** al objeto original.
- `%T>%`: ejecuta un paso “lateral” (como `plot`) y sigue pasando el objeto original.

16.2.1 Cómo funciona, exactamente

La regla es de una línea: **`x %>% f(y)` es lo mismo que `f(x, y)`**. Lo que está a la izquierda del pipe entra como **primer argumento** de la función que está a la derecha.

Eso funciona porque casi todas las funciones del tidyverse fueron diseñadas pensándolo: `filter()`, `mutate()`, `group_by()`, `summarise()` y `arrange()` reciben la tabla como primer argumento. Por eso encadenan tan bien.

¿Y cuando la función **no** recibe el dato en primer lugar? Para eso está el marcador de posición, el punto `.`, que dice “ponlo aquí y no al principio”:

```
# sin marcador: la tabla entra como primer argumento
países %>% filter(año == 2007)

# con marcador: lm() quiere la fórmula primero y los datos después
países %>% lm(esperanza_de_vida ~ pib_per_capita, data = .)
```

Lee el `%>%` en voz alta como “**y luego**”. `datos %>% filter(...)` `%>% summarise(...)` se lee “toma los datos, y luego filtra, y luego resume”. Si la frase no tiene sentido en voz alta, el código tampoco lo va a tener.

16.2.2 `%>%` y `|>`: cuál usar

Desde la versión 4.1, R trae su **propio** pipe, `|>`, que no necesita ningún paquete. Para el 95 % de lo que hacemos en el curso los dos son intercambiables. Las diferencias que sí importan:

- **Necesita un paquete:** `%>%` sí (`magrittr`, que viene con `tidyverse`); el nativo no, es parte de R.
- **Marcador de posición:** `%>%` usa el punto `.` en cualquier posición; el nativo usa el guion bajo `_` y solo en un argumento **con nombre**.

- **Función sin paréntesis:** `x %>% sqrt` se permite; con el nativo hay que escribir siempre `sqrt()`.
- **Pipes especiales** (`%$%`, `%<>%`, `%T>%`): existen solo del lado de `magrittr`.

En la práctica: usa `|>` cuando puedas, porque es parte de R y siempre va a estar ahí; usa `%>%` cuando necesites el punto o alguno de los pipes especiales de este capítulo. En este libro vas a ver los dos, a veces en el mismo bloque, y no es un error.

16.2.3 Un primer ejemplo: con pipe y sin pipe

Los ejemplos que siguen usan los conjuntos en inglés que vienen con R y con `ggplot2` (`diamonds`, `mtcars`, `iris`), no las versiones en español del paquete `datos`. Es a propósito: casi toda la documentación de `magrittr` que vas a encontrar en internet está escrita con estos mismos datos, y conviene que los reconozcas.

Primero, la manera de siempre, sin pipe: se crea una columna nueva escribiendo el nombre de la tabla tres veces en la misma línea.

Función básica para calcular el precio básico de diamantes por quilates

```
head(diamonds, n=3)
```

```
## # A tibble: 3 x 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Ideal    E      SI2     61.5   55   326   3.95   3.98   2.43
## 2  0.21 Premium E      SI1     59.8   61   326   3.89   3.84   2.31
## 3  0.23 Good    E      VS1     56.9   65   327   4.05   4.07   2.31
```

```
diamonds$Precio_q = diamonds$price/diamonds$carat # crear una nueva variable
```

```
head(diamonds, n=3)
```

```
## # A tibble: 3 x 11
##   carat cut      color clarity depth table price      x      y      z Precio_q
##   <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl> <dbl>
## 1  0.23 Ideal    E      SI2     61.5   55   326   3.95   3.98   2.43   1417.
## 2  0.21 Premium E      SI1     59.8   61   326   3.89   3.84   2.31   1552.
## 3  0.23 Good    E      VS1     56.9   65   327   4.05   4.07   2.31   1422.
```

Ahora lo mismo con pipe y `mutate()`. El nombre de la tabla aparece **una sola vez**, y la línea dice lo que hace:

```
diamonds2 <- diamonds %>%
  dplyr::mutate(price_per_carat = price / carat)
```

`dplyr::mutate()` con los dos puntos dobles significa “la función `mutate()` del paquete `dplyr`”. No hace falta si ya activaste `tidyverse`, pero se usa cuando dos paquetes tienen funciones con el mismo nombre y se quiere dejar claro cuáles.

Calcular el promedio de precio por la calidad de los diamantes usando pipes

Desde la versión 4.1, R trae un pipe nativo `|>` que no requiere paquetes. El `%>%` proviene de `magrittr` y añade funciones extra como el marcador de posición `..`

```
diamonds %>%
  group_by(cut) %>%
  summarize(Precio_color= mean(price))
```

```
## # A tibble: 5 x 2
##   cut      Precio_color
##   <ord>          <dbl>
## 1 Fair           4359.
## 2 Good           3929.
## 3 Very Good      3982.
## 4 Premium        4584.
## 5 Ideal          3458.
```

Calcular el promedio de precio por color de los diamantes usando pipes

16.3 El ejemplo clásico de magrittr

Los tres bloques que siguen hacen exactamente lo mismo con `mtcars` y están aquí para comparar. Vale la pena leerlos despacio, porque es el ejemplo con el que `magrittr` se presentó al mundo.

Un ejemplo básico del pipe

```
car_data <-
  mtcars %>%
  subset(hp > 100) %>%
  aggregate(. ~ cyl, data = ., FUN = . %>% mean %>% round(2)) %>%
  transform(kpl = mpg %>% multiply_by(0.4251)) %>%
  print
```

```
##   cyl  mpg  disp    hp drat   wt  qsec    vs  am gear carb      kpl
## 1   4 25.90 108.05 111.00 3.94 2.15 17.75 1.00 1.00 4.50 2.00 11.010090
## 2   6 19.74 183.31 122.29 3.59 3.12 17.98 0.57 0.43 3.86 3.43  8.391474
## 3   8 15.10 353.10 209.21 3.23 4.00 16.77 0.00 0.14 3.29 3.50  6.419010
```

Fíjate en dos cosas. Primero, `data = .: aggregate()` no recibe los datos en primer lugar, así que hace falta el marcador de posición. Segundo, `FUN = . %>% mean %>% round(2)`: una cadena de pipes que empieza con un punto no es un cálculo, es una **función anónima**, una receta guardada para aplicarla después a lo que llegue.

Ahora la misma cuenta escrita a la antigua, con las funciones anidadas:

Un ejemplo sin usar el pipe para calcular la misma información

```
car_data <-
  transform(aggregate(. ~ cyl,
                      data = subset(mtcars, hp > 100),
                      FUN = function(x) round(mean(x), 2)),
            kpl = mpg*0.4251)

car_data
```

```
##   cyl  mpg  disp    hp  drat    wt  qsec    vs  am gear carb      kpl
## 1    4 25.90 108.05 111.00 3.94  2.15 17.75  1.00 1.00 4.50 2.00 11.010090
## 2    6 19.74 183.31 122.29 3.59  3.12 17.98  0.57 0.43 3.86 3.43  8.391474
## 3    8 15.10 353.10 209.21 3.23  4.00 16.77  0.00 0.14 3.29 3.50  6.419010
```

Las dos dan el mismo resultado. La diferencia está en cómo se leen: la primera se lee de arriba abajo en el orden en que ocurren las cosas; la segunda hay que leerla de adentro hacia afuera, empezando por `subset()`, que está enterrado en el centro.

Y una tercera versión, con las funciones de `dplyr` en vez de las de base R:

```
car_data <-
  mtcars %>%
  subset(hp > 100) %>%
  group_by(cyl) %>%
  summarise(across(everything(), list(mean))) %>%
  transform(kpl = mpg_1 %>% multiply_by(0.4251)) %>%
  print
```

```
##   cyl  mpg_1  disp_1    hp_1  drat_1    wt_1  qsec_1    vs_1    am_1
## 1    4 25.90000 108.0500 111.0000 3.940000  2.146500 17.75000  1.0000000 1.0000000
## 2    6 19.74286 183.3143 122.2857 3.585714  3.117143 17.97714  0.5714286 0.4285714
## 3    8 15.10000 353.1000 209.2143 3.229286  3.999214 16.77214  0.0000000 0.1428571
##   gear_1  carb_1      kpl
## 1 4.500000 2.000000 11.010090
## 2 3.857143 3.428571  8.392689
## 3 3.285714 3.500000  6.419010
```

16.4 Alternativas a los pipes

El pipe no es obligatorio y no siempre es la mejor opción. Las dos alternativas son:

- **Pasos intermedios.** Guardar cada paso en un objeto con nombre (`vuelos_limpios`, `resumen_por_mes`). Ocupa más líneas, pero deja el resultado de cada paso disponible para revisarlo, y eso es exactamente lo que hace falta cuando algo sale mal en el medio de una cadena larga.
- **Sobre escribir el original.** Reasignar sobre el mismo nombre (`tabla <- filter(tabla, ...)`, y otra vez, y otra vez). Ahorra nombres, pero es peligroso: si corres una línea dos veces por accidente, ya no sabes en qué estado quedó la tabla, y hay que volver a leer los datos desde el principio.

Cuándo **no** usar el pipe: cuando la cadena pasa de unos diez pasos (parte en dos con un objeto intermedio bien nombrado); cuando hay varias entradas o varias salidas, porque el pipe supone un solo objeto que viaja; y cuando estás depurando, porque un objeto intermedio con nombre se puede mirar y un paso en el medio de una cadena no.

16.5 %\$%: el pipe de exposición

Algunas funciones no reciben una tabla: reciben **vectores sueltos**. `cor()` es el ejemplo típico: quiere `cor(x, y)`, dos columnas, no un data frame. Con el pipe normal eso no funciona, porque `cor()` recibiría la tabla entera.

El pipe de exposición `%$%` resuelve justo eso: **expone** las columnas de la tabla para que se puedan nombrar directamente, sin escribir `tabla$columna` cada vez.

`%$%` (`magrittr`): expone las columnas de la tabla como si fueran objetos sueltos.

- `tabla %$% cor(a, b)` equivale a `cor(tabla$a, tabla$b)`.
- **Para qué sirve:** usar funciones de base R que esperan vectores (`cor()`, `t.test()`, `table()`, `ts.plot()`) dentro de una cadena de pipes.
- **Ojo:** solo funciona con `%>%`; el pipe nativo `|>` no tiene equivalente.

En el bloque siguiente, las líneas comentadas son las que **no** funcionan, y están ahí a propósito para que veas el contraste: con el pipe nativo `cor()` recibiría la tabla completa y fallaría.

```
library(magrittr)

#iris %$%
# subset(Sepal.Length > mean(Sepal.Length)) |>
# cor(Sepal.Length, Sepal.Width)
```

```
iris %>%
  subset(Sepal.Length > mean(Sepal.Length)) %$%
  cor(Sepal.Length, Sepal.Width)
```

```
## [1] 0.3361992
```

```
iris %>%
  subset(Sepal.Length > mean(Sepal.Length)) |>
  summarize(corr= cor(Sepal.Length, Sepal.Width))
```

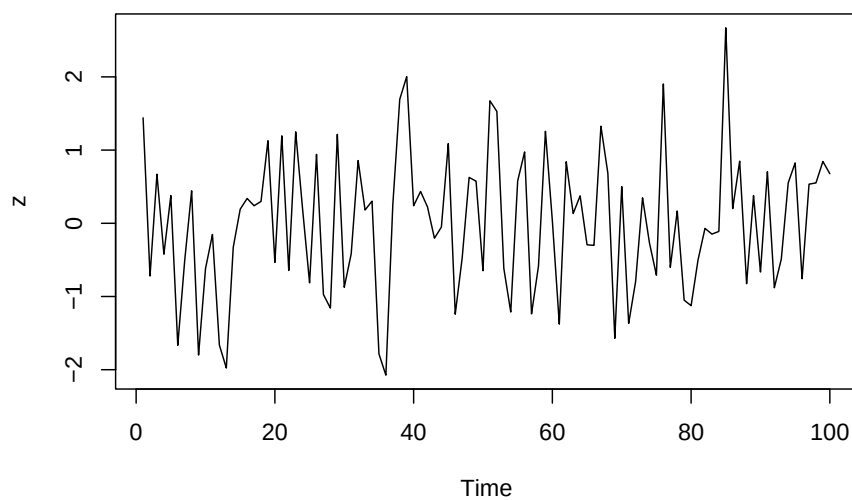
```
##           corr
## 1 0.3361992
```

La segunda parte del bloque enseña la alternativa tidyverse: si en vez de `cor()` usas `summarize()`, ya no hace falta exponer nada, porque `summarize()` sí sabe buscar las columnas dentro de la tabla. Casi siempre hay una salida tidyverse que evita `%$%`.

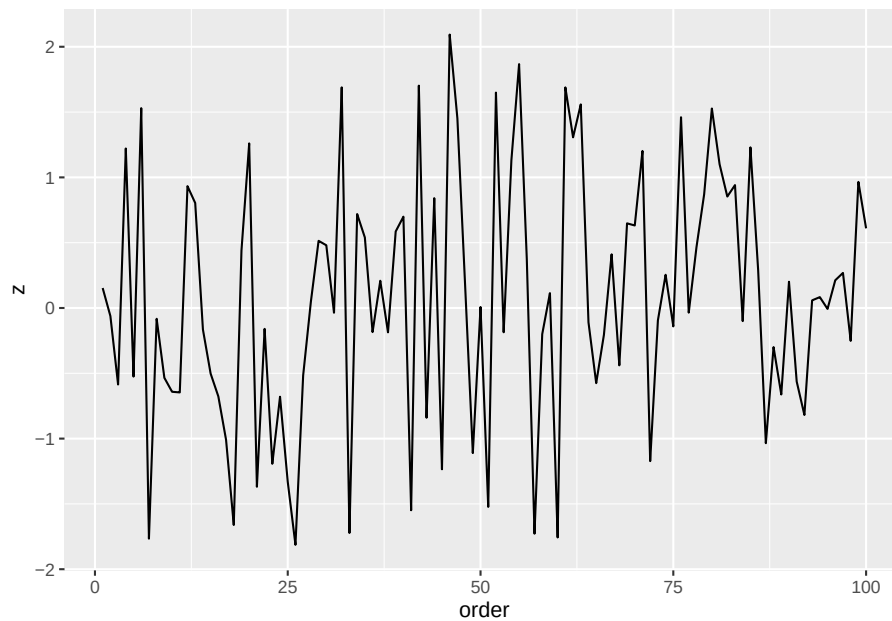
16.5.1 Otro ejemplo de Pipe: `%$%`

`ts.plot()` es una función de base R que dibuja una serie de tiempo y espera un vector, no una tabla: otro caso donde `%$%` es lo que hace falta. Después del ejemplo viene la versión tidyverse de la misma gráfica, con `ggplot()`, donde todo se resuelve dentro de la tabla.

```
data.frame(z = rnorm(100)) %$%
  ts.plot(z)
```

```
#data.frame(z = rnorm(100)) |>  
# ts.plot(z)  
  
#Para hacerlo la misma grafico con Tidyverse  
z1=data.frame(z = rnorm(100))  
library(data.table)  
z1=data.table(z1)  
z1$order= c(1:100)  
z1|>  
  ggplot(aes(order, z))+  
  geom_line()
```



16.6 %<>%: el pipe de asignación

Este pipe hace dos cosas de un golpe: pasa el objeto a la función **y guarda el resultado encima del objeto original**. Es decir, `x %<>% sqrt` es lo mismo que escribir `x <- x %>% sqrt`.

`lhs %<>% rhs`

- **lhs**: el objeto, que sirve a la vez de **valor inicial** y de **destino**.
- **rhs**: la llamada a la función, con la sintaxis de `magrittr`.

%<>% modifica el objeto original y no hay manera de deshacerlo. En el Ejemplo 1 de abajo, la columna `Sepal.Length` de `iris` queda cambiada de verdad en tu sesión: si vuelves a correr el bloque, le sacarás la raíz cuadrada otra vez, y luego otra, sin ningún aviso. Por eso casi nunca se usa en un análisis de verdad: rompe la regla de que correr el mismo script dos veces dé el mismo resultado. Si necesitas el valor transformado, guárdalo en una **columna nueva** con `mutate()`.

```
# Example 1
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           5.1         3.5          1.4         0.2   setosa
```

```
## 2      4.9      3.0      1.4      0.2 setosa
## 3      4.7      3.2      1.3      0.2 setosa
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

```
head(iris$Sepal.Length %<>% sqrt)
```

```
## [1] 2.258318 2.213594 2.167948 2.144761 2.236068 2.323790
```

```
# Example 2
```

```
x <- rnorm(100)
```

```
head(x, n=10)
```

```
## [1] 1.361612477 -0.189292059 0.462969745 -0.007385678 2.689108440
```

```
## [6] 1.459546809 -1.653300235 -0.203078277 0.618497216 0.597135379
```

```
x %<>%
  abs %>%
  sort
```

```
head(x, 20)
```

```
## [1] 0.007385678 0.029096523 0.030999873 0.035387591 0.040816992 0.054441440
```

```
## [7] 0.069795203 0.081581732 0.087776321 0.110958305 0.119075263 0.120358116
```

```
## [13] 0.120542457 0.131861167 0.133266112 0.145019780 0.145731942 0.150293091
```

```
## [19] 0.155538095 0.164714712
```

16.7 %in%: esto no es un pipe

%in% aparece en la lista del principio porque se escribe con signos de porcentaje a los lados, igual que los pipes, pero no tiene nada que ver con ellos. No pasa nada de la izquierda a la derecha: es un operador de **pertenencia**, que contesta la pregunta “¿este valor está en esta lista?” y devuelve TRUE o FALSE.

- Sirve para filtrar por **varios valores específicos** de una variable a la vez.

%in% (base R): pertenencia a un conjunto.

- **x %in% c("a", "b")** devuelve TRUE para cada elemento de x que sea "a" o "b".
- **Para qué sirve:** sustituye a una cadena larga de |. En vez de `filter(d, sp == "a" | sp == "b" | sp == "c")`, se escribe `filter(d, sp %in% c("a", "b", "c"))`.
- **Ojo con los NA:** `NA %in% c(1, 2)` devuelve FALSE, no NA, que es lo contrario de lo que hacen casi todas las comparaciones en R.

En el ejemplo se filtran dos especies de *Iris*, *Iris setosa* e *Iris virginica*, dejando fuera la tercera.

```
# Example 1
names(iris)
```

```
## [1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

iris |>
  filter(Species %in% c("setosa", "virginica")) |>
  head()
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      2.258318        3.5          1.4          0.2   setosa
## 2      2.213594        3.0          1.4          0.2   setosa
## 3      2.167948        3.2          1.3          0.2   setosa
## 4      2.144761        3.1          1.5          0.2   setosa
## 5      2.236068        3.6          1.4          0.2   setosa
## 6      2.323790        3.9          1.7          0.4   setosa
```

16.8 %T>%: el pipe en T

El pipe en T, %T>%, sirve cuando en medio de una cadena hay una función que **no devuelve nada**, como `plot()`, `print()` o `write_csv()`. Con un pipe normal la cadena se rompe ahí, porque el paso siguiente recibiría el NULL que esa función devuelve.

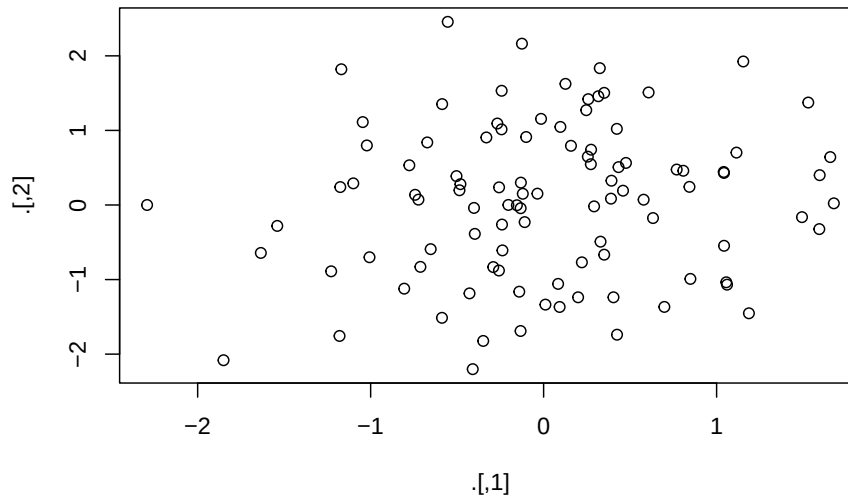
%T>% hace el paso “de lado”, como el palo corto de una T: ejecuta la función y sigue pasando **el objeto que entró**, no el resultado.

%T>% (magrittr): paso lateral dentro de una cadena.

- **x %T>% plot() %>% colSums()**: dibuja x y después le calcula las sumas de columna, todo en una cadena.
- **Cuándo se usa**: para dibujar, imprimir o guardar en medio de un análisis, sin cortarlo en dos.
- **Sin él** habría que guardar el objeto en una variable, dibujarlo, y volver a usarlo.

Referencia: <https://stackoverflow.com/questions/61196304/magrittr-tee-pipe-t-equivalent>

```
rnorm(200) %>%
matrix(ncol = 2) %T>%
plot %>% # plot usually does not return anything.
colSums
```



```
## [1] 1.395464 5.606604
```

16.9 %!>%: el pipe impaciente

Aparece en la lista del principio y casi nunca hace falta, pero conviene saber que existe. %>% es **perezoso**: no calcula un paso hasta que alguien necesita su resultado. %!>%, añadido en magrittr 2.0, es **impaciente**: obliga a calcular cada paso en el momento en que se escribe.

La diferencia solo se nota cuando los pasos tienen **efectos secundarios**, es decir, cuando además de calcular hacen algo más: imprimir un mensaje, escribir un archivo, avanzar un número al azar. Ahí el orden en que ocurren las cosas importa, y %!>% lo garantiza. En un análisis normal, %>% y %!>% dan el mismo resultado.

16.10 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan *millas* y *países* del paquete *datos*, en español,

y no `mtcars`, `iris` ni `diamonds`, que son los del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_16_pipes.Rmd`.

16.10.1 Ejercicio 1. De adentro hacia afuera, a de arriba hacia abajo

Este código anidado funciona pero es ilegible:

```
arrange(summarise(group_by(filter(países, año == 2007), continente),
esperanza_media = mean(esperanza_de_vida)), desc(esperanza_media))
```

Reescribelo con pipes, un paso por línea.

Verificación: tu tabla debe tener 5 filas, una por continente, y el mismo orden que produce la versión anidada.

En palabras: ¿en qué orden hay que leer la versión anidada para entenderla? ¿Y la tuya?

Escribe tu código aquí

16.10.2 Ejercicio 2. El marcador de posición

Datos: `países` del año 2007. Ajusta un modelo lineal de `esperanza_de_vida` en función de `pib_per_capita` dentro de una cadena de pipes, usando `lm()`. `lm()` quiere la fórmula primero, así que vas a necesitar `data = ..`

Verificación: el coeficiente del `pib_per_capita` debe salir **positivo**.

En palabras: ¿qué habría pasado si escribieras `países2007 %>% lm(...)` sin el punto? Explica qué habría recibido `lm()` como primer argumento.

Escribe tu código aquí

16.10.3 Ejercicio 3. `%>%` y `|>` lado a lado

Datos: `millas`. Escribe la **misma** cadena dos veces, una con `%>%` y otra con `|>`: filtra los vehículos de 2008, agrupa por `clase`, calcula el promedio de `autopista` y ordena de mayor a menor. Después intenta escribir `millas %>% nrow` y `millas |> nrow` (sin paréntesis) y mira qué pasa.

Verificación: las dos cadenas deben dar exactamente la misma tabla; una de las dos versiones de `nrow` sin paréntesis da error.

En palabras: ¿cuál de las dos falló y por qué? ¿Cuál de los dos pipes usarías por defecto, y en qué caso cambiarías de opinión?

Escribe tu código aquí

16.10.4 Ejercicio 4. `%%` con una función que quiere vectores

Datos: `millas`. Calcula la correlación entre `cilindrada` y `autopista` de tres maneras: con `cor(millas$cilindrada, millas$autopista)`, con `%%`, y con `summarise()` dentro de una cadena normal. Prueba también qué pasa si usas `%>` en vez de `%%`.

Verificación: las tres maneras que funcionan dan el mismo número, negativo y con valor absoluto mayor que 0.7.

En palabras: ¿qué mensaje da la versión con `%>` y por qué? ¿Cuál de las tres versiones que sí funcionan usarías en un análisis tuyo?

```
# Escribe tu código aquí
```

16.10.5 Ejercicio 5. Por qué `%<>` es peligroso

Haz una copia: `millas_copia <- millas`. Aplícale `millas_copia$autopista %<> sqrt` y mira el resultado. Después **corre la misma línea otra vez** y vuelve a mirar.

Verificación: `millas` sin tocar sigue igual, pero los valores de `millas_copia` cambiaron **las dos veces**.

En palabras: explica por qué esto rompe la reproducibilidad de un script. Escribe debajo la versión con `mutate()` que hace lo mismo sin ese problema.

```
# Escribe tu código aquí
```

16.10.6 Ejercicio 6. `%T>`, el paso de lado

Datos: `millas`. En **una sola cadena**: toma `millas`, quédate con `cilindrada` y `autopista`, dibuja el par con `plot()` en el medio de la cadena usando `%T>`, y termina calculando el promedio de las dos columnas con `colMeans()`. Prueba primero con `%>` en vez de `%T>` para ver el error.

Verificación: con `%T>` sale la gráfica y un vector de dos promedios.

En palabras: ¿qué recibió `colMeans()` cuando usaste `%>`? ¿Por qué?

```
# Escribe tu código aquí
```

16.10.7 Ejercicio 7 (BONO). `%in%` no es un pipe

Datos: `países` del año 2007. Filtra los países de tres continentes de dos maneras: con `|` repetido tres veces, y con `%in%`. Después contesta con código: ¿qué devuelve `NA %in% c(1, 2)` y qué devuelve `NA == 1`?

Verificación: las dos maneras de filtrar dan la misma tabla.

En palabras: `%in%` se escribe como un pipe pero no lo es. Explica en una oración qué hace realmente, y por qué la diferencia entre `NA %in% ...` y `NA == ...` puede cambiar el resultado de un filtro.

```
# Escribe tu código aquí
```


Chapter 17

Tibbles

Fecha de la última revisión

[1] "2026-09-06"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/tibbles.html>

Español: <https://es.r4ds.hadley.nz/10-tibble.html>

17.1 Temas: El paquete Tibble

Los `tibble` son la versión moderna del `data.frame` dentro del tidyverse: hacen menos «magia» automática (no cambian nombres ni tipos de columna) y ofrecen una impresión más legible en la consola.

- Por qué existe el tibble
- Crear tablas: `tibble()`, `tribble()`, `as_tibble()`
- Cómo se imprime un tibble, y cómo pedirle que imprima más
- Las cuatro diferencias con el `data.frame` que de verdad importan
- Sacar una columna: `$`, `[[`, `[` y el marcador `.`
- Cuando hay que volver a un `data.frame`

17.2 ¿Por qué existe el tibble?

El `data.frame` es de 1990 y se diseñó cuando las tablas cabían en una pantalla. Con los años se le fueron acumulando comportamientos que en su momento eran cómodos y que hoy hacen perder tiempo:

- imprime la tabla **entera** en la consola, así que un archivo de 50 000 filas llena la pantalla y hay que subir a buscar el principio;
- hace **coincidencia parcial** de nombres: si le pides `df$a` y la única columna que empieza por “a” es `abc`, te la da sin avisar, y una errata pasa por buena;
- durante muchos años convertía el texto en factores por su cuenta, y arreglaba los nombres de columna sin decirlo;
- guarda **nombres de fila**, que son datos escondidos fuera de las columnas.

El `tibble` hace lo mismo que el `data.frame` pero **sin la magia**: imprime poco, no adivina nombres, no cambia tipos y no guarda nombres de fila. Se queja cuando hay algo raro en vez de arreglarlo en silencio, y eso es exactamente lo que uno quiere de una herramienta de análisis.

Un tibble **es** un data frame: `is.data.frame(mi_tibble)` devuelve `TRUE`. Todo lo que funciona con un data frame funciona con un tibble. Al revés no siempre, y de eso trata la última sección del capítulo.

- `as_tibble()`

17.3 Crear un tibble

`tibble()`, `tribble()`, `as_tibble()`: crear tablas modernas.

- `tibble(col = valores, ...)`: crea un tibble por columnas.
- `tribble(~col1, ~col2, v, v, ...)`: lo crea **fila por fila** (cómodo para tablas pequeñas escritas a mano).
- `as_tibble(df)`: convierte un `data.frame` en tibble.
- `tibble()`

`tibble()` construye la tabla **por columnas**: cada argumento es un vector, y todos tienen que tener el mismo largo (o largo 1, que se repite). A diferencia de `data.frame()`, no toca los nombres, no convierte el texto en factores y no recicla vectores de largo raro.

Crear un tibble con la siguiente información. Una columna que tiene los valores del 1 al 8 que se llama “secuencia”, una segunda columna que tiene los siguientes valores 23,26, 24,26,27, 11,20,21 que se llama edad y una tercera columna con la siguiente información, Jose, Maria, Carol, Moncho, Liz, Maria, Jorge, Miguel, que se llama Nombre

El bloque de abajo construye solo las **seis** primeras filas de las ocho que pide el enunciado, y deja un NA en la última edad. Completarlo es el primer ejercicio del capítulo.

```
library(tidyverse)
#library(datos)

tibble(
  secuencia = c(1:6),
  edad = c(23,26, 24,26,27, NA),
  nombre = c("Jose", "Maria", "Carol", "Moncho", "Liz", "Maria")
)
```

```
## # A tibble: 6 x 3
##   secuencia edad nombre
##   <int> <dbl> <chr>
## 1      1    23 Jose
## 2      2    26 Maria
## 3      3    24 Carol
## 4      4    26 Moncho
## 5      5    27 Liz
## 6      6    NA Maria
```

17.4 Crear un tribble

`tribble()` construye la tabla **fila por fila**, que es como uno la escribiría a mano en una libreta. Los nombres de las columnas van con una virgulilla delante (`~secuencia`), y después vienen los valores en el mismo orden. Es la forma más cómoda de meter en un script una tabla pequeña: los datos de un experimento, una tabla de equivalencias, los códigos de las parcelas.

Alinea los valores en columnas cuando escribas un `tribble()`. No cambia nada para R, pero hace que un dato fuera de sitio salte a la vista.

- `tribble()`

Haga un tribble con las primeras 3 líneas del tibble anterior

```
tribble(
  ~secuencia, ~edad, ~nombre,
  1, 23, "Jose",
  2, 26, "Maria",
  3, 23, "Carol"
)
```

```
## # A tibble: 3 x 3
##   secuencia edad nombre
##   <dbl> <dbl> <chr>
## 1      1      23 Jose
## 2      2      26 Maria
## 3      3      23 Carol
```

17.5 tibble vs. data.frame

Los dos bloques siguientes construyen la misma tabla de 100 filas y la imprimen de las dos maneras. Compara la salida: el tibble enseña **diez filas**, el tipo de cada columna debajo del nombre (`<dtm>`, `<date>`, `<int>`, `<dbl>`, `<chr>`) y una nota de cuántas filas quedaron sin imprimir. El data frame las suelta las cien.

Esa línea con los tipos es la razón principal para preferir el tibble mientras se explora: la mitad de los problemas de un análisis son una columna que uno creía numérica y era texto, y aquí se ve sin tener que pedir nada.

```
tb = tibble(
  a = lubridate::now() + runif(1e2) * 86400,
  b = lubridate::today() + runif(1e2) * 30,
  c = 1:100,
  d = runif(1e2),
  e = sample(letters, 1e2, replace = TRUE)
)
tb

## # A tibble: 100 x 5
##       a              b              c          d e
##   <dtm>          <date>        <int>   <dbl> <chr>
## 1 2026-09-06 20:28:56 2026-09-24      1 0.559 o
## 2 2026-09-07 02:58:10 2026-09-06      2 0.250 o
## 3 2026-09-06 21:51:47 2026-09-08      3 0.0653 d
## 4 2026-09-07 19:38:01 2026-09-07      4 0.573 v
## 5 2026-09-07 03:10:16 2026-09-28      5 0.651 k
## 6 2026-09-07 11:22:06 2026-09-17      6 0.581 p
## 7 2026-09-07 16:00:11 2026-09-30      7 0.0175 x
## 8 2026-09-07 15:00:10 2026-09-17      8 0.0133 f
## 9 2026-09-07 09:14:54 2026-09-20      9 0.661 c
## 10 2026-09-07 15:23:06 2026-09-29     10 0.380 y
## # i 90 more rows

df_tb=as.data.frame(tb)
df_tb

##              a              b              c          d e
```

```

## 1 2026-09-06 20:28:56 2026-09-24 1 0.55895788 o
## 2 2026-09-07 02:58:10 2026-09-06 2 0.24952694 o
## 3 2026-09-06 21:51:47 2026-09-08 3 0.06531608 d
## 4 2026-09-07 19:38:01 2026-09-07 4 0.57306761 v
## 5 2026-09-07 03:10:16 2026-09-28 5 0.65098699 k
## 6 2026-09-07 11:22:06 2026-09-17 6 0.58124937 p
## 7 2026-09-07 16:00:11 2026-09-30 7 0.01748210 x
## 8 2026-09-07 15:00:10 2026-09-17 8 0.01331984 f
## 9 2026-09-07 09:14:54 2026-09-20 9 0.66064715 c
## 10 2026-09-07 15:23:06 2026-09-29 10 0.38021846 y
## 11 2026-09-07 19:32:23 2026-09-28 11 0.55374044 o
## 12 2026-09-07 16:59:57 2026-09-17 12 0.57734132 r
## 13 2026-09-07 17:24:36 2026-09-08 13 0.89874595 s
## 14 2026-09-07 11:44:37 2026-09-19 14 0.02335350 s
## 15 2026-09-07 06:23:46 2026-09-29 15 0.36233644 a
## 16 2026-09-07 08:08:52 2026-09-13 16 0.79219053 c
## 17 2026-09-07 09:26:44 2026-09-13 17 0.79451792 q
## 18 2026-09-06 21:20:12 2026-09-16 18 0.03461087 h
## 19 2026-09-07 01:58:25 2026-09-07 19 0.83758986 y
## 20 2026-09-07 08:13:44 2026-09-17 20 0.08880666 h
## 21 2026-09-07 00:33:55 2026-09-19 21 0.81779321 m
## 22 2026-09-06 22:58:18 2026-10-03 22 0.88725936 k
## 23 2026-09-07 11:52:39 2026-09-07 23 0.26002925 a
## 24 2026-09-06 23:03:21 2026-09-17 24 0.31542900 t
## 25 2026-09-07 05:40:25 2026-09-11 25 0.91124396 m
## 26 2026-09-07 01:00:14 2026-09-20 26 0.69990520 u
## 27 2026-09-07 10:11:22 2026-10-05 27 0.83810166 o
## 28 2026-09-07 07:41:19 2026-09-09 28 0.39910988 q
## 29 2026-09-07 17:36:03 2026-09-29 29 0.99993174 t
## 30 2026-09-07 09:19:04 2026-09-15 30 0.66218230 y
## 31 2026-09-07 04:16:52 2026-09-10 31 0.98046836 c
## 32 2026-09-07 08:26:44 2026-09-18 32 0.13905381 o
## 33 2026-09-07 12:28:38 2026-10-02 33 0.34755612 m
## 34 2026-09-07 01:23:16 2026-09-25 34 0.99072892 t
## 35 2026-09-07 16:36:38 2026-09-28 35 0.01158322 g
## 36 2026-09-07 06:50:26 2026-09-26 36 0.59905566 w
## 37 2026-09-07 04:51:39 2026-09-26 37 0.66253655 m
## 38 2026-09-07 16:11:00 2026-09-14 38 0.81108752 p
## 39 2026-09-06 23:50:22 2026-10-03 39 0.86941914 m
## 40 2026-09-07 09:35:58 2026-09-30 40 0.18531162 v
## 41 2026-09-07 17:24:26 2026-09-10 41 0.93845374 p
## 42 2026-09-06 21:54:35 2026-09-16 42 0.98626153 o
## 43 2026-09-06 21:51:33 2026-09-29 43 0.36978729 g
## 44 2026-09-06 21:39:28 2026-09-17 44 0.40597546 h
## 45 2026-09-06 20:04:03 2026-09-08 45 0.45127090 p
## 46 2026-09-07 12:40:02 2026-09-10 46 0.83761479 d

```

```
## 47 2026-09-07 09:32:15 2026-10-05 47 0.20941751 c
## 48 2026-09-07 05:23:55 2026-10-01 48 0.99204004 y
## 49 2026-09-07 05:41:31 2026-09-07 49 0.50421065 r
## 50 2026-09-06 23:16:12 2026-09-17 50 0.33143023 g
## 51 2026-09-07 18:39:47 2026-09-16 51 0.73076078 w
## 52 2026-09-07 09:35:38 2026-09-12 52 0.46601006 v
## 53 2026-09-07 06:04:48 2026-09-26 53 0.93914242 z
## 54 2026-09-07 10:58:56 2026-10-02 54 0.88173091 s
## 55 2026-09-07 03:57:39 2026-09-30 55 0.34532379 i
## 56 2026-09-06 22:00:34 2026-09-24 56 0.58456342 l
## 57 2026-09-06 20:02:39 2026-10-02 57 0.88058255 v
## 58 2026-09-06 21:55:24 2026-09-07 58 0.22127013 v
## 59 2026-09-07 11:36:01 2026-09-24 59 0.89288290 o
## 60 2026-09-07 03:43:35 2026-09-12 60 0.02081485 y
## 61 2026-09-07 00:21:21 2026-09-22 61 0.27143203 n
## 62 2026-09-07 05:18:48 2026-09-13 62 0.95400736 c
## 63 2026-09-07 03:25:35 2026-09-14 63 0.86419811 j
## 64 2026-09-07 04:20:02 2026-09-15 64 0.25909042 i
## 65 2026-09-07 12:18:04 2026-09-16 65 0.71488890 t
## 66 2026-09-07 12:49:58 2026-09-15 66 0.14559841 m
## 67 2026-09-07 09:56:51 2026-09-20 67 0.33884110 a
## 68 2026-09-07 13:22:40 2026-09-19 68 0.85914380 c
## 69 2026-09-07 10:26:09 2026-09-27 69 0.78041364 o
## 70 2026-09-07 05:03:16 2026-09-26 70 0.83340269 v
## 71 2026-09-07 15:53:04 2026-09-27 71 0.46138001 k
## 72 2026-09-07 15:10:15 2026-09-28 72 0.95366063 x
## 73 2026-09-07 01:48:03 2026-10-03 73 0.58594067 i
## 74 2026-09-07 03:57:51 2026-09-22 74 0.07427914 i
## 75 2026-09-06 20:17:13 2026-10-04 75 0.41196391 p
## 76 2026-09-07 17:25:15 2026-09-16 76 0.64245513 l
## 77 2026-09-07 05:31:53 2026-09-19 77 0.88572561 a
## 78 2026-09-07 00:03:20 2026-10-05 78 0.27894255 z
## 79 2026-09-07 05:57:28 2026-09-26 79 0.11545121 p
## 80 2026-09-06 21:01:06 2026-10-02 80 0.27069348 w
## 81 2026-09-07 10:46:50 2026-09-07 81 0.09039557 m
## 82 2026-09-07 09:59:14 2026-10-05 82 0.84375956 u
## 83 2026-09-07 01:26:30 2026-09-18 83 0.44452948 t
## 84 2026-09-06 23:46:40 2026-09-14 84 0.17054338 a
## 85 2026-09-07 05:47:36 2026-10-05 85 0.94064577 n
## 86 2026-09-07 08:15:15 2026-09-11 86 0.55504992 s
## 87 2026-09-06 23:28:28 2026-10-04 87 0.74138904 o
## 88 2026-09-06 23:10:58 2026-09-17 88 0.69146950 m
## 89 2026-09-07 11:18:55 2026-09-28 89 0.95055655 v
## 90 2026-09-07 15:14:34 2026-10-02 90 0.50241356 h
## 91 2026-09-07 15:34:11 2026-10-01 91 0.94041471 l
## 92 2026-09-07 09:38:06 2026-09-17 92 0.66040567 e
```

```
## 93 2026-09-07 03:30:12 2026-09-22 93 0.74075930 s
## 94 2026-09-07 15:05:24 2026-09-22 94 0.49824345 q
## 95 2026-09-06 19:44:31 2026-10-04 95 0.10436335 j
## 96 2026-09-07 04:43:04 2026-09-06 96 0.06179632 g
## 97 2026-09-07 07:45:41 2026-09-15 97 0.21776393 s
## 98 2026-09-07 19:05:28 2026-09-06 98 0.15662152 x
## 99 2026-09-07 05:48:57 2026-09-24 99 0.62424786 s
## 100 2026-09-07 10:46:12 2026-09-08 100 0.39037936 c
```

Cuando de verdad hacen falta más filas o más columnas, se piden:

print() en un tibble: decidir cuánto se imprime.

- **print(n = 4)**: cuatro filas en vez de diez. **n = Inf** las imprime todas.
- **width = Inf**: todas las columnas, aunque no quepan a lo ancho.
- **View(tabla)**: abre la tabla completa en una ventana de RStudio, que casi siempre es lo que uno quería.

```
tb %>%
  print(n = 4, width = Inf)

## # A tibble: 100 x 5
##   a                b                c      d e
##   <dtm>          <date>         <int> <dbl> <chr>
## 1 2026-09-06 20:28:56 2026-09-24     1 0.559 o
## 2 2026-09-07 02:58:10 2026-09-06     2 0.250 o
## 3 2026-09-06 21:51:47 2026-09-08     3 0.0653 d
## 4 2026-09-07 19:38:01 2026-09-07     4 0.573 v
## # i 96 more rows

lubridate::now()

## [1] "2026-09-06 19:41:14 AST"

lubridate::today()

## [1] "2026-09-06"
```

- Para convertir un data frame en tibble se usa **as_tibble()**.
- Para convertir un tibble en data frame se usa **as.data.frame()**.

as_tibble() y **as.data.frame()**: pasar de un formato al otro.

- **as_tibble(df)**: envuelve un data frame como tibble. No copia los datos ni los cambia; solo cambia cómo se imprime y cómo se comporta.
- **as.data.frame(tb)**: el camino de vuelta. Hace falta cuando una función vieja no acepta tibbles.
- **Ojo con los nombres de fila**: **as_tibble()** los **descarta**. Si los necesitas, guárdalos antes en una columna con **tibble::rownames_to_column()**.

17.6 Diferencias entre un tibble y un data frame

Son cuatro, y conviene tenerlas presentes porque explican casi todos los tropiezos del principio.

1. La impresión.

- Tibbles tiene un buen método de impresión que muestra solo las primeras 10 filas y todas las columnas que caben en la pantalla.
- Tibbles no admite nombres de filas. Se eliminan al convertir a tibble o al crear subconjuntos:

2. Operaciones aritméticas en tibbles.

A diferencia de los data frame, los tibbles NO admiten operaciones aritméticas en todas las columnas: el resultado se fuerza silenciosamente a un data frame.

Es decir: multiplicar la tabla entera por 2 funciona, pero lo que sale ya no es un tibble. No es un problema grave, pero sí explica por qué a veces una tabla “deja de imprimirse bonito” en medio de un análisis. La manera correcta de hacer aritmética sobre una tabla es columna por columna, con `mutate()`, como en el segundo bloque.

```
tbl <- tibble(a = 1:3, b = 4:6)
result=tbl * 2

is_tibble(result)
```

```
## [1] FALSE
```

```
is.data.frame(result)
```

```
## [1] TRUE
```

```
result2=tbl |> mutate(c=b*2)
result2
```

```
## # A tibble: 3 x 3
##       a     b     c
##   <int> <int> <dbl>
## 1     1     4     8
## 2     2     5    10
## 3     3     6    12
```

```
is_tibble(result2)
```

```
## [1] TRUE
```

3. Nada de coincidencias parciales.

- Los tibbles también son más estrictos con `$`. Tibbles nunca realiza coincidencias parciales y generará una advertencia y devolverá NULL si la columna no existe:


```
df <- data.frame(abc = 1)
df$a
```

```
## [1] 1
```

```
#> [1] 1
```

```
df2 <- tibble(abc = 1)
df2$abc # el nombre completo: funciona igual en los dos
```

```
## [1] 1
```

Ahora la comparación que importa. La misma errata, `$a` en vez de `$abc`, en las dos clases de tabla:

```
df$a      # data.frame: adivina y devuelve 1, sin avisar
```

```
## [1] 1
```

```
df2$a     # tibble: devuelve NULL y avisa
```

```
## Warning: Unknown or uninitialised column: `a`.
```

```
## NULL
```

Este es el mejor argumento a favor del tibble. En un data frame, escribir mal el nombre de una columna puede devolverte **otra columna** sin decir nada, y el análisis sigue con la variable equivocada. El tibble te devuelve NULL y una advertencia, así que el error aparece de inmediato.

4. Sacar una columna: `$`, `[[` y `[`.

- Para extraer una columna de un tibble se usa `[[`. Es más seguro que `$` porque acepta el nombre como texto y nunca hace coincidencias parciales:

`$`, `[[` y `[`: tres maneras de sacar datos de una tabla, y no dan lo mismo.

- **`tabla$columna`**: devuelve el **vector**. Cómodo, pero el nombre va escrito directamente en el código.
- **`tabla[["columna"]]`**: devuelve el **vector**. Acepta el nombre entre comillas o el número de posición, así que sirve dentro de una función o de un bucle.
- **`tabla["columna"]`**: con un solo corchete devuelve una **tabla de una columna**, no un vector. En un data frame, `df[, "columna"]` devuelve un vector; en un tibble devuelve un tibble, y esa diferencia rompe código viejo.
- **Dentro de un pipe de magrittr**: `tabla %>% .$columna` o `tabla %>% .[["columna"]]`, donde el punto es la tabla que viene por el pipe.

17.7 Selección de subconjuntos de datos

El bloque siguiente saca la misma columna de `vuelos` de cuatro maneras distintas, dos directas y dos dentro de un pipe. Todas devuelven exactamente lo mismo; están juntas para que reconozcas las cuatro cuando las veas en código ajeno.

En una cadena del tidyverse casi siempre es mejor `pull(columna)` que `.$columna`: hace lo mismo, se lee mejor y funciona igual con el pipe nativo.

```
library(datos)

head(vuelos$salida_programada, n=20)

## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600

vuelos[["salida_programada"]] |> head(n=20)

## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600

#Alternativas usar esta función dentro de pipe

vuelos %>% .$salida_programada |> head(n=20)

## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600

vuelos %>% .[["salida_programada"]] |> head(n=20)

## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600

vuelos %>% .[["atraso_salida"]] |>
  head(n=20)

## [1] 2 4 2 -1 -6 -4 -5 -3 -3 -2 -2 -2 -2 -2 -1 0 -1 0 0 1
```

17.8 Cuándo hay que volver a un `data.frame`

Casi todo el código de R moderno acepta tibbles sin problema. Los que no, son paquetes viejos que suponen que `tabla[, "columna"]` devuelve un vector, o que esperan nombres de fila. Cuando una función se queja de forma rara, la prueba rápida es pasarle `as.data.frame(mi_tibble)` y ver si el problema desaparece.

El `data.frame` viene del lenguaje S, de los Laboratorios Bell, y pasó a R casi sin

cambios. El paquete **tibble** lo reescribió en 2016, dentro del tidyverse liderado por **Hadley Wickham**, con una regla explícita: “*hacer menos y quejarse más*”. La palabra **tibble** sale de cómo suena “tbl”, la abreviatura que ya usaba **dplyr** para sus tablas.

17.9 Ejercicios del libro R4DS

Hacer los ejercicios en la sección 10.5 del libro en español.

17.10 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan millas y países del paquete **datos**, y tablas que escribirás a mano, no vuelos, que es el conjunto del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_17_tibbles.Rmd`.

17.10.1 Ejercicio 1. Completar el tibble

El bloque del principio del capítulo construye solo seis de las ocho filas que pedía el enunciado. Complétalo: **secuencia** del 1 al 8; **edad** con 23, 26, 24, 26, 27, 11, 20, 21; y **nombre** con Jose, Maria, Carol, Moncho, Liz, Maria, Jorge y Miguel.

Verificación: tu tibble debe tener 8 filas y 3 columnas, y la línea de tipos debe decir `<int>` o `<dbl>` para las dos primeras y `<chr>` para la tercera.

En palabras: ¿qué pasa si le das a una columna siete valores en vez de ocho? Pruébalo en un bloque con `error=TRUE` y explica el mensaje. ¿Por qué **sí** funciona si le das uno solo?

```
# Escribe tu código aquí
```

17.10.2 Ejercicio 2. Una tabla escrita a mano

Con `tribble()`, escribe una tabla de cuatro parcelas de campo con tres columnas: **parcela** (texto), **n_plantas** (número entero) y **humedad** (texto: “alta” o “baja”). Invéntate los valores.

Verificación: 4 filas, 3 columnas, y los tipos correctos en la línea de tipos.

En palabras: ¿en qué situación preferirías `tribble()` a `tibble()`? Da un ejemplo de tu propio trabajo o de la clase.

```
# Escribe tu código aquí
```

17.10.3 Ejercicio 3. Ida y vuelta

Convierte `millas` en data frame con `as.data.frame()`, guárdalo, y conviértelo otra vez en tibble con `as_tibble()`. Comprueba las dos veces con `is_tibble()` y con `is.data.frame()`.

Verificación: `is.data.frame()` da TRUE en los tres objetos; `is_tibble()` solo en dos de los tres.

En palabras: ¿por qué `is.data.frame()` dice TRUE incluso para el tibble? ¿Qué relación hay entre las dos clases?

```
# Escribe tu código aquí
```

17.10.4 Ejercicio 4. Imprimir lo justo

Imprime `países` de cuatro maneras: tal cual, con `print(n = 3)`, con `print(n = Inf, width = Inf)` (no lo hagas dentro del documento final, es enorme), y como data frame con `as.data.frame(países)`.

Verificación: la impresión normal enseña 10 filas y dice cuántas quedaron fuera.

En palabras: además del número de filas, ¿qué información te da el tibble que el data frame no te da? ¿Por qué esa información ahorra tiempo al principio de un análisis?

```
# Escribe tu código aquí
```

17.10.5 Ejercicio 5. La errata que no avisa

Crea `pais_df <- as.data.frame(países)` y `pais_tb <- as_tibble(países)`. En los dos, pide la columna con el nombre **incompleto**: `$pib` en vez de `$pib_per_capita`.

Verificación: una de las dos devuelve datos y la otra devuelve NULL con una advertencia.

En palabras: imagina que esa línea está en el medio de un análisis de 200 líneas. ¿Cuál de los dos comportamientos prefieres y por qué? Este es el argumento más fuerte del capítulo a favor del tibble: escríbelo con tus palabras.

```
# Escribe tu código aquí
```

17.10.6 Ejercicio 6. \$, [[y [no dan lo mismo

Con `pais_tb` y `pais_df` del ejercicio anterior, saca la columna `esperanza_de_vida` de cuatro maneras: con `$`, con `[[ "..."]]`, con `["..."]` (un solo corchete) y con `[, "..."]`. Usa `class()` en cada resultado.

Verificación: dos de las cuatro devuelven un **vector** y las otras dos una **tabla**, y `[, "..."]` no se comporta igual en el data frame que en el tibble.

En palabras: ¿cuál de las cuatro usarías dentro de una función que recibe el nombre de la columna como texto? ¿Por qué esa diferencia de `[, "..."]` puede romper un script viejo?

```
# Escribe tu código aquí
```

17.10.7 Ejercicio 7 (BONO). Los nombres de fila se pierden

`mtcars` guarda los modelos de carro en los **nombres de fila**, no en una columna. Míralos con `rownames(mtcars)`, conviértelo con `as_tibble()` y vuelve a mirar. Después hazlo bien con `tibble::rownames_to_column()`.

Verificación: el tibble hecho con `as_tibble()` tiene 11 columnas y ya no sabes qué carro es cada fila; el hecho con `rownames_to_column()` tiene 12.

En palabras: ¿por qué el tibble decidió no guardar nombres de fila? Piensa en qué son realmente esos nombres y en dónde deberían estar según lo que aprendiste sobre datos ordenados.

```
# Escribe tu código aquí
```


Chapter 18

Importar datos

Fecha de la última revisión

[1] "2026-09-06"

18.1 Temas

- De dónde salen los datos: la web, un archivo tuyo, un paquete
 - `read_csv()` y su familia, y en qué se diferencia de `read.csv()`
 - Los argumentos que de verdad se usan: `skip`, `na`, `col_names`, `col_types`
 - Cuando la columna llega con el tipo equivocado: las funciones `parse_*()`
 - Acentos y codificación
 - Escribir un archivo: `write_csv()`
 - Otros formatos: Excel, SPSS, bases de datos
-

18.2 De dónde salen los datos

Hasta ahora todos los datos venían de un paquete: escribías `library(datos)` y `vuelos` aparecía. Eso es cómodo para aprender y no se parece en nada a la vida real. En un proyecto de verdad los datos llegan de tres sitios:

1. **De internet**, con una dirección que apunta a un archivo `.csv`. Es rápido y siempre está al día, pero depende de que haya conexión y de que quien publica no cambie el archivo debajo de ti.
2. **De un archivo tuyo**, dentro de la carpeta del proyecto. Es lo más común y lo más reproducible.
3. **De un paquete**, como hemos hecho en clase.

La regla práctica que conviene adoptar desde ahora: **baja el archivo una vez, guárdalo en Datos/, y de ahí en adelante trabaja con tu copia**. Así tu análisis corre igual dentro de un año, sin internet y aunque la fuente original haya desaparecido.

Importar es el paso donde se cuelan la mitad de los errores de un análisis, y son errores silenciosos: una columna que llega como texto, un decimal leído como separador de miles, un acento convertido en símbolo raro. Después de **cada** importación, mira la tabla antes de calcular nada.

18.3 Datos importados de la web.

Los ejemplos de esta sección **descargan datos en vivo desde internet** con `fread("https://...")`, así que necesitas conexión para ejecutarlos. Si ves `Error in download.file(...): cannot open URL` o un `HTTP error`, revisa tu conexión o inténtalo más tarde: el servidor puede estar caído o tardar en responder. Una buena práctica es descargar el archivo una vez y guardarlo con `write_csv()` en `Datos/` para no depender de la red. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

18.3.1 Usar datos que provienen de una base de datos del web.

Aquí un website que recopila la cantidad de personas vacunadas en los EEUU.

```
library(tidyverse)
library(data.table)
```

Seleccionar solamente los datos de Puerto Rico

18.3.2 COVID world

Otro website de datos de COVID, estos representan los datos de todos los países del mundo

Para encontrar datos de PR vea el siguiente enlace: Este sitio del gobierno de Puerto Rico almacena datos muchos diferentes tipos que están relacionados a la isla.

https://datos.estadisticas.pr/dataset?res_format=CSV

Nota que para leer los datos de un archivo CSV, se usa la función “`fread()`” del paquete “`data.table`”.

- Cuando entre a la página, selecciona uno de los archivos, por ejemplo **Facultad -Conservatorio de Musica**
- Presione el botón “Explore” y selecciona “Preview”
- Arriba de la página verá el enlace de **https://.....** para poner en un script.
- Copie y pegue en el script.

Otra alternativa es bajar los datos en formato .csv o Excel y subirlo a su proyecto en RStudio.

18.4 Ejercicio

- Selecciona solamente los datos de los bebés que nacieron primer mes del año, pero agrupado por el día de la semana y contabilizado

COVID_PR

- Calcula la cantidad de muertes por día en PR, y el máximo de muerto en un día por COVID
-

18.5 Datos importando de su computadora

Esta es la manera que vas a usar el 90 % de las veces: el archivo está dentro de la carpeta del proyecto y se lee con una ruta relativa, "Datos/archivo.csv".

18.5.1 Temás: Funciones para importar datos con el paquete readr

Cada función de la lista corresponde a un formato de archivo distinto. Lo único que cambia entre ellas es **qué separa una columna de la siguiente**:

- `read_csv()` # separado por coma
- `read_csv2()` # separado por punto y coma
- `read_tsv()` # separado por tab
- `read_delim()` # separado por delimitador
- `read_fwf()`
- `read_width()`
- `read_positions()`
- `read_table()`

read_csv() (readr) y **fread()** (data.table): importar datos.

- `read_csv("archivo.csv")`: lee un CSV a un tibble; variantes: `read_csv2()` (separado por `;`), `read_tsv()` (tabs), `read_delim()` (delimitador a elección).
- `fread("https://...")`: rápido y práctico para leer directamente desde una URL.
- `write_csv(objeto, "archivo.csv")`: guarda tus datos para no depender de la red.

18.5.2 `read_csv()` frente a `read.csv()`

Se parecen tanto que es fácil confundirlas, y no son lo mismo. `read.csv()` es la de base R, con punto; `read_csv()` es la de `readr`, con guion bajo. Prefiere siempre la segunda:

- devuelve un **tibble**, así que ves los tipos de columna al imprimirla;
- es bastante más rápida con archivos grandes;
- **no** convierte el texto en factores ni cambia los nombres de las columnas (`Area Policiaca` sigue llamándose así, no `Area.Policiaca`);
- te dice en un mensaje cómo interpretó cada columna, que es justo lo que hay que revisar.

18.5.3 Los argumentos que de verdad se usan

Argumentos de `read_csv()`: lo que hay que saber para archivos del mundo real.

- `skip = 2`: salta las primeras líneas. Sirve cuando el archivo trae un título o un logo antes de la tabla.
- `col_names = FALSE`: el archivo no tiene fila de nombres. También acepta un vector con los nombres que quieras ponerle.
- `na = c("", "NA", "n/a", ".")`: qué textos hay que tratar como valor faltante. Por defecto solo `"` y `"NA"`.
- `col_types = cols(codigo = col_character())`: obliga a una columna a leerse con el tipo que tú digas. Es la solución cuando un código postal o un identificador que empieza con cero se lee como número y pierde el cero.
- `comment = "#"`: ignora las líneas que empiecen con ese símbolo.

`read_csv()` adivina el tipo de cada columna mirando las **primeras mil filas**. Si el problema aparece en la fila 5000 (una celda que dice “no medido” en una columna de números), la adivinanza sale mal. Cuando una columna llegue con el tipo equivocado, no la arregles después con `as.numeric()`: dile a `read_csv()` cómo leerla desde el principio con `col_types`.

```
library(tidyverse)
library(readr)
Newborn_Karn_Penrose <- read_csv("Datos/Newborn_Karn_Penrose.csv")
```

```
#library(readxl)
#Newborn_Karn_Penrose1 <- read_excel("~/Google Drive/GitHub_Google_Drive/GitHub/Vintage_DATA/DATA")
head(Newborn_Karn_Penrose)
```

```
## # A tibble: 6 x 4
##   Mother_age Count_Non_survivors Count_Total_Birth Birth_weight
##   <dbl>         <dbl>         <dbl>         <dbl>
## 1         16             0             1             9.5
## 2         16             0             2             9
## 3         16             0             1             8.5
## 4         16             0             3             8
## 5         16             0             7             7.5
## 6         16             0             2             7
```

Estos datos vienen de un estudio clásico de peso al nacer: cada fila es una combinación de edad de la madre y peso del bebé, con cuántos nacimientos hubo y cuántos no sobrevivieron. Fíjate en que **no** es una fila por bebé: son conteos agrupados, y eso cambia cómo hay que leerlos.

Hacer una columna de la proporción de los niños que no sobrevivieron.

La versión larga, con las columnas seleccionadas y ordenadas:

```
Newborn_Karn_Penrose %>%
  dplyr::select(Count_Non_survivors, Count_Total_Birth) %>%
  arrange(Count_Total_Birth, Count_Non_survivors) %>%
  mutate(Perc_sobrevivieron = Count_Non_survivors/Count_Total_Birth)
```

Y la versión corta, que es la que corre aquí:

```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth)
```

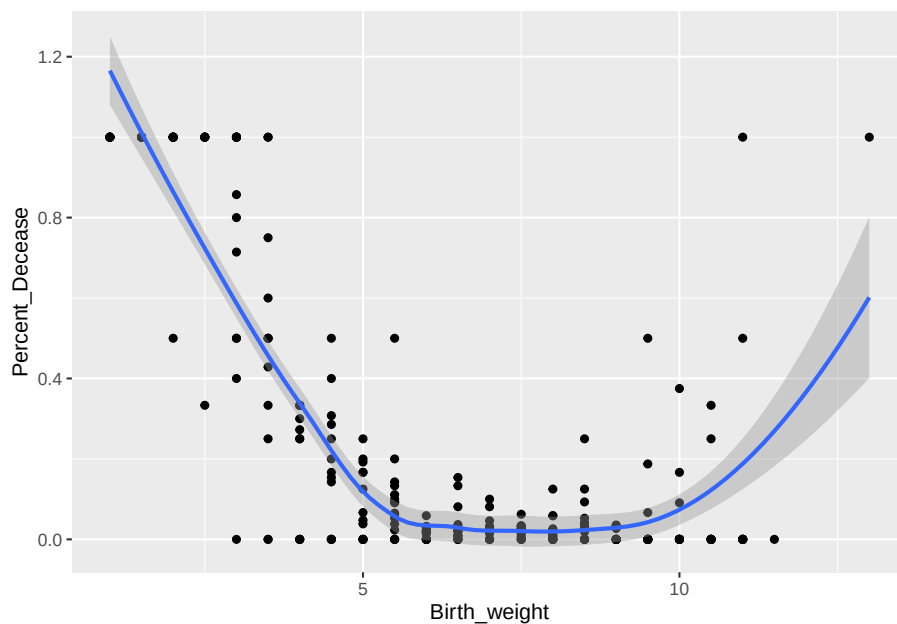
```
## # A tibble: 261 x 5
##   Mother_age Count_Non_survivors Count_Total_Birth Birth_weight Percent_Decease
##   <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
## 1         16             0             1             9.5             0
## 2         16             0             2             9             0
## 3         16             0             1             8.5             0
## 4         16             0             3             8             0
## 5         16             0             7             7.5             0
## 6         16             0             2             7             0
## 7         16             0             4             6.5             0
## 8         16             0             2             6             0
## 9         16             0             2             5.5             0
## 10        16             0             1             5             0
## # i 251 more rows
```

Lo mismo, ahora en gráfica. Primero la versión larga:

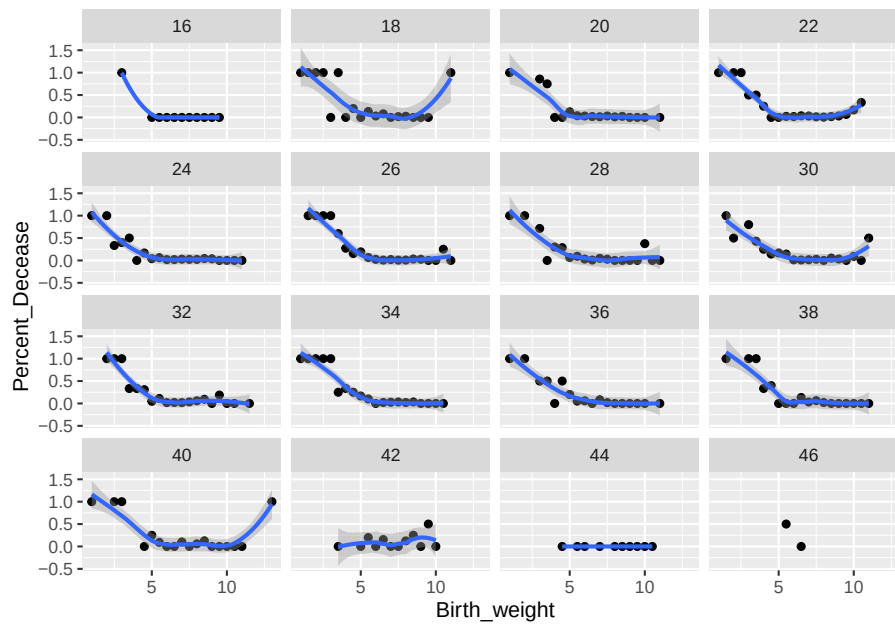
```
Newborn_Karn_Penrose %>%
  dplyr::select(Count_Non_survivors, Count_Total_Birth, Birth_weight) %>%
  arrange(Count_Total_Birth, Count_Non_survivors) %>%
  mutate(porc_sobrevivieron = Count_Non_survivors / Count_Total_Birth) %>%
  ggplot(aes(x = Birth_weight, y = porc_sobrevivieron)) +
  geom_point() +
  geom_smooth()
```

Y la corta, que es la que se ejecuta:

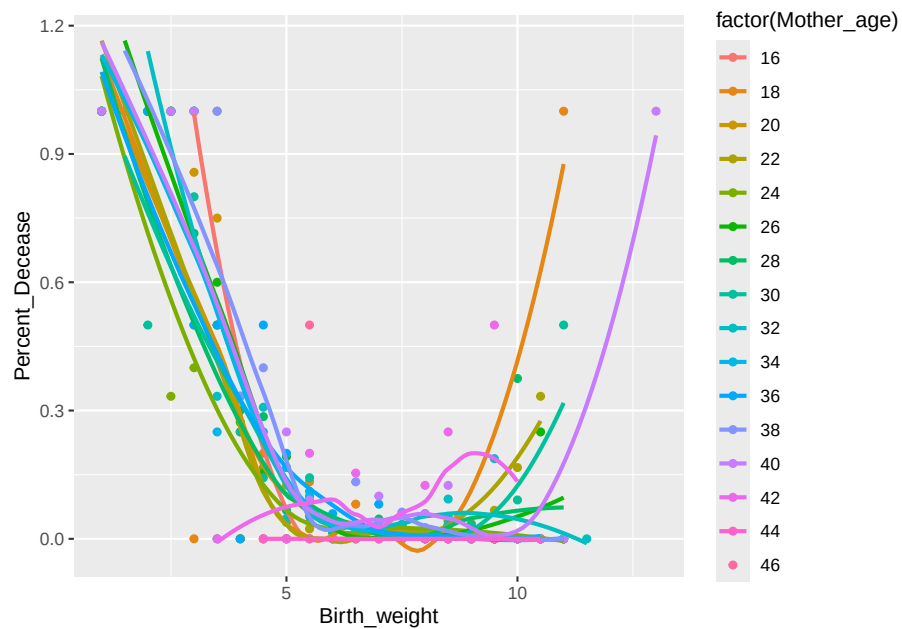
```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth) %>%
  ggplot(aes(Birth_weight, Percent_Decease)) +
  geom_point() +
  geom_smooth()
```



```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth) %>%
  ggplot(aes(Birth_weight, Percent_Decease)) +
  geom_point() +
  geom_smooth() +
  facet_wrap(~Mother_age)
```



```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth) %>%
  ggplot(aes(Birth_weight, Percent_Decease, color=Mother_age)) +
  geom_point(aes(color=factor(Mother_age))) +
  geom_smooth(se=FALSE, aes(color=factor(Mother_age)))
```



18.6 Comparar con base R

	<code>read.csv()</code> (base R)	<code>read_csv()</code> (readr)
Qué devuelve	un <code>data.frame</code>	un <code>tibble</code>
Nombres de columna	los “arregla”: <code>Area Policiaca</code> pasa a <code>Area.Policiaca</code>	los deja tal cual
Texto	en versiones viejas lo convertía en factor	siempre texto
Velocidad	lenta con archivos grandes	mucho más rápida
Informe	ninguno	dice cómo interpretó cada columna

En resumen: `read.csv()` funciona, pero toma decisiones por su cuenta y no te avisa. `read_csv()` hace menos y te cuenta lo que hizo, que es lo que uno quiere cuando los datos son de verdad.

18.7 Segmentar un vector

Cuando `read_csv()` no acierta con el tipo de una columna, o cuando los datos llegan de otra parte, hay que convertir el texto a mano. De eso se encargan las funciones `parse_*()`: cada una toma un vector de **texto** y devuelve un vector del tipo que dice su nombre.

Son también las funciones que `read_csv()` usa por dentro, así que entenderlas explica por qué a veces adivina mal.

`parse_*()` (readr): convertir texto al tipo correcto.

- **Qué hacen:** `parse_integer()`, `parse_double()`, `parse_number()`, `parse_logical()`, `parse_date()`, `parse_factor()` convierten cadenas de texto a números, fechas, etc.
- `parse_number("$123.45")`: extrae el número ignorando símbolos como \$ o %.
- `locale = locale(...)`: controla la codificación, el separador de miles y el formato de fecha.
- `str(parse_logical())`

En el bloque siguiente, fíjate en el contraste: `parse_logical()` funciona con `x`, que contiene textos que R reconoce como lógicos, y falla con `y`, que contiene nombres de personas. Y fíjate sobre todo en `mean(z)`: `z` parece numérico pero está entre comillas, así que es texto, y el promedio no se puede calcular.

```
x= c("TRUE", "FALSE", "NA")
x
```

```
## [1] "TRUE" "FALSE" "NA"
```

```
str(parse_logical(x))
```

```
## logi [1:3] TRUE FALSE NA
```

```
y= c("Carlos", "Kelvin")
```

```
y
```

```
## [1] "Carlos" "Kelvin"
```

```
str(parse_logical(y))
```

```
## logi [1:2] NA NA
```

```
## - attr(*, "problems")= tibble [2 x 4] (S3: tbl_df/tbl/data.frame)
```

```
## ..$ row      : int [1:2] 1 2
```

```
## ..$ col      : int [1:2] NA NA
```

```
## ..$ expected: chr [1:2] "1/0/T/F/TRUE/FALSE" "1/0/T/F/TRUE/FALSE"
```

```
## ..$ actual  : chr [1:2] "Carlos" "Kelvin"
```

```
str(parse_logical(c("TRUE", "FALSE", "NA")))
```

```
## logi [1:3] TRUE FALSE NA
```

```
z=c("1", "2", "3")
```

```
z
```

```
## [1] "1" "2" "3"
```

```
mean(z)
```

```
## [1] NA
```

```
z1=str(parse_integer(z))
```

```
## int [1:3] 1 2 3
```

```
str(parse_integer(c("1", "2", "3")))
```

```
## int [1:3] 1 2 3
```

```
str(parse_date(c("2010-01-01", "1979-10-14")))
```

```
## Date[1:2], format: "2010-01-01" "1979-10-14"
```

`parse_number()` es la más útil de todas en la práctica: se queda con el número y tira todo lo demás. Con ella se limpian de un golpe las columnas de precios, de porcentajes y de cantidades escritas con símbolos.

```
m=c("$4000", "$500" )
```

```
parse_number(m)
```

```
## [1] 4000 500
```

```
parse_number("$100")
```

```
## [1] 100
```

```
#> [1] 100
```

```
parse_number("20%")
```

```
## [1] 20
```

```
#> [1] 20
```

```
parse_number("It cost $123.45")
```

```
## [1] 123.45
```

```
#> [1] 123.45
```

- `str(parse_integer())`
- `str(parse_date())`

18.8 Números

Con los números hay dos trampas, y las dos son cuestión de país.

La primera es el **separador decimal**: en inglés se escribe 1.5 y en español 1,5. La segunda es el **separador de miles**: en inglés 1,000,000 y en español 1.000.000. Es decir, el mismo signo significa cosas contrarias según de dónde venga el archivo.

Se resuelven con `locale()`, que le dice a `readr` en qué convención está escrito el archivo:

- `parse_double()`
- `parse_number()`
- `parse_number(locale=locale(grouping_mark = "."))`

```
parse_double("1,5", locale = locale(decimal_mark = ","))
parse_number("1.000.000", locale = locale(grouping_mark = "."))
```

Este error es traicionero porque **no da ningún mensaje**: si un archivo en español se lee con la convención inglesa, 1,5 se convierte en 15 y el análisis sigue con números diez veces más grandes. Cuando importes datos de una agencia de gobierno, mira siempre un par de valores y compáralos con el archivo original.

18.9 Cadena de texto

Aquí aparece el problema de los acentos. Una computadora no guarda letras, guarda números, y la tabla que traduce números a letras se llama **codificación**. Hoy casi todo usa UTF-8, que tiene sitio para la ñ, los acentos y los signos de apertura. Los archivos viejos, sobre todo los que salieron de un Excel en Windows, suelen venir en Latin1.

Si abres un archivo con la codificación equivocada, la ñ se convierte en símbolos raros. `charToRaw()` enseña los números que hay detrás de un texto, y `guess_encoding()` intenta adivinar en qué codificación está escrito.

- `parse_character()`
- `charToRaw()`

```
charToRaw("Ramn")
```

```
## [1] 52 61 6d 6e
```

```
x1 <- "El Ni\xf1o was particularly bad this year"
x2 <- "\x82\xb1\x82\xf1\x82\xc9\x82xbf\x82xcd"
x3 <- "The boy was particularly bad this year"
```

```
library(readr)
parse_character(x1, locale = locale(encoding = "Latin1"))

## [1] "El Niño was particularly bad this year"
#> [1] "El Niño was particularly bad this year"
parse_character(x2, locale = locale(encoding = "Shift-JIS"))

## [1] "  "
parse_character(x2, locale = locale(encoding = "Shift-JIS"))

## [1] "  "
#> [1] "  "
```

Cuando veas `Ni o` o `Ni√±o` en una tabla recién importada, no es que los datos estén dañados: es la codificación. Prueba `read_csv(archivo, locale = locale(encoding = "Latin1"))`, y si no sabes cuál es, pregúntaselo a `guess_encoding()`.

- `guess_encoding(charToRaw())`

```
guess_encoding(charToRaw(x3))
```

```
## # A tibble: 1 x 2
##   encoding confidence
##   <chr>           <dbl>
## 1 ASCII             1
```

18.10 Factores

`parse_factor()` convierte texto en factor, pero pidiéndote la lista de niveles que esperas. Eso es útil precisamente porque **avisa** cuando aparece un valor que no estaba en tu lista: una errata, un “Carolina” con acento entre mil “Carolina” sin él.

- `parse_factor()`

```
parse_factor(c("alta", "baja", "media"), levels = c("alta", "baja"))
# avisa de que "media" no estaba en los niveles
```

18.11 Fechas, fechas-horas, horas

Una fecha leída como texto no sirve de nada: no se puede ordenar bien, no se le puede restar otra fecha, no se puede agrupar por mes. Convertirla es de los primeros pasos después de importar.

- `parse_datetime()`

- `library(hms)`
- `parse_time()`

```
parse_date("2016-08-01")
parse_date("01/08/2016", format = "%d/%m/%Y")
parse_time("15:30:00")
```

01/08/2016 es el 1 de agosto para media humanidad y el 8 de enero para la otra mitad. Si no dices el `format`, R usa el suyo y puede acertar por casualidad durante los primeros doce días del mes. Este capítulo solo abre el tema: las fechas tienen su propio capítulo más adelante, con el paquete `lubridate`.

18.12 Segmentar un archivo

- `guess_parser`

`guess_parser()` contesta la pregunta “¿qué tipo diría `readr` que es esto?”. Es la manera de entender por qué una columna llegó como texto cuando tú la querías numérica: casi siempre hay una celda con un valor raro escondida en el medio.

```
guess_parser(c("1", "2", "3"))
guess_parser(c("1", "2", "no medido"))
```

18.13 Escribir un Archivo

Guardar es tan importante como leer, y por dos razones distintas: para no depender de la red, y para dejarle a otra persona (o a un revisor) el resultado de tu limpieza sin que tenga que correr todo tu script.

Después de descargar datos de internet, guárdalos con `write_csv()` para trabajar sin depender de la conexión en la próxima sesión.

- `write_csv()`
- `write_tsv()`

Un `.csv` **no guarda los tipos**: al volver a leerlo, `readr` tiene que adivinar otra vez, y una columna que tú habías convertido en factor o en fecha vuelve a ser texto. Si necesitas conservar los tipos exactamente, guarda con `write_rds()` (formato propio de R) o con `arrow::write_parquet()`. Para compartir con gente que no usa R, el `.csv` sigue siendo la mejor opción.

18.14 Otros tipos de datos y paquetes

R lee casi cualquier formato, siempre con la misma idea: un paquete por familia de archivos.

- `library(haven)` # SPSS, Stata y SAS
- `readxl()` lee archivo Excel en formato .xls, .xlsx
- BDI lee archivo RMySQL y otros

Paquetes para otros formatos: cuál usar según de dónde vengan los datos.

- **`readxl::read_excel("archivo.xlsx", sheet = 2)`:** hojas de Excel. El argumento `sheet` acepta el número o el nombre de la hoja.
- **`haven`:** archivos de SPSS (.sav), Stata (.dta) y SAS. Es el camino habitual cuando los datos vienen de una encuesta o de otra disciplina.
- **`DBI` y `RMariaDB` / `RPostgres`:** bases de datos, cuando la tabla es demasiado grande para caber en la memoria.
- **`googlesheets4`:** hojas de Google, útiles cuando varias personas están llenando la tabla a la vez.

Con un Excel, el problema casi nunca es leerlo: es que la hoja trae celdas combinadas, un título en la primera fila y notas al pie debajo de la tabla. Los argumentos `skip` y `range` de `read_excel()` resuelven la mayoría de esos casos.

18.15 Ejercicios del libro R4DS

Hacer los ejercicios de las siguientes secciones del libro en español:

- sección 11.2.2 (leer archivos),
- sección 11.3.5 (segmentar vectores).

18.16 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan `Datos/crimen_pr.csv`, un archivo real del gobierno de Puerto Rico que está en el repositorio del curso, y archivos que vas a crear tú mismo. No usan `Newborn_Karn_Penrose.csv`, que es el del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_18_importar.Rmd`.

18.16.1 Ejercicio 1. Mirar antes de calcular

Lee `Datos/crimen_pr.csv` con `read_csv()`. Antes de tocar nada, mira **qué llegó**: imprime el tibble, corre `glimpse()` y `spec()`, y cuenta los valores faltantes de cada columna.

Verificación: la tabla debe tener **8870 filas** y **7 columnas**, y dos de las columnas deben traer **733 valores faltantes** cada una.

En palabras: `readr` convirtió esos faltantes solo porque en el archivo estaban escritos como NA. ¿Qué habría pasado si el archivo los hubiera escrito como "sin dato"? ¿Con cuál argumento lo arreglarías?

Escribe tu código aquí

18.16.2 Ejercicio 2. Los nombres mienten

Sigue con la misma tabla. Mira los valores de `Delito` y de `Delitos_code` con `count()`.

Verificación: cada una tiene 9 valores distintos, pero `Delito` contiene números del 1 al 9 y `Delitos_code` contiene nombres como `Asesinato` o `Apropiacion Ilegal`. Los nombres de las dos columnas están intercambiados.

Renómbralas con `rename()` para que digan la verdad, y de paso quítale el espacio al nombre de `Area Policiaca`.

En palabras: para usar `Area Policiaca` tal como viene hay que escribirla entre acentos graves. ¿Por qué? ¿Qué le habría hecho `read.csv()` a ese nombre?

Escribe tu código aquí

18.16.3 Ejercicio 3. Un código no es un número

La columna con los números del 1 al 9 es un **código**, no una cantidad: no tiene sentido calcular su promedio. Vuelve a leer el archivo obligando a que esa columna se lea como texto, con `col_types = cols(...)` y `col_character()`.

Verificación: `class()` de esa columna debe pasar de "numeric" a "character", y el resto de la tabla debe quedar igual (8870 por 7).

En palabras: da otro ejemplo de una columna que parece numérica y no lo es. ¿Qué se pierde si un código postal de Puerto Rico se lee como número?

Escribe tu código aquí

18.16.4 Ejercicio 4. Ida y vuelta por el disco

Toma millas del paquete `datos`. Convierte `clase` en factor con `mutate(clase = factor(clase))`, guarda la tabla con `write_csv()` en `Datos/millas_prueba.csv`, y vuelve a leerla con `read_csv()`.

Verificación: la tabla que vuelve tiene las mismas 234 filas, pero `clase` ya no es factor: es texto.

En palabras: explica por qué se perdió el tipo. ¿Qué formato usarías si necesitaras conservarlo? ¿Y por qué, aun así, seguimos usando `.csv` para compartir datos?

```
# Escribe tu código aquí
```

18.16.5 Ejercicio 5. Un archivo sucio, hecho a mano

Crea con `writeLines()` un archivo de texto que empiece con dos líneas de basura antes de la tabla y que use `n/d` para los faltantes:

```
Reporte mensual
```

```
Generado el 1 de agosto
```

```
parcela,plantas
```

```
P1,34
```

```
P2,n/d
```

```
P3,27
```

Léelo primero con `read_csv()` sin argumentos y mira el desastre. Después arréglalo con `skip` y `na`.

Verificación: la versión arreglada tiene 3 filas, 2 columnas, `plantas` es numérica y hay exactamente **un** NA.

En palabras: en la primera lectura, ¿cuántas columnas creó `readr` y de dónde sacó el nombre? Copia la advertencia que salió y explica qué te estaba diciendo.

```
# Escribe tu código aquí
```

18.16.6 Ejercicio 6. Números escritos en español

Convierte a número estos textos, que vienen escritos a la manera española: `c("1.234", "2.500.000", "3,75")`. Hazlo primero sin `locale()` y después con el `locale()` correcto.

Verificación: con el `locale()` correcto, "2.500.000" tiene que dar dos millones y medio, y "3,75" tiene que dar tres con setenta y cinco.

En palabras: sin `locale()`, ninguno de los tres da error, pero todos dan un número equivocado. Explica por qué este es el tipo de fallo más peligroso de todo el capítulo.

```
# Escribe tu código aquí
```

18.16.7 Ejercicio 7 (BONO). Fechas y horas de verdad

Vuelve a `crimen_pr.csv`. Comprueba con `class()` cómo llegaron `Fecha` y `Hora`. Después cuenta cuántos incidentes hay por mes.

Verificación: hay tres meses, todos de 2016, y el tercero tiene muchísimos menos casos que los otros dos.

En palabras: ¿por qué el tercer mes tiene tan pocos? Míralo con `range(Fecha)` y explica qué te dice eso sobre el archivo. ¿Qué error de análisis cometería alguien que comparara los tres meses sin darse cuenta?

Escribe tu código aquí

Chapter 19

Datos Ordenados

```
## [1] "2026-09-06"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/tidy-data.html>

Español: <https://es.r4ds.hadley.nz/12-tidy.html>

19.1 Temas: Datos Ordenados

- Las tres reglas de los datos ordenados
 - Por qué conviene ordenarlos, aunque cueste trabajo
 - `pivot_longer()`: de ancho a largo
 - `pivot_wider()`: de largo a ancho
 - `separate()` y `unite()`: partir y juntar columnas
 - Valores faltantes implícitos y explícitos
-

19.2 Paquetes

```
library(tidyverse)
library(datos)
```

19.3 Las tres reglas

Datos ordenados (tidy data): cada variable es una columna, cada observación es una fila y cada valor ocupa su propia celda. Con este formato el tidyverse trabaja de forma más natural.

Las tres reglas están relacionadas: si cumples dos, la tercera sale sola. En la práctica se reducen a dos instrucciones:

1. **Pon cada conjunto de datos en su propia tabla.**
2. **Pon cada variable en su propia columna.**

Lo difícil no es la definición, es reconocer **cuál es la variable** en un conjunto concreto. Y eso depende de la pregunta: en una tabla de mediciones repetidas, ¿la variable es “el peso” o son “el peso en enero” y “el peso en julio”? Casi siempre la respuesta correcta es la primera, y la fecha es otra variable.

El concepto de *tidy data* (datos ordenados) fue formalizado por **Hadley Wickham** en un influyente artículo de 2014. La idea (una variable por columna, una observación por fila) unificó la forma de preparar datos para el análisis.

19.3.1 ¿Por qué molestarse?

Ordenar una tabla da trabajo y no produce ningún resultado nuevo, así que la pregunta es legítima. Hay dos razones.

La primera es **consistencia**: si todos los conjuntos tienen la misma forma, las herramientas que aprendiste sirven para todos. No hay que reaprender nada al cambiar de proyecto.

La segunda es que R **trabaja por columnas**. `mutate()` opera sobre una columna entera, `group_by()` agrupa por los valores de una columna, `ggplot()` mapea una columna a un eje. Cuando la variable que te interesa está repartida entre cinco columnas, ninguna de esas herramientas sirve: no puedes agrupar por “el año” si el año es el nombre de la columna en vez de un valor.

19.4 Las cinco tablas del ejemplo

Los cinco conjuntos siguientes contienen **exactamente la misma información**: casos de tuberculosis y población de tres países en 1999 y en 2000. Lo único que cambia es la forma. Míralos con calma antes de seguir: solo uno de ellos está ordenado.

```
tabla1
```

```
## # A tibble: 6 x 4
##   pais      anio  casos poblacion
##   <chr>    <dbl> <dbl>    <dbl>
```

```
## 1 Afganistán 1999 745 19987071
## 2 Afganistán 2000 2666 20595360
## 3 Brasil 1999 37737 172006362
## 4 Brasil 2000 80488 174504898
## 5 China 1999 212258 1272915272
## 6 China 2000 213766 1280428583
```

```
tabla2
```

```
## # A tibble: 12 x 4
##   pais      anio tipo      cuenta
##   <chr>    <dbl> <chr>    <dbl>
## 1 Afganistán 1999 casos      745
## 2 Afganistán 1999 población 19987071
## 3 Afganistán 2000 casos      2666
## 4 Afganistán 2000 población 20595360
## 5 Brasil 1999 casos      37737
## 6 Brasil 1999 población 172006362
## 7 Brasil 2000 casos      80488
## 8 Brasil 2000 población 174504898
## 9 China 1999 casos      212258
## 10 China 1999 población 1272915272
## 11 China 2000 casos      213766
## 12 China 2000 población 1280428583
```

```
tabla3
```

```
## # A tibble: 6 x 3
##   pais      anio tasa
##   <chr>    <dbl> <chr>
## 1 Afganistán 1999 745/19987071
## 2 Afganistán 2000 2666/20595360
## 3 Brasil 1999 37737/172006362
## 4 Brasil 2000 80488/174504898
## 5 China 1999 212258/1272915272
## 6 China 2000 213766/1280428583
```

```
tabla4a
```

```
## # A tibble: 3 x 3
##   pais      `1999` `2000`
##   <chr>    <dbl> <dbl>
## 1 Afganistán 745 2666
## 2 Brasil 37737 80488
## 3 China 212258 213766
```

```
tabla4b
```

```
## # A tibble: 3 x 3
```

```
##   pais          `1999`      `2000`
##   <chr>          <dbl>      <dbl>
## 1 Afganistán    19987071    20595360
## 2 Brasil        172006362    174504898
## 3 China         1272915272    1280428583
```

La respuesta es `tabla1`: tiene cuatro columnas (`pais`, `anio`, `casos`, `poblacion`) y cada una es una variable. Las otras cuatro están mal, cada una a su manera:

- `tabla2` mete dos variables distintas (`casos` y `población`) en una sola columna de valores, y usa otra columna para decir cuál es cuál. Cada observación quedó partida en dos filas.
- `tabla3` guarda dos valores en la misma celda: 745/19987071 es texto, no un número, y no se puede dividir ni graficar.
- `tabla4a` y `tabla4b` usan los años como nombres de columna. El año es una variable y está escondido en la cabecera. Además hacen falta dos tablas para lo que debería ser una.

Una señal casi infalible de que una tabla no está ordenada: los nombres de las columnas son **valores** (1999, 2000, enero, febrero, control, tratamiento) en vez de nombres de variables.

Con `tabla1`, calcular la tasa de casos por cada diez mil habitantes es una línea:

```
tabla1 |>
  mutate(tasa = casos / poblacion * 10000)
```

```
## # A tibble: 6 x 5
##   pais      anio  casos  poblacion  tasa
##   <chr>    <dbl> <dbl>      <dbl> <dbl>
## 1 Afganistán 1999    745    19987071 0.373
## 2 Afganistán 2000   2666    20595360 1.29
## 3 Brasil    1999  37737    172006362 2.19
## 4 Brasil    2000  80488    174504898 4.61
## 5 China     1999 212258    1272915272 1.67
## 6 China     2000 213766    1280428583 1.67
```

Con cualquiera de las otras cuatro, hay que ordenarlas primero. De eso trata el resto del capítulo.

19.5 El paquete tidyr

Las funciones `pivot_longer()` y `pivot_wider()` de `tidyr` son las herramientas principales para pasar los datos entre el formato ancho y el formato largo.

`pivot_longer()` y `pivot_wider()` (`tidyr`): cambiar la forma de la tabla.

- **pivot_longer()**: pasa de **ancho a largo** (junta varias columnas en pares nombre-valor).
- **pivot_wider()**: pasa de **largo a ancho** (reparte una columna en varias).
- **Uso**: dejar los datos *ordenados* para poder graficar y resumir con facilidad.

Las dos son opuestas: lo que hace una, la otra lo deshace. Casi siempre se usa `pivot_longer()`, porque el problema más común con diferencia es el de los nombres de columna que en realidad son valores.

19.5.1 pivot_longer(): de ancho a largo

`tabla4a` tiene los años como nombres de columna. Para arreglarla hay que contestar tres preguntas:

1. **¿Cuáles columnas hay que mover?** Aquí, 1999 y 2000. Van entre acentos graves porque un nombre que empieza por número no es un nombre válido en R.
2. **¿Cómo se llama la variable escondida en los nombres?** Se llama `anio`, y eso va en `names_to`.
3. **¿Cómo se llama la variable escondida en los valores?** Se llama `casos`, y eso va en `values_to`.

```
tabla4a |>
  pivot_longer(
    cols      = c(`1999`, `2000`),
    names_to  = "anio",
    values_to = "casos"
  )
```

```
## # A tibble: 6 x 3
##   pais      anio  casos
##   <chr>    <chr> <dbl>
## 1 Afganistán 1999     745
## 2 Afganistán 2000    2666
## 3 Brasil     1999   37737
## 4 Brasil     2000   80488
## 5 China      1999  212258
## 6 China      2000  213766
```

La tabla pasó de 3 filas y 3 columnas a 6 filas y 3 columnas. Es más **larga** y más **estrecha**, y de ahí el nombre de la función.

El `anio` que sale de `pivot_longer()` es **texto**, no un número, porque venía de un nombre de columna y los nombres son texto. Si vas a ordenar por año o restar años, conviértelo: `mutate(anio = as.integer(anio))`, o usa `names_transform = list(anio = as.integer)` dentro de la misma llamada.

Lo mismo con la tabla de población, y después se unen las dos por sus columnas comunes:

```

tabla4a_larga <- tabla4a |>
  pivot_longer(c(`1999`, `2000`), names_to = "anio", values_to = "casos")

tabla4b_larga <- tabla4b |>
  pivot_longer(c(`1999`, `2000`), names_to = "anio", values_to = "poblacion")

left_join(tabla4a_larga, tabla4b_larga)

```

```

## # A tibble: 6 x 4
##   pais      anio  casos poblacion
##   <chr>    <chr> <dbl>    <dbl>
## 1 Afganistán 1999     745  19987071
## 2 Afganistán 2000    2666  20595360
## 3 Brasil     1999   37737  172006362
## 4 Brasil     2000   80488  174504898
## 5 China      1999  212258 1272915272
## 6 China      2000  213766 1280428583

```

El resultado es `tabla1`. Ese `left_join()` es el tema del capítulo siguiente; por ahora basta con ver que dos tablas ordenadas se combinan sin esfuerzo, y dos tablas desordenadas no.

19.5.2 `pivot_wider()`: de largo a ancho

El problema de `tabla2` es el contrario: una observación está repartida en varias filas. Aquí las preguntas son dos:

1. ¿Cuál columna tiene los nombres de las variables nuevas? tipo, que va en `names_from`.
2. ¿Cuál columna tiene los valores? cuenta, que va en `values_from`.

```

tabla2 |>
  pivot_wider(
    names_from = tipo,
    values_from = cuenta
  )

## # A tibble: 6 x 4
##   pais      anio  casos población
##   <chr>    <dbl> <dbl>    <dbl>
## 1 Afganistán 1999     745  19987071
## 2 Afganistán 2000    2666  20595360
## 3 Brasil     1999   37737  172006362
## 4 Brasil     2000   80488  174504898
## 5 China      1999  212258 1272915272
## 6 China      2000  213766 1280428583

```

De 12 filas a 6. Fíjate en que la columna nueva se llama `población`, con acento,

porque ese era el valor que estaba dentro de la columna `tipo`: `pivot_wider()` usa los valores tal como vienen para nombrar las columnas.

`pivot_wider()` se usa mucho menos que `pivot_longer()` para ordenar datos, pero es la herramienta de siempre para **presentar** un resultado. Una tabla larga es la que R quiere; una tabla ancha es la que un lector entiende de un vistazo.

19.6 `separate()` y `unite()`

Estas dos resuelven el problema de `tabla3`: dos valores metidos en la misma celda.

`separate()` y `unite()` (tidyr): partir una columna en varias, y juntar varias en una.

- `separate(columna, into = c("a", "b"), sep = "/")`: parte por el separador que le digas. Si no le dices ninguno, parte por el primer carácter que no sea alfanumérico.
- `convert = TRUE`: después de partir, adivina el tipo de las columnas nuevas. Sin esto, todo sale como texto.
- `unite(nueva, a, b, sep = "_")`: el camino contrario, junta varias columnas en una.

```
tabla3 |>
  separate(tasa, into = c("casos", "poblacion"), sep = "/")
```

```
## # A tibble: 6 x 4
##   pais      anio casos poblacion
##   <chr>    <dbl> <chr>   <chr>
## 1 Afganistán 1999 745   19987071
## 2 Afganistán 2000 2666  20595360
## 3 Brasil    1999 37737 172006362
## 4 Brasil    2000 80488 174504898
## 5 China     1999 212258 1272915272
## 6 China     2000 213766 1280428583
```

Funciona, pero mira los tipos: `casos` y `poblacion` son texto. Con `convert = TRUE` quedan numéricos y ya se puede calcular con ellos:

```
tabla3 |>
  separate(tasa, into = c("casos", "poblacion"), sep = "/", convert = TRUE) |>
  mutate(tasa = casos / poblacion * 10000)
```

```
## # A tibble: 6 x 5
##   pais      anio casos poblacion tasa
##   <chr>    <dbl> <int>   <int> <dbl>
```

```
## 1 Afganistán 1999 745 19987071 0.373
## 2 Afganistán 2000 2666 20595360 1.29
## 3 Brasil 1999 37737 172006362 2.19
## 4 Brasil 2000 80488 174504898 4.61
## 5 China 1999 212258 1272915272 1.67
## 6 China 2000 213766 1280428583 1.67
```

`unite()` hace el camino de vuelta:

```
tabla3 |>
  separate(tasa, into = c("casos", "poblacion"), sep = "/") |>
  unite(tasa_otra_vez, casos, poblacion, sep = "/")
```

```
## # A tibble: 6 x 3
##   pais      anio tasa_otra_vez
##   <chr>    <dbl> <chr>
## 1 Afganistán 1999 745/19987071
## 2 Afganistán 2000 2666/20595360
## 3 Brasil 1999 37737/172006362
## 4 Brasil 2000 80488/174504898
## 5 China 1999 212258/1272915272
## 6 China 2000 213766/1280428583
```

En las versiones recientes de `tidyr` estas dos funciones tienen sucesoras con nombres más precisos: `separate_wider_delim()`, `separate_wider_position()` y `separate_longer_delim()`. Hacen lo mismo con más control y mejores mensajes de error. `separate()` y `unite()` siguen funcionando, y son las que usa el libro de R4DS en el que se basa el curso.

Ver vignettes: <https://tidyr.tidyverse.org/articles/pivot.html>

19.7 Valores faltantes implícitos y explícitos

Un valor puede faltar de dos maneras, y la diferencia importa.

- **Explícito:** ahí está la fila, con un `NA` en la celda. Se ve.
- **Implícito:** la fila sencillamente no existe. No se ve, y por eso es más peligroso.

Cambiar la forma de una tabla convierte una clase en la otra. Al pasar a ancho, las combinaciones que no existían aparecen como `NA`; al pasar a largo, esos `NA` se pueden volver a quitar.

```
mediciones <- tibble(
  parcela = c("P1", "P1", "P2"),
  anio    = c(2023, 2024, 2023),
  plantas = c(12, 15, 30)
```



```
)

mediciones

## # A tibble: 3 x 3
##   parcela  anio plantas
##   <chr>    <dbl>   <dbl>
## 1 P1      2023     12
## 2 P1      2024     15
## 3 P2      2023     30

mediciones |>
  pivot_wider(names_from = anio, values_from = plantas)

## # A tibble: 2 x 3
##   parcela `2023` `2024`
##   <chr>    <dbl>   <dbl>
## 1 P1      12     15
## 2 P2      30     NA
```

La parcela P2 no tenía fila para 2024: era un faltante **implícito**. Al ensanchar la tabla se hizo **explícito**, y ahora se ve.

`complete(parcela, anio)` hace explícitos todos los faltantes sin cambiar la forma de la tabla, y es la manera de contestar la pregunta “¿me falta algún dato que debería estar?”. Es de las primeras cosas que conviene correr con datos de campo.

19.8 Ejercicios del libro R4DS

Hacer los ejercicios de las secciones 12.2.1, 12.3.3 y 12.4.3 del libro en español.

19.9 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan **países** y **millas** del paquete **datos**, y tablas que escribirás a mano, no las `tabla1` a `tabla4b` del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_19_datos-ordenados.Rmd`.

19.9.1 Ejercicio 1. ¿Está ordenada?

Mira `países` con `glimpse()`. Contesta, columna por columna, si cada una es una variable, y decide si la tabla está ordenada.

Verificación: `países` tiene 1704 filas y 6 columnas.

En palabras: justifica tu respuesta con las tres reglas. ¿Cuál es la observación en esta tabla: un país, o un país en un año?

```
# Escribe tu código aquí
```

19.9.2 Ejercicio 2. De largo a ancho

Con `países`, quédate solo con `pais`, `anio` y `esperanza_de_vida`, y ensancha la tabla con `pivot_wider()` para que cada año sea una columna.

Verificación: la tabla ancha debe tener **142 filas** y **13 columnas** (el país más los doce años).

En palabras: esta tabla ancha, ¿está ordenada? ¿Para qué sirve mejor que la original, y para qué sirve peor?

```
# Escribe tu código aquí
```

19.9.3 Ejercicio 3. Y de vuelta

Toma la tabla ancha del ejercicio anterior y devuélvela a la forma larga con `pivot_longer()`. Ocupate de que `anio` vuelva a ser numérico.

Verificación: la tabla debe volver a tener 1704 filas, y `anio` debe ser numérico, no texto.

En palabras: si no conviertes `anio`, ¿qué pasa al ordenar con `arrange()`? Pruébalo y explica el resultado.

```
# Escribe tu código aquí
```

19.9.4 Ejercicio 4. Dos columnas que son una variable

En `millas`, `ciudad` y `autopista` son dos columnas, pero miden **lo mismo** en dos condiciones distintas. Pásalas a formato largo con `names_to = "tipo_de_via"` y `values_to = "millas_por_galon"`.

Verificación: la tabla larga debe tener **468 filas** y **11 columnas**.

En palabras: con la tabla larga, haz un diagrama de caja del consumo por `tipo_de_via`. ¿Podrías haber hecho esa gráfica con la tabla original? Explica por qué no de una sola vez.

```
# Escribe tu código aquí
```

19.9.5 Ejercicio 5. Juntar y volver a partir

Con `países`, usa `unite()` para crear una columna `pais_anio` que junte `pais` y `anio` con un guion bajo. Después usa `separate()` para deshacerlo.

Verificación: después de `separate()` sin `convert = TRUE`, `anio` es **texto**; con `convert = TRUE`, vuelve a ser numérico.

En palabras: ¿por qué `separate()` no puede adivinar el tipo por su cuenta? ¿Y qué pasaría si algún país tuviera un guion bajo en el nombre?

```
# Escribe tu código aquí
```

19.9.6 Ejercicio 6. El faltante que no se ve

Escribe a mano una tabla de cuatro parcelas medidas en tres años, pero **quita tres combinaciones a propósito**. Ensánchala con `pivot_wider()` y después usa `complete()` sobre la tabla original.

Verificación: la tabla ancha debe enseñar exactamente tres NA, y `complete()` debe llevar la tabla larga a 12 filas.

En palabras: las dos maneras enseñan los mismos tres huecos. ¿Cuál usarías en un análisis de datos de campo y por qué? ¿Qué habría pasado si nunca los hubieras hecho explícitos y hubieras calculado un promedio por parcela?

```
# Escribe tu código aquí
```

19.9.7 Ejercicio 7 (BONO). Desordenar a propósito

Convierte `millas` en una tabla del estilo de `tabla4a`: una fila por `clase`, una columna por `anio`, y en las celdas el promedio de `autopista`.

Verificación: 7 filas y 3 columnas, y los nombres de dos de las columnas son años.

En palabras: acabas de producir una tabla **desordenada** a propósito, y sin embargo es la que pondrías en un informe. Explica esa contradicción: ¿cuándo conviene una tabla ordenada y cuándo una ancha?

```
# Escribe tu código aquí
```


Chapter 20

Datos Relacionados

[1] "2026-09-06"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/relational-data.html>

Español: <https://es.r4ds.hadley.nz/13-relational-data.html>

20.1 Temas: Datos Relacionados

- Qué es una clave: primaria, foránea y sustituta
 - Cómo se comprueba que una clave lo es de verdad
 - Uniones de transformación (*mutating joins*): `left_join()`, `inner_join()`, `right_join()`, `full_join()`
 - Uniones de filtro (*filtering joins*): `semi_join()`, `anti_join()`
 - Definir las columnas clave con `by`
 - Los tres problemas que arruinan una unión
-

20.2 Por qué hace falta más de una tabla

Hasta ahora cada análisis cabía en una tabla. En un proyecto real casi nunca es así: los datos llegan repartidos en varias, y hay que juntarlos. Y eso está bien hecho, no es un descuido. Si en `vuelos` guardáramos el nombre completo de la aerolínea en cada fila, escribiríamos “United Air Lines Inc.” **336 776 veces**, y el día que hubiera que corregir una letra habría que corregirla en todas.

La solución es guardar cada cosa una sola vez, en su propia tabla, y conectarlas por una columna común. A un conjunto de tablas conectadas así se le llama **datos relacionados**, y es como funcionan todas las bases de datos del mundo.

En este capítulo trabajamos con cinco tablas del paquete **datos**, que juntas describen los vuelos que salieron de Nueva York en 2013:

- **vuelos** es la tabla central: una fila por vuelo.
- **aerolineas** traduce el código de dos letras al nombre completo de la compañía.
- **aeropuertos** da el nombre, la latitud y la longitud de cada aeropuerto.
- **aviones** describe cada avión: año de fabricación, modelo, número de asientos.
- **clima** da las condiciones del tiempo en cada aeropuerto de origen, hora por hora.

Y se conectan así:

- `vuelos$aerolinea` apunta a `aerolineas$aerolinea`
- `vuelos$codigoCola` apunta a `aviones$codigoCola`
- `vuelos$origen` y `vuelos$destino` apuntan a `aeropuertos$codigo_aeropuerto`
- `vuelos` se conecta con `clima` por **cinco** columnas a la vez: `anio`, `mes`, `dia`, `hora` y `origen`

Antes de escribir una sola línea de código, dibuja en papel las tablas y las flechas entre ellas. Los errores de unión casi siempre vienen de no tener claro cuál columna apunta a cuál.

```
library(tidyverse)
library(datos)
```

Mira las cinco tablas antes de unir nada. Fíjate sobre todo en qué columnas comparten:

```
vuelos # data set #1
```

```
## # A tibble: 336,776 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>          <int>          <int>          <dbl>
## 1  2013     1     1             517             515             2
## 2  2013     1     1             533             529             4
## 3  2013     1     1             542             540             2
## 4  2013     1     1             544             545            -1
## 5  2013     1     1             554             600            -6
## 6  2013     1     1             554             558            -4
## 7  2013     1     1             555             600            -5
## 8  2013     1     1             557             600            -3
## 9  2013     1     1             557             600            -3
```

```
## 10 2013      1      1          558          600          -2
## # i 336,766 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
names(vuelos)
```

```
## [1] "anio" "mes" "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea" "vuelo" "codigoCola"
## [13] "origen" "destino" "tiempo_vuelo"
## [16] "distancia" "hora" "minuto"
## [19] "fecha_hora"
```

aerolineas # data set #2

```
## # A tibble: 16 x 2
##   aerolinea nombre
##   <chr>         <chr>
## 1 9E           Endeavor Air Inc.
## 2 AA           American Airlines Inc.
## 3 AS           Alaska Airlines Inc.
## 4 B6           JetBlue Airways
## 5 DL           Delta Air Lines Inc.
## 6 EV           ExpressJet Airlines Inc.
## 7 F9           Frontier Airlines Inc.
## 8 FL           AirTran Airways Corporation
## 9 HA           Hawaiian Airlines Inc.
## 10 MQ          Envoy Air
## 11 OO          SkyWest Airlines Inc.
## 12 UA          United Air Lines Inc.
## 13 US          US Airways Inc.
## 14 VX          Virgin America
## 15 WN          Southwest Airlines Co.
## 16 YV          Mesa Airlines Inc.
```

```
names(aerolineas)
```

```
## [1] "aerolinea" "nombre"
```

aeropuertos

```
## # A tibble: 1,458 x 8
##   codigo_aeropuerto nombre  latitud longitud altura zona_horaria horario_verano
##   <chr>             <chr>    <dbl>   <dbl>   <dbl>    <dbl> <chr>
## 1 04G               Lansdo~   41.1    -80.6   1044      -5 A
```

```
## 2 06A Moton ~ 32.5 -85.7 264 -6 A
## 3 06C Schaum~ 42.0 -88.1 801 -6 A
## 4 06N Randal~ 41.4 -74.4 523 -5 A
## 5 09J Jekyll~ 31.1 -81.4 11 -5 A
## 6 0A9 Elizab~ 36.4 -82.2 1593 -5 A
## 7 0G6 Willia~ 41.5 -84.5 730 -5 A
## 8 0G7 Finger~ 42.9 -76.8 492 -5 A
## 9 OP2 Shoest~ 39.8 -76.6 1000 -5 U
## 10 OS9 Jeffer~ 48.1 -123. 108 -8 A
```

```
## # i 1,448 more rows
```

```
## # i 1 more variable: zona_horaria_iana <chr>
```

```
names(aeropuertos)
```

```
## [1] "codigo_aeropuerto" "nombre" "latitud"
## [4] "longitud" "altura" "zona_horaria"
## [7] "horario_verano" "zona_horaria_iana"
```

```
aviones
```

```
## # A tibble: 3,322 x 9
```

```
##   codigoCola anio tipo fabricante modelo motores asientos velocidad
##   <chr> <int> <chr> <chr> <chr> <int> <int> <int>
## 1 N10156 2004 Ala fija mult~ EMBRAER EMB-1~ 2 55 NA
## 2 N102UW 1998 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
## 3 N103US 1999 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
## 4 N104UW 1999 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
## 5 N10575 2002 Ala fija mult~ EMBRAER EMB-1~ 2 55 NA
## 6 N105UW 1999 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
## 7 N107US 1999 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
## 8 N108UW 1999 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
## 9 N109UW 1999 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
## 10 N110UW 1999 Ala fija mult~ AIRBUS IN~ A320~ 2 182 NA
```

```
## # i 3,312 more rows
```

```
## # i 1 more variable: tipo_motor <chr>
```

```
clima
```

```
## # A tibble: 26,115 x 15
```

```
##   origen anio mes dia hora temperatura punto_rocio humedad
##   <chr> <int> <int> <int> <int> <dbl> <dbl> <dbl>
## 1 EWR 2013 1 1 1 39.0 26.1 59.4
## 2 EWR 2013 1 1 2 39.0 27.0 61.6
## 3 EWR 2013 1 1 3 39.0 28.0 64.4
## 4 EWR 2013 1 1 4 39.9 28.0 62.2
## 5 EWR 2013 1 1 5 39.0 28.0 64.4
## 6 EWR 2013 1 1 6 37.9 28.0 67.2
## 7 EWR 2013 1 1 7 39.0 28.0 64.4
```



```
## 8 EWR      2013      1      1      8      39.9      28.0      62.2
## 9 EWR      2013      1      1      9      39.9      28.0      62.2
## 10 EWR     2013      1      1     10      41       28.0      59.6
## # i 26,105 more rows
## # i 7 more variables: direccion_viento <dbl>, velocidad_viento <dbl>,
## #   velocidad_rafaga <dbl>, precipitacion <dbl>, presion <dbl>,
## #   visibilidad <dbl>, fecha_hora <dtm>
```

vuelos

```
## # A tibble: 336,776 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>           <int>         <dbl>
## 1 2013     1     1           517             515             2
## 2 2013     1     1           533             529             4
## 3 2013     1     1           542             540             2
## 4 2013     1     1           544             545            -1
## 5 2013     1     1           554             600            -6
## 6 2013     1     1           554             558            -4
## 7 2013     1     1           555             600            -5
## 8 2013     1     1           557             600            -3
## 9 2013     1     1           557             600            -3
## 10 2013     1     1           558             600            -2
## # i 336,766 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

aviones

```
## # A tibble: 3,322 x 9
##   codigoCola anio tipo fabricante modelo motores asientos velocidad
##   <chr>      <int> <chr>      <chr>      <chr>      <int>      <int>      <int>
## 1 N10156    2004 Ala fija mult~ EMBRAER    EMB-1~      2        55        NA
## 2 N102UW    1998 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## 3 N103US    1999 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## 4 N104UW    1999 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## 5 N10575    2002 Ala fija mult~ EMBRAER    EMB-1~      2        55        NA
## 6 N105UW    1999 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## 7 N107US    1999 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## 8 N108UW    1999 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## 9 N109UW    1999 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## 10 N110UW   1999 Ala fija mult~ AIRBUS    IN~ A320--      2       182        NA
## # i 3,312 more rows
## # i 1 more variable: tipo_motor <chr>
```

```
aviones %>%
  count(codigoCola) %>%
  filter(n > 1)
```

```
## # A tibble: 0 x 2
## # i 2 variables: codigoCola <chr>, n <int>
```

20.3 Claves

Una **clave** es la columna, o el conjunto de columnas, que identifica una observación sin ambigüedad. Hay tres tipos, y conviene distinguirlos:

- Una **clave primaria** identifica una observación **en su propia tabla**. `codigoCola` es la clave primaria de `aviones`: cada avión aparece una sola vez.
- Una **clave foránea** identifica una observación **de otra tabla**. `codigoCola` en `vuelos` es una clave foránea: ahí se repite miles de veces, una por cada vuelo que hizo ese avión.
- Una **clave sustituta** es la que uno se inventa cuando la tabla no tiene ninguna, casi siempre un número de fila (`mutate(id = row_number())`).

A veces la clave es **una sola columna** y a veces hacen falta varias. `vuelos` no tiene clave primaria: ni el número de vuelo ni la fecha bastan, porque el mismo vuelo sale todos los días y varios vuelos salen a la misma hora.

20.3.1 Cómo se comprueba una clave

Hay una sola manera, y es la que aparece una y otra vez en los bloques de abajo: **contar y quedarse con lo que se repite**. Si la columna es de verdad una clave, el resultado tiene que estar **vacío**.

La receta para verificar una clave: `count()` seguido de `filter(n > 1)`.

- **tabla |> count(columna) |> filter(n > 1)**: si devuelve cero filas, esa columna es una clave.
- **Con varias columnas:** `count(a, b, c) |> filter(n > 1)`, que comprueba si la **combinación** es única.
- **Ojo con los NA:** una columna con valores faltantes **no** sirve como clave, porque un NA no identifica nada. Compruébalo aparte con `filter(is.na(columna))`.

Ese `filter(n > 1)` que devuelve una tabla vacía es una prueba superada, no un fallo. Si devuelve filas, ahí tienes tus duplicados.

Identify the keys in the following datasets

Lahman::Batting, babynames::babynames nasaweather::atmos fueleconomy::vehicles ggplot2::diamonds

```
head(Lahman::Batting)
```

```
##   playerID yearID stint teamID lgID  G AB R H X2B X3B HR RBI SB CS BB SO IBB
## 1 aardsda01  2004     1   SFN   NL 11  0 0 0    0  0  0  0  0  0  0  0  0
## 2 aardsda01  2006     1   CHN   NL 45  2 0 0    0  0  0  0  0  0  0  0  0
## 3 aardsda01  2007     1   CHA   AL 25  0 0 0    0  0  0  0  0  0  0  0  0
## 4 aardsda01  2008     1   BOS   AL 47  1 0 0    0  0  0  0  0  0  0  1  0
## 5 aardsda01  2009     1   SEA   AL 73  0 0 0    0  0  0  0  0  0  0  0  0
## 6 aardsda01  2010     1   SEA   AL 53  0 0 0    0  0  0  0  0  0  0  0  0
##   HBP SH SF GIDP
## 1   0  0  0    0
## 2   0  1  0    0
## 3   0  0  0    0
## 4   0  0  0    0
## 5   0  0  0    0
## 6   0  0  0    0
```

```
head(datos::bateadores)
```

```
##   id_jugador id_anio orden_equipos id_equipo id_liga juegos al_bate carreras
## 1 aardsda01  2004              1      SFN      NL    11      0      0
## 2 aardsda01  2006              1      CHN      NL    45      2      0
## 3 aardsda01  2007              1      CHA      AL    25      0      0
## 4 aardsda01  2008              1      BOS      AL    47      1      0
## 5 aardsda01  2009              1      SEA      AL    73      0      0
## 6 aardsda01  2010              1      SEA      AL    53      0      0
##   golpes dobles triples cuadrangulares carreras_empujadas bases_robadas
## 1      0      0      0              0              0              0
## 2      0      0      0              0              0              0
## 3      0      0      0              0              0              0
## 4      0      0      0              0              0              0
## 5      0      0      0              0              0              0
## 6      0      0      0              0              0              0
##   atrapado_robando base_bolas ponches base_intencional golpeado
## 1              0      0      0              0      0
## 2              0      0      0              0      0
## 3              0      0      0              0      0
## 4              0      0      1              0      0
## 5              0      0      0              0      0
## 6              0      0      0              0      0
##   toque_sacrificio elavado_sacrificio doble_matanza
## 1              0              0              0
## 2              1              0              0
## 3              0              0              0
```

```
## 4      0      0      0
## 5      0      0      0
## 6      0      0      0
```

```
Lahman::Batting %>%
  count(playerID) %>%
  filter(n>1) |>
  head()
```

```
##   playerID  n
## 1 aardsda01  9
## 2 aaronha01 23
## 3 aaronto01  7
## 4 aasedo01 13
## 5 abadan01  3
## 6 abadfe01 12
```

```
bateadores %>%
count(id_jugador,id_anio,orden_equipos) %>%
filter(n > 1) |>
  head()
```

```
## [1] id_jugador    id_anio      orden_equipos n
## <0 rows> (or 0-length row.names)
```

```
Lahman::Batting %>%
  count(playerID, yearID, stint) %>%
  filter(n>1) |>
  head()
```

```
## [1] playerID yearID  stint    n
## <0 rows> (or 0-length row.names)
```

```
#babynames::babynames
```

```
babynames::babynames %>%
  count(year, name, sex) %>%
  filter(n>1)
```

```
## # A tibble: 0 x 4
## # i 4 variables: year <dbl>, name <chr>, sex <chr>, n <int>
```

¿Cuál fue el primer año que su nombre aparece en la base de datos?

```
babynames::babynames %>%
  filter(name == "Abimelys")
```

```
## # A tibble: 0 x 5
## # i 5 variables: year <dbl>, sex <chr>, name <chr>, n <int>, prop <dbl>
```

```
max(babynames::babynames$year)
```

```
## [1] 2017
```

```
nasaweather::atmos
```

```
## # A tibble: 41,472 x 11
```

```
##       lat long year month surftemp temp pressure ozone cloudlow cloudmid
##   <dbl> <dbl> <int> <int>   <dbl> <dbl>   <dbl> <dbl>   <dbl>   <dbl>
## 1  36.2 -114. 1995     1    273.  272.    835  304     7.5    34.5
## 2  33.7 -114. 1995     1    280.  282.    940  304    11.5    32.5
## 3  31.2 -114. 1995     1    285.  285.    960  298    16.5     26
## 4  28.7 -114. 1995     1    289.  291.    990  276    20.5    14.5
## 5  26.2 -114. 1995     1    292.  293.   1000  274     26    10.5
## 6  23.7 -114. 1995     1    294.  294.   1000  264     30     9.5
## 7  21.2 -114. 1995     1    295.  295.   1000  258    29.5     11
## 8  18.7 -114. 1995     1    298.  297.   1000  252    26.5    17.5
## 9  16.2 -114. 1995     1    300.  298.   1000  250    27.5    18.5
## 10 13.7 -114. 1995     1    300.  299.   1000  250     26    16.5
```

```
## # i 41,462 more rows
```

```
## # i 1 more variable: cloudhigh <dbl>
```

```
nasaweather::atmos %>%
```

```
  count(lat, long, year, month) %>%
```

```
  filter(n>1)
```

```
## # A tibble: 0 x 5
```

```
## # i 5 variables: lat <dbl>, long <dbl>, year <int>, month <int>, n <int>
```

```
fueleconomy::vehicles
```

```
## # A tibble: 33,442 x 12
```

```
##       id make model year class trans drive cyl displ fuel hwy cty
##   <dbl> <chr> <chr>   <dbl> <chr> <chr> <chr> <dbl> <dbl> <chr> <dbl> <dbl>
## 1 13309 Acura 2.2CL/3.0CL 1997 Subc~ Auto~ Fron~ 4 2.2 Regu~ 26 20
## 2 13310 Acura 2.2CL/3.0CL 1997 Subc~ Manu~ Fron~ 4 2.2 Regu~ 28 22
## 3 13311 Acura 2.2CL/3.0CL 1997 Subc~ Auto~ Fron~ 6 3 Regu~ 26 18
## 4 14038 Acura 2.3CL/3.0CL 1998 Subc~ Auto~ Fron~ 4 2.3 Regu~ 27 19
## 5 14039 Acura 2.3CL/3.0CL 1998 Subc~ Manu~ Fron~ 4 2.3 Regu~ 29 21
## 6 14040 Acura 2.3CL/3.0CL 1998 Subc~ Auto~ Fron~ 6 3 Regu~ 26 17
## 7 14834 Acura 2.3CL/3.0CL 1999 Subc~ Auto~ Fron~ 4 2.3 Regu~ 27 20
## 8 14835 Acura 2.3CL/3.0CL 1999 Subc~ Manu~ Fron~ 4 2.3 Regu~ 29 21
## 9 14836 Acura 2.3CL/3.0CL 1999 Subc~ Auto~ Fron~ 6 3 Regu~ 26 17
## 10 11789 Acura 2.5TL 1995 Comp~ Auto~ Fron~ 5 2.5 Prem~ 23 18
```

```
## # i 33,432 more rows
```

```
fueleconomy::vehicles %>%
```

```
  count(id) %>%
```

```

filter(n>1)

## # A tibble: 0 x 2
## # i 2 variables: id <dbl>, n <int>

ggplot2::diamonds

## # A tibble: 53,940 x 10
##   carat cut      color clarity depth table price     x     y     z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Ideal    E     SI2     61.5   55   326  3.95  3.98  2.43
## 2  0.21 Premium  E     SI1     59.8   61   326  3.89  3.84  2.31
## 3  0.23 Good    E     VS1     56.9   65   327  4.05  4.07  2.31
## 4  0.29 Premium  I     VS2     62.4   58   334  4.2   4.23  2.63
## 5  0.31 Good    J     SI2     63.3   58   335  4.34  4.35  2.75
## 6  0.24 Very Good J     VVS2     62.8   57   336  3.94  3.96  2.48
## 7  0.24 Very Good I     VVS1     62.3   57   336  3.95  3.98  2.47
## 8  0.26 Very Good H     SI1     61.9   55   337  4.07  4.11  2.53
## 9  0.22 Fair    E     VS2     65.1   61   337  3.87  3.78  2.49
## 10 0.23 Very Good H     VS1     59.4   61   338  4     4.05  2.39
## # i 53,930 more rows

ggplot2::diamonds %>%
  count(carat, cut, color, clarity, depth, price, x, y, z) %>%
  filter(n>1)

## # A tibble: 283 x 10
##   carat cut      color clarity depth price     x     y     z     n
##   <dbl> <ord>    <ord> <ord>    <dbl> <int> <dbl> <dbl> <dbl> <int>
## 1  0.23 Very Good E     VVS2     60.9   530  3.96  3.99  2.42     2
## 2  0.24 Very Good E     VVS1     61.7   485  3.95  3.99  2.45     2
## 3  0.24 Very Good G     VVS2     62     449  4     4.03  2.49     2
## 4  0.26 Ideal    G     SI1     62     394  4.08  4.11  2.54     2
## 5  0.3   Good    J     VS1     63.4   394  4.23  4.26  2.69     2
## 6  0.3   Very Good D     SI1     62.5   552  4.26  4.28  2.67     2
## 7  0.3   Very Good E     SI1     62.9   526  4.25  4.3   2.69     2
## 8  0.3   Very Good E     VVS2     61.4   789  4.25  4.28  2.62     2
## 9  0.3   Very Good G     VS2     63     526  4.29  4.31  2.71     2
## 10 0.3   Very Good H     SI1     62.9   421  4.28  4.31  2.7     2
## # i 273 more rows

```

Draw a diagram illustrating the connections between the Batting, People, and Salaries tables in the Lahman package. Draw another diagram that shows the relationship between People, Managers, AwardsManagers.

Dibuja un diagrama que ilustre las conexiones entre las tablas bateadores, personas y salarios incluidas en el paquete datos. Dibuja otro diagrama que muestre la relación entre personas, dirigentes y premios_dirigentes.

```
names(Lahman::Batting)
```

```
## [1] "playerID" "yearID" "stint" "teamID" "lgID" "G"
## [7] "AB" "R" "H" "X2B" "X3B" "HR"
## [13] "RBI" "SB" "CS" "BB" "SO" "IBB"
## [19] "HBP" "SH" "SF" "GIDP"
```

```
names(Lahman::Salaries)
```

```
## [1] "yearID" "teamID" "lgID" "playerID" "salary"
```

```
names(Lahman::People)
```

```
## [1] "playerID" "birthYear" "birthMonth" "birthDay" "birthCity"
## [6] "birthCountry" "birthState" "deathYear" "deathMonth" "deathDay"
## [11] "deathCountry" "deathState" "deathCity" "nameFirst" "nameLast"
## [16] "nameGiven" "weight" "height" "bats" "throws"
## [21] "debut" "bbrefID" "finalGame" "retroID" "deathDate"
## [26] "birthDate"
```

20.4 Uniones de transformaciones

Una **unión de transformación** (*mutating join*) añade columnas de una tabla a otra, emparejando las filas por la clave. El nombre viene de que hace lo mismo que `mutate()`: la tabla sale con más columnas que entró.

- mutating join

`left_join()`, `inner_join()`, `full_join()`, `right_join()` (dplyr): combinar tablas.

- **Qué hacen:** unen dos tablas emparejando filas por una columna **clave** (`by =`).
- `left_join(x, y)`: conserva todas las filas de `x`; `inner_join` solo las coincidencias; `full_join` todas las de ambas.
- **De filtro:** `semi_join()` y `anti_join()` no añaden columnas, solo filtran filas de `x`.

```
vuelos2 <- vuelos %>%
```

```
  dplyr::select(anio, dia, hora, origen, destino, codigoCola, aerolinea)
vuelos2
```

```
## # A tibble: 336,776 x 7
```

```
##   anio dia hora origen destino codigoCola aerolinea
##   <int> <int> <dbl> <chr> <chr> <chr> <chr>
## 1 2013 1 5 EWR IAH N14228 UA
## 2 2013 1 5 LGA IAH N24211 UA
```

```
## 3 2013 1 5 JFK MIA N619AA AA
## 4 2013 1 5 JFK BQN N804JB B6
## 5 2013 1 6 LGA ATL N668DN DL
## 6 2013 1 5 EWR ORD N39463 UA
## 7 2013 1 6 EWR FLL N516JB B6
## 8 2013 1 6 LGA IAD N829AS EV
## 9 2013 1 6 JFK MCO N593JB B6
## 10 2013 1 6 LGA ORD N3ALAA AA
## # i 336,766 more rows
```

```
names(aerolineas)
```

```
## [1] "aerolinea" "nombre"
```

```
aerolineas
```

```
## # A tibble: 16 x 2
##   aerolinea nombre
##   <chr>      <chr>
## 1 9E      Endeavor Air Inc.
## 2 AA      American Airlines Inc.
## 3 AS      Alaska Airlines Inc.
## 4 B6      JetBlue Airways
## 5 DL      Delta Air Lines Inc.
## 6 EV      ExpressJet Airlines Inc.
## 7 F9      Frontier Airlines Inc.
## 8 FL      AirTran Airways Corporation
## 9 HA      Hawaiian Airlines Inc.
## 10 MQ     Envoy Air
## 11 OO     SkyWest Airlines Inc.
## 12 UA     United Air Lines Inc.
## 13 US     US Airways Inc.
## 14 VX     Virgin America
## 15 WN     Southwest Airlines Co.
## 16 YV     Mesa Airlines Inc.
```

```
vuelos2 %>%
  dplyr::select(-origen, -destino) %>%
  left_join(aerolineas, by = "aerolinea")
```

```
## # A tibble: 336,776 x 6
##   anio dia hora codigoCola aerolinea nombre
##   <int> <int> <dbl> <chr>      <chr>      <chr>
## 1 2013 1 5 N14228 UA United Air Lines Inc.
## 2 2013 1 5 N24211 UA United Air Lines Inc.
## 3 2013 1 5 N619AA AA American Airlines Inc.
## 4 2013 1 5 N804JB B6 JetBlue Airways
## 5 2013 1 6 N668DN DL Delta Air Lines Inc.
```



```
## 6 2013      1      5 N39463      UA      United Air Lines Inc.
## 7 2013      1      6 N516JB      B6      JetBlue Airways
## 8 2013      1      6 N829AS      EV      ExpressJet Airlines Inc.
## 9 2013      1      6 N593JB      B6      JetBlue Airways
## 10 2013     1      6 N3ALAA      AA      American Airlines Inc.
## # i 336,766 more rows
```

Ahora vamos a entender las cuatro uniones con dos tablas mínimas, de tres filas cada una. Fíjate en que la clave 1 y la 2 están en las dos, pero la 3 solo está en x y la 4 solo está en y. Todo el capítulo se resume en qué le pasa a esas dos filas sueltas.

```
x <- tribble(
  ~key, ~val_x,
  1, "x1",
  2, "x2",
  3, "x3"
)
y <- tribble(
  ~key, ~val_y,
  1, "y1",
  2, "y2",
  4, "y3"
)
x
```

```
## # A tibble: 3 x 2
##       key val_x
##   <dbl> <chr>
## 1     1  x1
## 2     2  x2
## 3     3  x3
y
```

```
## # A tibble: 3 x 2
##       key val_y
##   <dbl> <chr>
## 1     1  y1
## 2     2  y2
## 3     4  y3
```

`inner_join()` se queda solo con lo que coincide. La fila 3 de x y la fila 4 de y desaparecen, así que el resultado tiene dos filas:

```
x %>%
  inner_join(y, by="key")
```

```
## # A tibble: 2 x 3
##   key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
```

`inner_join()` **pierde filas sin avisar**. Es la causa número uno de análisis equivocados con datos relacionados: uno une dos tablas, sigue trabajando, y nunca se entera de que a mitad de camino se quedó sin la tercera parte de sus datos. Compara siempre `nrow()` antes y después de unir.

Las otras tres son **uniones exteriores**: conservan filas que no tienen pareja y rellenan con NA lo que falta. Se diferencian solo en cuál tabla es la que manda:

- `left_join(x, y)` conserva todas las filas de `x`. Es la que se usa el 90 % de las veces, porque casi siempre uno tiene una tabla principal y quiere añadirle información de una tabla de consulta.
- `right_join(x, y)` conserva todas las de `y`. Es lo mismo que `left_join(y, x)` con las columnas en otro orden.
- `full_join(x, y)` conserva todas las de las dos.

```
#left_join
```

```
x %>%
  left_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
```

```
#right_join
```

```
x %>%
  right_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 4 <NA> y3
```

```
#full_join
```

```
x %>%
  full_join(y, by="key")
```

```
## # A tibble: 4 x 3
##   key val_x val_y
##   <dbl> <chr> <chr>
## 1     1    x1    y1
## 2     2    x2    y2
## 3     3    x3   <NA>
## 4     4   <NA>   y3
```

El mismo ejemplo con nombres y edades, que se lee mejor. Fíjate en Orlando, que no tiene edad, y en la persona de 22 años, que no tiene nombre: son la fila 3 y la fila 4 de antes.

```
x <- tribble(
  ~key, ~nombre_x,
  1, "Juan",
  2, "Maria",
  3, "Orlando"
)
y <- tribble(
  ~key, ~edad_y,
  1, 20,
  2, 21,
  4, 22
)
x
```

```
## # A tibble: 3 x 2
##   key nombre_x
##   <dbl> <chr>
## 1     1    Juan
## 2     2    Maria
## 3     3    Orlando
y
```

```
## # A tibble: 3 x 2
##   key edad_y
##   <dbl> <dbl>
## 1     1     20
## 2     2     21
## 3     4     22
```

```
x %>%
  full_join(y, by="key")
```

```
## # A tibble: 4 x 3
##   key nombre_x edad_y
##   <dbl> <chr>    <dbl>
```

```
## 1      1 Juan      20
## 2      2 Maria     21
## 3      3 Orlando   NA
## 4      4 <NA>      22
```

```
x %>%
  inner_join(y, by="key")
```

```
## # A tibble: 2 x 3
##   key nombre_x edad_y
##   <dbl> <chr>   <dbl>
## 1      1 Juan      20
## 2      2 Maria     21
```

```
x %>%
  left_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key nombre_x edad_y
##   <dbl> <chr>   <dbl>
## 1      1 Juan      20
## 2      2 Maria     21
## 3      3 Orlando   NA
```

```
x %>%
  right_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key nombre_x edad_y
##   <dbl> <chr>   <dbl>
## 1      1 Juan      20
## 2      2 Maria     21
## 3      4 <NA>      22
```

```
x %>%
  full_join(y, by="key")
```

```
## # A tibble: 4 x 3
##   key nombre_x edad_y
##   <dbl> <chr>   <dbl>
## 1      1 Juan      20
## 2      2 Maria     21
## 3      3 Orlando   NA
## 4      4 <NA>      22
```

```
x %>%
  inner_join(y, by="key")
```

```
## # A tibble: 2 x 3
```

```
##      key nombre_x edad_y
##      <dbl> <chr>      <dbl>
## 1      1 Juan         20
## 2      2 Maria        21
```

20.5 Las uniones con los datos de verdad

Con las tablas de vuelos, la unión típica es exactamente la del principio: añadirle a `vuelos2` el nombre completo de la aerolínea, que vive en otra tabla.

```
vuelos2 %>%
  dplyr::select(-origen, -destino) %>%
  left_join(aerolineas, by = "aerolinea")
```

```
## # A tibble: 336,776 x 6
##   anio dia hora codigoCola aerolinea nombre
##   <int> <int> <dbl> <chr>      <chr>      <chr>
## 1 2013     1     5 N14228      UA      United Air Lines Inc.
## 2 2013     1     5 N24211      UA      United Air Lines Inc.
## 3 2013     1     5 N619AA      AA      American Airlines Inc.
## 4 2013     1     5 N804JB      B6      JetBlue Airways
## 5 2013     1     6 N668DN      DL      Delta Air Lines Inc.
## 6 2013     1     5 N39463      UA      United Air Lines Inc.
## 7 2013     1     6 N516JB      B6      JetBlue Airways
## 8 2013     1     6 N829AS      EV      ExpressJet Airlines Inc.
## 9 2013     1     6 N593JB      B6      JetBlue Airways
## 10 2013     1     6 N3ALAA      AA      American Airlines Inc.
## # i 336,766 more rows
```

```
names(vuelos2)
```

```
## [1] "anio"      "dia"      "hora"      "origen"    "destino"
## [6] "codigoCola" "aerolinea"
```

```
names(clima)
```

```
## [1] "origen"      "anio"      "mes"      "dia"
## [5] "hora"      "temperatura" "punto_rocio" "humedad"
## [9] "direccion_viento" "velocidad_viento" "velocidad_rafaga" "precipitacion"
## [13] "presion"      "visibilidad" "fecha_hora"
```

Cuando dos tablas comparten **varias** columnas con el mismo nombre y no se dice nada, `dplyr` las usa **todas** como clave. Eso se llama unión natural, y es cómodo pero peligroso: basta con que una de las tablas cambie de columnas para que la unión cambie de significado sin que nadie lo note. El segundo bloque hace lo mismo pero diciendo explícitamente cuáles columnas usar, que

es lo recomendable:

```
vuelos2 %>%
left_join(clima)
```

```
## # A tibble: 3,974,448 x 18
##   anio dia hora origen destino codigoCola aerolinea mes temperatura
##   <int> <int> <dbl> <chr> <chr> <chr> <chr> <int> <dbl>
## 1 2013 1 5 EWR IAH N14228 UA 1 39.0
## 2 2013 1 5 EWR IAH N14228 UA 2 28.0
## 3 2013 1 5 EWR IAH N14228 UA 3 35.1
## 4 2013 1 5 EWR IAH N14228 UA 4 45.0
## 5 2013 1 5 EWR IAH N14228 UA 5 44.1
## 6 2013 1 5 EWR IAH N14228 UA 6 72.0
## 7 2013 1 5 EWR IAH N14228 UA 7 75.0
## 8 2013 1 5 EWR IAH N14228 UA 8 72.0
## 9 2013 1 5 EWR IAH N14228 UA 9 73.9
## 10 2013 1 5 EWR IAH N14228 UA 10 53.1
## # i 3,974,438 more rows
## # i 9 more variables: punto_rocio <dbl>, humedad <dbl>, direccion_viento <dbl>,
## # velocidad_viento <dbl>, velocidad_rafaga <dbl>, precipitacion <dbl>,
## # presion <dbl>, visibilidad <dbl>, fecha_hora <dtm>

#> Joining, by = c("anio", "mes", "dia", "hora", "origen")
vuelos2 %>%
left_join(clima, by = c("anio", "dia", "hora", "origen"))
```

```
## # A tibble: 3,974,448 x 18
##   anio dia hora origen destino codigoCola aerolinea mes temperatura
##   <int> <int> <dbl> <chr> <chr> <chr> <chr> <int> <dbl>
## 1 2013 1 5 EWR IAH N14228 UA 1 39.0
## 2 2013 1 5 EWR IAH N14228 UA 2 28.0
## 3 2013 1 5 EWR IAH N14228 UA 3 35.1
## 4 2013 1 5 EWR IAH N14228 UA 4 45.0
## 5 2013 1 5 EWR IAH N14228 UA 5 44.1
## 6 2013 1 5 EWR IAH N14228 UA 6 72.0
## 7 2013 1 5 EWR IAH N14228 UA 7 75.0
## 8 2013 1 5 EWR IAH N14228 UA 8 72.0
## 9 2013 1 5 EWR IAH N14228 UA 9 73.9
## 10 2013 1 5 EWR IAH N14228 UA 10 53.1
## # i 3,974,438 more rows
## # i 9 more variables: punto_rocio <dbl>, humedad <dbl>, direccion_viento <dbl>,
## # velocidad_viento <dbl>, velocidad_rafaga <dbl>, precipitacion <dbl>,
## # presion <dbl>, visibilidad <dbl>, fecha_hora <dtm>
```

Y cuando las columnas se llaman **distinto** en cada tabla, se dice con `c("nombre_en_x" = "nombre_en_y")`. Aquí el aeropuerto de origen se llama

origen en vuelos2 y codigo_aeropuerto en aeropuertos:

```
names(vuelos2)
```

```
## [1] "anio"      "dia"      "hora"      "origen"    "destino"
## [6] "codigoCola" "aerolinea"
```

```
names(aeropuertos)
```

```
## [1] "codigo_aeropuerto" "nombre"      "latitud"
## [4] "longitud"          "altura"      "zona_horaria"
## [7] "horario_verano"    "zona_horaria_iana"
```

```
head(vuelos2, n=3)
```

```
## # A tibble: 3 x 7
##   anio dia hora origen destino codigoCola aerolinea
##   <int> <int> <dbl> <chr> <chr> <chr> <chr>
## 1 2013   1   5 EWR IAH N14228 UA
## 2 2013   1   5 LGA IAH N24211 UA
## 3 2013   1   5 JFK MIA N619AA AA
```

```
aeropuertos
```

```
## # A tibble: 1,458 x 8
##   codigo_aeropuerto nombre latitud longitud altura zona_horaria horario_verano
##   <chr> <chr> <dbl> <dbl> <dbl> <dbl> <chr>
## 1 04G Lansdo~ 41.1 -80.6 1044 -5 A
## 2 06A Moton ~ 32.5 -85.7 264 -6 A
## 3 06C Schaum~ 42.0 -88.1 801 -6 A
## 4 06N Randal~ 41.4 -74.4 523 -5 A
## 5 09J Jekyll~ 31.1 -81.4 11 -5 A
## 6 0A9 Elizab~ 36.4 -82.2 1593 -5 A
## 7 0G6 Willia~ 41.5 -84.5 730 -5 A
## 8 0G7 Finger~ 42.9 -76.8 492 -5 A
## 9 0P2 Shoest~ 39.8 -76.6 1000 -5 U
## 10 OS9 Jeffer~ 48.1 -123. 108 -8 A
```

```
## # i 1,448 more rows
```

```
## # i 1 more variable: zona_horaria_iana <chr>
```

```
vuelos2 %>%
```

```
  right_join(aeropuertos, c("origen" = "codigo_aeropuerto"))
```

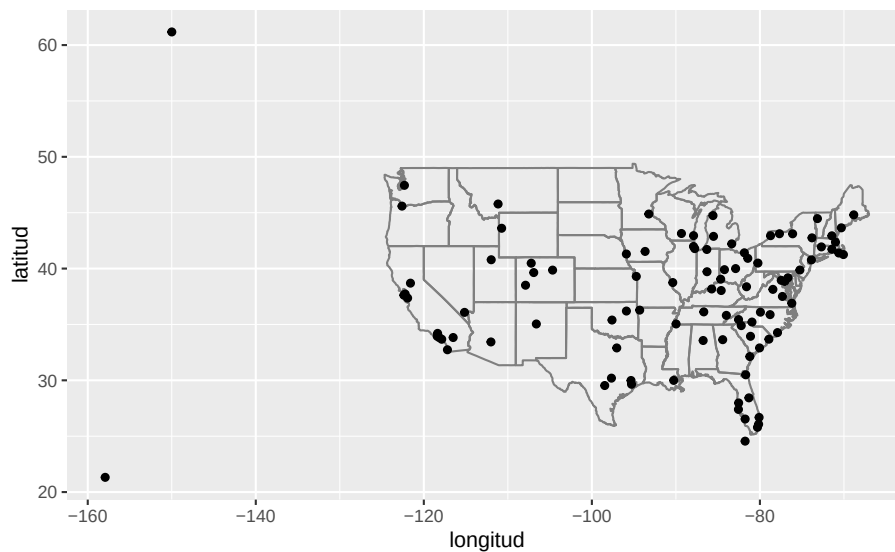
```
## # A tibble: 338,231 x 14
##   anio dia hora origen destino codigoCola aerolinea nombre latitud
##   <int> <int> <dbl> <chr> <chr> <chr> <chr> <chr> <dbl>
## 1 2013   1   5 EWR IAH N14228 UA Newark Libert~ 40.7
## 2 2013   1   5 LGA IAH N24211 UA La Guardia 40.8
## 3 2013   1   5 JFK MIA N619AA AA John F Kenned~ 40.6
```

```
## 4 2013 1 5 JFK BQN N804JB B6 John F Kenned~ 40.6
## 5 2013 1 6 LGA ATL N668DN DL La Guardia 40.8
## 6 2013 1 5 EWR ORD N39463 UA Newark Libert~ 40.7
## 7 2013 1 6 EWR FLL N516JB B6 Newark Libert~ 40.7
## 8 2013 1 6 LGA IAD N829AS EV La Guardia 40.8
## 9 2013 1 6 JFK MCO N593JB B6 John F Kenned~ 40.6
## 10 2013 1 6 LGA ORD N3ALAA AA La Guardia 40.8
## # i 338,221 more rows
## # i 5 more variables: longitud <dbl>, altura <dbl>, zona_horaria <dbl>,
## # horario_verano <chr>, zona_horaria_iana <chr>
```

Un ejemplo de para qué sirve todo esto: quedarse solo con los aeropuertos que de verdad reciben vuelos, y dibujarlos en un mapa. El `semi_join()` filtra sin añadir columnas, que es justo lo que hace falta aquí.

```
library(maps)

aeropuertos %>%
  semi_join(vuelos, c("codigo_aeropuerto" = "destino")) %>%
  ggplot(aes(longitud, latitud)) +
  borders("state") +
  geom_point() +
  coord_quickmap()
```



- `mutate()`
- `select()`

- `left_join()`
 - `match()`
-

20.6 Entendiendo las uniones

Todas las uniones funcionan igual por dentro. Imagina las dos tablas una al lado de la otra: para cada fila de la izquierda, R busca en la derecha las filas cuya clave coincida, y pega las columnas. Lo único que cambia entre las cinco funciones es **qué hacer con las filas que no encontraron pareja**.

20.6.1 Unión interiores

`inner_join()` es la unión interior: solo sobrevive lo que aparece en las dos tablas. Es la más limpia de entender y la más fácil de usar mal, porque el resultado nunca dice cuántas filas se quedaron por el camino.

Úsala cuando la ausencia de pareja signifique de verdad que esa fila no debe seguir en el análisis, y comprueba siempre cuántas filas perdiste.

20.6.2 Unión exteriores

Las tres uniones exteriores (`left_join()`, `right_join()`, `full_join()`) conservan filas sin pareja y ponen NA en las columnas que no pudieron llenar.

La regla práctica: **empieza siempre por `left_join()`**. Si tu tabla principal tiene 234 filas, después de un `left_join()` sigue teniendo 234 (salvo por el problema de la sección siguiente), y los NA que aparezcan te dicen exactamente qué no encontró pareja. Es la unión que no pierde datos y que además te informa.

Los NA que salen de un `left_join()` no son un fallo: son el resultado más informativo de la operación. Cuéntalos con `filter(is.na(columna_nueva))` y mira qué tienen en común. Casi siempre revelan un problema en la tabla de consulta: una categoría que se te olvidó, un código escrito de dos maneras, un espacio de más.

20.6.3 Claves duplicadas

Cuando la clave está duplicada en ambas tablas, la unión genera todas las combinaciones posibles (producto cartesiano) y el resultado tendrá más filas de las esperadas. Verifica con `count(clave) %>% filter(n > 1)` antes de unir.

Conviene distinguir dos situaciones, porque una es normal y la otra es un error.

Duplicada en una sola tabla: normal. Es el caso de `vuelos` y `aerolineas`: la aerolínea UA aparece miles de veces en `vuelos` y una sola vez en `aerolineas`.

El resultado tiene tantas filas como **vuelos**, y el nombre de la compañía se repite. Es exactamente lo que uno quiere.

Duplicada en las dos tablas: casi siempre un error. Si la clave se repite dos veces a cada lado, esa fila produce cuatro filas en el resultado. La tabla crece sin que nadie lo pida y los promedios que se calculen después estarán mal.

```
a <- tribble(
  ~clave, ~valor_a,
  1, "a1",
  1, "a2"
)

b <- tribble(
  ~clave, ~valor_b,
  1, "b1",
  1, "b2"
)

left_join(a, b, by = "clave")  # 4 filas, no 2
```

```
## # A tibble: 4 x 3
##   clave valor_a valor_b
##   <dbl> <chr>   <chr>
## 1     1 a1      b1
## 2     1 a1      b2
## 3     1 a2      b1
## 4     1 a2      b2
```

El hábito que evita el 90 % de estos problemas: después de **cada** unión, comprueba `nrow()`. Si cambió y no esperabas que cambiara, para y averigua por qué antes de seguir.

20.7 Definiendo las columnas clave

El argumento `by` tiene cuatro formas, y vale la pena conocerlas todas:

Forma	Qué hace
<code>by = NULL</code> (o no ponerlo)	Unión natural : usa todas las columnas con el mismo nombre en las dos tablas
<code>by = "clave"</code>	Usa solo esa columna
<code>by = c("a", "b")</code>	Usa las dos columnas a la vez

```
by = c("x" = "y")
```

La columna se llama `x` en la primera
tabla y `y` en la segunda

La unión natural (`by = NULL`) es cómoda para explorar y mala idea en un script que vas a guardar. `vuelos` y `clima` comparten `año`, `mes`, `día`, `hora` y `origen`, pero también podrían compartir mañana una columna nueva, y entonces la unión cambiaría de significado sin que tú tocaras nada. En un análisis serio, escribe siempre el `by` completo.

20.8 Otras implementaciones

- `merge()`

`merge()` es la función de base R que hace lo mismo. Funciona, pero es más lenta, tiene los argumentos al revés de lo que uno espera, y no distingue con claridad las cuatro uniones:

dplyr	<code>merge()</code> de base R
<code>inner_join(x, y)</code>	<code>merge(x, y)</code>
<code>left_join(x, y)</code>	<code>merge(x, y, all.x = TRUE)</code>
<code>right_join(x, y)</code>	<code>merge(x, y, all.y = TRUE)</code>
<code>full_join(x, y)</code>	<code>merge(x, y, all = TRUE)</code>

Los nombres de `dplyr` dicen lo que hacen; los argumentos de `merge()` hay que recordarlos. Si te encuentras un `merge()` en código ajeno, esta tabla te dice cuál unión era.

20.9 Uniones de filtro

Las uniones de filtro (`semi_join()` y `anti_join()`) nunca añaden columnas nuevas: solo conservan o descartan filas de la tabla de la izquierda según coincidan con la de la derecha.

- `semi_join()`
- `anti_join()`

`semi_join()` y `anti_join()` (`dplyr`): filtrar una tabla usando otra.

- `semi_join(x, y)`: conserva las filas de `x` que **sí** tienen pareja en `y`.
- `anti_join(x, y)`: conserva las filas de `x` que **no** la tienen.

- **Ninguna de las dos añade columnas**, y ninguna duplica filas aunque la clave esté repetida en y. Por eso son más seguras que un `inner_join()` cuando lo único que quieres es filtrar.

De las dos, la que más se usa en la práctica es `anti_join()`, y no para filtrar sino para **diagnosticar**: contesta la pregunta “¿qué hay en mi tabla que no encontré pareja?”. Es lo primero que hay que correr cuando un `left_join()` devuelve más NA de los esperados.

```
vuelos |>
  anti_join(aviones, by = "codigoCola") |>
  count(aerolinea, sort = TRUE)
```

```
## # A tibble: 10 x 2
##   aerolinea      n
##   <chr>      <int>
## 1 MQ        25397
## 2 AA        22558
## 3 UA         1693
## 4 9E         1044
## 5 B6          830
## 6 US          699
## 7 FL          187
## 8 DL          110
## 9 F9           50
## 10 WN           38
```

Los vuelos que no encontraron su avión no están repartidos al azar entre las aerolíneas: se concentran en unas pocas. Eso no es un error de R, es un dato sobre cómo se recogió la información, y es justo el tipo de hallazgo que un `inner_join()` habría borrado en silencio.

20.10 Problemas con las uniones

Tres cosas salen mal casi siempre, y las tres se detectan antes de unir:

1. **La clave no es única.** Comprueba las dos tablas con `count(clave) |> filter(n > 1)`. Si la clave se repite en las dos, la unión va a multiplicar filas.
2. **La clave tiene valores faltantes.** Un NA no empareja con nada, ni siquiera con otro NA. Búscalos con `filter(is.na(clave))` antes de unir.
3. **La clave está escrita de dos maneras.** Es el problema más común y el más invisible: "SJU " con un espacio al final no es "SJU", "Carolina" no es "CAROLINA", y un código leído como número perdió sus ceros de delante. `anti_join()` los saca a la luz enseguida.

La rutina completa, en cuatro líneas, antes de cualquier unión sería: comprobar la clave en la tabla A, comprobarla en la tabla B, correr el `anti_join()` en los dos sentidos para ver qué no empareja, y solo entonces unir. Cuesta un minuto y ahorra tardes enteras.

20.11 Ejercicios del libro R4DS

Hacer los ejercicios de las siguientes secciones del libro en español:

- sección 13.2.1 (los datos de vuelos),
 - sección 13.3.1 (claves),
 - sección 13.4.6 (uniones de transformación),
 - sección 13.5.1 (uniones de filtro).
-

20.12 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan `millas` y `países` del paquete `datos`, más una tabla de consulta que vas a escribir tú, no `vuelos` ni sus tablas acompañantes, que son las del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_20_datos-relacionados.Rmd`.

20.12.1 Ejercicio 1. Encontrar la clave

Datos: `países`. Averigua con `count()` y `filter(n > 1)` si `pais` por sí solo es una clave, y si la combinación de `pais` y `anio` lo es.

Verificación: la primera prueba devuelve 142 filas repetidas (una por país, con `n` igual a 12); la segunda devuelve una tabla **vacía**.

En palabras: una tabla vacía es la prueba superada. Explica por qué, y di cuál es entonces la clave primaria de `países`.

Escribe tu código aquí

20.12.2 Ejercicio 2. Tu propia tabla de consulta

Averigua cuántos fabricantes distintos hay en `millas` con `n_distinct()`. Después escribe con `tribble()` una tabla de consulta llamada `origen_fabricante` con dos columnas, `fabricante` y `pais_origen`, pero solo para la mitad de los fabricantes. Únela a `millas` con `left_join()`.

Verificación: el resultado tiene las mismas **234** filas que `millas`, y una columna nueva con `NA` en las filas de los fabricantes que no pusiste.

En palabras: los `NA` no son un error aquí. ¿Qué te están diciendo? ¿Por qué `left_join()` es la unión correcta para añadir una tabla de consulta?

```
# Escribe tu código aquí
```

20.12.3 Ejercicio 3. `left_join()` frente a `inner_join()`

Repite la unión del ejercicio anterior con `inner_join()` y compara `nrow()` de los dos resultados.

Verificación: el `inner_join()` devuelve **menos** de 234 filas; el `left_join()` devuelve exactamente 234.

En palabras: el `inner_join()` no dio ningún error ni ninguna advertencia. Explica por qué eso lo hace más peligroso que un error, y cuál costumbre te protege.

```
# Escribe tu código aquí
```

20.12.4 Ejercicio 4. `anti_join()` para diagnosticar

Usa `anti_join()` para averiguar **cuáles** fabricantes de `millas` no están en tu tabla de consulta, y cuántos vehículos representan.

Verificación: la lista tiene que coincidir exactamente con los fabricantes que dejaste fuera a propósito, y el número de vehículos tiene que ser igual a la diferencia entre las dos uniones del Ejercicio 3.

En palabras: `anti_join()` no añade ninguna columna. ¿Para qué sirve entonces? Describe una situación de tu propio trabajo donde lo usarías.

```
# Escribe tu código aquí
```

20.12.5 Ejercicio 5. La clave duplicada

Añade a tu tabla de consulta una **segunda fila** para un fabricante que ya estaba, con otro país. Vuelve a hacer el `left_join()`.

Verificación: el resultado tiene **más** de 234 filas, y los vehículos de ese fabricante aparecen duplicados.

En palabras: ¿cuántas filas de más salieron, y por qué exactamente esa cantidad? Si después de esta unión calcularas el consumo promedio por `clase`, ¿estaría mal? Explica en qué sentido.

```
# Escribe tu código aquí
```

20.12.6 Ejercicio 6. Cuando las columnas se llaman distinto

Cambia el nombre de la columna de tu tabla de consulta, de `fabricante` a `marca`. Intenta el `left_join()` sin tocar nada más, en un bloque con `error=TRUE`, y después arrégalo con la forma `by = c("a" = "b")`.

Verificación: el intento sin arreglar da error; la versión arreglada da otra vez 234 filas.

En palabras: copia el mensaje de error y explica qué te estaba diciendo. En `by = c("fabricante" = "marca")`, ¿cuál de los dos nombres pertenece a cuál tabla?

```
# Escribe tu código aquí
```

20.12.7 Ejercicio 7 (BONO). `semi_join()` frente a `%in%`

Quédate con los vehículos de `millas` cuyo fabricante **sí** está en tu tabla de consulta, de dos maneras: con `semi_join()` y con `filter(fabricante %in% ...)`.

Verificación: las dos dan exactamente la misma tabla, con el mismo número de filas y las mismas 11 columnas.

En palabras: si las dos dan lo mismo, ¿para qué existe `semi_join()`? Piensa en qué pasaría si la condición dependiera de **dos** columnas a la vez, y en qué pasaría si usaras `inner_join()` en lugar de `semi_join()`.

```
# Escribe tu código aquí
```


Chapter 21

Factores

Fecha de la última revisión

[1] "2026-09-06"

El tema proviene de los siguientes sitios. Aunque los ejemplos son distintos

English: <https://r4ds.had.co.nz/factors.html>

Español: <https://es.r4ds.hadley.nz/15-factors.html>

21.1 Temas

- Qué es un factor y en qué se diferencia de una variable de texto
 - Crear factores: `factor()`, `parse_factor()`, `fct_inorder()`
 - Ver y cambiar los niveles: `levels()`, `fct_relevel()`
 - Reordenar para graficar: `fct_reorder()`, `fct_infreq()`, `fct_rev()`
 - Agrupar categorías: `fct_recode()`, `fct_collapse()`, `fct_lump_n()`
 - Las constantes que ya trae R: `letters`, `month.name`, `pi`
-

21.2 ¿Qué es un factor?

Un **factor** es una variable categórica con una lista de valores permitidos, que se llaman **niveles**, y con un **orden** entre ellos. Por dentro R guarda números enteros (1, 2, 3) y una tabla que dice qué texto corresponde a cada número.

La pregunta razonable es por qué molestarse, si el texto ya funciona. Hay dos razones, y las dos aparecen en este capítulo:

1. **El orden.** El texto se ordena alfabéticamente, siempre. Con los meses del año eso significa que abril va antes que enero, lo cual es absurdo y sin embargo es lo que hace cualquier gráfica si la variable es texto.
2. **Los valores permitidos.** Con un factor, R sabe cuáles categorías existen aunque en los datos no aparezca ninguna. Si en tu muestra no hubo ningún individuo de la categoría “juvenil”, con texto esa categoría desaparece de la tabla y de la gráfica; con factor sigue ahí, con un cero, que es la información correcta.

Regla práctica: deja el texto como texto mientras estés limpiando datos, y conviértelo en factor **cuando llegues a graficar o a modelar**. Convertir demasiado pronto es la causa de la mitad de los enredos con factores.

```
library(tidyverse)
library(datos)
library(forcats)
```

21.3 El problema, en cuatro meses

El ejemplo que sigue tiene cuatro meses y cuatro valores. Fíjate desde ahora en que `month` es texto (`chr`) y `x` es un número entero (`int`): esa diferencia va a decidir cómo sale la gráfica.

```
month <- c("December", "January", "March", "April")
```

```
month
```

```
## [1] "December" "January" "March" "April"
```

```
x=c(1:4)
str(month)
```

```
## chr [1:4] "December" "January" "March" "April"
```

```
str(x)
```

```
## int [1:4] 1 2 3 4
```

```
valores <- c(7, 3, 2, 10)
```

Unir las dos listas. Ojo con `cbind()`: junta las columnas, pero como una de ellas es texto, **convierte todo a texto**, incluidos los números. Es un caso clásico y una razón más para preferir `tibble()` o `data.frame()`.

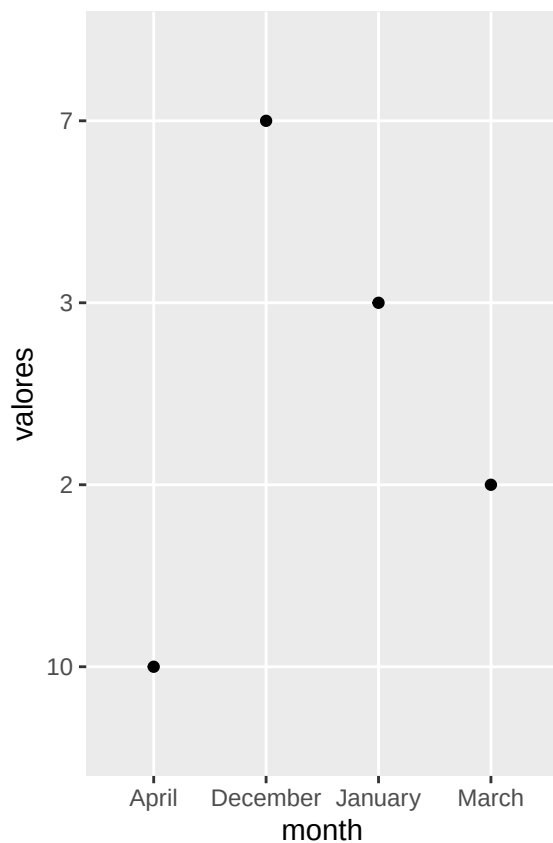
```
df=as.data.frame(cbind(month, valores, x))
df
```

```
##      month valores x
## 1 December      7  1
## 2  January      3  2
## 3   March      2  3
## 4   April     10  4
```

21.4 ¿Qué ocurrió en el gráfico?

```
library(tidyverse)

ggplot(df, aes(month, valores))+
  geom_point()
```



Los meses salieron en orden alfabético: April, December, January, March. No

es un error de `ggplot()`, es lo único que puede hacer con una variable de texto: ordenarla como se ordenan las palabras. Para que salgan en el orden del calendario hay que decirle a R cuál es ese orden, y eso es exactamente lo que hace un factor.

Ordenar los meses correctamente, usando **month.name**.

Hay variables ya descrita en R, para facilitar el uso de constantes

- `LETTERS`: Mayúscula
- `letters`: minúscula
- `month.abb`: abreviación de meses en Inglés
- `month.name`: nombre de meses en Inglés
- `pi` = el valor de pi

See this website for more details

<https://datacornering.com/how-to-get-the-month-name-from-the-number-in-r/>

`LETTERS`

```
## [1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"
## [20] "T" "U" "V" "W" "X" "Y" "Z"
```

`letters`

```
## [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
## [20] "t" "u" "v" "w" "x" "y" "z"
```

`month.abb`

```
## [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

`month.name`

```
## [1] "January" "February" "March" "April" "May" "June"
## [7] "July" "August" "September" "October" "November" "December"
```

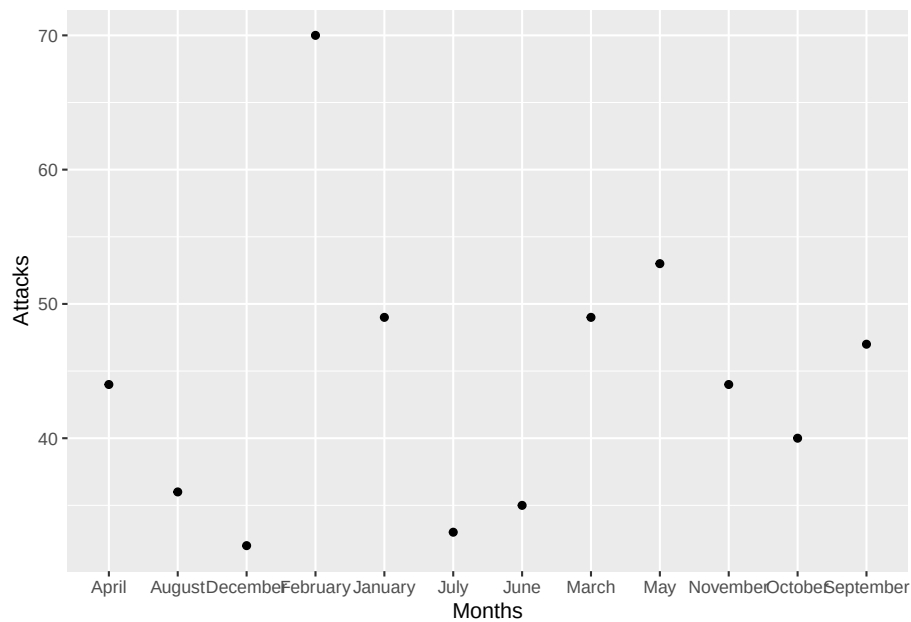
`pi`

```
## [1] 3.141593
```

El bloque siguiente es la receta completa, y vale la pena leerlo en tres pasos. Primero la gráfica mal, en orden alfabético. Después una línea de verificación que casi nadie escribe y que evita muchos problemas: `all(df$Months %in% month.name)` comprueba que **todos** los meses estén bien escritos antes de convertirlos. Y por último `factor(..., levels = month.name)`, que impone el orden correcto, y la gráfica buena.

```
df <- data.frame(Months=c("January", "February", "March", "April", "May", "June", "July",
                          "August", "September", "October", "November", "December"),
                 Attacks=c(49, 70, 49, 44, 53, 35, 33, 36, 47, 40, 44, 32))
```

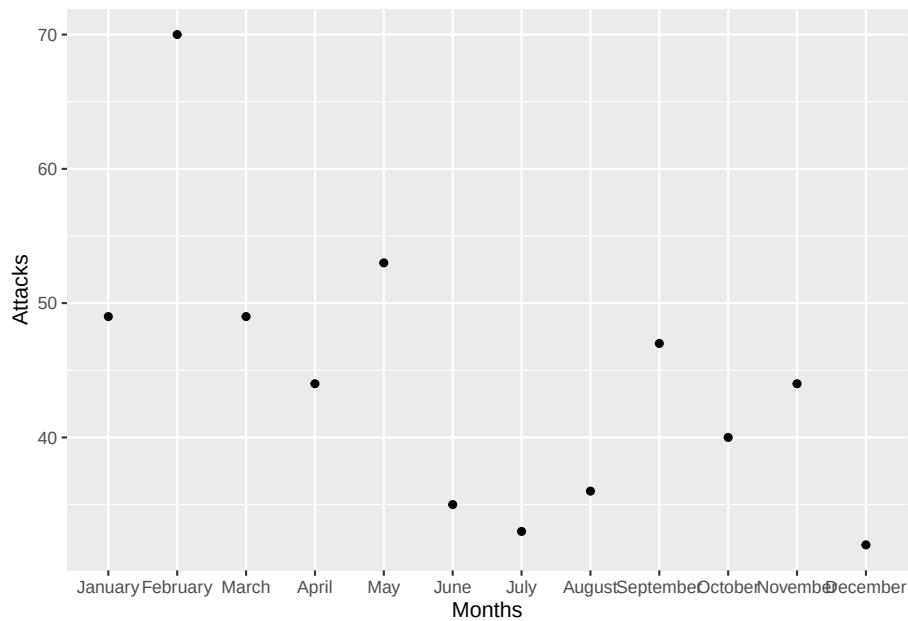
```
ggplot(df, aes(Months, Attacks)) +  
  geom_point()
```



```
### validar el supuesto que tengo información escrita correctamente para cada mes de año en el data frame  
all(df$Months %in% month.name)
```

```
## [1] TRUE
```

```
### convert to a factor, order defined by `month.name`.  
df$Months <- factor(df$Months, levels=month.name)  
  
ggplot(data=df, aes(x=Months, y=Attacks)) +  
  geom_point()
```



21.4.1 factor() y parse_factor()

Con `factor(x, levels = ...)` asignamos los niveles en el orden de la lista que demos. Y aquí aparece la diferencia importante entre las dos funciones que crean factores.

`factor()` (base R) y `parse_factor()` (readr): crear un factor.

- **Las dos** convierten un vector de texto en factor, con los niveles y el orden que les digas en `levels`.
- **La diferencia:** si un valor de los datos **no** está en la lista de niveles, `factor()` lo convierte en `NA` **en silencio**, y `parse_factor()` produce una **advertencia** que lo nombra.
- **Cuál usar:** `parse_factor()` mientras estés revisando datos, porque te enseña las erratas. `factor()` cuando ya sabes que los datos están limpios.

En el bloque de abajo, `x1` trae un mes inventado, "Dii", que no está en la lista de niveles. Compara lo que hace cada función con él: las dos producen un `NA`, pero solo una te avisa.

```
x1= c("Dic", "Ene", "Mar", "Dii") # Nota que esta lista tengo un mes que se llama "Dii"
niveles_meses=c("Ene", "Mar", "Dic")
y1 <-factor(x1, levels = niveles_meses)
y1
```

```
## [1] Dic Ene Mar <NA>
## Levels: Ene Mar Dic
y1 <- parse_factor(x1, levels = niveles_meses)
y1
```

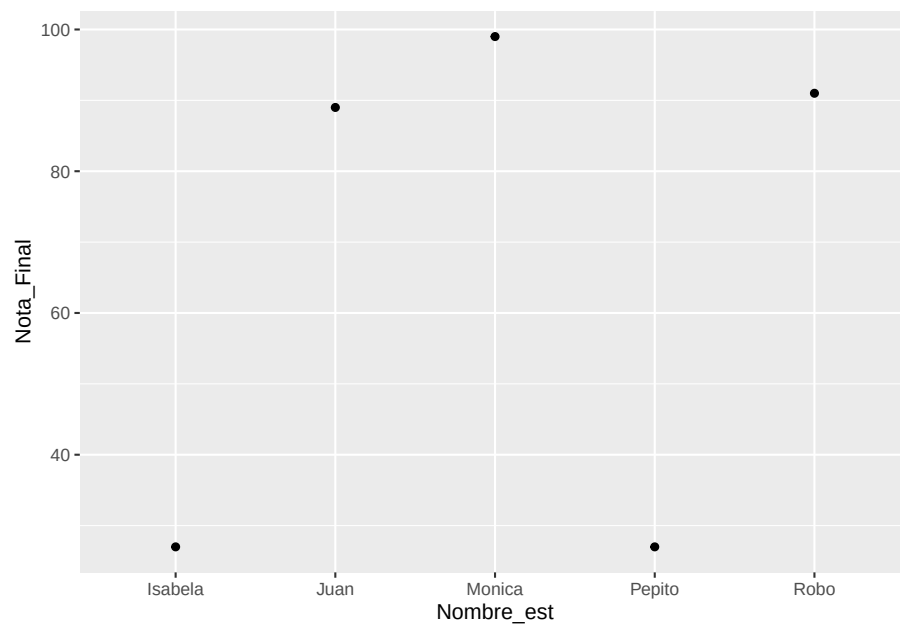
```
## [1] Dic Ene Mar <NA>
## attr("problems")
## # A tibble: 1 x 4
##   row   col expected      actual
##   <int> <int> <chr>      <chr>
## 1     4    NA value in level set Dii
## Levels: Ene Mar Dic
```

Ese NA silencioso de `factor()` es de los errores más caros del curso. Si en una columna de 5000 filas hay cincuenta escritas con una errata, `factor()` las convierte en cincuenta faltantes y el análisis sigue como si nada. Compruébalo siempre después de convertir: `sum(is.na(mi_factor))`.

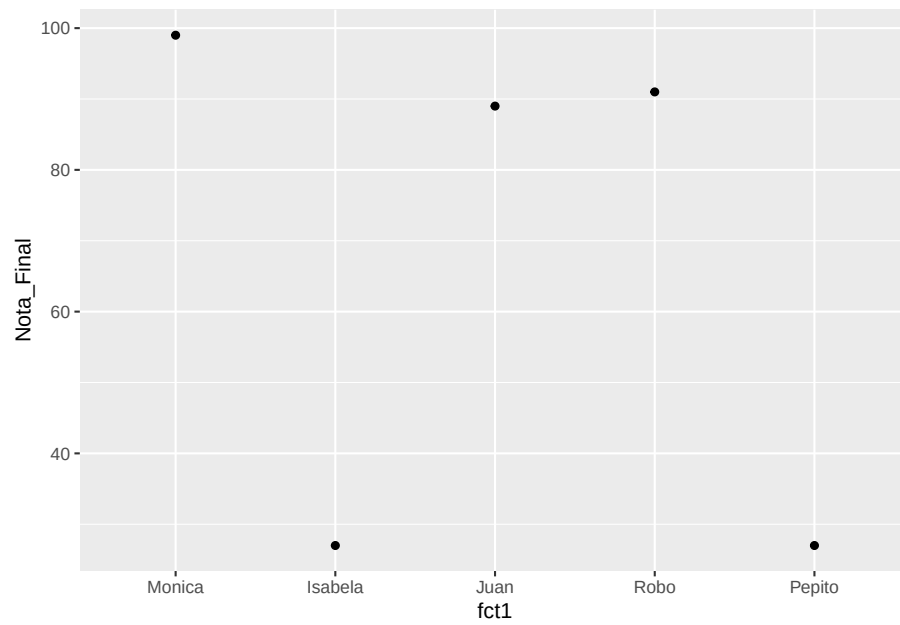
Si quiere que los valores aparezcan en el orden que aparecen en la base de datos use `unique(variable)`

```
df_est=data.frame(Nombre_est=c("Monica", "Isabela", "Juan", "Robo", "Pepito"),
                  Nota_Final=c(99, 27, 89, 91, 27))
df_est
```

```
##   Nombre_est Nota_Final
## 1   Monica          99
## 2  Isabela          27
## 3    Juan          89
## 4    Robo          91
## 5  Pepito          27
ggplot(df_est, aes(Nombre_est, Nota_Final))+
  geom_point()
```



```
df_est$fct1=factor(df_est$Nombre_est, levels = unique(df_est$Nombre_est))  
  
ggplot(df_est, aes(fct1, Nota_Final))+  
  geom_point()
```



`fct_inorder()` del paquete `forcats` hace lo mismo que ese `unique()` pero directamente sobre el factor, y se lee mejor dentro de una cadena de pipes:

```
f2 <- x1 %>% factor() %>% fct_inorder()
f2
```

```
## [1] Dic Ene Mar Dii
## Levels: Dic Ene Mar Dii
```

`levels()` es la función que hay que correr siempre que un factor se porte raro: enseña la lista de niveles y, sobre todo, **el orden** en que están.

```
levels(f2)
```

```
## [1] "Dic" "Ene" "Mar" "Dii"
```

21.5 El paquete forcats

- `library(forcats)`

`forcats` es el paquete del tidyverse dedicado a los factores (el nombre es un anagrama de *factors*). Todas sus funciones empiezan por `fct_`, reciben el factor como primer argumento y devuelven otro factor, así que encajan directamente dentro de `mutate()` y de `aes()`.

Los factores vienen del lenguaje **S** y en su momento fueron una buena idea por una razón que hoy suena rara: guardar enteros en vez de texto ahorra memoria, que en los años ochenta era escasa. Durante décadas R convirtió automáticamente todo el texto en factores al leer un archivo, lo que causó incontables dolores de cabeza. En 2020, con **R 4.0.0**, ese comportamiento por fin cambió: `stringsAsFactors = FALSE` pasó a ser el valor por defecto. `forcats`, de **Hadley Wickham** (2016), había resuelto antes la otra mitad del problema: cómo manipular los niveles sin pelear.

`factor()` y las funciones `fct_*` (`forcats`): manejar variables categóricas.

- **`factor(x, levels = ...)`:** define una variable categórica y el **orden** de sus niveles.
- **`fct_reorder(f, y)`:** reordena los niveles según otra variable (ideal para gráficos).
- **Otras:** `fct_infreq()` (por frecuencia), `fct_rev()` (invertir), `fct_relevel()` (mover niveles a mano).
- `sort()`
- `factor()`
- `parse_factor()`
- `unique()`
- `fct_inorder()`
- `levels()`

21.5.1 Datos: Encuesta Social General

`encuesta` es una muestra de la General Social Survey de Estados Unidos, y es el conjunto que usa R4DS para este tema. Tiene varias columnas categóricas con muchos niveles (`religion`, `estado_civil`, `ingreso`, `partido`), que es exactamente el terreno donde los factores hacen falta.

- `count()`

```
library(datos)
encuesta
```

```
## # A tibble: 21,483 x 9
##   anio estado_civil edad raza ingreso partido religion denominacion horas_tv
##   <int> <fct>      <int> <fct> <fct> <fct> <fct> <fct> <int>
## 1 2000 Nunca se ha~ 26 Blan~ 8000 -- Ind, p~ Protest~ Bautistas d~ 12
## 2 2000 Divorciado 48 Blan~ 8000 -- No fue~ Protest~ Bautista, n~ NA
## 3 2000 Viudo 67 Blan~ No apl~ Indepe~ Protest~ No denomina~ 2
## 4 2000 Nunca se ha~ 39 Blan~ No apl~ Ind, p~ Cristia~ No aplica 4
## 5 2000 Divorciado 25 Blan~ No apl~ No fue~ Ninguna No aplica 1
## 6 2000 Casado 25 Blan~ 20000 ~ Fuerte~ Protest~ Bautistas d~ NA
## 7 2000 Nunca se ha~ 36 Blan~ 25000 ~ No fue~ Cristia~ No aplica 3
## 8 2000 Divorciado 44 Blan~ 7000 -- Ind, p~ Protest~ Sínodo lute~ NA
## 9 2000 Casado 44 Blan~ 25000 ~ No fue~ Protest~ Otra 0
## 10 2000 Casado 47 Blan~ 25000 ~ Fuerte~ Protest~ Bautistas d~ 3
## # i 21,473 more rows
```

Contabilizar la cantidad de personas por “estado civil”

```
df=encuesta %>%
  count(estado_civil)
df
```

```
## # A tibble: 6 x 2
##   estado_civil      n
##   <fct>      <int>
## 1 Sin respuesta    17
## 2 Nunca se ha casado 5416
## 3 Separado        743
## 4 Divorciado      3383
## 5 Viudo           1807
## 6 Casado          10117
```

```
unique(df$estado_civil)
```

```
## [1] Sin respuesta      Nunca se ha casado Separado      Divorciado
## [5] Viudo              Casado
## 6 Levels: Sin respuesta Nunca se ha casado Separado Divorciado ... Casado
```

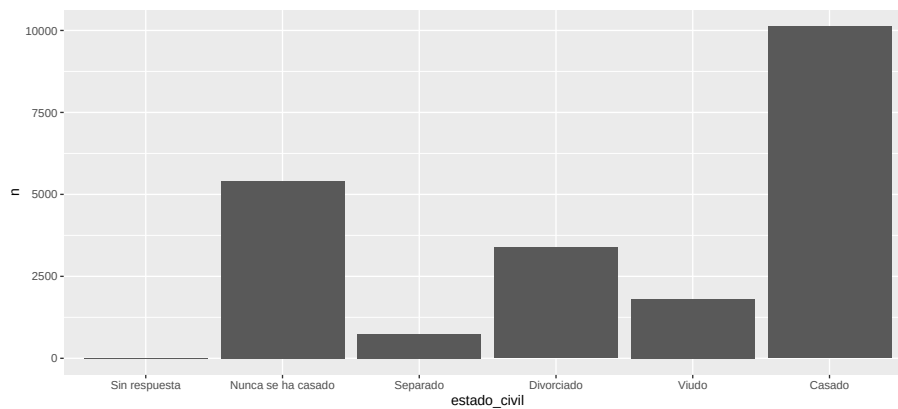
```
df$estado_est_civil <- factor(df$estado_civil, levels = df$estado_civil)

df$niveles_est_civil <- fct_reorder(df$estado_civil, df$n )

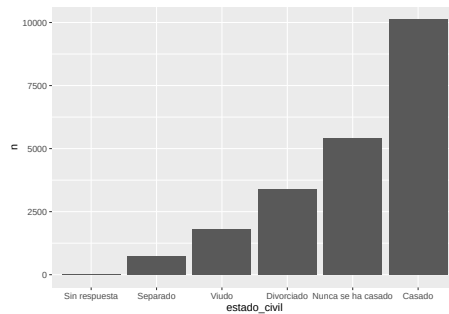
df
```

```
## # A tibble: 6 x 4
##   estado_civil      n estado_est_civil  niveles_est_civil
##   <fct>          <int> <fct>          <fct>
## 1 Sin respuesta      17 Sin respuesta    Sin respuesta
## 2 Nunca se ha casado 5416 Nunca se ha casado Nunca se ha casado
## 3 Separado           743 Separado        Separado
## 4 Divorciado        3383 Divorciado      Divorciado
## 5 Viudo             1807 Viudo           Viudo
## 6 Casado            10117 Casado          Casado
```

```
ggplot(df,
  aes(estado_civil,n )) +
  geom_col()
```



21.5.2 Una figura pequeña a la derecha, con el texto alrededor



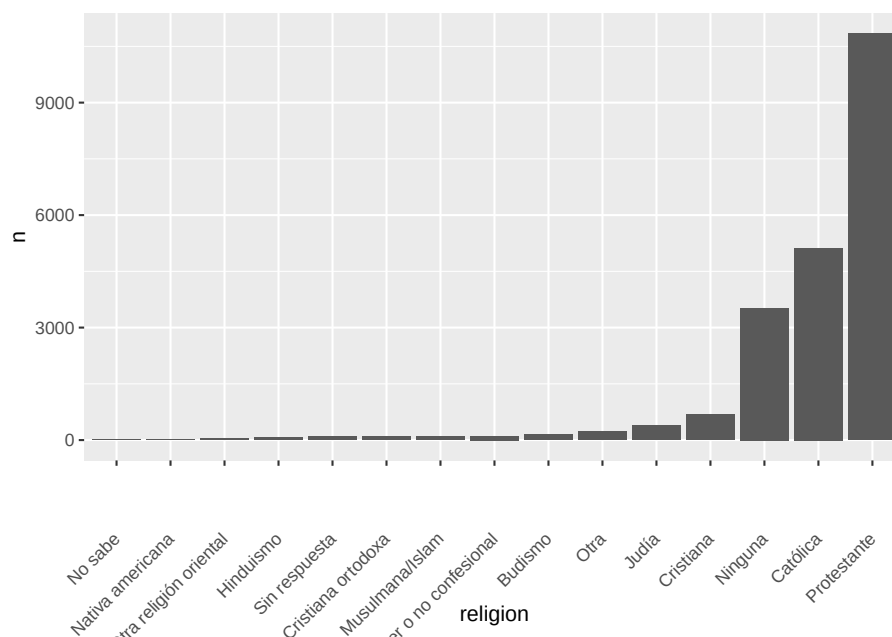
El bloque de arriba enseña dos cosas a la vez. La primera es la de siempre: `fct_reorder(estado_civil, n)` ordena las barras por su tamaño, y la gráfica pasa de ser una lista alfabética a contar algo. La segunda es de presentación: con las opciones `out.width` y `out.extra` del bloque, la figura sale a la mitad de ancho y flotando a la derecha, de manera que el texto la rodea en vez de quedarse debajo. Es un recurso útil cuando la gráfica es un apoyo del argumento y no el argumento entero.

Fíjate en el detalle de que `fct_reorder()` va dentro de `mutate()`, antes del `ggplot()`. También se puede escribir directamente dentro de `aes()`, y en los bloques siguientes se hace así; las dos maneras funcionan y conviene reconocer las dos.

21.6 Haga un re-order por la cantidad de personas por religión

```
df2=encuesta %>%
  count(religion)

df2 %>%
  mutate(religion = fct_reorder(religion, n)) %>%
  ggplot(aes(x = religion, y = n)) +
  geom_col() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.5, hjust=1))
```



21.7 Modificar el orden de los factores

Usa `fct_reorder()` para ordenar los niveles de un factor según otra variable (por ejemplo la frecuencia o la media). Los gráficos quedan mucho más fáciles de leer que con el orden alfabético por defecto.

- `fct_reorder()`
- `!is.na()` remover los “NA”
- `fct_infreq()`
- `fct_rev()`
- `fct_recode()`
- `fct_collapse()`
- `fct_lump()`

Conviene separarlas en dos grupos, porque hacen cosas distintas:

Reordenar los niveles (no cambian los datos, solo el orden en que se enseñan).

- **`fct_reorder(f, y)`**: ordena los niveles de `f` según la mediana de `y`. Es la que más se usa: convierte una gráfica de barras alfabética en una gráfica que se lee sola.
- **`fct_reorder2(f, x, y)`**: para gráficas de líneas, ordena la leyenda según el valor de `y` en el `x` más grande, es decir, según el orden en que las líneas terminan.

- `fct_infreq(f)`: ordena por frecuencia, de la categoría más común a la menos común.
- `fct_rev(f)`: invierte el orden actual.
- `fct_relevel(f, "algo")`: mueve a mano uno o varios niveles al principio.

Agrupar o renombrar niveles (estos sí cambian los datos).

- `fct_recode(f, nuevo = "viejo")`: cambia el nombre de un nivel. Si varios niveles reciben el mismo nombre nuevo, quedan fundidos en uno.
- `fct_collapse(f, grupo = c("a", "b", "c"))`: junta varios niveles en uno, y es más cómodo que `fct_recode()` cuando son muchos.
- `fct_lump_n(f, n = 5)`: conserva los `n` niveles más frecuentes y mete todo lo demás en una categoría “otros”. Útil cuando una variable tiene cuarenta categorías y treinta y cinco tienen dos observaciones cada una.

`fct_recode()`, `fct_collapse()` y `fct_lump_n()` **cambian los datos**, no solo su presentación. Agrupar categorías es una decisión de análisis, no de estética: di siempre en el texto del informe cuáles categorías juntaste y por qué.

Un ejemplo de las tres últimas, con la religión de la encuesta:

```
encuesta |>
  mutate(religion_corta = fct_lump_n(religion, n = 5, other_level = "Otras")) |>
  count(religion_corta, sort = TRUE)
```

```
## # A tibble: 6 x 2
##   religion_corta    n
##   <fct>          <int>
## 1 Protestante    10846
## 2 Católica       5124
## 3 Ninguna        3523
## 4 Otras          913
## 5 Cristiana      689
## 6 Judía          388
```

Y `fct_recode()`, que cambia el nombre de un nivel. Aquí le cambiamos el nombre al nivel "Otras" que acabamos de crear nosotros mismos:

```
encuesta |>
  mutate(
    religion_corta = fct_lump_n(religion, n = 5, other_level = "Otras"),
    religion_corta = fct_recode(religion_corta, "Otras religiones" = "Otras")
  ) |>
  count(religion_corta, sort = TRUE)
```

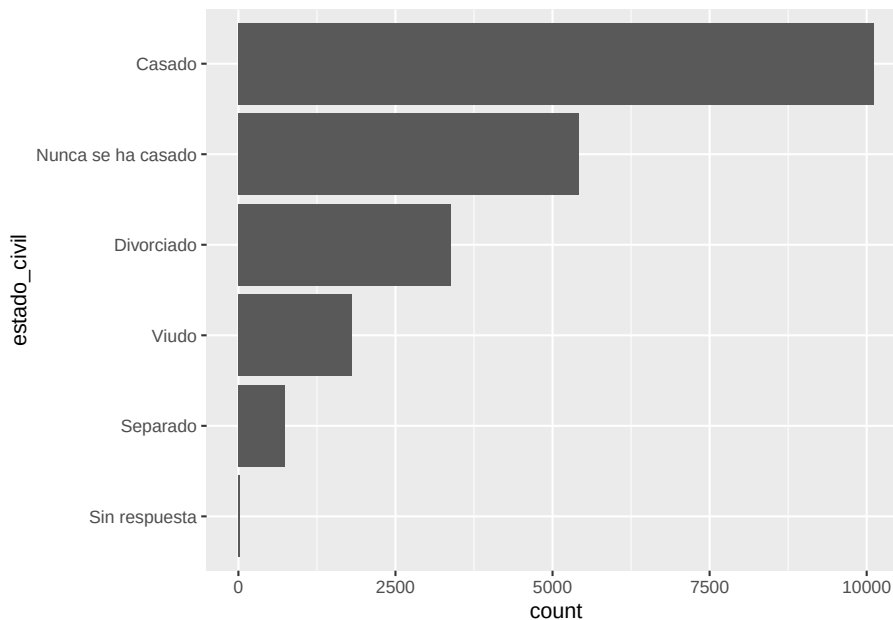
```
## # A tibble: 6 x 2
##   religion_corta    n
##   <fct>          <int>
```

```
## 1 Protestante      10846
## 2 Católica        5124
## 3 Ninguna         3523
## 4 Otras religiones 913
## 5 Cristiana        689
## 6 Judía           388
```

En `fct_recode()`, el nombre **nuevo** va a la izquierda del igual y el **viejo** a la derecha, que es al revés de lo que casi todo el mundo escribe la primera vez. Y si el nivel viejo no existe, no da error: da una advertencia (**Unknown levels in f**) y devuelve el factor sin tocar. Con `warning=FALSE` en las opciones del bloque, esa advertencia no se ve y uno se queda creyendo que renombró algo. Comprueba siempre con `levels()` después de renombrar.

Y `fct_infreq()` con `fct_rev()`, que juntas son la receta habitual para una gráfica de barras horizontales bien ordenada:

```
encuesta |>
  mutate(estado_civil = fct_rev(fct_infreq(estado_civil))) |>
  ggplot(aes(y = estado_civil)) +
  geom_bar()
```



```
#model=lm(y~x, df)
#summary(model)

resumen_religion <- encuesta %>%
```

```

group_by(religion) %>%
  summarise(
    edad = mean(edad, na.rm = TRUE),
    horas_tv = mean(horas_tv, na.rm = TRUE),
    sd_edad = sd(edad, na.rm = TRUE),
    n = n()
  )
resumen_religion

```

```

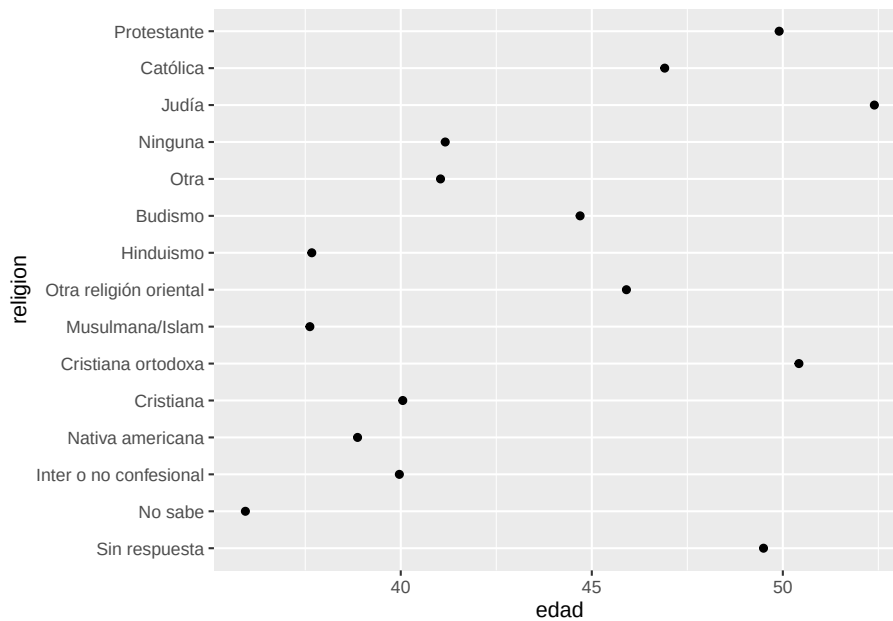
## # A tibble: 15 x 5
##   religion      edad horas_tv sd_edad      n
##   <fct>      <dbl>   <dbl>   <dbl> <int>
## 1 Sin respuesta  49.5     2.72    NA     93
## 2 No sabe       35.9     4.62    NA     15
## 3 Inter o no confesional 40.0     2.87    NA    109
## 4 Nativa americana 38.9     3.46    NA     23
## 5 Cristiana     40.1     2.79    NA    689
## 6 Cristiana ortodoxa 50.4     2.42    NA     95
## 7 Musulmana/Islam  37.6     2.44    NA    104
## 8 Otra religión oriental 45.9     1.67    NA     32
## 9 Hinduismo      37.7     1.89    NA     71
## 10 Budismo       44.7     2.38    NA    147
## 11 Otra          41.0     2.73    NA    224
## 12 Ninguna       41.2     2.71    NA   3523
## 13 Judía         52.4     2.52    NA    388
## 14 Católica      46.9     2.96    NA   5124
## 15 Protestante   49.9     3.15    NA  10846

```

```

ggplot(resumen_religion, aes(edad, religion)) +
  geom_point()

```

```
min(encuesta$horas_tv, na.rm = TRUE)
```

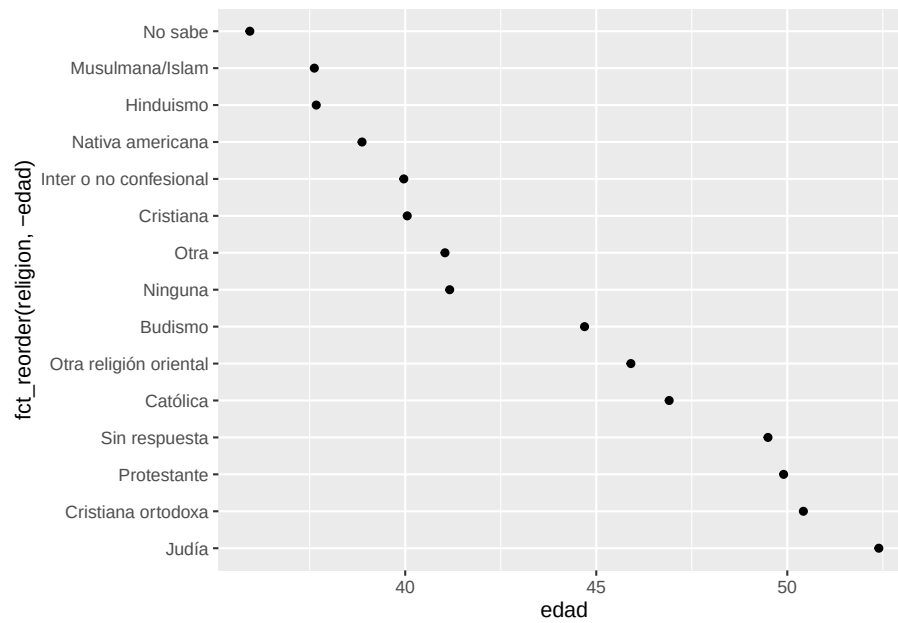
```
## [1] 0
```

Esta primera gráfica está en orden alfabético inverso, que es el que traía el factor, y por eso los puntos parecen repartidos al azar. Compárala con la siguiente.

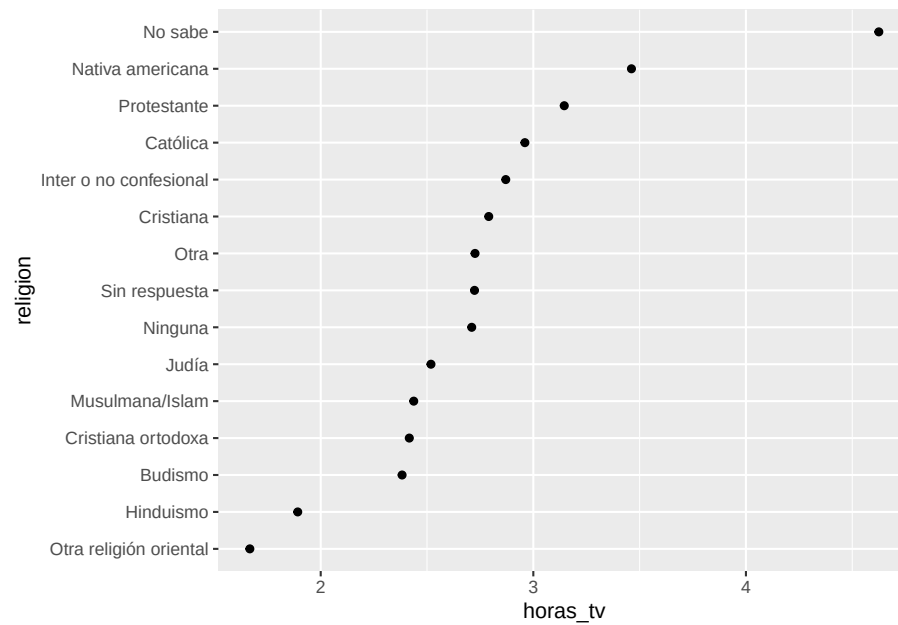
21.7.1 Nota el efecto de añadir un “-” antes de edad

`fct_reorder(religion, edad)` ordena de menor a mayor; con el signo menos delante, `fct_reorder(religion, -edad)`, el orden se invierte. Es el truco más corto para darle la vuelta a una gráfica, y funciona solo con variables numéricas. Para cualquier otro caso está `fct_rev()`.

```
ggplot(resumen_religion, aes(edad, fct_reorder(religion, -edad))) +  
  geom_point()
```



```
resumen_religion %>%
  mutate(religion = fct_reorder(religion, horas_tv)) %>%
  ggplot(aes(horas_tv, religion)) +
  geom_point()
```



21.7.2 fct_reorder() y fct_reorder2()

`fct_reorder()` sirve cuando el factor va en una **posición**, es decir, en uno de los dos ejes: es el caso de todas las gráficas de barras y de puntos de este capítulo.

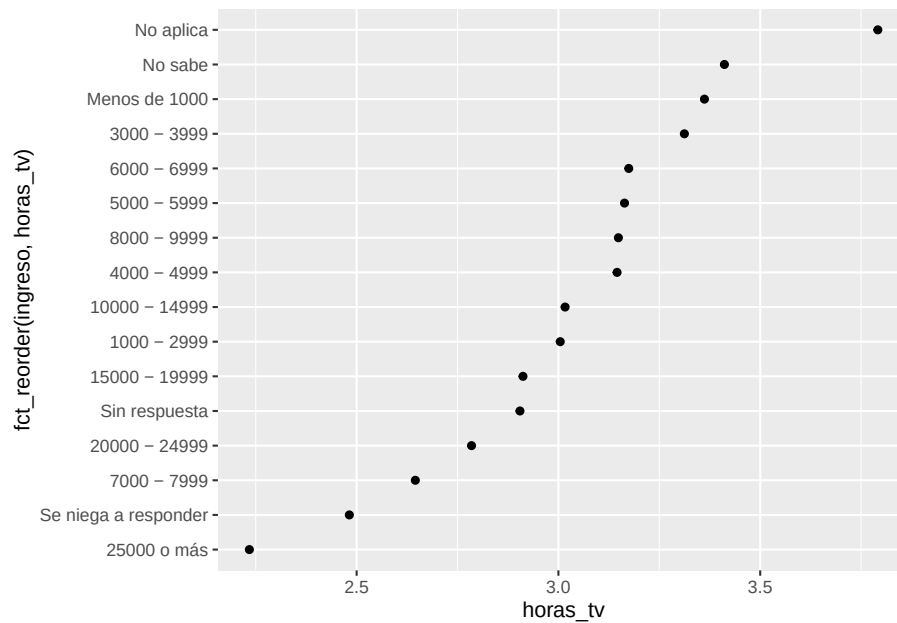
`fct_reorder2()` sirve cuando el factor va en un aspecto **que no es posición**, típicamente el color de una gráfica de líneas. Ordena la leyenda según el valor de y en el x más grande, o sea, según el orden en que las líneas terminan a la derecha. El resultado es que la leyenda y las líneas coinciden de arriba abajo, y el lector deja de tener que ir y venir entre las dos.

```
resumen_ingreso <- encuesta %>%
  group_by(ingreso) %>%
  summarise(
    edad = mean(edad, na.rm = TRUE),
    horas_tv = mean(horas_tv, na.rm = TRUE),
    n = n()
  )

resumen_ingreso
```

```
## # A tibble: 16 x 4
##   ingreso          edad horas_tv      n
##   <fct>          <dbl>   <dbl> <int>
## 1 Sin respuesta  45.5     2.90   183
## 2 No sabe       45.6     3.41   267
## 3 Se niega a responder 47.6     2.48   975
## 4 25000 o más   44.2     2.23  7363
## 5 20000 - 24999 41.5     2.78  1283
## 6 15000 - 19999 40.0     2.91  1048
## 7 10000 - 14999 41.1     3.02  1168
## 8 8000 - 9999   41.1     3.15   340
## 9 7000 - 7999   38.2     2.65   188
## 10 6000 - 6999  40.3     3.17   215
## 11 5000 - 5999  37.8     3.16   227
## 12 4000 - 4999  38.9     3.15   226
## 13 3000 - 3999  37.8     3.31   276
## 14 1000 - 2999  34.5     3.00   395
## 15 Menos de 1000 40.5     3.36   286
## 16 No aplica    56.1     3.79  7043
```

```
ggplot(resumen_ingreso, aes(horas_tv, fct_reorder(ingreso, horas_tv))) +
  geom_point()
```

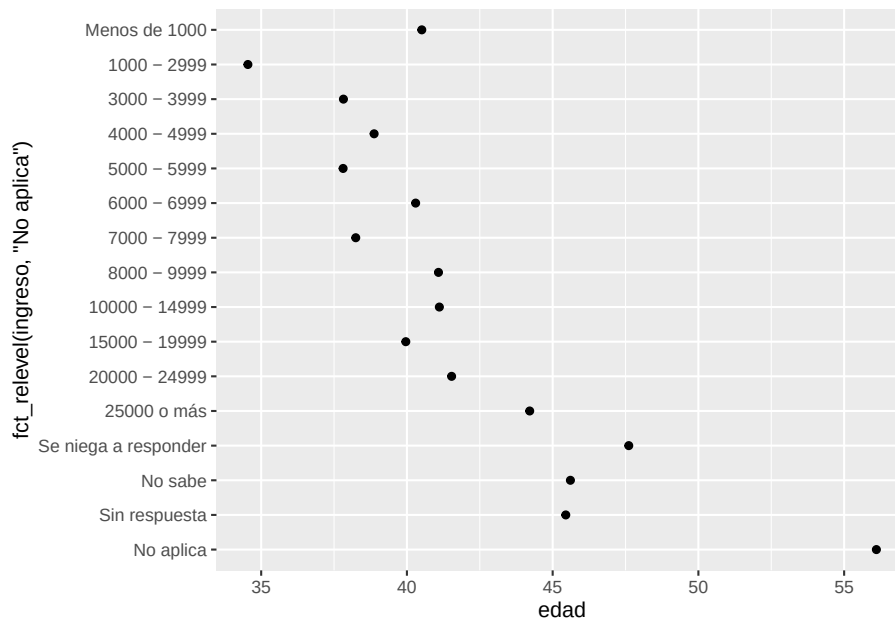


21.7.3 Reordenar a mano: `fct_relevel()`

A veces el orden que uno quiere no sale de ningún cálculo. Un caso muy común: mover al final las categorías que no son categorías de verdad, como “No aplica”, “No sé” o “Sin respuesta”, para que no estorben en el medio de la gráfica.

`fct_relevel()` es una generalización de `stats::relevel()` que permite mover cualquier número de niveles a cualquier posición.

```
ggplot(resumen_ingreso, aes(edad, fct_relevel(ingreso, "No aplica")) +  
  geom_point())
```



21.8 Ejercicios del libro R4DS

Hacer los ejercicios de las secciones 15.3.1 y 15.5.1 del libro en español.

21.9 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que la tabla o la gráfica se vean) y una **explicación en palabras**. Los ejercicios usan **millas** del paquete **datos**, no **encuesta**, que es el conjunto con el que se enseña aquí. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_21_factores.Rmd`.

21.9.1 Ejercicio 1. Texto contra factor

Datos: `millas`. Comprueba con `class()` que `clase` es texto. Haz una gráfica de barras. Después conviértela en factor con un orden que tenga sentido para ti (por ejemplo del vehículo más pequeño al más grande) y repite la gráfica.

Verificación: `levels()` debe devolver 7 niveles en el orden que tú escogiste, y la gráfica debe seguir ese orden.

En palabras: ¿en qué orden salían las barras antes? ¿Por qué ese orden no lo escogió nadie?

Escribe tu código aquí

21.9.2 Ejercicio 2. `fct_reorder()` para que la gráfica hable

Datos: `millas`. Haz un diagrama de puntos del consumo mediano en `autopista` para cada `clase`, primero sin ordenar y después con `fct_reorder(clase, autopista)`.

Verificación: en la versión ordenada, `pickup` debe quedar en un extremo y `compact` o `subcompact` en el otro.

En palabras: las dos gráficas tienen exactamente los mismos datos. ¿Qué se ve en la segunda que no se veía en la primera? ¿Por qué el orden alfabético es casi siempre el peor de los órdenes posibles?

Escribe tu código aquí

21.9.3 Ejercicio 3. Ordenar por frecuencia

Datos: `millas`, variable `fabricante`. Haz una gráfica de barras **horizontales** usando `fct_infreq()` y `fct_rev()`.

Verificación: la barra más larga tiene que quedar arriba del todo.

En palabras: prueba a quitar `fct_rev()` y mira qué pasa. Explica por qué en barras horizontales hace falta y en barras verticales no.

Escribe tu código aquí

21.9.4 Ejercicio 4. Cuando hay demasiadas categorías

Datos: `millas`. Averigua cuántos fabricantes distintos hay. Después usa `fct_lump_n()` para quedarte con los 5 más frecuentes y meter el resto en una categoría llamada `"Otros"`.

Verificación: el factor nuevo debe tener exactamente **6** niveles, y la suma de los conteos debe seguir dando 234.

En palabras: ¿cuántos vehículos acabaron en `"Otros"`? Esta función **cambia los datos**, no solo la gráfica. ¿En qué situación sería deshonesto usarla sin decirlo?

Escribe tu código aquí

21.9.5 Ejercicio 5. Agrupar a mano

Datos: `millas`. Con `fct_collapse()`, agrupa las siete clases en dos categorías que tengan sentido para ti (por ejemplo vehículos grandes y vehículos pequeños). Después compara el consumo en autopista entre los dos grupos.

Verificación: el factor nuevo tiene 2 niveles y la suma de los conteos sigue dando 234.

En palabras: justifica tu agrupación. ¿Qué información se perdió al pasar de siete categorías a dos? ¿Habías llegado a la misma conclusión con las siete?

Escribe tu código aquí

21.9.6 Ejercicio 6. El nivel que sigue ahí aunque no haya datos

Convierte `clase` en factor. Filtra fuera todos los `2seater`. Después mira `levels()`, `count(clase)` y `count(clase, .drop = FALSE)`.

Verificación: `levels()` sigue devolviendo 7 niveles, `count()` devuelve 6 filas, y `count(clase, .drop = FALSE)` devuelve 7, la última con un cero. Con `fct_drop()`, `levels()` baja a 6.

En palabras: las tres respuestas son distintas y las tres son correctas. Explica cuándo querrías ver el cero y cuándo no. Piensa en una especie que buscaste en el campo y no encontraste.

Escribe tu código aquí

21.9.7 Ejercicio 7 (BONO). La errata que se convierte en NA

Haz una copia de `millas` y cámbiale a mano la clase de tres vehículos a "compacto" (en español, que no es un nivel válido). Conviértela en factor de dos maneras: con `factor(..., levels = ...)` usando las siete clases correctas, y con `parse_factor()`.

Verificación: las dos producen exactamente 3 NA, pero solo una de ellas te avisa.

En palabras: copia la advertencia de `parse_factor()`. Si esto pasara en una columna de 5000 filas de datos de campo, ¿cuál de las dos funciones querrías haber usado, y qué línea correrías después para estar seguro?

Escribe tu código aquí

Chapter 22

Cadena de Caracteres = STRINGS

```
## [1] "2026-09-06"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/strings.html>

Español: <https://es.r4ds.hadley.nz/14-strings.html>

```
library(tidyverse)
library(datos)
library(stringr)
```

```
string1 <- "El Pequeño Principe; Saint Exupery"
string2 <- "El segundo sexo: Simone de Beauvoir"
string3 <- "El segundo sexo \\ Simone de Beauvoir"
```

```
string1
```

```
## [1] "El Pequeño Principe; Saint Exupery"
```

```
string2
```

```
## [1] "El segundo sexo: Simone de Beauvoir"
```

```
string3
```

```
## [1] "El segundo sexo \\ Simone de Beauvoir"
```

Caracteres especiales en cadenas de caracteres

```
comilla_doble <- '\"' # ó '''
comilla_simple <- '\'' # ó '''

comilla_ampersand <- "&"

comilla_doble
```

```
## [1] "\"
comilla_simple
```

```
## [1] ""
comilla_ampersand
```

```
## [1] "&"

&
```

Backslash

```
x <- c("\\", "\\")
x
```

```
## [1] "\" "\\ "
writeLines(x)
```

```
## "
## \
```

22.1 Caracteres especiales

```
"\n"
```

```
## [1] "\n"
```

```
"\t"
```

```
## [1] "\t"
```

22.2 Para ver las opciones de caracteres y como trabajar cadenas de caracteres use ;“” para ver

```
? " ' "

'\n'  newline (aka 'line feed')
'\r'  carriage return
'\t'  tab
'\b'  backspace
'\a'  alert (bell)
'\f'  form feed
'\v'  vertical tab
'\'\'  backslash '\'
'\''  ASCII apostrophe ''
'\''  ASCII quotation mark ''
'\'\'  ASCII grave accent (backtick) ``
```

22.3 Lista de caracteres especiales (unicode)

Vea el siguiente enlace para los unicodes

https://en.wikipedia.org/wiki/List_of_Unicode_characters

```
x <- "\u00b5" # mu
x
```

```
## [1] "µ"
```

```
y <- "\u00b6" # pilcrow
y
```

```
## [1] "¶"
```

```
z <- "\u00A1" # ¡
z
```

```
## [1] "¡"
```

```
w <- "\u01FC\u00b6" # Ě¶
w
```

```
## [1] "Ě¶"
```

Escribir su nombre en el alfabeto cirílico

- Vea este enlace para conocer el alfabeto cirílico

<<https://www.google.com/search?client=safari&rls=en&q=cyrillic+alphabet&ie=UTF-8&oe=UTF-8#vhid=cMLOx1fN0Zg5UM&vssid=1>>

```
Raymond = "\u048E\u0400\u042B\u043C\u043E\u043D\u0434, Asi se escribe mi nombre en ruso"
Raymond
```

```
## [1] "      , Asi se escribe mi nombre en ruso"
```

necesito poner una letra extraña en mi frase, "\u048E".

22.4 Temas: Cadenas de Caracteres

- Lo básico: largo, unir, cortar, mayúsculas y minúsculas
- Qué es una expresión regular y por qué vale la pena aprenderla
- Anclas: ^ y \$
- Clases de caracteres y alternativas
- Repetición: ?, +, *, {n,m}
- Agrupamiento y referencias previas
- Las cinco herramientas: detectar, extraer, agrupar, reemplazar y dividir
- `library(stringr)`

Solamente trabajaremos con el paquete `stringr`, donde las funciones escritas de forma que sean más intuitivas para recordar. Todas las funciones comienzan con `str_`

Las funciones `str_*` (`stringr`): trabajar con texto.

- `str_length(x)`: número de caracteres.
- `str_c(...)`: une cadenas (concatena).
- `str_sub(x, a, b)`: extrae o reemplaza una porción por posición.
- `str_to_lower()` / `str_to_upper()`: cambian mayúsculas/minúsculas.
- `str_detect()`, `str_replace()`, `str_view()`: buscan y reemplazan patrones (regex).

22.4.1 El largo de una cadena de caracteres con `str_length`

```
library(stringr)
str_length("Yo quiero un Lamborghini")
```

```
## [1] 24
```

22.4.2 Combinar cadenas con str_c

Nota aquí que une la primera oración con la segunda oración

```
str_c("Es una oración con muchas letras y palabras",
      "Mi película favorita es Cinema Paraíso")
```

```
## [1] "Es una oración con muchas letras y palabrasMi película favorita es Cinema Paraíso"
```

Usa el argumento `sep = "xxx"` para controlar cómo separar las oraciones

```
str_c("Es una oración con muchas letras y palabras",
      "Mi película favorita es Cinema Paraíso", sep = ", ")
```

```
## [1] "Es una oración con muchas letras y palabras, Mi película favorita es Cinema Paraíso"
```

- `str_replace_na()`

```
x <- c("Mi película favorita es Cinema Paraíso", NA)
x
```

```
## [1] "Mi película favorita es Cinema Paraíso"
## [2] NA
```

```
str_c("|-", x, "-|")
```

```
## [1] "|-Mi película favorita es Cinema Paraíso-|"
## [2] NA
```

22.5 Dividir cadenas basado en la posición de los caracteres en la cadena

- `str_sub()`

```
x= c("Piña", "Manzana", "Toronja")
str_sub(x, 1,1)
```

```
## [1] "P" "M" "T"
```

22.5.1 Seleccionar los caracteres al último de las palabras

```
x= c( "Piña", "Manzana", "Toronja", "Hello")
str_sub(x, -2,-1)
```

```
## [1] "ña" "na" "ja" "lo"
```

22.6 También se puede usar str_sub() para modificar una cadena

```
x <- "BBCDEF"
str_sub(x, 1, 1) <- "A"; x
```

```
## [1] "ABCDEF"
```

```
x <- "BBCDEF"
str_sub(x, -1, -1) <- "K"; x
```

```
## [1] "BBCDEK"
```

```
x <- "BBCDEF"
str_sub(x, -2, -2) <- "GHIJ"; x
```

```
## [1] "BBCDGHJF"
```

```
x <- "BBCDEF"
str_sub(x, -3, -3) <- "BC"; x
```

```
## [1] "BBCBCEF"
```

```
x <- "BBCDEF"
str_sub(x, 2, -2) <- ""; x
```

```
## [1] "BF"
```

22.6.1 Cambiar a todo minúscula

```
x= c( "Piña", "Manzana", "Toronja", "HAPPY")
str_to_lower(x)
```

```
## [1] "piña" "manzana" "toronja" "happy"
```

22.6.2 Cambiar a mayúscula

```
x= c( "Piña", "Manzana", "Toronja")
str_to_upper(x)

## [1] "PIÑA"      "MANZANA" "TORONJA"
```

22.7 Locales

- ¿Qué es un locale?
- str_to_lower()
- str_to_upper()
- str_order() # que hace esta función?
- str_sort() # que hace esta función?

```
x

## [1] "Piña"      "Manzana" "Toronja"
str_order(x) # regresa el orden de las palabras

## [1] 2 1 3
str_sort(x) # reorganiza las palabras en orden alfabético

## [1] "Manzana" "Piña"     "Toronja"
```

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.2.5 del libro en español

22.8 Buscar patrones

Las expresiones regulares surgieron en los años 1950 del trabajo del matemático Stephen Kleene sobre lenguajes formales, y hoy son el estándar para describir patrones de texto en casi todos los lenguajes de programación.

22.9 Coincidencia básica

Para visualizar patrones de letras

- str_view()
- str_view(, .x.)

```
library(stringr)
x <- c("manzana", "banana", "pera", "extraer")
str_view(x, "er")

## [3] | p<er>a
## [4] | extra<er>
```

Ejemplo de cómo uno puede utilizar esas funciones

```
str_detect()
x= c( "Piña", "Manzana", "Toronja", "Jugo", "Monique", "Raymond", "mami")
str_detect(x, "a") # detectar la a

## [1] TRUE TRUE TRUE FALSE FALSE TRUE TRUE
str_detect(x, "i") # detectar la i

## [1] TRUE FALSE FALSE FALSE TRUE FALSE TRUE
str_detect(x, "ñ")

## [1] TRUE FALSE FALSE FALSE FALSE FALSE FALSE
```

22.9.1 Detectar las palabras que comienzan con una letra específica con “^”. Nota el uso de mayúsculas y minúsculas y sumamos cuantas palabras cumplen con la condición.

```
# ¿Cuántas palabras comunes empiezan con m o M?
# str_detect(x, "^M")
x

## [1] "Piña" "Manzana" "Toronja" "Jugo" "Monique" "Raymond" "mami"
sum(str_detect(x, "^m"))

## [1] 1
sum(str_detect(x, "^M"))

## [1] 2
sum(str_detect(x, "[mM]")) # ambas mayusculas y minusculas

## [1] 3
```


22.9.2 Detectar las palabras que terminan con una letra específica con “\$”

```
# ¿Qué proporción de palabras comunes terminan con una vocal?

library(datos)
mean(str_detect(palabras, "[aáeéiíoóuú]$"))

## [1] 0.561

# ¿Qué proporción de palabras comunes terminan con una vocal específica?
mean(str_detect(palabras, "[ao]$"))

## [1] 0.428

mean(str_detect(x, "[d]$")) # Raymond

## [1] 0.1428571
```

Dataset: palabras

- Comienza para mirar el archivo palabras
- Cuentas palabras tiene ese archivo?

```
library(stringr)
library(datos)
palabras
```

```
## [1] "a" "abril" "acción" "acciones"
## [5] "acerca" "actitud" "actividad" "actividades"
## [9] "acto" "actual" "acuerdo" "adelante"
## [13] "además" "administración" "afirmó" "agua"
## [17] "ahí" "ahora" "aire" "al"
## [21] "algo" "alguien" "algún" "alguna"
## [25] "algunas" "algunos" "allá" "allí"
## [29] "alrededor" "alta" "alto" "ambiente"
## [33] "ambos" "américa" "amigo" "amigos"
## [37] "amor" "análisis" "animales" "ante"
## [41] "anterior" "antes" "antonio" "año"
## [45] "años" "aparece" "apenas" "apoyo"
## [49] "aquel" "aquella" "aquellas" "aquellos"
## [53] "aquí" "área" "argentina" "armas"
## [57] "arriba" "arte" "artículo" "así"
## [61] "asimismo" "asociación" "aspecto" "aspectos"
## [65] "asunto" "atención" "atrás" "aumento"
## [69] "aun" "aún" "aunque" "autor"
## [73] "autoridades" "ayer" "ayuda" "b"
```

##	[77]	"baja"	"bajo"	"banco"	"barcelona"
##	[81]	"base"	"bastante"	"bien"	"blanca"
##	[85]	"blanco"	"boca"	"buen"	"buena"
##	[89]	"bueno"	"buenos"	"busca"	"buscar"
##	[93]	"c"	"cabeza"	"cabo"	"cada"
##	[97]	"calidad"	"calle"	"cama"	"cámara"
##	[101]	"cambio"	"cambios"	"camino"	"campaña"
##	[105]	"campo"	"cantidad"	"capacidad"	"capaz"
##	[109]	"capital"	"cara"	"carácter"	"características"
##	[113]	"cargo"	"carlos"	"carne"	"carrera"
##	[117]	"carta"	"casa"	"casi"	"caso"
##	[121]	"casos"	"causa"	"central"	"centro"
##	[125]	"centros"	"cerca"	"chile"	"ciencia"
##	[129]	"ciento"	"cierta"	"cierto"	"cinco"
##	[133]	"cine"	"ciudad"	"civil"	"claro"
##	[137]	"clase"	"club"	"color"	"comenzó"
##	[141]	"comercio"	"comisión"	"como"	"cómo"
##	[145]	"compañía"	"común"	"comunicación"	"comunidad"
##	[149]	"con"	"concepto"	"conciencia"	"condiciones"
##	[153]	"congreso"	"conjunto"	"conocer"	"conocimiento"
##	[157]	"consecuencia"	"conseguir"	"consejo"	"considera"
##	[161]	"constitución"	"construcción"	"consumo"	"contenido"
##	[165]	"contra"	"contrario"	"control"	"corazón"
##	[169]	"corte"	"cosa"	"cosas"	"costa"
##	[173]	"creación"	"crecimiento"	"creo"	"crisis"
##	[177]	"cuadro"	"cual"	"cuales"	"cualquier"
##	[181]	"cuando"	"cuanto"	"cuarto"	"cuatro"
##	[185]	"cuba"	"cuenta"	"cuerpo"	"cuestión"
##	[189]	"cultura"	"cultural"	"curso"	"cuya"
##	[193]	"cuyo"	"d"	"da"	"dado"
##	[197]	"dan"	"dar"	"datos"	"de"
##	[201]	"debe"	"deben"	"debía"	"debido"
##	[205]	"decía"	"decir"	"decisión"	"defensa"
##	[209]	"deja"	"dejar"	"dejó"	"del"
##	[213]	"demás"	"demasiado"	"democracia"	"dentro"
##	[217]	"derecha"	"derecho"	"derechos"	"desarrollo"
##	[221]	"desde"	"deseo"	"después"	"destino"
##	[225]	"día"	"diario"	"días"	"dice"
##	[229]	"dicen"	"dicho"	"diciembre"	"diez"
##	[233]	"diferencia"	"diferentes"	"difícil"	"dijo"
##	[237]	"dinero"	"dio"	"dios"	"dirección"
##	[241]	"director"	"distintas"	"distintos"	"diversas"
##	[245]	"diversos"	"doctor"	"dólares"	"dolor"
##	[249]	"domingo"	"don"	"donde"	"dónde"
##	[253]	"dos"	"duda"	"durante"	"e"
##	[257]	"economía"	"económica"	"económico"	"edad"

##	[261]	"educación"	"efecto"	"efectos"	"ejemplo"
##	[265]	"ejército"	"el"	"él"	"elecciones"
##	[269]	"electoral"	"elementos"	"ella"	"ellas"
##	[273]	"ello"	"ellos"	"embargo"	"empresa"
##	[277]	"empresas"	"en"	"encima"	"encontrar"
##	[281]	"encuentra"	"encuentran"	"encuentro"	"energía"
##	[285]	"enero"	"enfermedad"	"entonces"	"entrada"
##	[289]	"entrar"	"entre"	"época"	"equipo"
##	[293]	"era"	"eran"	"es"	"esa"
##	[297]	"esas"	"escuela"	"ese"	"esfuerzo"
##	[301]	"eso"	"esos"	"espacio"	"españa"
##	[305]	"español"	"española"	"españoles"	"especial"
##	[309]	"especialmente"	"especie"	"espera"	"esta"
##	[313]	"está"	"ésta"	"estaba"	"estaban"
##	[317]	"estado"	"estados"	"estamos"	"están"
##	[321]	"estar"	"estas"	"este"	"éste"
##	[325]	"estilo"	"esto"	"estos"	"estoy"
##	[329]	"estructura"	"estudio"	"estudios"	"estuvo"
##	[333]	"etapa"	"etc"	"europa"	"europea"
##	[337]	"evitar"	"ex"	"existe"	"existen"
##	[341]	"existencia"	"éxito"	"experiencia"	"explicó"
##	[345]	"expresión"	"exterior"	"fácil"	"falta"
##	[349]	"familia"	"favor"	"fecha"	"felipe"
##	[353]	"fernando"	"figura"	"fin"	"final"
##	[357]	"finalmente"	"fiscal"	"flores"	"fondo"
##	[361]	"forma"	"formación"	"formas"	"francia"
##	[365]	"francisco"	"frente"	"fue"	"fuego"
##	[369]	"fuentes"	"fuera"	"fueron"	"fuerte"
##	[373]	"fuerza"	"fuerzas"	"función"	"fútbol"
##	[377]	"futuro"	"g"	"garcía"	"general"
##	[381]	"generales"	"gente"	"gobierno"	"gonzález"
##	[385]	"gracias"	"grado"	"gran"	"grande"
##	[389]	"grandes"	"grupo"	"grupos"	"guerra"
##	[393]	"ha"	"haber"	"había"	"habían"
##	[397]	"habla"	"hablar"	"habrá"	"habría"
##	[401]	"hace"	"hacen"	"hacer"	"hacerlo"
##	[405]	"hacia"	"hacía"	"haciendo"	"han"
##	[409]	"has"	"hasta"	"hay"	"haya"
##	[413]	"he"	"hecho"	"hechos"	"hemos"
##	[417]	"hermano"	"hicieron"	"hija"	"hijo"
##	[421]	"hijos"	"historia"	"hizo"	"hombre"
##	[425]	"hombres"	"hora"	"horas"	"hospital"
##	[429]	"hoy"	"hubiera"	"hubo"	"humana"
##	[433]	"humano"	"humanos"	"i"	"iba"
##	[437]	"idea"	"ideas"	"iglesia"	"igual"
##	[441]	"ii"	"imagen"	"imágenes"	"importancia"

##	[445]	"importante"	"importantes"	"imposible"	"incluso"
##	[449]	"industria"	"información"	"informe"	"instituciones"
##	[453]	"instituto"	"interés"	"intereses"	"interior"
##	[457]	"internacional"	"investigación"	"ir"	"izquierda"
##	[461]	"j"	"jamás"	"jefe"	"jorge"
##	[465]	"josé"	"joven"	"jóvenes"	"juan"
##	[469]	"juego"	"juez"	"juicio"	"julio"
##	[473]	"junio"	"junto"	"justicia"	"la"
##	[477]	"lado"	"larga"	"largo"	"las"
##	[481]	"le"	"lejos"	"lenguaje"	"les"
##	[485]	"ley"	"libertad"	"libre"	"libro"
##	[489]	"libros"	"líder"	"línea"	"llama"
##	[493]	"llamado"	"llega"	"llegado"	"llegar"
##	[497]	"llegó"	"lleva"	"llevar"	"lo"
##	[501]	"local"	"lópez"	"los"	"lucha"
##	[505]	"luego"	"lugar"	"luis"	"luz"
##	[509]	"m"	"madre"	"madrid"	"mal"
##	[513]	"manera"	"mano"	"manos"	"mantener"
##	[517]	"manuel"	"mañana"	"mar"	"marcha"
##	[521]	"marco"	"maría"	"martín"	"marzo"
##	[525]	"más"	"materia"	"material"	"máximo"
##	[529]	"mayo"	"mayor"	"mayores"	"mayoría"
##	[533]	"me"	"media"	"mediante"	"médico"
##	[537]	"medida"	"medidas"	"medio"	"medios"
##	[541]	"mejor"	"mejores"	"memoria"	"menor"
##	[545]	"menos"	"mercado"	"mes"	"mesa"
##	[549]	"meses"	"metros"	"méxico"	"mi"
##	[553]	"mí"	"miedo"	"miembros"	"mientras"
##	[557]	"miguel"	"mil"	"militar"	"militares"
##	[561]	"millones"	"ministerio"	"ministro"	"minutos"
##	[565]	"mira"	"mirada"	"mis"	"misma"
##	[569]	"mismo"	"mismos"	"mitad"	"modelo"
##	[573]	"modo"	"momento"	"momentos"	"movimiento"
##	[577]	"mucho"	"muchas"	"mucho"	"muchos"
##	[581]	"muerte"	"muerto"	"muestra"	"mujer"
##	[585]	"mujeres"	"mundial"	"mundo"	"música"
##	[589]	"muy"	"n"	"nacional"	"nada"
##	[593]	"nadie"	"natural"	"naturaleza"	"necesario"
##	[597]	"necesidad"	"negro"	"ni"	"ningún"
##	[601]	"ninguna"	"niño"	"niños"	"nivel"
##	[605]	"niveles"	"no"	"noche"	"nombre"
##	[609]	"norte"	"nos"	"nosotros"	"noviembre"
##	[613]	"nuestra"	"nuestras"	"nuestro"	"nuestros"
##	[617]	"nueva"	"nuevas"	"nuevo"	"nuevos"
##	[621]	"número"	"nunca"	"o"	"objetivo"
##	[625]	"objeto"	"obra"	"obras"	"obstante"

##	[629]	"ocasión"	"ocasiones"	"ocho"	"octubre"
##	[633]	"oficial"	"ojos"	"operación"	"opinión"
##	[637]	"oposición"	"orden"	"organización"	"origen"
##	[641]	"oro"	"otra"	"otras"	"otro"
##	[645]	"otros"	"p"	"pablo"	"paciente"
##	[649]	"pacientes"	"padre"	"padres"	"país"
##	[653]	"países"	"palabra"	"palabras"	"papel"
##	[657]	"para"	"parece"	"parecía"	"parís"
##	[661]	"parte"	"partes"	"participación"	"particular"
##	[665]	"partido"	"partidos"	"partir"	"pasa"
##	[669]	"pasado"	"pasar"	"paso"	"pasó"
##	[673]	"paz"	"pedro"	"película"	"pensar"
##	[677]	"peor"	"pequeña"	"pequeño"	"perdido"
##	[681]	"período"	"permite"	"pero"	"persona"
##	[685]	"personal"	"personas"	"pesar"	"pese"
##	[689]	"pesetas"	"peso"	"pie"	"piel"
##	[693]	"plan"	"plaza"	"plazo"	"población"
##	[697]	"poco"	"pocos"	"podemos"	"poder"
##	[701]	"podía"	"podrá"	"podría"	"policia"
##	[705]	"política"	"políticas"	"político"	"políticos"
##	[709]	"pone"	"poner"	"popular"	"por"
##	[713]	"porque"	"posibilidad"	"posibilidades"	"posible"
##	[717]	"posición"	"pp"	"práctica"	"precio"
##	[721]	"precios"	"precisamente"	"pregunta"	"premio"
##	[725]	"prensa"	"presencia"	"presenta"	"presente"
##	[729]	"presidente"	"primer"	"primera"	"primeras"
##	[733]	"primero"	"primeros"	"principal"	"principales"
##	[737]	"principio"	"principios"	"problema"	"problemas"
##	[741]	"proceso"	"producción"	"produce"	"producto"
##	[745]	"productos"	"profesional"	"programa"	"programas"
##	[749]	"pronto"	"propia"	"propio"	"propios"
##	[753]	"propuesta"	"próximo"	"proyecto"	"proyectos"
##	[757]	"prueba"	"psoe"	"pública"	"público"
##	[761]	"pudo"	"pueblo"	"pueda"	"puede"
##	[765]	"pueden"	"puedo"	"puerta"	"puerto"
##	[769]	"pues"	"puesto"	"punto"	"puntos"
##	[773]	"puso"	"que"	"qué"	"queda"
##	[777]	"quedó"	"quería"	"quien"	"quién"
##	[781]	"quienes"	"quiere"	"quiero"	"quizá"
##	[785]	"r"	"radio"	"rafael"	"razón"
##	[789]	"razones"	"real"	"realidad"	"realizar"
##	[793]	"recuerdo"	"recursos"	"reforma"	"régimen"
##	[797]	"región"	"relación"	"relaciones"	"república"
##	[801]	"respecto"	"respuesta"	"resto"	"resulta"
##	[805]	"resultado"	"resultados"	"reunión"	"revolución"
##	[809]	"rey"	"riesgo"	"río"	"rodríguez"

##	[813]	"rosa"	"s"	"sabe"	"saber"
##	[817]	"sabía"	"sala"	"salida"	"salir"
##	[821]	"salud"	"san"	"sangre"	"santa"
##	[825]	"santiago"	"se"	"sé"	"sea"
##	[829]	"sean"	"secretario"	"sector"	"sectores"
##	[833]	"seguir"	"según"	"segunda"	"segundo"
##	[837]	"seguridad"	"seguro"	"seis"	"semana"
##	[841]	"semanas"	"sentido"	"señaló"	"señor"
##	[845]	"señora"	"septiembre"	"ser"	"será"
##	[849]	"serán"	"sería"	"serie"	"servicio"
##	[853]	"servicios"	"si"	"sí"	"sido"
##	[857]	"siempre"	"siendo"	"siete"	"siglo"
##	[861]	"sigue"	"siguiente"	"siguientes"	"silencio"
##	[865]	"sin"	"sino"	"siquiera"	"sistema"
##	[869]	"sistemas"	"situación"	"sobre"	"social"
##	[873]	"sociales"	"socialista"	"sociedad"	"sol"
##	[877]	"sola"	"solamente"	"solo"	"sólo"
##	[881]	"solución"	"somos"	"son"	"soy"
##	[885]	"su"	"suelo"	"sueño"	"suerte"
##	[889]	"suficiente"	"superior"	"supuesto"	"sur"
##	[893]	"sus"	"tal"	"tales"	"también"
##	[897]	"tampoco"	"tan"	"tanto"	"tarde"
##	[901]	"te"	"teatro"	"técnica"	"televisión"
##	[905]	"tema"	"temas"	"tendrá"	"tenemos"
##	[909]	"tener"	"tengo"	"tenía"	"tenían"
##	[913]	"tenido"	"teoría"	"tercera"	"términos"
##	[917]	"texto"	"tiempo"	"tiempos"	"tiene"
##	[921]	"tienen"	"tierra"	"tipo"	"tipos"
##	[925]	"título"	"toda"	"todas"	"todavía"
##	[929]	"todo"	"todos"	"toma"	"tomar"
##	[933]	"torno"	"total"	"trabajadores"	"trabajar"
##	[937]	"trabajo"	"tras"	"trata"	"tratamiento"
##	[941]	"través"	"tres"	"tribunal"	"tu"
##	[945]	"tú"	"tuvo"	"u"	"última"
##	[949]	"último"	"últimos"	"un"	"una"
##	[953]	"unas"	"única"	"único"	"unidad"
##	[957]	"unidos"	"unión"	"universidad"	"uno"
##	[961]	"unos"	"uso"	"usted"	"va"
##	[965]	"valor"	"valores"	"vamos"	"van"
##	[969]	"varias"	"varios"	"ve"	"veces"
##	[973]	"ver"	"verdad"	"vez"	"vía"
##	[977]	"viaje"	"victoria"	"vida"	"viejo"
##	[981]	"viene"	"vino"	"vio"	"violencia"
##	[985]	"visita"	"vista"	"visto"	"vivir"
##	[989]	"voluntad"	"volver"	"volvió"	"voy"
##	[993]	"voz"	"vuelta"	"vuelve"	"y"

```
## [997] "ya" "yo" "zona" "zonas"
```

```
# Encuentra todas las palabras que contengan al menos una vocal, y luego niégalo
sin_vocales_1 <- !str_detect(palabras, "[aáeéiíoóuúüü]")
```

```
sin_vocales_1
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [73] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE
## [97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [157] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [169] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [181] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [193] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [205] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [217] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [229] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [241] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [253] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [265] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [277] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [289] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [301] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [313] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [325] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [337] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [349] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [361] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [373] FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE
## [385] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [397] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [409] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [421] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [433] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

[illegible]


```
## [997] FALSE FALSE FALSE FALSE
# Encuentra todas las palabras consistentes solo en consonantes (no vocales)
sin_vocales_2 <- str_detect(palabras, "[^aáeéííoóuúü]+")

head(identical(sin_vocales_1, sin_vocales_2), 100)

## [1] TRUE
```

Suma la cantidad de palabras que no tienen vocales?

```
library(tidyverse)
df <- tibble(
  palabra = palabras,
  i = seq_along(palabra)
) ## poner los datos en un dataframe

df %>%
  filter(str_detect(palabras, "x$")) # filtrar todas la palabras para solamente las terminan en x

## # A tibble: 1 x 2
##   palabra      i
##   <chr>    <int>
## 1 ex      338

df %>%
  filter(str_detect(palabras, "z$")) # filtrar todas la palabras para solamente las terminan en z

## # A tibble: 10 x 2
##   palabra      i
##   <chr>    <int>
## 1 capaz      108
## 2 diez       232
## 3 gonzález   384
## 4 juez       470
## 5 lópez      502
## 6 luz        508
## 7 paz        673
## 8 rodríguez  812
## 9 vez        975
## 10 voz       993
```

```
library(readr)
fragmentos_neruda_sep <- read_csv("Datos/fragmentos_neruda_sep.csv")

fragmentos_neruda_sep
```

```
## # A tibble: 10 x 2
##       id texto
##   <dbl> <chr>
## 1     1 "\"Puedo\" \"escribir\" \"los\" \"versos\" \"más\" \"tristes\" \"esta\" ~
## 2     2 "\"Es\" \"tan\" \"corto\" \"el\" \"amor,\" \"y\" \"es\" \"tan\" \"larg~
## 3     3 "\"Me\" \"gustas\" \"cuando\" \"callas\" \"porque\" \"estás\" \"como\" ~
## 4     4 "\"Para\" \"que\" \"nada\" \"nos\" \"separe\" \"que\" \"nada\" \"nos\" ~
## 5     5 "\"En\" \"un\" \"beso\" \"sabrás\" \"todo\" \"lo\" \"que\" \"he\" \"ca~
## 6     6 "\"Te\" \"amo\" \"como\" \"se\" \"aman\" \"ciertas\" \"cosas\" \"oscur~
## 7     7 "\"Quiero\" \"hacer\" \"contigo\" \"lo\" \"que\" \"la\" \"primavera\" ~
## 8     8 "\"Algún\" \"día\" \"en\" \"cualquier\" \"parte,\" \"en\" \"cualquier\" ~
## 9     9 "\"Si\" \"nada\" \"nos\" \"salva\" \"de\" \"la\" \"muerte,\" \"al\" \"~
## 10    10 "\"Podrán\" \"cortar\" \"todas\" \"las\" \"flores,\" \"pero\" \"no\" \"~
```

Contar cuantas “a” hay en cada palabra con `str_count()`

```
#x= c( "Piña", "Manzana", "Toronja", "Jugo")
```

```
str_count(fragmentos_neruda_sep, "e")
```

```
##       id texto
##       0      57
```

```
# En promedio, ¿cuántas vocales hay por palabra?
mean(str_count(palabras, "[aáeéííoóuü]"))
```

```
## [1] 2.72
```

```
mean(str_count(palabras, "[x]"))
```

```
## [1] 0.013
```

Contar cuantas vocales y consonantes hay en cada palabra

```
df %>%
  mutate(
    vocales = str_count(palabra, "[aáeéííoóuü]"), # Cuenta cuantas vocales
    consonantes = str_count(palabra, "[^aáeéííoóuü]") # cuenta cuantos consonantes
  ) |> head(n=30)
```

```
## # A tibble: 30 x 4
##   palabra      i vocales consonantes
##   <chr>      <int>   <int>      <int>
## 1 a          1       1          0
## 2 abril      2       2          3
## 3 acción      3       3          3
```

```
## 4 acciones      4      4      4
## 5 acerca       5      3      3
## 6 actitud      6      3      4
## 7 actividad    7      4      5
## 8 actividades  8      5      6
## 9 acto         9      2      2
## 10 actual      10     3      3
## # i 20 more rows
```

- los puntos y caracteres especiales

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.2.7.1 del libro en español

22.10 Anclas

Sin ancla, un patrón se busca **en cualquier parte** de la cadena. Las anclas dicen dónde tiene que estar:

- `^` al principio
- `$` al final

`^` y `$`: anclar el patrón a un extremo de la cadena.

- `"^a"`: empieza por a.
- `"a$"`: termina en a.
- `"^abc$"`: la cadena es **exactamente** abc, ni una letra más.
- **En inglés hay una regla mnemotécnica que se recuerda sola: *if you begin with power (^), you end up with money (\$)*.**

```
palabras |> str_subset("^y") |> head()      # empiezan por y
```

```
## [1] "y" "ya" "yo"
```

```
palabras |> str_subset("z$") |> head()      # terminan en z
```

```
## [1] "capaz" "diez" "gonzález" "juez" "lópez" "luz"
```

```
palabras |> str_subset("^...$") |> head()   # exactamente tres letras
```

```
## [1] "ahí" "año" "así" "aun" "aún" "con"
```

Sin anclas, `str_detect(x, "sol")` encuentra también "solución", "consola" y "girasol". Es el error más común al empezar con expresiones regulares: uno cree que está buscando una palabra y en realidad está buscando un pedazo de palabra.

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.2.8.1 del libro en español

22.11 Clases de caracteres y alternativas

Una **clase de caracteres** dice “aquí va uno cualquiera de estos”. Una **alternativa** dice “aquí va esto o aquello”.

- `\d`
- `\s`
- `[abc]`
- `[^abc]`

Clases de caracteres: un solo carácter, escogido de un conjunto.

- `\d`: cualquier dígito. `\s`: cualquier espacio en blanco (espacio, tabulador, salto de línea).
- `[abc]`: la a, la b o la c. `[^abc]`: cualquier carácter **menos** esos tres.
- `[a-z]`, `[0-9]`: rangos. `.`: cualquier carácter, sea el que sea.
- `gato|perro`: alternativa. Con paréntesis se acota el alcance, como en `c(ó|o)mo`, que es lo que hace el ejemplo de abajo.

Estos son los caracteres que tienen significado especial y que hay que **escapar** si quieres buscarlos tal cual:

- `$ . | ? * + ( ) [ {`

```
str_view(c("cómo", "como", "Raymond"), "c(ó|o)mo")
```

```
## [1] | <cómo>
## [2] | <como>
```

Para buscar un punto de verdad hay que escribir el patrón con **dos** barras invertidas. La razón es que hay dos traducciones seguidas: primero R lee la cadena y convierte las dos barras en una, y solo entonces la expresión regular lee esa barra como “el carácter siguiente es literal”. Es el tropiezo clásico del capítulo, y si te pasa no lo estás haciendo mal: le pasa a todo el mundo.

```
palabras |> str_subset("^[aeiou]") |> head() # empiezan por vocal
```

```
## [1] "a"          "abril"      "acción"     "acciones"   "acerca"     "actitud"
```

```
str_detect("El precio es $19.99", "\\$") # un signo de dólar literal
```

```
## [1] TRUE
```

```
str_detect(c("R.4", "R4"), "R\\.4") # un punto literal
```

```
## [1] TRUE FALSE
```

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.2.9.1 del libro en español

22.12 Repetición

Los cuantificadores dicen **cuántas veces** se repite lo que va justo antes: una letra, una clase de caracteres o un grupo entre paréntesis.

- `?`, 0 o 1, presencia o ausencia
- `+`, 1 o más
- `*`, 0 o más
- `str_view()`
- `{n}` : exactamente `n`
- `{n, }`: `n` o más
- `{ , m}`: no más de `m`
- `{n, m}`: entre `n` y `m`
- `{n, m}?`: la cadena más larga
- `{n, m}+?`: la cadena más corta

Los tres primeros se usan el 95 % de las veces y vale la pena aprenderlos de memoria: `?` es “quizás”, `+` es “al menos uno” y `*` es “los que sea, incluso ninguno”. Las llaves son para cuando necesitas un número exacto, como en un código de tres letras ("`[A-Z]{3}`") o un año ("`\\d{4}`").

```
palabras |> str_subset("[aeiou]{3}") |> head()      # tres vocales seguidas

## [1] "alguien"    "cualquier" "izquierda" "quien"      "quienes"    "quiere"
palabras |> str_subset("^.{10,}$") |> head()        # diez letras o más

## [1] "actividades"      "administración" "asociación"      "autoridades"
## [5] "características" "comunicación"

str_extract("Muestreo del 2016 al 2024", "\\d{4}")

## [1] "2016"
```

Por defecto los cuantificadores son **glotones**: se llevan todo lo que pueden. En "`<a>`" y "`<b>`", el patrón "`<.+>`" no encuentra `<a>` sino `<a>` y `<b>`, porque `+` se comió también lo del medio. Añadiendo un `?` detrás ("`<.+?>`") se vuelven perezosos y paran en cuanto pueden.

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.2.10.1 del libro en español

22.13 Agrupamiento y referencias previas

Los paréntesis hacen dos cosas a la vez. La primera ya la vimos: **agrupan**, para que un cuantificador o una alternativa se aplique a todo el grupo. La segunda es que R **recuerda** lo que encontró dentro de cada paréntesis, y se puede volver a usar más adelante en el mismo patrón con `\\1`, `\\2`, etc. Eso se llama una **referencia previa**.

`(..)\\1` quiere decir: “dos caracteres cualesquiera, y otra vez esos mismos dos”. Es decir, busca pares de letras repetidos, como el **nana** de banana.

```
• str_view(frutas, "(.)\\1", match=TRUE)

palabras |> str_subset("(.)\\1") |> head()           # un par de letras repetido

## [1] "alrededor" "análisis" "crisis" "intereses" "meses" "tienen"
palabras |> str_subset("(.)\\1") |> head()           # una letra doble: ll, rr, cc

## [1] "acción" "acciones" "allá" "allí" "aquella" "aquellas"
```

Las referencias previas sirven sobre todo para **encontrar duplicados**: palabras con una sílaba repetida, nombres escritos dos veces seguidas en una celda, un código que se copió por error. Y con `str_replace()` sirven para reordenar: el patrón captura dos pedazos y el reemplazo los pone al revés con `\\2 \\1`.

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.2.11.1 del libro en español

22.14 Herramientas: Objetivos

Con las expresiones regulares aprendidas, **stringr** ofrece cinco familias de funciones. Casi todo lo que se hace con texto es una de estas cinco cosas:

Objetivo	Función	Qué devuelve
¿Coincide?	<code>str_detect()</code> , <code>str_subset()</code>	TRUE/FALSE, o las cadenas que sí
¿Dónde?	<code>str_locate()</code>	las posiciones
¿Qué encontró?	<code>str_extract()</code> , <code>str_extract_all()</code>	el texto que coincidió
Por partes	<code>str_match()</code>	una columna por cada grupo entre paréntesis
Cambiar	<code>str_replace()</code> , <code>str_replace_all()</code>	la cadena con el cambio hecho
Partir	<code>str_split()</code>	una lista de pedazos

Casi todas tienen una versión `_all`: la corriente actúa sobre la **primera** coincidencia de cada cadena y la `_all` sobre **todas**.

- Determinar qué cadenas coinciden con un patrón.
- Encontrar la posición de una coincidencia.
- Extraer el contenido de las coincidencias.
- Remplazar coincidencias con nuevos valores.
- Dividir una cadena de acuerdo a una coincidencia.

22.14.1 Detectar coincidencias

- `str_detect(x, "e")`

```
library(datos)
#fruit
head(palabras)

## [1] "a"          "abril"      "acción"     "acciones"  "acerca"    "actitud"
# Encuentra todas las palabras que contengan al menos una vocal, y luego niégalo
sin_vocales_1 <- !str_detect(palabras, "[aáeéííoóuúü]")

#sin_vocales_1
# Encuentra todas las palabras consistentes solo en consonantes (no vocales)
sin_vocales_2 <- str_detect(palabras, "^[^aáeéííoóuúü]+$")
#sin_vocales_1
head(identical(sin_vocales_1, sin_vocales_2), 30)

## [1] TRUE
#> [1] TRUE
```

- `str_subset()`
- `seq_along()`
- `str_count()`

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.3.2 del libro en español

Ejercicio

22.15 Extraer coincidencias

`str_detect()` contesta sí o no; `str_extract()` devuelve el **pedazo de texto** que coincidió. Es la función que convierte texto sucio en datos: sacar el año

de una cita, el número de una etiqueta de herbario, el código de parcela de un nombre de archivo.

- `str_extract()`
- `str_extract_all()`

`str_extract()` y `str_extract_all()` (stringr): sacar lo que coincidió.

- **`str_extract(x, patron)`**: devuelve la **primera** coincidencia de cada cadena, y NA si no hubo ninguna.
- **`str_extract_all(x, patron)`**: devuelve **todas**, en una lista. Con `simplify = TRUE` devuelve una matriz.
- **Ojo**: que devuelva NA no es un error, significa que esa cadena no tenía nada que extraer. Cuéntalos.

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.3.3.1 del libro en español

22.16 Coincidencias agrupadas:

`str_match()` va un paso más allá que `str_extract()`: además de la coincidencia completa, devuelve **una columna por cada paréntesis** del patrón. Es la manera de partir un texto con estructura en varias variables de una sola vez.

En el ejemplo, el patrón tiene dos grupos: el artículo (`el|la|los|...`) y la palabra que le sigue. `str_extract()` devolvería las dos cosas pegadas; `str_match()` las devuelve en columnas separadas.

- `str_match*`

```
sustantivo <- "(el|la|los|las|lo|un|una|unos|unas) ([^ ]+)"
```

```
tiene_sustantivo <- oraciones %>%
  str_subset(sustantivo)
```

```
#tiene_sustantivo
tiene_sustantivo %>%
  str_extract(sustantivo)
```

## [1]	"los de"	"el camión"	"la mejor"	"la cuenta"	"las ruinas."
## [6]	"la hoja"	"la cocina."	"la taza"	"el tanque."	"el calor"
## [11]	"el aire"	"lo en"	"el papel"	"la parte"	"el sofá"
## [16]	"lo de"	"lo curó"	"los actores."	"la azul"	"el banco"
## [21]	"una llama"	"la cabeza."	"la ventana."	"la hierba"	"el sol"
## [26]	"el lugar"	"el sofá"	"un suéter"	"el gráfico"	"los encajes"
## [31]	"el verde"	"la caja"	"la pistola"	"los niños"	"una delgada"
## [36]	"lo en"	"la hierba"	"la primavera"	"la es"	


```
#> [1] "los de"      "el camión"    "la mejor"     "la cuenta"    "las ruinas."
#> [6] "la hoja"     "la cocina."  "la taza"      "el tanque."   "el calor"
```

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.3.4.1 del libro en español

22.17 Reemplazar coincidencias

- `str_replace()`
- `str_replace_all()`

`str_replace()` y `str_replace_all()` (stringr): cambiar lo que coincidió.

- `str_replace(x, patron, nuevo)`: cambia la **primera** coincidencia de cada cadena.
- `str_replace_all(x, patron, nuevo)`: cambia **todas**. Es la que se usa casi siempre.
- **Varios cambios a la vez**: `str_replace_all(x, c("a" = "1", "b" = "2"))`, con un vector con nombres.
- **En el reemplazo** se pueden usar las referencias previas: `\\1`, `\\2`, para reordenar lo que el patrón capturó.

```
str_replace_all("uno-dos-tres", "-", " ")
```

```
## [1] "uno dos tres"
```

```
str_replace_all(c("2016-08-01", "2016-06-15"), "(\\d{4})-(\\d{2})-(\\d{2})", "\\3/\\2/\\1")
```

```
## [1] "01/08/2016" "15/06/2016"
```

Antes de reemplazar en serio, **mira lo que vas a cambiar**: corre primero `str_subset()` o `str_view()` con el mismo patrón. Un reemplazo mal hecho es de las pocas operaciones de este curso que no se pueden deshacer.

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.3.5.1 del libro en español

22.18 Divisiones

`str_split()` parte una cadena en pedazos. La diferencia con `separate()`, del capítulo de datos ordenados, es que `separate()` trabaja sobre una **columna de una tabla** y `str_split()` sobre un **vector cualquiera**, y devuelve una lista porque cada cadena puede partirse en un número distinto de pedazos.

`boundary("word")` es la manera correcta de partir un texto en palabras: en vez de partir por el espacio, entiende dónde acaba una palabra de verdad, sin dejarse la puntuación pegada. Es lo que usan los ejemplos del Quijote más adelante.

- `str_split()`
- `str_split(" ", simplify=TRUE)`
- `str_split(x, boundary("word"))[[1]]`

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.3.6.1 del libro en español

22.19 Otro tipos de patrones

Todas las funciones de **stringr** aceptan, en vez de un patrón normal, una de estas envolturas:

- `regex("x", ignore_case=TRUE)`
- separar números telefónicos

Envolturas del patrón: cambiar cómo se interpreta lo que buscas.

- **`regex(x, ignore_case = TRUE)`:** no distinguir mayúsculas de minúsculas. Con `comments = TRUE` puedes escribir el patrón en varias líneas y comentarlo, cosa que se agradece en patrones largos.
- **`fixed("a.b")`:** buscar el texto **tal cual**, sin interpretar nada. Es más rápido y evita tener que escapar; el problema es que con acentos y letras no inglesas puede fallar.
- **`coll()`:** comparar respetando las reglas del idioma. Es la más lenta y la única correcta cuando hay que comparar texto en español teniendo en cuenta acentos y ñ.
- **`boundary("word")`:** buscar límites de palabra en vez de un patrón.

22.20 stringi:

stringr está construido encima de **stringi**, que es mucho más grande: unas 250 funciones frente a las 50 y pico de **stringr**. **stringr** escogió a propósito un subconjunto pequeño y con nombres consistentes, que es lo que hace falta el 95 % de las veces. Cuando te falte algo (transliterar, ordenar según un idioma, detectar el idioma de un texto), búscalo en **stringi**, donde las funciones empiezan por **stri_**.

Ejercicios del libro R4DS.

Hacer los ejercicios en la sección 14.3.6.1 del libro en español. Buscar 3 funciones en **stringi** y aplica y explica las funciones con un set de datos.

22.21 El texto de Quijote

22.21.1 Cómo importar un archivo .txt en R

```
quijote <- read_lines("Datos/el_quijote.txt")

head(quijote, n=10)

## [1] "DON QUIJOTE DE LA MANCHA"
## [2] "Miguel de Cervantes Saavedra"
## [3] ""
## [4] "PRIMERA PARTE"
## [5] "CAPÍTULO 1: Que trata de la condición y ejercicio del famoso hidalgo D. Quijote de la Ma
## [6] "En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que viv
## [7] "Tuvo muchas veces competencia con el cura de su lugar (que era hombre docto graduado en
## [8] "En resolución, él se enfrascó tanto en su lectura, que se le pasaban las noches leyendo
## [9] "historia más cierta en el mundo."
## [10] "Decía él, que el Cid Ruy Díaz había sido muy buen caballero; pero que no tenía que ver c
```

22.21.2 Cuántas oraciones tiene el text primer capitulo de Quijote?

```
length(quijote)
```

```
## [1] 2186
```

22.22 Cuántas palabras tiene el primer capítulo de Quijote?

```
quijote_palabras <- str_split(quijote, " ")

quijote_palabras_vector <- unlist(quijote_palabras)

head(quijote_palabras_vector) # observar las primeras palabras

## [1] "DON"      "QUIJOTE" "DE"      "LA"      "MANCHA"  "Miguel"
length(quijote_palabras_vector) # cuantas palabras hay en total

## [1] 187019
```

22.23 Cuántas veces aparece la palabra “caballero” en el primer capítulo de Quijote?

```
sum(str_detect(quijote_palabras_vector, "caballero"))
```

```
## [1] 475
```

22.24 ¿Cuántas palabras tienen más de 10 letras?

```
sum(str_length(quijote_palabras_vector) > 10)
```

```
## [1] 5326
```

22.25 Cual es la palabra más larga

```
quijote_palabras_vector %>%  
  str_length() %>%  
  which.max() %>%  
  quijote_palabras_vector[.]
```

```
## [1] "procuremos.Levántate,"
```

22.26 Cuántas veces aparece “Sancho Panza” en el primer capítulo de Quijote?

```
sum(str_detect(quijote, "Sancho Panza"))
```

```
## [1] 95
```

22.27 Cuántas veces aparece “Dios” en el primer capítulo de Quijote?

```
sum(str_detect(quijote, "Dios"))
```

```
## [1] 166
```

```
# Cuántas palabras tiene “ñ” en cualquier posición de la palabra?  
!str_detect(palabras, "[áéííoóuúü]")
```

```
str_detect(quijote_palabras_vector, "e") |> head() # cuantas veces aparece la letra e en cada pa

## [1] FALSE FALSE FALSE FALSE FALSE TRUE
sum(str_detect(quijote_palabras_vector, "e")) # cuantas veces aparece la letra e en total

## [1] 96484
```

22.28 Cuantas palabras tienen ñ?

```
sum(str_detect(quijote_palabras_vector, "[aáeéííoóuúü]")) # cuantas veces aparece la letra ñ en

## [1] 177633
sum(str_detect(quijote_palabras_vector, "[ñ]")) # cuantas veces aparece la letra ñ en total

## [1] 49227
```

22.29 Remover del texto todas las siguientes palabras

las palabras como “que”, “de”, “y”, “la”, “el”, “en”, “a”, “los”, “del”, “se”, “las”, “un”, “por”, “con”, “no”, “una”, “su”, “para”, “es”, “al”, “lo”, “como”, “más”, “pero”, “sus”, “le”, “ya”, “o”, “este”, “sí”, “porque”, “esta”, “entre”, “cuando”, “muy”, “sin”, “sobre”, “también”, “me”, “hasta”, “hay”, “donde”, “quien”, etc.

```
stop_words <- c("que", "de", "y", "la", "el", "en", "a", "los", "del", "se",
  "las", "un", "por", "con", "no", "una", "su", "para", "es",
  "al", "lo", "como", "más", "pero", "sus", "le", "ya", "o",
  "este", "sí", "porque", "esta", "entre", "cuando", "muy",
  "sin", "sobre", "también", "me", "hasta", "hay", "donde",
  "quien")

quijote_palabras_vector_limpio <- quijote_palabras_vector[!str_detect(quijote_palabras_vector,
  str_c("^", stop_words, "$", collapse = "|"))]

library(cranlogs)
cranlogs_badge("ggversa", summary = "grand-total")

## [1] "[! [CRAN RStudio mirror downloads] (https://cranlogs.r-pkg.org/badges/grand-total/ggversa?c)]"
cran_downloads(packages = "ggversa", when = "last-month")

##           date count package
## 1 2026-08-07      2 ggversa
## 2 2026-08-08     21 ggversa
## 3 2026-08-09      7 ggversa
```

```
## 4 2026-08-10      8 ggversa
## 5 2026-08-11      6 ggversa
## 6 2026-08-12     20 ggversa
## 7 2026-08-13      2 ggversa
## 8 2026-08-14      1 ggversa
## 9 2026-08-15      6 ggversa
## 10 2026-08-16     3 ggversa
## 11 2026-08-17     4 ggversa
## 12 2026-08-18     1 ggversa
## 13 2026-08-19     4 ggversa
## 14 2026-08-20     6 ggversa
## 15 2026-08-21    15 ggversa
## 16 2026-08-22     2 ggversa
## 17 2026-08-23     2 ggversa
## 18 2026-08-24     9 ggversa
## 19 2026-08-25     3 ggversa
## 20 2026-08-26     9 ggversa
## 21 2026-08-27    16 ggversa
## 22 2026-08-28     7 ggversa
## 23 2026-08-29    11 ggversa
## 24 2026-08-30     3 ggversa
## 25 2026-08-31     4 ggversa
## 26 2026-09-01     7 ggversa
## 27 2026-09-02    21 ggversa
## 28 2026-09-03     8 ggversa
## 29 2026-09-04    11 ggversa
## 30 2026-09-05     8 ggversa

# Replace start date with the earliest date ggversa is available on CRAN
df <- cran_downloads("ggversa", from = "2017-08-05", to = Sys.Date())

# Sum all daily downloads
total_downloads <- sum(df$count, na.rm = TRUE)
print(total_downloads)

## [1] 36824

# Desactivado para el build: install_github requiere credenciales de GitHub.
#install.packages("devtools")
devtools::install_github("arunsrinivasan/cran.stats")
library(cran.stats)

# Get logs for a specific day
dt <- read_logs(start = as.Date("2025-11-09"),
                 end   = as.Date("2025-11-09"))

# Filter for ggversa and summarize downloads by country
```

```

library(data.table)
dt <- as.data.table(dt)
dt_ggversa <- dt[package == "ggversa", .N, by = country]
print(dt_ggversa)

#install.packages("packageRank")
library(packageRank)

# Define target countries
target_countries <- c(
  "United States", "Puerto Rico", "Spain", "Mexico", "Argentina", "Colombia", "Chile", "Peru",
  "Venezuela", "Ecuador", "Guatemala", "Cuba", "Bolivia", "Honduras", "Paraguay", "El Salvador",
  "Nicaragua", "Costa Rica", "Panama", "Uruguay", "Dominican Republic", "Equatorial Guinea"
)

# Downloads for ggversa by country (latest day)
dates <- seq(as.Date("2025-10-01"), as.Date("2025-11-09"), by = "day")
results <- lapply(dates, function(d) {
  tryCatch({
    df <- packageCountry(packages = "ggversa", date = d)
    df$date <- d
    df
  }, error = function(e) NULL)
})

# Combine and filter
country_data <- bind_rows(results)
filtered_data <- country_data %>% filter(country %in% target_countries)

# Summarize
summary <- filtered_data %>%
  group_by(country) %>%
  summarise(total_downloads = sum(downloads)) %>%
  arrange(desc(total_downloads))

print(summary)

```

22.30 Contabilizar las palabras más comunes en el primer capítulo de Quijote

```
table(quijote_palabras_vector_limpio) %>%
  sort(decreasing = TRUE) %>%
  head(40)
```

##	quijote_palabras_vector_limpio							
##	si	más	mi	yo	tan	don	había	él
##	899	882	851	814	750	713	653	627
##	ni	todo	ser	ha	era	bien	vuestra	Y
##	617	509	466	460	452	451	445	407
##	todos	dijo	Don	fue	te	cual	así	sino
##	372	348	345	343	340	326	321	313
##	esto	Sancho	que,	Quijote	aquel	merced	dos	hacer
##	312	312	310	310	293	286	282	279
##	señor	ella	aunque	Quijote,	otra	he	nos	cosa
##	275	269	265	265	262	246	241	240

```
clean=str_replace_all(quijote_palabras_vector_limpio, "[,\\.\\!\\:\\'\\'\\?]", "")

table(clean) %>%
  sort(decreasing = TRUE) %>%
  head(40)
```

```
## clean
```

##	más	si	yo	mi	tan	él	don	había
##	958	917	886	876	750	737	718	668
##	Quijote	ni	dijo	Sancho	todo	bien	ser	así
##	668	620	579	560	560	507	503	489
##	era	esto	ha	vuestra	Y	todos	cual	merced
##	475	462	460	453	422	416	399	395
##	señor	fue	ella	Don	pues	te	sino	hacer
##	363	358	347	345	344	340	316	312
##	que	caballero	dos	aquel	otra	cosa	aunque	decir
##	310	309	299	296	280	274	265	264

```
\\b[^\s]*;[^\s]*\\b matches:
```

```
\\b: word boundary
```

```
[^\s]*: any number of non-space characters
```

```
;: the semicolon
```

```
[^\s]*: more non-space characters
```

```
\\b: word boundary
```

```
words_with_semicolon <- str_extract_all(clean, "\\b[^\s]*;[^\s]*\\b")[[1]]
```

```
words_with_semicolon
```

```
## character(0)
```


22.30. CONTABILIZAR LAS PALABRAS MÁS COMUNES EN EL PRIMER CAPITULO DE QUIJOTE329

Contabilizar la cantidad de veces los numeros aparecen en el texto

```
str(clean)
```

```
## chr [1:111341] "DON" "QUIJOTE" "DE" "LA" "MANCHA" "Miguel" "Cervantes" ...
```

```
library(purrr)
```

```
numeros <- c("uno", "dos", "tres", "cuatro", "cinco", "seis", "siete", "ocho", "nueve", "diez")
```

```
clean <- str_replace_all(clean, "[[:punct:]]", "")
```

```
clean_text <- paste(clean, collapse = " ")
```

```
counts <- sapply(numeros, function(n) str_count(tolower(clean_text), fixed(n)))
```

```
names(counts) <- numeros
```

```
print(counts)
```

```
##      uno      dos      tres cuatro      cinco      seis      siete      ocho      nueve      diez
##      445     1563      117       72       13       31        9       17        5       18
```

```
## [1] "*** START OF THE PROJECT GUTENBERG EBOOK 11 ***"
```

```
## [2] ""
```

```
## [3] "[Illustration]"
```

```
## [4] ""
```

```
## [5] ""
```

```
## [6] ""
```

```
## [7] ""
```

```
## [8] "Alice's Adventures in Wonderland"
```

```
## [9] ""
```

```
## [10] "by Lewis Carroll"
```

```
## [1] "childhood: and how she would gather about her other little children,"
```

```
## [2] "and make _their_ eyes bright and eager with many a strange tale,"
```

```
## [3] "perhaps even with the dream of Wonderland of long ago: and how she"
```

```
## [4] "would feel with all their simple sorrows, and find a pleasure in all"
```

```
## [5] "their simple joys, remembering her own child-life, and the happy summer"
```

```
## [6] "days."
```

```
## [7] ""
```

```
## [8] "THE END"
```

```
## [9] ""
```

```
## [10] "*** END OF THE PROJECT GUTENBERG EBOOK 11 ***"
```

22.30.1 Cuántas oraciones tiene el text de Alice?

```
## [1] 3384
```

22.30.2 Cuantas palabras tiene el texto de Alice?

```
## [1] "***"      "START"      "OF"         "THE"         "PROJECT"     "GUTENBERG"
## [1] 28017
```

22.30.3 Convertir todo a minúsculas

Determinar cuantas veces aparece la palabra “alice” en el texto

```
## [1] 399
```

22.30.4 ¿Cuántas palabras tienen más de 10 letras?

```
## [1] 391
```

22.30.5 Remover todos las puntuaciones en el texto, usando “[[:punct:]]”

```
## [1] ""      "start"  "of"     "the"    "project" "gutenberg"
```

22.30.6 Haz una lista de las 20 palabras más comunes en el texto de Alice

```
## Alice_palabras_vector_limpio
## the and to a she it of said i alice in you
## 1640 1538 846 721 632 537 526 511 462 401 385 367 360
## was that as her at on with
## 357 276 262 248 209 193 181
```

Remover los espacios vacíos

Haga una lista de las 20 palabras más comunes en el texto de Alice sin los espacios vacíos, para averguar que removiste los espacios vacíos

```
## Alice_palabras_vector_limpio
## the and to a she it of said i alice in you was
## 1640 846 721 632 537 526 511 462 401 385 367 360 357
## that as her at on with all
## 276 262 248 209 193 181 179
```

Ahora remueve todos los stopwords en ingles del texto de Alice usando la libreria stopwords

Ahora enseñe las 20 palabras más comunes en el texto de Alice sin los stopwords

```
## Alice_palabras_vector_limpio2
## said alice little one know like went thought queen time
## 462 385 129 101 85 85 83 74 68 68
## see king dont well began im mock now turtle gryphon
```

66 61 60 60 58 57 57 57 56 55

Using the following list of main characters in the book, count how many times each character is mentioned in the text of Alice in Wonderland

- Alice
- White Rabbit
- Mad Hatter
- Hatter
- Queen of Hearts
- Queen
- Cheshire Cat
- March Hare
- Caterpillar
- Dormouse
- King of Hearts
- Gryphon
- Mock Turtle
- Dodo
- Bill the Lizard
- Knave of Hearts

```
characters <- c("Alice", "White Rabbit", "Rabbit", "Mad Hatter", "Hatter", "Queen of Hearts", "Qu
               "Cheshire Cat", "Cat", "March Hare", "Caterpillar", "Dormouse",
               "King of Hearts", "King", "Gryphon", "Mock Turtle", "Dodo", "Knave of Hearts")
```

##	Alice	White Rabbit	Rabbit	Mad Hatter	Hatter
##	399	22	54	0	57
##	Queen of Hearts	Queen	Cheshire Cat	Cat	March Hare
##	4	77	6	90	31
##	Caterpillar	Dormouse	King of Hearts	King	Gryphon
##	29	40	0	179	55
##	Mock Turtle	Dodo	Knave of Hearts		
##	57	13	3		

22.31 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan `millas` y `países` del paquete `datos`, más un vector de códigos sucios que vas a escribir tú, no palabras, oraciones, el Quijote ni Alicia, que son los del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_22_strings.Rmd`.

Un aviso técnico: algunas columnas de texto llegan como **factor**, y las funciones `str_*` esperan texto. Si una te da un error raro, comprueba con `class()` y

envuélvela en `as.character()`.

22.31.1 Ejercicio 1. Lo básico

Datos: `millas`. Crea una columna `etiqueta` que junte `fabricante` y `modelo` con un espacio, en mayúsculas, y otra columna con su largo en caracteres.

Verificación: las dos columnas nuevas deben tener 234 valores y ningún NA. La etiqueta más larga debe pasar de 25 caracteres.

En palabras: ¿cuál es el modelo con el nombre más largo y cuál el más corto? ¿Por qué `str_length()` no es lo mismo que contar las letras a mano en un texto con acentos?

Escribe tu código aquí

22.31.2 Ejercicio 2. Anclas

Datos: los nombres de país de `países`, sin repetir. Encuentra los que empiezan por vocal, los que terminan en `a`, y los que cumplen **las dos** cosas.

Verificación: la tercera lista tiene que ser más corta que las otras dos, y todos sus elementos deben aparecer también en ellas.

En palabras: haz la misma búsqueda **sin** anclas, solo con `"a"`, y compara. Explica con un ejemplo concreto qué encontró de más.

Escribe tu código aquí

22.31.3 Ejercicio 3. Sacar el número de un código sucio

Escribe a mano este vector, que imita cómo llegan los códigos de parcela de una libreta de campo:

```
codigos <- c("P-01", "p 2", "PARCELA_3", " P04 ", "parcela 5", "P6")
```

Extrae el número de cada uno con `str_extract()` y conviértelo a numérico.

Verificación: debes obtener los números del 1 al 6, en ese orden, sin ningún NA.

En palabras: ¿por qué `"P-01"` y `"P6"` se resuelven con el mismo patrón, aunque uno tenga guion y ceros y el otro no? ¿Qué pasaría si un código fuera `"P-01-bis"`?

Escribe tu código aquí

22.31.4 Ejercicio 4. Normalizar

Sigue con `codigos`. Conviértelos todos a la forma P01, P02, ..., P06: quita los espacios de los extremos, pásalos a mayúscula, quita guiones y guiones bajos, y rellena el número a dos dígitos.

Verificación: los seis resultados deben ser exactamente "P01" "P02" "P03" "P04" "P05" "P06", y `n_distinct()` debe dar 6.

En palabras: antes de normalizar, `n_distinct(codigos)` también daba 6, pero eso era una coincidencia. Explica qué habría pasado si la libreta tuviera "P-01" y "p 1", que son la misma parcela.

Escribe tu código aquí

22.31.5 Ejercicio 5. Contar palabras

Toma esta frase: "El muestreo se hizo en 2016, 2017 y 2018.". Pártela en palabras de dos maneras: con `str_split(" ")` y con `str_split(boundary("word"))`.

Verificación: las dos maneras dan un número **distinto** de elementos.

En palabras: compara las dos listas elemento por elemento y explica qué hace distinto `boundary("word")`. ¿Cuál de las dos usarías para contar las palabras de un texto y por qué?

Escribe tu código aquí

22.31.6 Ejercicio 6. Las cinco herramientas, una sola pregunta

Datos: `millas`, columna `modelo`. La pregunta es siempre la misma, "¿cuáles modelos tienen un número en el nombre?", contestada de cinco maneras: `str_detect()`, `str_subset()`, `str_which()`, `str_extract()` y `str_replace()`.

Verificación: `sum(str_detect(...))`, `length(str_subset(...))` y `length(str_which(...))` tienen que dar **el mismo número**.

En palabras: las cinco funciones usan el mismo patrón y devuelven cosas distintas. Haz una tabla de dos columnas diciendo qué devuelve cada una y en qué situación la usarías.

Escribe tu código aquí

22.31.7 Ejercicio 7 (BONO). Referencias previas

Con los nombres de país de `países`, encuentra los que tienen una **letra doble** (dos iguales seguidas) y los que tienen un **par de letras repetido**.

Verificación: las dos listas son **distintas**, y ninguna contiene a la otra: tiene que haber al menos un nombre en cada una que no esté en la otra. Compruébalo con `setdiff()` en los dos sentidos.

En palabras: explica la diferencia entre los dos patrones. Después describe un caso de tus propios datos donde te serviría buscar algo repetido: piensa en nombres de especie tecleados dos veces, o en un código copiado por error.

Escribe tu código aquí

Chapter 23

Ecuaciones Matemáticas

[1] "2026-09-06"

23.1 Ecuaciones matemáticas

TeX es la mejor manera de escribir funciones matemáticas en R. Donald Knuth diseñó el paquete *T_EX* cuando se sintió frustrado por el tiempo que les tomaba a los tipógrafos y terminar su libro, que contenía muchas funciones matemáticas. Una característica interesante de *R Markdown* es su capacidad para leer código LaTeX directamente.

Si está haciendo una tesis, artículo científico, presentaciones o quiere aclarar en su documento las funciones que estás usando, típicamente esto involucrará muchas ecuaciones matemáticas. Por consecuencia querrá leer la siguiente sección que ha sido comentada.

El sistema *TeX* fue creado por Donald Knuth en 1978 porque no le convenía la calidad tipográfica de su libro *The Art of Computer Programming*. *LaTeX*, la variante que usamos hoy, fue construida sobre *TeX* por Leslie Lamport.

23.2 Temas

- Dónde se escribe una ecuación: en la línea o aparte
- Los seis comandos que resuelven el 90 % de las fórmulas
- Letras griegas y símbolos
- `\mathrm{}`: cuándo la cursiva estorba
- Ecuaciones numeradas y referencias cruzadas
- Dónde buscar un símbolo que no te sabes

23.3 Lo mínimo que hay que saber

Antes del catálogo de ejemplos, conviene tener claras seis cosas. Con estas seis se escriben casi todas las fórmulas de un artículo de biología.

Los comandos de LaTeX que más vas a usar.

- **Subíndice `_` y superíndice `^`:** `$x_i$` da x_i y `$x^2$` da x^2 .
- **Llaves `{}` para agrupar:** todo lo que ocupe **más de un carácter** va entre llaves. `$x_{10}$` da x_{10} , que está mal; `$x_{10}$` da x_{10} , que es lo que querías.
- **Fracción `\frac{arriba}{abajo}`:** `$\frac{a}{b}$` da $\frac{a}{b}$.
- **Raíz `\sqrt{}`:** `$\sqrt{n}$` da $\sqrt{n}$; con índice, `$\sqrt[3]{n}$`.
- **Sumatoria `\sum`, con límites en `_` y `^`:** `$\sum_{i=1}^n x_i$` da $\sum_{i=1}^n x_i$.
- **Acentos sobre una letra:** `$\bar{x}$` da $\bar{x}$ (promedio), `$\hat{y}$` da $\hat{y}$ (valor estimado), `$\tilde{x}$` da $\tilde{x}$.

El error número uno, con diferencia, son **las llaves que faltan**. `$e^{-2x}$` no es lo que parece: solo el signo menos sube al exponente. Hay que escribir `$e^{\{-2x\}}$`. Cada vez que un subíndice o un exponente tenga más de un carácter, llaves.

23.3.1 Griegas y otros símbolos

Las letras griegas se escriben con su nombre en inglés precedido de barra invertida. La minúscula va en minúscula y la mayúscula en mayúscula:

Se escribe	Sale	Se escribe	Sale
<code>\$\alpha\$</code>	α	<code>\$\mu\$</code>	μ
<code>\$\beta\$</code>	β	<code>\$\sigma\$</code>	σ
<code>\$\lambda\$</code>	λ	<code>\$\Sigma\$</code>	Σ
<code>\$\theta\$</code>	θ	<code>\$\Delta\$</code>	Δ
<code>\$\chi^2\$</code>	χ^2	<code>\$\infty\$</code>	∞
<code>\$\leq\$</code>	$\leq$	<code>\$\neq\$</code>	$\neq$
<code>\$\approx\$</code>	$\approx$	<code>\$\pm\$</code>	$\pm$

Ojo con `$\lambda$`: en modelos matriciales de población, la tasa de crecimiento λ es la letra griega, no la letra ele. Y `$\Sigma$` (la mayúscula, que significa “matriz de covarianza”) no es lo mismo que `$\sum$` (el operador de sumatoria), aunque se parezcan mucho en la pantalla.

23.3.2 Matemática y Física

El promedio $\bar{x} = \frac{\sum x_i}{n}$ se escribe como $\bar{x} = \frac{\sum x_i}{n}$.
 Nota que para poner una ecuación en una línea se usa solamente una \$ antes y después, si queremos que sea en una línea aparte se usa dos \$\$ antes y después.

$$\bar{x} = \frac{\sum x_i}{n}$$

$$\sum_{j=1}^n (\delta\theta_j)^2 \leq \frac{\beta_i^2}{\delta_i^2 + \rho_i^2} \left[2\rho_i^2 + \frac{\delta_i^2 \beta_i^2}{\delta_i^2 + \rho_i^2} \right] \equiv \omega_i^2$$

De Informational Dynamics, tenemos lo siguiente (Dave Braden):

Después de n tales encuentros, la densidad posterior de θ es

$$\pi(\theta|X_1 < y_1, \dots, X_n < y_n) \propto \pi(\theta) \prod_{i=1}^n \int_{-\infty}^{y_i} \exp\left(-\frac{(x-\theta)^2}{2\sigma^2}\right) dx$$

Otra Ecuaciones

$$\det \begin{vmatrix} c_0 & c_1 & c_2 & \cdots & c_n \\ c_1 & c_2 & c_3 & \cdots & c_{n+1} \\ c_2 & c_3 & c_4 & \cdots & c_{n+2} \\ \vdots & \vdots & \vdots & & \vdots \\ c_n & c_{n+1} & c_{n+2} & \cdots & c_{2n} \end{vmatrix} > 0$$

Lapidus y Píndaro, Solución numérica de ecuaciones diferenciales parciales en ciencia y Ingeniería. Página 54

$$\int_t \left\{ \sum_{j=1}^3 T_j \left(\frac{d\phi_j}{dt} + k\phi_j \right) - kT_e \right\} w_i(t) dt = 0, \quad i = 1, 2, 3.$$

Funciones de ponderación del método L&P Galerkin. Página 55

$$\sum_{j=1}^3 T_j \int_0^1 \left\{ \frac{d\phi_j}{dt} + k\phi_j \right\} \phi_i dt = \int_0^1 k T_e \phi_i dt, \quad i = 1, 2, 3$$

Otra de L&P (p145)

$$\int_{-1}^1 \int_{-1}^1 \int_{-1}^1 f(\xi, \eta, \zeta) = \sum_{k=1}^n \sum_{j=1}^n \sum_{i=1}^n w_i w_j w_k f(\xi, \eta, \zeta).$$

Otra de L&P (p126)

$$\int_{A_e} (\cdot) dx dy = \int_{-1}^1 \int_{-1}^1 (\cdot) \det[J] d\xi d\eta.$$

23.4 Símbolos de LATEX

Nadie se aprende los símbolos de memoria, ni hace falta. Lo que sí hay que saber es **dónde buscarlos**, y hay dos maneras. La primera es una lista como esta:

Pueden encontrar los símbolos de Latex matemáticos en la siguiente pagina web: https://oeis.org/wiki/List_of_LaTeX_mathematical_symbols

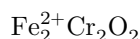
La segunda, y casi siempre la más rápida, es **dibujar el símbolo** en <https://detexify.kirelabs.org/classify.html>: uno lo traza con el ratón y la página contesta cómo se escribe. Es la herramienta que hay que enseñarles a los estudiantes el primer día.

23.5 Química 101: Símbolos

Las fórmulas químicas se verán mejor si no están en cursiva. Evite la cursiva automática del modo matemático en LaTeX usando el argumento `$\mathrm{fórmula aquí}$` , con su fórmula dentro de las llaves. (Observe el uso de comillas invertidas aquí que encierran texto y que actúa como código para ver el código original).

Entonces, $\text{Fe}_2^{2+}\text{Cr}_2\text{O}_4$ es escrito como `$\mathrm{Fe_2^{2+}Cr_{20_4}}$` .

Esta fórmula es parte de la frase



- Exponente or Sobrescrito: O^- y se escribe como `$\mathrm{O^{-}}$`
- Subíndice: CH_4 y se escribe como `$\mathrm{CH_4}$`
- Para apilar números o letras como en Fe_2^{2+} , primero se define el subíndice y luego el superíndice y se escribe como `$\mathrm{Fe_2^{2+}}$` .
- punto se usa la palabra **bullet**:

$\text{CuCl} \bullet 7\text{H}_2\text{O}$

donde se escribe como `$\mathrm{CuCl \bullet 7H_{20}}$`

23.6 Símbolos comunes

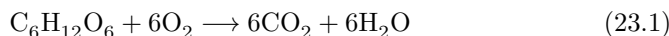
- Delta: Δ y se escribe como `$\Delta$`

- Gamma mayúscula Γ
- Gamma minúscula γ
- Aquí está mi súper fórmula de ϕ que voy a usar en el examen de matemáticas.
- Flecha de Reacción: $\longrightarrow$ or $\xrightarrow{\textit{fabulous}}$ y se escribe como `\longrightarrow` or `\xrightarrow{solution}`
- Flecha de Resonancia: $\leftrightarrow$ y se escribe como `\leftrightharpoonow`
- Flechas de reacción reversibles: $\rightleftharpoons$ y se escribe como `\rightleftharpoons`

23.6.1 Tipografía de reacciones

Es posible que desees poner tu reacción en un entorno de ecuaciones, lo que significa que LaTeX colocará la reacción donde encaje y numerará las ecuaciones por ti.

La numeración automática y la referencia cruzada (el comando `\@ref` seguido de la etiqueta de la ecuación entre paréntesis) son funciones de **bookdown**, no de R Markdown a secas. Funcionan en este libro porque está hecho con bookdown; en un `.Rmd` normal con salida `output: html_document` la ecuación sale sin número y la referencia aparece como texto sin resolver. Para que funcionen en un documento suelto hay que poner `output: bookdown:html_document2`.



y se escribe como

```
\begin{equation}
  \mathrm{C_6H_{12}O_6 + 6O_2} \longrightarrow \mathrm{6CO_2 + 6H_2O}
  (\#eq:reaction)
\end{equation}
```

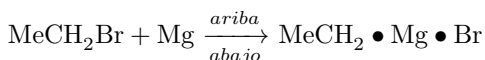
Podemos hacer referencia a esta reacción de combustión de glucosa mediante la ecuación (23.1).

23.6.2 Otros ejemplos de reacciones



y se escribe como

```
\mathrm{NH_4Cl_{(s)}} \rightleftharpoons \mathrm{NH_{3(g)}+HCl_{(g)}}
```



y se escribe como

$\mathrm{MeCH_2Br} + \mathrm{Mg} \xrightarrow[\text{abajo}]{\text{arriba}} \mathrm{MeCH_2\bullet Mg\bullet Br}$

23.7 Física

Muchos de los símbolos que necesitará se pueden encontrar en la página de matemáticas.

<https://web.reed.edu/cis/help/latex/math.html> y la guía completa de símbolos LaTeX (<https://mirror.utexas.edu/ctan/info/symbols/comprehensive/symbols-letter.pdf>).

23.8 Biología

Probablemente encontrará los recursos en <https://www.lecb.ncifcrf.gov/~toms/latex.html> útil, particularmente los enlaces a bsts de varias revistas. Quizás también le interese TeXShade para composición tipográfica de nucleótidos (<https://homepages.uni-tuebingen.de/beitz/txe.html>). Asegúrese de leer el capítulo anterior sobre gráficos y tablas.

La fórmula del promedio

$\mu = \frac{\sum x_i}{n}$ se escribe así $\mu = \frac{\sum\{x\}_{i}}{n}$

NOTE la diferencia en tipo de letra.....entre las dos fórmulas

Nota aquí que la formula se escribe en la oración $\mu = \frac{\sum x_i}{n}$ se escribe así $\mathrm{\mu = \frac{\sum\{x\}_{i}}{n}}$

Pero si quiere que la formula este en una linea aparte lo escribe así dentro de $\begin{equation}$, la formula aquí y termina con $\end{equation}$

$$\mu = \frac{\sum x_i}{n} \quad (23.2)$$

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$$

Sitios en el web para preparar las ecuaciones Latex

1. <https://latex.codecogs.com/eqneditor/editor.php>
2. <https://latexeditor.lagrida.com>
3. <https://www.hostmath.com>
4. https://www.overleaf.com/learn/latex/Mathematical_expressions

23.9 Ejercicios del capítulo

Sobre los ejercicios de este capítulo. Este capítulo no enseña funciones de R sino cómo se escribe una fórmula, así que la entrega es distinta: para cada ejercicio hay que poner **el código LaTeX** (dentro de acentos graves, para que se vea) y **la fórmula ya compuesta** debajo. Los ejercicios piden fórmulas descritas **en palabras**, no copiadas: la parte difícil, y la que se evalúa, es traducir la descripción al código. Ninguna aparece escrita en el capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_23_ecuaciones.Rmd`.

23.9.1 Ejercicio 1. En la línea y aparte

Escribe la fórmula del promedio (equis barra igual a la suma de las equis sub i , dividida entre n) de las dos maneras: **dentro de una oración y en una línea aparte**.

Verificación: las dos tienen que verse iguales salvo por el tamaño y la posición. Si la versión aparte no queda centrada en su propia línea, te faltó un signo de dólar en cada extremo.

En palabras: ¿cuándo conviene cada una? Piensa en una fórmula corta dentro de un párrafo y en una fórmula larga en la sección de métodos de un artículo.

23.9.2 Ejercicio 2. Las llaves que faltan

Escribe la varianza muestral: **ese al cuadrado** igual a la suma, desde i igual a 1 hasta n , de **(equis sub i menos equis barra) al cuadrado**, dividida entre **ene menos uno**.

Escribe después, a propósito, la versión **sin llaves** en el exponente y en el denominador, y ponlas una debajo de la otra.

Verificación: en la versión correcta, el cuadrado tiene que estar fuera del paréntesis y el denominador completo tiene que ser $n - 1$. En la versión mala se ve exactamente qué se rompe.

En palabras: explica qué le pasó a la versión sin llaves, símbolo por símbolo.

23.9.3 Ejercicio 3. Sumatorias con límites

Escribe el índice de diversidad de Shannon: **hache prima** igual a **menos** la suma, desde i igual a 1 hasta n **ese** (el número de especies), de **pe sub i por el logaritmo natural de pe sub i** .

Escribe debajo, también, la fórmula de la equidad de Pielou: hache prima dividida entre el logaritmo natural de ese.

Verificación: el límite inferior y el superior de la sumatoria tienen que salir debajo y encima del signo, no a los lados. El logaritmo natural tiene que salir en letra recta (`\ln`), no en cursiva.

En palabras: ¿por qué `\ln` sale recto y `ln` escrito a mano sale en cursiva? ¿Qué te dice eso sobre cómo interpreta LaTeX las letras sueltas?

23.9.4 Ejercicio 4. Fracciones y raíces

Escribe el error estándar de la media: **ese sub equis barra** igual a **ese** dividido entre la **raíz cuadrada de ene**.

Escribe después el estadístico *t* de una muestra: **te** igual a **(equis barra menos mu)** dividido entre **ese sobre raíz de ene**, con la fracción de abajo dentro de la fracción de arriba.

Verificación: en la segunda, la raíz tiene que cubrir solo la ene, y la fracción pequeña tiene que quedar dentro del denominador de la grande.

En palabras: una fracción dentro de otra se ve pequeña y apretada. ¿Cómo la escribirías si tuviera que ir dentro de un párrafo, en la línea? Escribe esa segunda versión también.

23.9.5 Ejercicio 5. Una ecuación numerada

Escribe la ecuación de crecimiento logístico dentro de un entorno `\begin{equation}` con su etiqueta: **de ene sobre de te** igual a **erre por ene por (uno menos ene sobre ka)**. Después haz una referencia cruzada a ella en una oración del texto.

Verificación: en este libro, hecho con bookdown, la ecuación sale numerada y la referencia se convierte en ese número. En tu hoja de entrega, que es un `html_document` normal, **no** va a funcionar: la referencia va a salir como texto sin resolver.

En palabras: eso último no es un error tuyo. Explica por qué pasa y qué tendrías que cambiar en el encabezado YAML de tu documento para que funcionara.

23.9.6 Ejercicio 6. Cuando la cursiva estorba

Escribe el equilibrio de Hardy-Weinberg (**pe al cuadrado más dos pe cu más cu al cuadrado igual a uno**) y la reacción global de la fotosíntesis (seis moléculas de dióxido de carbono más seis de agua, flecha, una de glucosa más seis de oxígeno), las dos con `\mathrm{}`.

Escribe la fotosíntesis también **sin** `\mathrm{}` y ponlas una al lado de la otra.

Verificación: en la versión con `\mathrm{}` las fórmulas químicas salen en letra recta; en la otra salen en cursiva, que es incorrecto para una fórmula química.

En palabras: en Hardy-Weinberg, ¿deben ir *p* y *q* en cursiva o rectas? Justifica tu respuesta con la regla general: en notación científica, las **variables** van en cursiva y los **nombres de cosas** van rectos.

23.9.7 Ejercicio 7 (BONO). Una matriz de población

Escribe la ecuación de proyección de un modelo matricial de población: **ene de te más uno** igual a la matriz **A** por **ene de te**, con la matriz escrita como una matriz de tres por tres cuyas entradas son a_{ij} .

Usa el entorno `\begin{bmatrix} ... \end{bmatrix}`, con `&` entre las columnas y `\\` al final de cada fila.

Verificación: la matriz tiene que salir con corchetes rectos a los lados, tres filas y tres columnas alineadas.

En palabras: ¿qué cambia si usas `pmatrix` en vez de `bmatrix`? ¿Y `vmatrix`? Pruébalos y describe la diferencia. En un artículo sobre matrices de proyección, ¿cuál usarías para la matriz y cuál para su determinante?

Chapter 24

Opciones de Knitr

[1] "2026-09-06"

La mayoría de los ejemplos y ideas de esta sección proviene de **yihui** en el siguiente website <<https://yihui.org/knitr/>>

el paquete **knitr** fue diseñado para ser transparente para generar reporte de R, añadiendo componente de animación y Latex (Ecuaciones) y otras funciones.

Hay muchas opciones para determinar lo que hace un **chunk**

- Code evaluation: evaluación de código
 - Text output: Formateo de texto
 - Code decoration: Decoración del código
 - Cache: Cache
 - Plots: Gráficos
 - Animation: Animaciones
 - Code chunk: Código del chunk
 - Child Document: Documento asociados
 - Language engines: Asociación de lenguaje
 - Option Template:
 - Extracting source code:
 - Other chunk options:
 - Package Options:
 - Global R Options:
-

24.1 Temas

- Qué es una opción de bloque y dónde se escribe
- Los tres niveles: global, por bloque, y en el YAML

- Las cuatro opciones que esconden cosas: `eval`, `echo`, `include`, `results`
- Avisos y mensajes
- Figuras: tamaño, título, alineación
- `cache`: no repetir lo que ya se calculó
- `results='asis'`: generar texto desde R

24.2 Qué es una opción de bloque

Una **opción de bloque** es un ajuste que se escribe en la primera línea del bloque, después de la etiqueta y separado por comas: en la primera línea escribes `{r mi-bloque, echo=FALSE, fig.width=4}` entre acentos graves, donde `mi-bloque` es la etiqueta y lo demás son las opciones.

Todo lo que hay ahí dentro controla **qué hace R con ese bloque y qué acaba apareciendo en el documento final**. Y hay que tener clara una distinción que confunde a todo el mundo al principio: una cosa es que el código **se ejecute** y otra que **se vea**. Son dos decisiones independientes, y por eso hacen falta varias opciones y no una sola.

Las opciones se pueden poner en tres niveles, y el más específico gana:

1. **Global**, con `knitr::opts_chunk$set(...)` en el primer bloque del documento. Es lo que hace la primera línea de todos los capítulos de este libro. Vale para todos los bloques que vengan después.
2. **Por bloque**, en la primera línea de ese bloque. Sobrescribe la global solo para él.
3. **En el YAML**, para cosas que afectan al documento entero (el formato de salida, la tabla de contenido, el tema).

La costumbre que conviene adoptar: pon en global lo que quieras casi siempre (típicamente `warning = FALSE`, `message = FALSE`) y deja las excepciones bloque por bloque. Así el documento se lee sin ruido y no tienes que repetir lo mismo cuarenta veces.

24.3 Para ver las opciones de knitr de los chunk

knitr fue creado por **Yihui Xie** en 2012; es el motor que ejecuta el código de R dentro de los documentos R Markdown y “teje” (*knit*) el resultado junto con el texto. Sustituyó a Sweave con una sintaxis más flexible.

Opciones de los chunks: controlan qué se muestra.

- `echo=FALSE`: oculta el código (muestra solo el resultado).
- `eval=FALSE`: muestra el código pero no lo ejecuta.

- **include=FALSE**: ejecuta el código pero no muestra nada.
- **warning=FALSE** / **message=FALSE**: ocultan avisos y mensajes.
- **fig.width** / **fig.height** / **out.width**: tamaño de las figuras.

echo=FALSE y **eval=FALSE** no son lo mismo y se confunden constantemente:

- **echo=FALSE** **ejecuta** el código y esconde el código; el resultado sí sale.
- **eval=FALSE** **no ejecuta** nada y enseña el código; no hay resultado.
- **include=FALSE** ejecuta el código y no enseña **nada**, ni código ni resultado.

Es decir, **include=FALSE** sí crea los objetos y **eval=FALSE** no. Esa diferencia es la causa del problema que vas a ver unas líneas más abajo.

Este **chunk** es para que la información de los chunk en la parte de **knitr** se vea.

```
library(knitr)
hook_chunk = knit_hooks$get('chunk')
knit_hooks$set(chunk = function(x, options) {
  if (!is.null(options$echo_opts)) {
    return(paste0("~~~~ ~~~~{r ", options$params.src, "} ~~~~", x, "~~~~ ~~~~"))
  } else {
    return(hook_chunk(x, options)) # pass to default hook
  }
})

opts_knit$set(eval.after = 'fig.cap')
```

Los ejemplos aquí serán limitados a las alternativas para los documentos **.Rmd**

- Las opciones están escrita de forma de **tag=selección de la alternativa**.

Opciones de bloques

Se personaliza los bloques con “opciones” **options** de los chunks.

En la rueda de a la derecha de cada **chunk** hay opciones para seleccionar opciones.

24.3.1 Cómo se lee la tabla

En la tabla siguiente, cada columna es una cosa que puede aparecer o no en el documento, y un guion significa que esa opción **la suprime**:

- **Ejecuta**: el código corre de verdad.
- **Muestra**: se ve el código escrito.
- **Output**: se ve el resultado de texto.
- **Gráficos, Mensajes, Advertencias**: lo que dice cada nombre.

Fíjate en la primera fila: **eval=FALSE** suprime todo menos el código, porque no hay nada que ejecutar. Y en la tercera: **echo=FALSE** suprime solo el código, y deja pasar todo lo demás.

24.4 Tablas de opciones principales

Opción	Ejecuta	Muestra	Output	Gráficos	Mensajes	Advertencias
eval=FALSE	-		-	-	-	-
include=FALSE		-	-	-	-	-
echo=FALSE		-				
results="hide"			-			
fig.show ="hide"				-		
message=FALSE					-	
warning=FALSE						-
error=FALSE						-

- Comenzando con dándole un nombre al chunk “mi-chunk”

Ponerle nombre a cada chunk (como `mi-chunk`) facilita localizar errores, navegar por el documento y reutilizar el caché de los cálculos costosos.

```
library(tidyverse)
```

```
algun_nombre, echo=TRUE
```

Eso es lo que va a ver cuando hace el knit

```
library(tidyverse)
```

Nota aquí tengo `include=FALSE`, por consecuencia cuando se hace un knit, no se ve lo que hay en el chunk.

- Ahora cambialo a `TRUE`

```
head(cars)
```

```
##   speed dist
## 1     4    2
## 2     4   10
## 3     7    4
## 4     7   22
## 5     8   16
## 6     9   10
```

```
```{r c22-7, include=TRUE, echo_opts=TRUE}
```

```
head(cars)
```

```
speed dist
1 4 2
```

```
2 4 10
3 7 4
4 7 22
5 8 16
6 9 10
...

```

---

### 24.4.1 Figuras

Con las figuras hay dos parejas de opciones que se confunden:

- **fig.width** y **fig.height** son el tamaño con el que **R** dibuja la figura, en pulgadas. Cambian el tamaño relativo del texto y de los puntos dentro de la gráfica: una figura de 3 pulgadas tiene las letras proporcionalmente más grandes que una de 8.
- **out.width** y **out.height** son el tamaño al que se **muestra** la imagen ya hecha, normalmente en porcentaje (**out.width='50%'**). Es un simple estirar o encoger, y el texto de dentro se estira con ella.

La regla práctica: ajusta primero **fig.width** hasta que las letras se vean del tamaño correcto, y usa **out.width** solo para colocarla en la página.

Las otras dos que se usan a diario son **fig.cap**, que pone el título al pie de la figura (y en bookdown la numera y permite referenciarla), y **fig.align**, que la coloca a la izquierda, a la derecha o centrada.

Añade una figura

Y ahora un ejemplo que sale mal a propósito, y que vale por todo el capítulo. El bloque siguiente lleva **eval=FALSE**, así que **el objeto data nunca se crea**:

```
data=c(1:100)
```

El bloque de abajo sí se ejecuta, y pide el promedio de **data**:

```
mean(data)
```

```
[1] 9.6
```

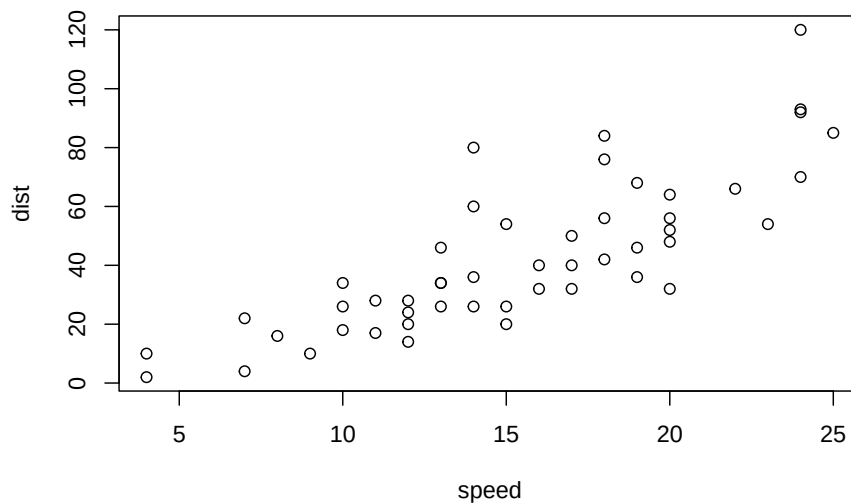
Mira bien el resultado: no es 50.5, es NA con una advertencia. Como el objeto **data** no se creó, R fue a buscar ese nombre a otro sitio y encontró la **función data()** de base R, que existe siempre. Y el promedio de una función no se puede calcular.

Es el mismo **object of type 'closure'** del capítulo de errores comunes, provocado aquí por una opción de bloque. La lección: **eval=FALSE** no es cosmética. Si el bloque que apagas creaba algo, todo lo que venga después trabaja sin ello, y no siempre da un error limpio.

Este es el argumento definitivo a favor de la prueba de la sesión limpia del capítulo de scripts: reiniciar R y hacer *Knit* del documento completo. Un `eval=FALSE` olvidado se detecta ahí y en ningún otro sitio.

---

```
```{r c22-11, include=TRUE, fig.cap='El nombre de mi figura.',
fig.topcaption=TRUE, echo_opts=TRUE}
plot(cars)
```

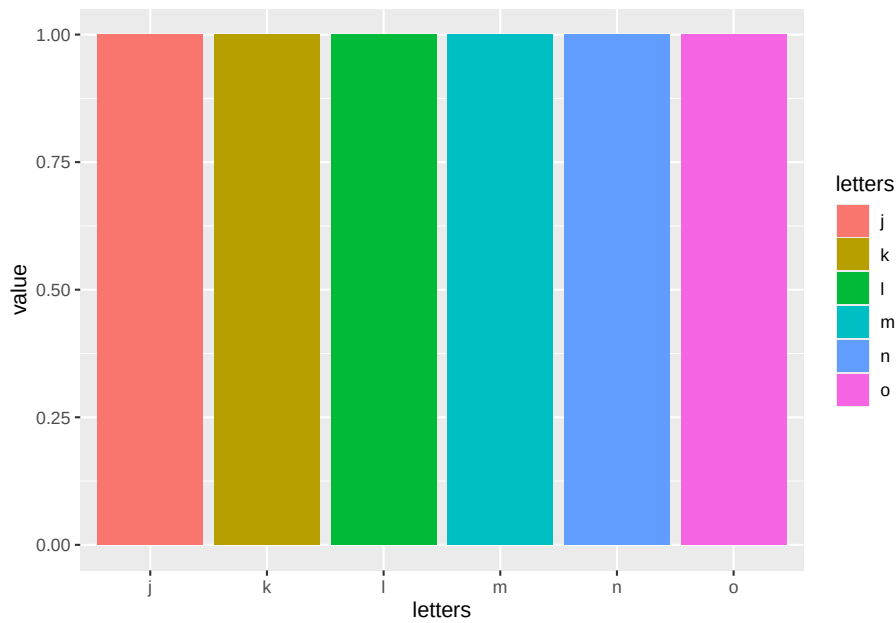


```
```
```

---

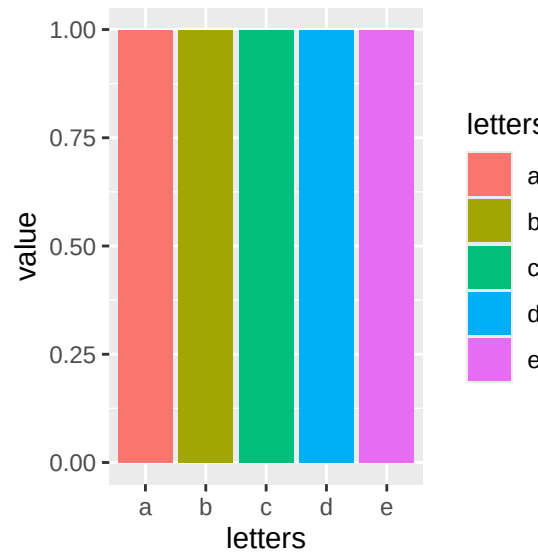
```
```{r c22-12, include=TRUE, fig.cap= "An incredible figure",
echo_opts=TRUE}
library(ggplot2)
df <- data.frame(letters = letters[10:15], value = 1)

ggplot(df, aes(letters, value, fill = letters)) +
  geom_bar(stat = "identity")
```



...

```
```{r c22-13, include=TRUE, fig.cap= "An incredible figure",
```



```
fig.width = 3, fig.height = 3, echo_opts=TRUE, echo=FALSE}
```
```

Figure alignment: default, left, right, and center

fig.align

```
```{r c22-14, include=TRUE, fig.cap= "caption", fig.width = 3,
fig.height = 3, fig.align='right', echo_opts=TRUE}
```

```
library(ggplot2)
df <- data.frame(letters = letters[1:5], value = 1)

ggplot(df, aes(letters, value, fill = letters)) +
 geom_bar(stat = "identity")
```

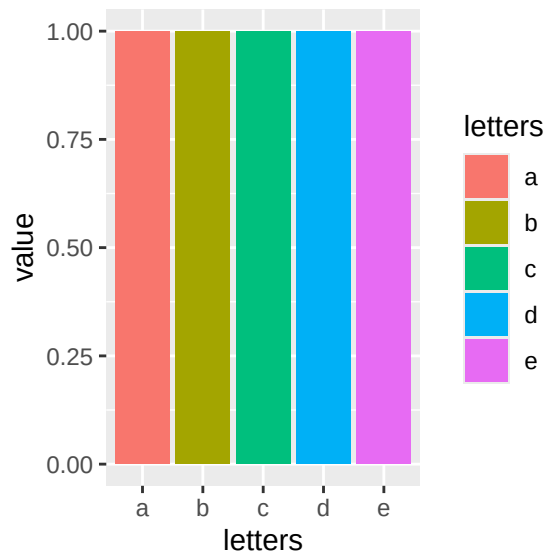


Figure 24.1: caption

...

Wrap text around figure

Use ese código para que el texto “wraps” alrededor de las figura (ese código es de otro idioma de computadora de se llama **css**)

AQUI el CHUNK

#### 24.4.1.0.1 R Markdown wrapping código css

El bloque `<div style="float:right; ...">` de arriba no es de R ni de knitr: es **CSS**, el lenguaje con el que se le da estilo a una página web. `float:right`



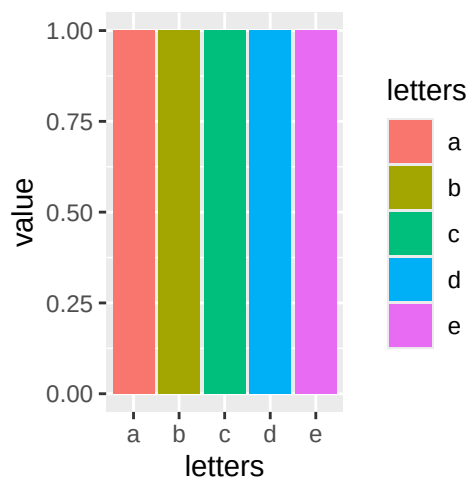


Figure 24.2: caption

saca la figura del flujo normal del texto y la pega a la derecha, y `position` y `top` la suben o la bajan unos píxeles para que quede a la altura del párrafo que le corresponde.

Esto solo funciona en la salida **HTML**. Si el mismo documento se compila a PDF o a Word, el `div` se ignora y la figura vuelve a su sitio. Es el precio de usar CSS: da control fino, pero solo en un formato.

La alternativa que sí funciona en los tres formatos es la opción `out.extra` del bloque, condicionada al formato de salida, que es lo que hace el capítulo de factores. Cuesta más de escribir y no se rompe al cambiar de salida.

Para más detalles sobre R Markdown, <http://rmarkdown.rstudio.com>.

### 24.4.2 OTRAS opciones de knitr

- `eval=` : ejecutar o no el código (TRUE o FALSE)
- `echo=` : mostrar o no el código (TRUE o FALSE)
- `message=` : mostrar o no los mensajes
- `warning=` : mostrar o no las advertencias
- `error=` : si es TRUE, el documento **sigue compilando** aunque el bloque dé error, y el mensaje sale impreso. Es lo que usa el capítulo de errores comunes para enseñar los errores sin que el libro se caiga.
- `results='asis'` : no envolver el resultado como salida de consola, sino meterlo en el documento **tal cual**, como si lo hubieras escrito a mano.

`results='asis'` es la puerta a algo muy útil: **generar el documento desde el código**. Con él, un `for` puede escribir un encabezado y un párrafo por cada especie de tu tabla, y el informe entero se rehace solo cuando cambian los datos. Es lo que hace el último bloque de esta sección, en pequeño.

```
[1] 1 2 3 4 5 6 7 8 9 10

library(MASS)

library(dplyr)
library(MASS)

cat("I'm raw Markdown content.\n",
 "\n",
 " My poetry of Puerto Rico\n")
```

I'm raw **Markdown** content.

My poetry of Puerto Rico

---



---

## 24.5 Emojis y símbolos

### 24.5.1 En Mac se abren desde Edit Emoji & Symbols.

En Windows, con la tecla de Windows más el punto.

**24.5.1.0.1 Recuerda que puedes añadir emojis a tu documento** A veces uno quiere un poco y ponerle un par de al documento. Y si le añades emojis interesantes a tu trabajo hay premio, sobre todo si eres .

Un aviso práctico: los emojis se ven bien en HTML, pero al compilar a **PDF** con LaTeX suelen fallar o salir como cajas vacías, porque la fuente por defecto no los tiene. Para un trabajo que vaya a imprimirse, mejor no usarlos.

---

## 24.6 cache: no repetir lo que ya se calculó

Cuando un bloque tarda (un modelo bayesiano, una simulación, leer un archivo enorme), compilar el documento se vuelve insoportable, porque **todo** se vuelve a calcular cada vez.

`cache=TRUE`: guardar el resultado de un bloque para no repetirlo.

- **Qué hace:** la primera vez ejecuta el bloque y guarda el resultado en disco. Las veces siguientes, si el código del bloque **no cambió**, lo lee del disco en vez de calcularlo.
- **dependson = "otro-bloque":** obliga a recalcular cuando cambia otro bloque del que este depende.
- **cache.lazy = FALSE:** hace falta con objetos muy grandes.

El caché mira si cambió el **código del bloque**, no si cambiaron los datos. Si editas el archivo `.csv` que lee un bloque cacheado, el bloque no se entera y sigue devolviendo lo viejo. Cuando un resultado no cambie por más que edites, borra la carpeta `_cache` y vuelve a compilar. Es de los pocos casos del curso en que “apágalo y enciéndelo” es la respuesta correcta.

## 24.7 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Aquí lo que se evalúa no es un resultado sino el **documento**: las opciones que pusiste en cada bloque y lo que eso hizo aparecer o desaparecer. Por eso el `.html` que entregues cuenta tanto como el `.Rmd`, y hay que mirar los dos lado a lado. Los ejercicios usan `millas` del paquete `datos`, no `cars` ni las letras del abecedario, que son los del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_24_knitr.Rmd`.

### 24.7.1 Ejercicio 1. Las cuatro maneras de esconder algo

Escribe **cuatro** bloques que hagan exactamente lo mismo (calcular el promedio de autopista en millas), uno con `echo=FALSE`, otro con `eval=FALSE`, otro con `include=FALSE` y otro con `results='hide'`.

**Verificación:** en el `.html` final, **uno** enseña solo el número, **dos** enseñan solo el código, y **uno** no enseña absolutamente nada.

**En palabras:** haz una tabla de cuatro filas diciendo, para cada opción, si el código **se ejecutó** y si se **vio**. ¿Cuáles dos crean el objeto en la memoria y cuáles no?

### 24.7.2 Ejercicio 2. Avisos y mensajes

Escribe un bloque con `library(dplyr)` y `library(MASS)` sin ninguna opción, y mira lo que sale. Después repítelo con `message=FALSE` y con `warning=FALSE`.

**Verificación:** sin opciones aparece un aviso sobre funciones enmascaradas; con `message=FALSE` desaparece.

**En palabras:** ese aviso dice que `MASS::select()` está tapando a `dplyr::select()`. Esconderlo con `message=FALSE`, ¿resuelve el problema o solo lo oculta? ¿Qué harías tú en un análisis de verdad?

### 24.7.3 Ejercicio 3. La trampa del `eval=FALSE`

Reproduce el problema del capítulo con tus propios datos: un bloque con `eval=FALSE` que cree un objeto llamado `datos_millas`, y un bloque después que lo use.

**Verificación:** el segundo bloque tiene que dar `object 'datos_millas' not found`. Ponle `error=TRUE` para que el documento compile de todos modos.

**En palabras:** en el capítulo, el mismo experimento con un objeto llamado `data` **no** dio ese error, dio un NA. Explica la diferencia. ¿Cuál de los dos comportamientos prefieres y por qué?

### 24.7.4 Ejercicio 4. El tamaño de una figura

Haz la misma gráfica de `millas` (dispersión de `cilindrada` contra `autopista`) tres veces: con `fig.width=8`, con `fig.width=3`, y con `fig.width=8` más `out.width='40%'`.

**Verificación:** las tres se ven de tamaño distinto, pero **dos** de ellas tienen las letras de los ejes del mismo tamaño relativo y una no.

**En palabras:** ¿cuáles dos, y por qué? Explica la diferencia entre `fig.width` y `out.width` con lo que acabas de ver. ¿Cuál usarías para una figura que va en un artículo a una columna?

### 24.7.5 Ejercicio 5. Global contra local

En el primer bloque de tu documento, pon `knitr::opts_chunk$set(echo = FALSE)`. Después escribe dos bloques: uno normal y otro con `echo=TRUE`.

**Verificación:** en el `.html`, el primero no enseña su código y el segundo sí.

**En palabras:** explica qué nivel ganó y por qué. ¿Qué ventaja tiene poner las opciones comunes en global en vez de repetirlas en cada bloque? ¿Y qué desventaja, para alguien que abra tu documento por el medio?

### 24.7.6 Ejercicio 6. `cache`

Escribe un bloque lento a propósito, con `Sys.sleep(10)` y algún cálculo sobre `millas`. Compila el documento y anota cuánto tardó. Añade `cache=TRUE`, compila dos veces más y anota los tiempos.

**Verificación:** la segunda compilación con `cache=TRUE` tiene que ser claramente más rápida que la primera.

**En palabras:** cambia ahora **una sola letra** de un comentario dentro de ese bloque y vuelve a compilar. ¿Se recalculó? Explica qué mira `cache` para decidir, y por qué eso lo hace peligroso si el bloque lee un archivo de datos.

### 24.7.7 Ejercicio 7 (BONO). Escribir el documento desde el código

Con `results='asis'` y un `for`, genera automáticamente un encabezado de nivel 3 por cada `clase` de `millas`, seguido de una oración con el número de vehículos y el consumo promedio de esa clase.

**Verificación:** en el `.html` final tienen que aparecer 7 encabezados, y tienen que salir en la tabla de contenido como si los hubieras escrito a mano.

**En palabras:** ¿qué pasa si quitas `results='asis'`? Pruébalo y describe la diferencia. ¿En qué situación de tu trabajo te serviría generar así un informe?



## Chapter 25

# Fechas, horas y minutas

```
[1] "2026-09-06"
```

---

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/dates-and-times.html>

Español: <https://es.r4ds.hadley.nz/16-datetimes.html>

---

### 25.1 Temas:

- Por qué las fechas son más difíciles de lo que parecen
  - Los tres tipos: `<date>`, `<time>` y `<dtm>`
  - Convertir texto a fecha con `ymd()` y compañía
  - Extraer partes: `year()`, `month()`, `wday()`, `hour()`
  - Redondear: `floor_date()`, `ceiling_date()`, `round_date()`
  - Duraciones, períodos e intervalos: los tres no son lo mismo
  - Zonas horarias
  - `library(lubridate)`
  - `library(datos)`
  - `library(tidyverse)`
-

## 25.2 Por qué las fechas son difíciles

Contar tiempo parece trivial y no lo es, porque el calendario que usamos está lleno de excepciones que el código tiene que respetar:

- **los meses no duran lo mismo**, así que “un mes después del 31 de enero” no tiene una respuesta obvia;
- **hay años bisiestos**, así que un año no siempre son 365 días;
- **hay cambios de horario de verano**, así que un día no siempre son 24 horas;
- **hay zonas horarias**, así que dos personas pueden estar en fechas distintas en el mismo instante;
- y hasta hay **segundos intercalares**, que se añaden de vez en cuando para cuadrar el reloj con la rotación de la Tierra.

De ahí sale la idea central del capítulo, que conviene tener clara desde el principio: hay **dos maneras distintas de medir tiempo**, y hacen falta las dos. Una cuenta segundos exactos, sin excepciones, y la otra cuenta en unidades humanas respetando el calendario. Casi todos los errores con fechas vienen de usar la que no era.

Puerto Rico no cambia de horario de verano y usa siempre la zona "America/Puerto\_Rico" (AST, cuatro horas menos que UTC). Eso nos ahorra un problema que sufren los que trabajan con datos de Estados Unidos continental, pero aparece en cuanto se comparan datos de aquí con datos de allá.

---

## 25.3 Creando fechas/horas

En R y en los sistemas tipo Unix el tiempo se cuenta como el número de segundos (o días) transcurridos desde la medianoche del 1 de enero de 1970, conocida como la época Unix. Por eso `as.numeric()` de una fecha devuelve los días desde ese punto.

R distingue tres cosas, y conviene usar la más sencilla que sirva:

- `date`,
- `time`,
- `date-time`,
- **<date>**: solo el día, sin hora. Por dentro es un número: los **días** transcurridos desde el 1 de enero de 1970.
- **<time>**: solo la hora del día, sin fecha. Por dentro, los **segundos** desde la medianoche.
- **<dtm>**: fecha y hora juntas. Por dentro, los **segundos** desde la época Unix, y además arrastra una zona horaria.



Usa `<dtm>` solo cuando de verdad necesites la hora: las fechas-hora arrastran zona horaria y complican todo lo demás.

<https://github.com/edgararuiz/guias-rapidas/blob/master/fechas.pdf>

```
library(tidyverse)
library(lubridate) # paquete para convertir fechas

tidyverse_packages() # lista completa de los paquetes activado con **tidyverse**

[1] "broom" "conflicted" "cli" "dbplyr"
[5] "dplyr" "dtplyr" "forcats" "ggplot2"
[9] "googledrive" "googlesheets4" "haven" "hms"
[13] "httr" "jsonlite" "lubridate" "magrittr"
[17] "modelr" "pillar" "purrr" "ragg"
[21] "readr" "readxl" "reprex" "rlang"
[25] "rstudioapi" "rvest" "stringr" "tibble"
[29] "tidyr" "xml2" "tidyverse"
```

## 25.4 El paquete lubridate

`ymd()`, `mdy()`, `dmy()` (`lubridate`): convertir texto a fechas.

- **Qué hacen:** interpretan una cadena según el **orden** de sus partes: `ymd("2024-03-01")`, `mdy("3/1/2024")`, `dmy("1/3/2024")`.
- **Con hora:** añade `_hms`, `_hm` o `_h` (p. ej. `dmy_hms()`).
- **Extraer partes:** `year()`, `month()`, `day()`, `wday()` (día de la semana).

El truco de `lubridate` es que el **nombre de la función dice el orden de las partes**, no el formato exacto. `dmy()` significa “día, mes, año”, y con eso le basta: acepta “1/3/2024”, “01-03-2024”, “1 marzo 2024” y “1.3.24”. No hay que escribir formatos como `%d/%m/%Y`, que es lo que hacía falta en base R y lo que casi nadie recuerda.

“01/03/2024” es el 1 de marzo o el 3 de enero según de dónde venga el archivo, y **ninguna función puede adivinarlo**. `dmy()` y `mdy()` las dos van a darte una fecha válida, y una de las dos estará mal. Antes de convertir, averigua de dónde salieron los datos. Si hay valores con día mayor que 12 en la columna, salen NA con una de las dos funciones, y esa es tu pista.

La función `today()` regresa la fecha de hoy, similar a `Sys.date()`

```
today()

[1] "2026-09-06"
```

Use la función de `tribble()`, del paquete `tibble`, para crear una tabla manualmente

```
fechas <- tribble(
 ~ codigo, ~ fecha,
 "001", "01/01/2019 00:00:00",
 "002", "31/03/2019 01:05:00",
 "003", "14/06/2019 20:00:00",
 "004", "01/12/1859 11:32:13",
 "005", "01/01/0501 05:04:33"
)
fechas
```

```
A tibble: 5 x 2
codigo fecha
<chr> <chr>
1 001 01/01/2019 00:00:00
2 002 31/03/2019 01:05:00
3 003 14/06/2019 20:00:00
4 004 01/12/1859 11:32:13
5 005 01/01/0501 05:04:33
```

Hay varias opciones para convertir texto a fechas, el más usado es `mdy_hms()`, pero en este caso el formato utilizado no funciona bien.

Un error muy común es usar `mdy()` cuando las fechas están en formato día/mes/año. Como no existe un mes 31, obtendrás `NA` o fechas equivocadas; elige la función (`dmy()`, `mdy()`, `ymd()`) según el orden real de la fecha.

### 25.4.1 Un error típico

```
fechas %>%
 mutate(fecha = mdy_hms(fecha))
```

```
A tibble: 5 x 2
codigo fecha
<chr> <dtm>
1 001 2019-01-01 00:00:00
2 002 NA
3 003 NA
4 004 1859-01-12 11:32:13
5 005 0501-01-01 05:04:33
```

Ya que el día es el primero, y no el mes, usamos `dmy_hms()`

```
nueva_fechas <- fechas %>%
 mutate(fecha = dmy_hms(fecha))
```

```
nueva_fechas
```

```
A tibble: 5 x 2
```

```
codigo fecha
<chr> <dtm>
1 001 2019-01-01 00:00:00
2 002 2019-03-31 01:05:00
3 003 2019-06-14 20:00:00
4 004 1859-12-01 11:32:13
5 005 0501-01-01 05:04:33
```

lubridate tiene varias funciones para extraer partes de la fecha, por ejemplo: año, mes, día, hora, minuto, y cuatrimestre.

```
nueva_fechas %>%
 mutate(a = year(fecha),
 m = month(fecha),
 d = day(fecha),
 h = hour(fecha),
 mn = minute(fecha),
 q = quarter(fecha)
)
```

```
A tibble: 5 x 8
codigo fecha a m d h mn q
<chr> <dtm> <int> <int> <int> <int> <int> <int>
1 001 2019-01-01 00:00:00 2019 1 1 0 0 1
2 002 2019-03-31 01:05:00 2019 3 31 1 5 1
3 003 2019-06-14 20:00:00 2019 6 14 20 0 2
4 004 1859-12-01 11:32:13 1859 12 1 11 32 4
5 005 0501-01-01 05:04:33 501 1 1 5 4 1
```

Las funciones `round_date()`, `ceiling_date()` y `floor_date()` permiten redondear la fecha al número más cercano de la unidad especificada

```
nueva_fechas %>%
 mutate(
 redondear = round_date(fecha, unit = "month"), # round_date es redondea al mes más cercano
 techo = ceiling_date(fecha, unit = "day"), # ceiling_date redondea al día más cercano
 suelo = floor_date(fecha, unit = "month"), # floor_date redondea al mes más cercano
 suelo_hour = floor_date(fecha, unit = "hour") # redondea a la hora más cercana
)
```

```
A tibble: 5 x 6
codigo fecha redondear techo
<chr> <dtm> <dtm> <dtm>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 2019-01-01 00:00:00
2 002 2019-03-31 01:05:00 2019-04-01 00:00:00 2019-04-01 00:00:00
3 003 2019-06-14 20:00:00 2019-06-01 00:00:00 2019-06-15 00:00:00
4 004 1859-12-01 11:32:13 1859-12-01 00:00:00 1859-12-02 00:00:00
5 005 0501-01-01 05:04:33 0501-01-01 00:00:00 0501-01-02 00:00:00
```

```
i 2 more variables: suelo <dtm>, suelo_hour <dtm>
```

## 25.5 Intervalos y duraciones

La función `interval()` crea un objeto R de intervalo de tiempo. En este caso, el intervalo entre la fecha en la tabla, y el día de hoy. Nota que esto puede ser muy útil si quiere calcular la cantidad de tiempo entre dos fechas.

```
nueva_fechas %>%
 mutate(intervalo = interval(fecha, today())) # intervalo entre la fecha y hoy
```

```
A tibble: 5 x 3
codigo fecha intervalo
<chr> <dtm> <Interval>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 UTC--2026-09-06 UTC
2 002 2019-03-31 01:05:00 2019-03-31 01:05:00 UTC--2026-09-06 UTC
3 003 2019-06-14 20:00:00 2019-06-14 20:00:00 UTC--2026-09-06 UTC
4 004 1859-12-01 11:32:13 1859-12-01 11:32:13 UTC--2026-09-06 UTC
5 005 0501-01-01 05:04:33 0501-01-01 05:04:33 UTC--2026-09-06 UTC
```

`int_length()` regresa el número de segundos dentro del intervalo

```
nueva_fechas %>%
 mutate(
 intervalo = interval(fecha, today()), #
 segundos = int_length(intervalo) # número de segundos en el intervalo
)
```

```
A tibble: 5 x 4
codigo fecha intervalo segundos
<chr> <dtm> <Interval> <dbl>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 UTC--2026-09-06 UTC 242352000
2 002 2019-03-31 01:05:00 2019-03-31 01:05:00 UTC--2026-09-06 UTC 234658500
3 003 2019-06-14 20:00:00 2019-06-14 20:00:00 UTC--2026-09-06 UTC 228110400
4 004 1859-12-01 11:32:13 1859-12-01 11:32:13 UTC--2026-09-06 UTC 5262582467
5 005 0501-01-01 05:04:33 0501-01-01 05:04:33 UTC--2026-09-06 UTC 48145776927
```

## 25.6 El operador `%--%` simplifica el cálculo del intervalo

Compara con el script anterior

```
nueva_fechas %>%
 mutate(intervalo = fecha %--% today())
```

```
A tibble: 5 x 3
codigo fecha intervalo
```

```
<chr> <dtm> <Interval>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 UTC--2026-09-06 UTC
2 002 2019-03-31 01:05:00 2019-03-31 01:05:00 UTC--2026-09-06 UTC
3 003 2019-06-14 20:00:00 2019-06-14 20:00:00 UTC--2026-09-06 UTC
4 004 1859-12-01 11:32:13 1859-12-01 11:32:13 UTC--2026-09-06 UTC
5 005 0501-01-01 05:04:33 0501-01-01 05:04:33 UTC--2026-09-06 UTC
```

Para saber el número de días en el intervalo, divida el intervalo por la función que corresponde a días, `days()`

```
nueva_fechas %>%
 mutate(dias = fecha %--% today() / days(),
 anio= fecha %--% today() / years()) # número de días en el intervalo
```

```
A tibble: 5 x 4
codigo fecha dias anio
<chr> <dtm> <dbl> <dbl>
1 001 2019-01-01 00:00:00 2805 7.68
2 002 2019-03-31 01:05:00 2716. 7.44
3 003 2019-06-14 20:00:00 2640. 7.23
4 004 1859-12-01 11:32:13 60910. 167.
5 005 0501-01-01 05:04:33 557243. 1526.
```

*#Cuántos años de diferencia en el intervalo?*

Intervalo entre una lista de fecha y otra fecha y hora específica

```
nueva_fechas %>%
 mutate(dias = fecha %--% "2020-01-01 00:00:00" / days())
```

```
A tibble: 5 x 3
codigo fecha dias
<chr> <dtm> <dbl>
1 001 2019-01-01 00:00:00 365
2 002 2019-03-31 01:05:00 276.
3 003 2019-06-14 20:00:00 200.
4 004 1859-12-01 11:32:13 58470.
5 005 0501-01-01 05:04:33 554803.
```

## 25.7 Ejercicio

Usa esta función y calcula el número de día que ha transcurrido entre el día de su nacimiento y el día de hoy

```
segundos = "2018-08-14 00:00:00" %--% today()/seconds()
segundos
```

```
[1] 254448000
```

```
dias = "2018-08-14 00:00:00" %--% today() / days()
dias

[1] 2945

mes= "2018-08-14 00:00:00" %--% today() / months(1)
mes

[1] 96.74194

anio = "2018-08-14 00:00:00" %--% today() / years()
anio

[1] 8.063014

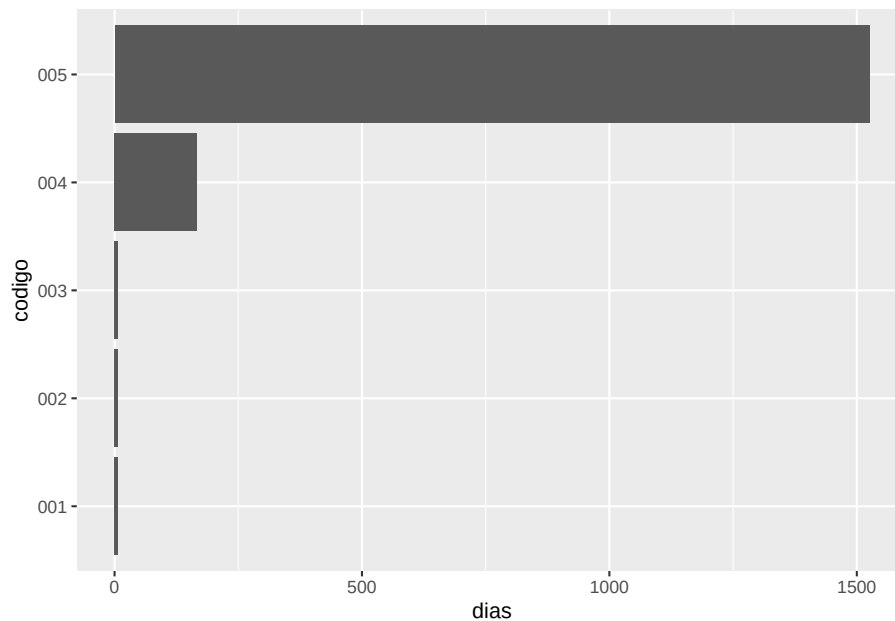
millisecond = "2018-08-14 00:00:00" %--% today() / milliseconds()

millisecond

[1] 2.54448e+11
```

Los resultados se pueden visualizar usando ggplot2

```
nueva_fechas %>%
 mutate(dias = fecha %--% today() / years()) %>%
 ggplot() +
 geom_col(aes(codigo, dias)) +
 coord_flip()
```



## 25.8 Serie de funciones para calcular la duración de un intervalo de tiempo.

- `dyears()` # años
- `dmonths()` # meses
- `dweeks()` # semanas
- `ddays()` # días
- `dhours()` # horas
- `dminutes()` # minutos
- `dseconds()` # segundos
- `dmilliseconds()` # milisegundos
- `dmicroseconds()` # microsegundos
- `dnanoseconds()` # nanosegundos
- `dpicoseconds()` # picosegundos

## 25.9 `as.duration`

`as.duration()` crea un objeto en R que contiene la duración del intervalo de tiempo.

```
nueva_fechas %>%
 mutate(desde_hoy = as.duration(fecha %--% today())) # duración desde la fecha hasta hoy
```

```
A tibble: 5 x 3
codigo fecha desde_hoy
<chr> <dtm> <Duration>
1 001 2019-01-01 00:00:00 242352000s (~7.68 years)
2 002 2019-03-31 01:05:00 234658500s (~7.44 years)
3 003 2019-06-14 20:00:00 228110400s (~7.23 years)
4 004 1859-12-01 11:32:13 5262582467s (~166.76 years)
5 005 0501-01-01 05:04:33 48145776927s (~1525.65 years)
```

El objeto de duración de tiempo se puede filtrar fácilmente basado en una variedad de tipo de tiempos. En este caso, semanas `dweeks()` crea un objeto de duración de la largura especificada

```
nueva_fechas %>%
 mutate(desde_hoy = as.duration(fecha %--% today())) %>%
 filter(desde_hoy > dyears(7))
```

```
A tibble: 5 x 3
codigo fecha desde_hoy
<chr> <dtm> <Duration>
1 001 2019-01-01 00:00:00 242352000s (~7.68 years)
2 002 2019-03-31 01:05:00 234658500s (~7.44 years)
3 003 2019-06-14 20:00:00 228110400s (~7.23 years)
4 004 1859-12-01 11:32:13 5262582467s (~166.76 years)
```

```
5 005 0501-01-01 05:04:33 48145776927s (~1525.65 years)
```

Otra opción es `ddays()`.

```
nueva_fechas %>%
 mutate(desde_hoy = as.duration(fecha %--% today())) %>%
 filter(desde_hoy < ddays(2500)) # Nota que se puede filtrar por el número de días q
```

```
A tibble: 0 x 3
```

```
i 3 variables: codigo <chr>, fecha <dtm>, desde_hoy <Duration>
```

### 25.9.1 Funciones

- `today()`
- `now()`

```
today()
```

```
[1] "2026-09-06"
```

```
now()
```

```
[1] "2026-09-06 19:41:38 AST"
```

### 25.9.2 Fechas con años, meses y días

- `ymd()`
- `ydm()`
- `mdy()`
- `myd()`
- `dmy()`
- `dym()`

### 25.9.3 Fechas con horas, minutos y segundos

- `ymd_hms()`
- `ymd_hm()`
- `ymd_h()`
- `ydm_hms()`
- `ydm_hm()`
- `etc`

### 25.9.4 Fechas

- `yq()` Year quarter (quarters are 1, 2, 3, 4, Jan-March, April-Jun, Jul-Sept, Oct-Dec)



```
x =c("2012.1", "1970.4")
yq(x)

[1] "2012-01-01" "1970-10-01"
yq("2012.1")

[1] "2012-01-01"
```

- **Cuidado** con el paquete **hms** que tiene funciones igual como **lubridate** (**hms**, **hm**, y **ms**) , pero no necesariamente hace lo mismo.

## 25.10 Crear una fecha desde columnas individuales

- `make_date()`
- `make_datetime()`

Unir el año, mes, día, hora y minutos que estén en diferentes columnas en uno

- sumamente practico cuando se tiene una base de datos con las fechas en diferentes columnas

```
library(datos)
head(vuelos)

A tibble: 6 x 19
anio mes dia horario_salida salida_programada atraso_salida
<int> <int> <int> <int> <int> <dbl>
1 2013 1 1 517 515 2
2 2013 1 1 533 529 4
3 2013 1 1 542 540 2
4 2013 1 1 544 545 -1
5 2013 1 1 554 600 -6
6 2013 1 1 554 558 -4
i 13 more variables: horario_llegada <int>, llegada_programada <int>,
atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
hora <dbl>, minuto <dbl>, fecha_hora <dtm>

vuelos %>%
 dplyr::select(anio, mes, dia, hora, minuto) %>%
 mutate(salida_dia=make_date(anio, mes, dia),
 salida_dia_minutos = make_datetime(anio, mes, dia, hora, minuto))

A tibble: 336,776 x 7
anio mes dia hora minuto salida_dia salida_dia_minutos
```

```
<int> <int> <int> <dbl> <dbl> <date> <dtm>
1 2013 1 1 5 15 2013-01-01 2013-01-01 05:15:00
2 2013 1 1 5 29 2013-01-01 2013-01-01 05:29:00
3 2013 1 1 5 40 2013-01-01 2013-01-01 05:40:00
4 2013 1 1 5 45 2013-01-01 2013-01-01 05:45:00
5 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
6 2013 1 1 5 58 2013-01-01 2013-01-01 05:58:00
7 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
8 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
9 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
10 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
i 336,766 more rows
```

Unir el año, mes, día, hora que estén en diferentes columnas en uno

```
vuelos %>%
 dplyr::select(anio, mes, dia, hora, minuto) %>%
 mutate(salida2 = make_date(anio, mes, dia))
```

```
A tibble: 336,776 x 6
anio mes dia hora minuto salida2
<int> <int> <int> <dbl> <dbl> <date>
1 2013 1 1 5 15 2013-01-01
2 2013 1 1 5 29 2013-01-01
3 2013 1 1 5 40 2013-01-01
4 2013 1 1 5 45 2013-01-01
5 2013 1 1 6 0 2013-01-01
6 2013 1 1 5 58 2013-01-01
7 2013 1 1 6 0 2013-01-01
8 2013 1 1 6 0 2013-01-01
9 2013 1 1 6 0 2013-01-01
10 2013 1 1 6 0 2013-01-01
i 336,766 more rows
```

## Conversión de fechas que no son estándar

```
Datos_raros=tribble(~Ind, ~Persona, ~fecha_de_nacimiento,
 1, "Abuela", "1/1/30",
 2, "Abuelo", "1/30/01",
 3, "Tio", "1/30/09")

Datos_raros
```

```
A tibble: 3 x 3
Ind Persona fecha_de_nacimiento
<dbl> <chr> <chr>
1 1 Abuela 1/1/30
2 2 Abuelo 1/30/01
```

```
3 3 Tio 1/30/09
```

```
Datos_raros %>%
 mutate(nueva_fecha = mdy(fecha_de_nacimiento))
```

```
A tibble: 3 x 4
Ind Persona fecha_de_nacimiento nueva_fecha
<dbl> <chr> <chr> <date>
1 1 Abuela 1/1/30 2030-01-01
2 2 Abuelo 1/30/01 2001-01-30
3 3 Tio 1/30/09 2009-01-30
as.Date(0, origin = "1970-01-01")
```

```
[1] "1970-01-01"
```

```
Returns "1970-01-01"
```

```
df <- tibble(
 Persona = c("Abuela", "Abuelo", "Tio"),
 fecha_de_nacimiento = c("1/1/1930", "1/30/1901", "1/30/1909")
)

df <- df %>%
 mutate(
 nueva_fecha = mdy(fecha_de_nacimiento),
 nueva_fecha = if_else(year(nueva_fecha) > year(today()),
 nueva_fecha - years(100),
 nueva_fecha)
)
df
```

```
A tibble: 3 x 3
Persona fecha_de_nacimiento nueva_fecha
<chr> <chr> <date>
1 Abuela 1/1/1930 1930-01-01
2 Abuelo 1/30/1901 1901-01-30
3 Tio 1/30/1909 1909-01-30
```

```
#Datos_raros$fecha2=as.Date(as.Date(format(as.Date(Datos_raros$fecha_de_nacimiento, format="%m/%d/%Y"), format="%m/%d/%Y"))
```

```
#Datos_raros
```

## 25.11 %/% y %%: división entera y residuo

Estos dos operadores no son de fechas, pero aparecen constantemente al trabajar con ellas, porque muchos archivos guardan la hora como un **número**: las 5:30 de la tarde vienen como 1730.

- **%/% es la división entera**: se queda con la parte de arriba y tira el resto. 1730 %/% 100 da 17, la hora.
- **%% es el residuo**: se queda con lo que sobra. 1730 %% 100 da 30, los minutos.

Con los dos juntos se parte un 1730 en hora y minutos sin tocar texto.

```
vuelos |> dplyr::select(tiempo_vuelo) |>
 mutate(t=tiempo_vuelo %/% 100) |> # Division
 mutate(m= tiempo_vuelo %% 100) # modulus... Cambia el 100 para 1, 10, 1000... y
```

```
A tibble: 336,776 x 3
tiempo_vuelo t m
<dbl> <dbl> <dbl>
1 227 2 27
2 227 2 27
3 160 1 60
4 183 1 83
5 116 1 16
6 150 1 50
7 158 1 58
8 53 0 53
9 140 1 40
10 138 1 38
i 336,766 more rows
```

### 25.11.1 El operador %% devuelve el residuo de una división

- Por ejemplo, 5 %% 2 devuelve 1, porque el residuo de dividir 5 entre 2 es 1.
- Y 5 %/% 2 devuelve 2, la parte entera de la división.

%% sirve también para preguntar “¿es múltiplo de?”: `x %% 2 == 0` es la manera de saber si un número es par, y `anio %% 4 == 0` es el primer paso de la regla de los años bisiestos.

```
names(vuelos)
```

```
[1] "anio" "mes" "dia"
[4] "horario_salida" "salida_programada" "atraso_salida"
[7] "horario_llegada" "llegada_programada" "atraso_llegada"
[10] "aerolinea" "vuelo" "codigoCola"
[13] "origen" "destino" "tiempo_vuelo"
```

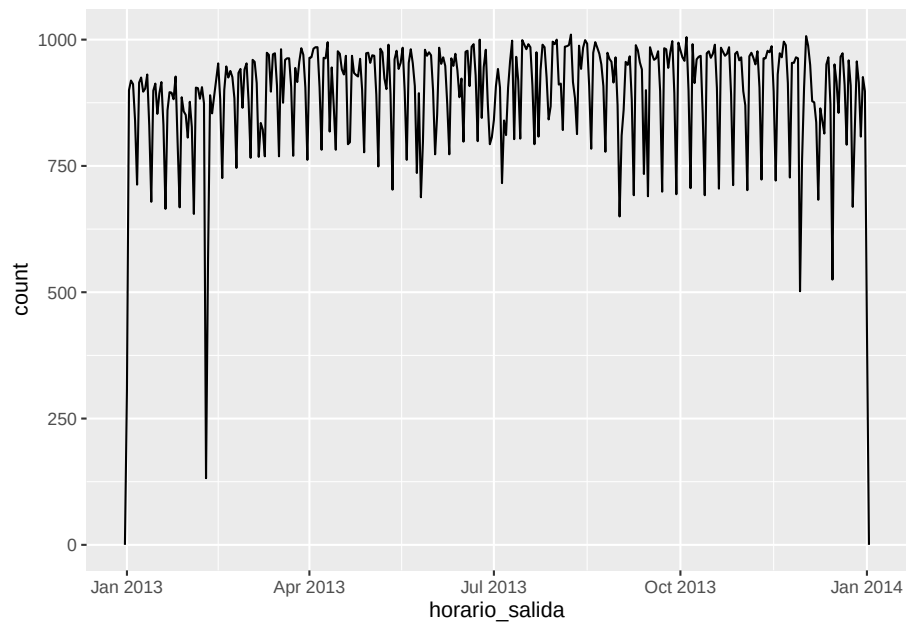
```
[16] "distancia" "hora" "minuto"
[19] "fecha_hora"

hacer_fechahora_100 <- function(anio, mes, dia, tiempo) {
 make_datetime(anio, mes, dia, tiempo %/% 100, tiempo %% 100)
}

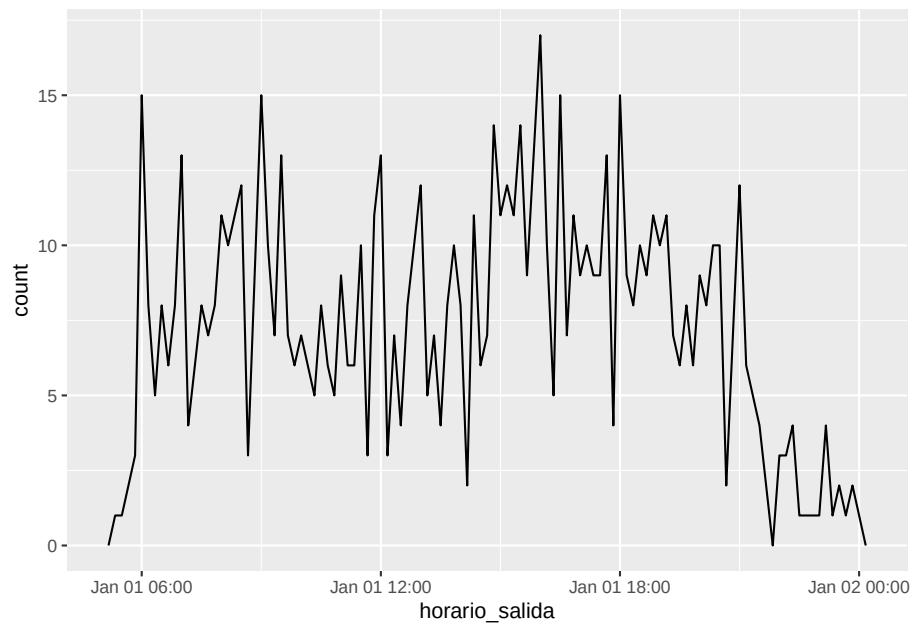
vuelos_dt <- vuelos %>%
 filter(!is.na(horario_salida), !is.na(horario_llegada)) %>%
 mutate(
 horario_salida = hacer_fechahora_100(anio, mes, dia, horario_salida),
 horario_llegada = hacer_fechahora_100(anio, mes, dia, horario_llegada),
 salida_programada = hacer_fechahora_100(anio, mes, dia, salida_programada),
 llegada_programada = hacer_fechahora_100(anio, mes, dia, llegada_programada)
) %>%
 dplyr::select(origen, destino, starts_with("atraso"), starts_with("horario"), ends_with("programa"))
head(vuelos_dt)
```

```
A tibble: 6 x 9
origen destino atraso_salida atraso_llegada horario_salida
<chr> <chr> <dbl> <dbl> <dtm>
1 EWR IAH 2 11 2013-01-01 05:17:00
2 LGA IAH 4 20 2013-01-01 05:33:00
3 JFK MIA 2 33 2013-01-01 05:42:00
4 JFK BQN -1 -18 2013-01-01 05:44:00
5 LGA ATL -6 -25 2013-01-01 05:54:00
6 EWR ORD -4 12 2013-01-01 05:54:00
i 4 more variables: horario_llegada <dtm>, salida_programada <dtm>,
llegada_programada <dtm>, tiempo_vuelo <dbl>
```

```
vuelos_dt %>%
 ggplot(aes(horario_salida)) +
 geom_freqpoly(binwidth = 86400) # 86400 segundos = 1 día
```



```
vuelos_dt %>%
 filter(horario_salida < ymd(20130102)) %>%
 ggplot(aes(horario_salida)) +
 geom_freqpoly(binwidth = 600) # 600 segundos = 10 minutos
```



Nota variables que no funciona

```
ymd(c("2010-10-10", "bananas"))
```

```
[1] "2010-10-10" NA
```

```
d1 <- "Jan 1, 2010"
d1
```

```
[1] "Jan 1, 2010"
```

```
mdy(d1)
```

```
[1] "2010-01-01"
```

```
d2 <- "2015-Mar-07"
d2
```

```
[1] "2015-Mar-07"
```

```
ymd(d2)
```

```
[1] "2015-03-07"
```

```
d3 <- "06-Jun-2017"
d3
```

```
[1] "06-Jun-2017"
```

```
dmy(d3)
```

```
[1] "2017-06-06"
```

```
d4 <- c("Aug 19 (2015)", "Jul 1 (2015)")
d4
```

```
[1] "Aug 19 (2015)" "Jul 1 (2015)"
```

```
mdy(d4)
```

```
[1] "2015-08-19" "2015-07-01"
```

```
d5 <- "12/30/14" # Diciembre 30, 2014
d5
```

```
[1] "12/30/14"
```

```
mdy(d5)
```

```
[1] "2014-12-30"
```

```
d6 <- "ene 1, 2010"
d6
```

```
[1] "ene 1, 2010"
```

```
mdy(d6)

[1] NA
d7 <- c("Agosto 19 (2015)", "Julio 1 (2015)")
d7

[1] "Agosto 19 (2015)" "Julio 1 (2015)"
mdy(d7)

[1] NA NA
mdy(d1)

[1] "2010-01-01"
ymd(d2)

[1] "2015-03-07"
dmy(d3)

[1] "2017-06-06"
mdy(d4)

[1] "2015-08-19" "2015-07-01"
mdy(d5)

[1] "2014-12-30"
mdy(d6)

[1] NA
mdy(d7)

[1] NA NA
```

### 1. Ejercicios:

Hacer los ejercicios en la sección 16.2.4 del libro en español

---

## 25.12 Extrayendo parte de las fecha-hora

- `year()`
- `month()`
- `mday()`
- `yday()`



- `wday()`

**Extraer y redondear** (`lubridate`): sacar una parte de una fecha, o llevarla a un borde.

- **`year()`, `month()`, `mday()`, `yday()`, `wday()`**: el año, el mes, el día del mes, el día del año (1 a 366) y el día de la semana.
- **`month(x, label = TRUE)` y `wday(x, label = TRUE)`**: devuelven el nombre en vez del número, como factor ordenado. Es lo que hace falta para graficar.
- **`hour()`, `minute()`, `second()`**: las partes de la hora.
- **`ceiling_date()`**
- **`floor_date()`**
- **`round_date()`**
- **`floor_date(x, "month")`**: lleva la fecha **hacia atrás** al principio del mes. Es la manera de agrupar por mes sin perder el año.
- **`ceiling_date()`**: hacia adelante; **`round_date()`**: al borde más cercano.

Para contar por mes, `floor_date(fecha, "month")` es mejor que `month(fecha)`. `month()` devuelve un número del 1 al 12 y **mezcla los años**: el enero de 2016 y el de 2017 caen en el mismo grupo. `floor_date()` conserva el año, así que la serie sale en orden y sin mezclas.

## 2. Ejercicios:

Hacer estos ejercicios en la sección 16.3.4 del libro en español. Entregar por MTeams.

¿Cómo cambia la demora promedio durante el curso de un día? ¿Deberías usar `horario_salida` o `salida_programada`? ¿Por qué?

¿En qué día de la semana deberías salir si quieres minimizar las posibilidades de una demora?

Confirma nuestra hipótesis de que las salidas programadas en los minutos 20-30 y 50-60 están causadas por los vuelos programados que salen más temprano. Pista: crea una variable binaria que te diga si un vuelo tuvo o no demora.

## 25.13 Lapso de tiempo

Aquí está la idea más importante del capítulo. `lubridate` ofrece **tres** maneras de hablar de un lapso de tiempo, y no son intercambiables:

**Duraciones, períodos e intervalos**: tres maneras de medir un lapso.

- **Duración** (`dseconds()`, `ddays()`, `dyears()`): un número exacto de **segundos**. Un `dyears(1)` son siempre 31 536 000 segundos, haya bisiesto o no. Es tiempo de físico.
- **Período** (`days()`, `months()`, `years()`): unidades **humanas**, que se estiman según el calendario. `years(1)` sumado al 1 de marzo da el 1 de marzo siguiente, tenga ese año 365 o 366 días.
- **Intervalo** (`inicio %--% fin`): un lapso **con puntos de inicio y fin concretos**. Solo con un intervalo se puede saber cuántos días tuvo de verdad, porque se sabe entre qué fechas está.

La regla práctica: **períodos para hablar con humanos, duraciones para hacer física, intervalos cuando importa exactamente cuándo empezó.**

El ejemplo que lo deja claro: si sumas `dyears(1)` (una duración) a una fecha justo antes de un 29 de febrero, caes en un día distinto del que esperabas, porque un año bisiesto tiene un día más y la duración no lo sabe. Con `years(1)` (un período) caes donde cualquier persona diría. Los dos son correctos; contestan preguntas distintas.

## 25.14 duraciones

```
¿Qué edad tiene Charles Darwin? fecha de nacimiento February 12, 1809
edad_h <- today() - ymd("1809-02-12")
edad_h <- today() - ymd(18090212)
edad_h
```

```
Time difference of 79464 days
```

```
as.duration(edad_h)
```

```
[1] "6865689600s (~217.56 years)"
```

```
x=as.duration(edad_h)
```

Otras funciones de duración

```
dseconds(15)
```

```
[1] "15s"
```

```
dminutes(10)
```

```
[1] "600s (~10 minutes)"
```

```
dhours(c(12, 24))
```

```
[1] "43200s (~12 hours)" "86400s (~1 days)"
```

```
ddays(0:5)
```

```
[1] "0s" "86400s (~1 days)" "172800s (~2 days)"
```

```
[4] "259200s (~3 days)" "345600s (~4 days)" "432000s (~5 days)"
dweeks(3)

[1] "1814400s (~3 weeks)"
dyears(1)

[1] "31557600s (~1 years)"
```

## 25.15 Puede agregar periodos de tiempo

```
2 * dyears(1)

[1] "63115200s (~2 years)"
dyears(1) + dweeks(12) + dhours(15)

[1] "38869200s (~1.23 years)"
ayer <- today() - ddays(1)
anio_pasado <- today() - dyears(1)

ayer

[1] "2026-09-05"
anio_pasado

[1] "2025-09-05 18:00:00 UTC"
```

## 25.16 períodos

```
una_pm <- ymd_hms("2016-03-12 13:00:00", tz = "America/New_York")
una_pm

[1] "2016-03-12 13:00:00 EST"
#> [1] "2016-03-12 13:00:00 EST"
una_pm + ddays(1)

[1] "2016-03-13 14:00:00 EDT"
#> [1] "2016-03-13 14:00:00 EDT"

seconds(15)

[1] "15S"
```

```

#> [1] "15S"
minutes(10)

[1] "10M 0S"
#> [1] "10M 0S"
hours(c(12, 24))

[1] "12H 0M 0S" "24H 0M 0S"
#> [1] "12H 0M 0S" "24H 0M 0S"
days(7)

[1] "7d 0H 0M 0S"
#> [1] "7d 0H 0M 0S"
months(1:6)

[1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S" "4m 0d 0H 0M 0S"
[5] "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"
#> [1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S" "4m 0d 0H 0M 0S"
#> [5] "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"
weeks(3)

[1] "21d 0H 0M 0S"
#> [1] "21d 0H 0M 0S"
years(1)

[1] "1y 0m 0d 0H 0M 0S"
#> [1] "1y 0m 0d 0H 0M 0S"

10 * (months(6) + days(1))

[1] "60m 10d 0H 0M 0S"
#> [1] "60m 10d 0H 0M 0S"
days(50) + hours(25) + minutes(2)

[1] "50d 25H 2M 0S"
#> [1] "50d 25H 2M 0S"

Un año bisiesto
ymd("2016-01-01") + dyears(1)

[1] "2016-12-31 06:00:00 UTC"
#> [1] "2016-12-31 06:00:00 UTC"
ymd("2016-01-01") + years(1)

```

```
[1] "2017-01-01"
#> [1] "2017-01-01"

Horarios de verano
una_pm + ddays(1)

[1] "2016-03-13 14:00:00 EDT"
#> [1] "2016-03-13 14:00:00 EDT"
una_pm + days(1)

[1] "2016-03-13 13:00:00 EDT"
#> [1] "2016-03-13 13:00:00 EDT"

10 * (months(6) + days(1))

[1] "60m 10d 0H 0M 0S"
#> [1] "60m 10d 0H 0M 0S"
days(50) + hours(25) + minutes(2)

[1] "50d 25H 2M 0S"
#> [1] "50d 25H 2M 0S"
```

- intervalos

### 25.16.1 Multiplicaciones y sumas de fechas

## 25.17 Períodos

- seconds()
- minutes()
- hours()
- days()
- months()
- weeks()
- years()

### 25.17.1 Multiplicaciones y sumas de periodos

## 25.18 Intervalos

- %-% start end

Un intervalo es lo único que sabe **entre qué dos fechas** está, y por eso es lo único que puede convertirse a cualquier unidad sin ambigüedad. La receta es

siempre la misma: se construye el intervalo con `%--%` y se divide entre la unidad que se quiera.

## 25.19 Resumen

Para escoger entre los tres, la pregunta es qué quieres que pase con las excepciones del calendario:

Quieres	Usa	Ejemplo
Un número exacto de segundos, sin excepciones	duración	tiempo de reacción, decaimiento
Lo que diría una persona con un calendario	período	“la cita es dentro de un mes”
Cuánto tiempo hay entre dos fechas concretas	intervalo	edad, duración de un experimento

### 2. Ejercicios:

Hacer los ejercicios en la sección 16.4.5 del libro en español

## 25.20 Zonas horarias

- `Sys.timezone()`
- `head(OlsonNames())`
- `tz= “ ”`

Una fecha-hora sin zona horaria está incompleta: las 3 de la tarde en San Juan y las 3 de la tarde en Madrid son dos instantes distintos. R guarda la zona como un atributo de la fecha-hora, y por defecto usa la de tu computadora.

**Zonas horarias** (base R y lubridate).

- **`Sys.timezone()`**: en cuál zona está tu computadora ahora mismo.
- **`OlsonNames()`**: la lista completa de nombres válidos, unos 600. Los nombres van en la forma "Continente/Ciudad", como "America/Puerto\_Rico", y **no** con siglas como "AST", que son ambiguas.
- **`with_tz(x, "zona")`**: el **mismo instante**, mostrado en otra zona. Cambia lo que se lee, no el momento.
- **`force_tz(x, "zona")`**: la **misma hora del reloj**, declarada en otra zona. Cambia el momento.

`with_tz()` y `force_tz()` se confunden y hacen cosas opuestas. `with_tz()` es “esta reunión, ¿a qué hora es en Madrid?”. `force_tz()` es “esta hora que anoté sin decir dónde, en realidad era hora de Madrid”. Si te equivocas de función, los datos se mueven varias horas y nada avisa.

### 3. Ejercicios:

Seleccionar 3 archivos de los vuelos que salen o llegan a PR, (el código del aeropuerto es “SJU”) de la base de datos de [https://www.transtats.bts.gov/DL\\_SelectFields.asp?Table\\_ID=236](https://www.transtats.bts.gov/DL_SelectFields.asp?Table_ID=236) Pueden ser el mismo mes en 3 diferentes años o 3 diferentes meses en el mismo año.

- Repite la mayoría de los análisis enseñado arriba (como práctica).
- Evaluar el tiempo de retraso de los vuelos que salen de SJU en cada periodo seleccionado, y haz *una* gráfica para visualizar el patrón
- Cuál es el día preferible para no tener retraso
- Cuál es la mejor hora de salida para no tener retraso
- Compara por lo menos 3 diferentes líneas saliendo de SJU y el periodo de retraso.

---



---

## 25.21 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan `Datos/crimen_pr.csv`, que está en el repositorio del curso y trae fecha y hora de verdad, no **vuelos** ni la fecha de nacimiento de Darwin, que son los del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_25_fechas.Rmd`.

### 25.21.1 Ejercicio 1. La misma fecha, escrita de cinco maneras

Convierte a fecha estos cinco textos, que son **todos el mismo día**, cada uno con la función de `lubridate` que le corresponda: "2016-08-01", "01/08/2016", "August 1, 2016", "20160801" y "01.08.2016".

**Verificación:** los cinco tienen que dar exactamente la misma fecha. Compruébalo con `n_distinct()`, que debe dar 1.

**En palabras:** prueba ahora `mdy("01/08/2016")` y mira qué sale. ¿Dio error? Explica por qué este es el error más peligroso de todo el capítulo y qué comprobación harías en una columna de 5000 fechas para detectarlo.

```
Escribe tu código aquí
```

### 25.21.2 Ejercicio 2. Extraer las partes

Lee `Datos/crimen_pr.csv`. Sacá el día de la semana y la hora de cada incidente, y cuenta cuántos hay de cada uno. Usa `label = TRUE` para que los días salgan con su nombre.

**Verificación:** tienen que salir **7** días de la semana y **24** horas del día, todas con algún caso.

**En palabras:** ¿cuál día y cuál hora concentran más incidentes? Haz una gráfica de barras de cada uno. ¿Qué habría pasado si hubieras usado `wday()` sin `label = TRUE` para graficar?

```
Escribe tu código aquí
```

### 25.21.3 Ejercicio 3. `%%` y `%`

Muchos archivos guardan la hora como un número de la forma HHMM. Toma este vector: `horas <- c(1730, 905, 2359, 15, 1200)`.

Sepáralo en hora y minutos con `%%` y `%`, y calcula los **minutos transcurridos desde la medianoche** de cada uno. Después convierte los mismos valores a hora de verdad con `hm()` de `lubridate` y comprueba que coinciden.

**Verificación:** los minutos desde la medianoche tienen que ser 1050, 545, 1439, 15, 720. La suma de los cinco es 3769.

**En palabras:** el valor 15 significa las 00:15, no las 15:00. Explica por qué `%%` y `%` lo resuelven bien sin ningún caso especial. ¿Qué habría pasado si hubieras intentado partir el número con `str_sub()`, cortando por posición?

```
Escribe tu código aquí
```

### 25.21.4 Ejercicio 4. Duración contra período

Toma el 1 de marzo de 2024 (un año bisiesto) y réstale un año de las dos maneras: con `years(1)` y con `dyears(1)`. Haz lo mismo sumando un mes al 31 de enero, con `months(1)`.

**Verificación:** las dos restas dan **días distintos**, y la suma al 31 de enero da NA.

**En palabras:** ninguno de los tres resultados es un error del programa. Explica qué contesta cada uno, y por qué el NA es la respuesta más honesta que puede dar R a “¿qué día es un mes después del 31 de enero?”.

```
Escribe tu código aquí
```



### 25.21.5 Ejercicio 5. Intervalos

Con `crimen_pr.csv`, construye el intervalo que va de la primera a la última fecha del archivo con `%--%`, y exprésalo en días, en semanas y en meses.

**Verificación:** el intervalo tiene que ser de unos dos meses, y en días tiene que dar un número entre 60 y 65.

**En palabras:** divide también entre `dmonths(1)` en vez de `months(1)` y compara. ¿Por qué no dan lo mismo? ¿Cuál de los dos números pondrías en la sección de métodos de un informe?

*# Escribe tu código aquí*

### 25.21.6 Ejercicio 6. Redondear para agrupar

Con el mismo archivo, cuenta los incidentes por **semana** usando `floor_date(Fecha, "week")`, y haz una gráfica de líneas de la serie.

**Verificación:** deben salir unas nueve o diez semanas, y la primera y la última tienen que tener bastantes menos casos que las del medio.

**En palabras:** ¿por qué los extremos son más bajos? Ya viste este mismo problema en el capítulo de importar datos. ¿Qué harías con esas dos semanas antes de enseñarle la gráfica a alguien?

*# Escribe tu código aquí*

### 25.21.7 Ejercicio 7 (BONO). Zonas horarias

Averigua en cuál zona horaria está tu computadora con `Sys.timezone()`. Después toma el instante "2016-08-01 15:30:00" en "America/Puerto\_Rico" y míralo en "Europe/Madrid" y en "America/Los\_Angeles" con `with_tz()`. Prueba también `force_tz()` con el mismo instante.

**Verificación:** con `with_tz()` el instante es el mismo y solo cambia la hora que se lee; con `force_tz()` cambia el instante.

**En palabras:** explica la diferencia con tus palabras. Puerto Rico no cambia de horario de verano y el este de Estados Unidos sí: ¿qué le pasa a la diferencia de horas entre San Juan y Nueva York a lo largo del año, y qué problema causaría eso al comparar datos de vuelos de los dos sitios?

*# Escribe tu código aquí*



## Chapter 26

# Funciones

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/functions.html>

Español: <https://es.r4ds.hadley.nz/funciones.html>

---

Fecha de la última revisión

## [1] "2026-09-06"

### 26.1 Temas:

- ¿Cuándo escribir una función?
- La anatomía de una función: nombre, argumentos y cuerpo
- Cómo se le pone nombre a una función
- Ejecución condicional: `if` y `else`
- Argumentos y valores por defecto
- Comprobar los argumentos con `stopifnot()`
- Los puntos suspensivos `...`
- Valores de retorno
- Lo que pasa dentro se queda dentro: el ámbito
- Un vistazo a los vectores

En el capítulo de *tidyverse avanzado* verás funciones más sofisticadas que usan *tidy evaluation* (el operador `{{ }}`). Aquí construimos la base.

## 26.2 ¿Cuándo escribir una función?

Una **función** te permite darle un nombre a un bloque de código para poder reutilizarlo. La regla práctica es: **si copiaste y pegaste el mismo código más de dos veces, escribe una función**. Copiar y pegar es una de las fuentes de errores más comunes, porque es fácil olvidar cambiar el nombre de una variable en una de las copias.

Regla de tres: si copias y pegas el mismo bloque de código más de dos veces, conviértelo en una función. Así reduces errores y solo corriges el código en un lugar.

Observa este código que reescala tres columnas para que queden entre 0 y 1:

```
df <- data.frame(
 a = c(2, 5, 8, 1),
 b = c(10, 20, 30, 40),
 c = c(-5, 0, 5, 10)
)

df$a <- (df$a - min(df$a)) / (max(df$a) - min(df$a))
df$b <- (df$b - min(df$b)) / (max(df$b) - min(df$b))
df$c <- (df$c - min(df$c)) / (max(df$c) - min(df$c))
df

a b c
1 0.1428571 0.0000000 0.0000000
2 0.5714286 0.3333333 0.3333333
3 1.0000000 0.6666667 0.6666667
4 0.0000000 1.0000000 1.0000000
```

¿Notaste el patrón repetido? Eso es una señal clara de que conviene una función.

## 26.3 La anatomía de una función

**R** es un lenguaje de *programación funcional*: las funciones son “ciudadanos de primera clase” (se pueden guardar en variables y pasar a otras funciones). Heredó esta filosofía del lenguaje **S** y de **Scheme**, que inspiraron a sus creadores.

**function(argumentos) { cuerpo }**: crear tu propia función.

- **Estructura:** `nombre <- function(x) { ... }`; se llama luego con `nombre(valor)`.
- **Argumentos:** pueden tener valores por defecto (`function(x, recorte = 0.1)`).
- **Retorno:** devuelve el último valor evaluado, o el que indiques con `return()`.

Toda función tiene tres partes:

1. Un **nombre** (aquí, `reescalar01`).
2. Los **argumentos** que van dentro de `function( )` (aquí, `x`).
3. El **cuerpo**, el código dentro de las llaves `{ }`.

```
reescalar01 <- function(x) {
 rango <- range(x, na.rm = TRUE)
 (x - rango[1]) / (rango[2] - rango[1])
}

Probar la función con un ejemplo cuya respuesta ya conocemos
reescalar01(c(0, 5, 10))

[1] 0.0 0.5 1.0
```

Ahora el código repetido se reduce a esto, mucho más fácil de leer y de corregir:

### 26.3.1 Cómo se le pone nombre a una función

El nombre de una función es parte del código, no una etiqueta: se va a leer muchas más veces de las que se escribe. Tres reglas resuelven casi todo:

1. **Un verbo, o algo que describa lo que devuelve.** Las funciones **hacen** cosas: `reescalar01()`, `contar_especies()`, `calcular_tasa()`. Los sustantivos se reservan para los objetos.
2. **En minúsculas y con guion bajo** (`snake_case`), como el resto del tidyverse: `media_recortada`, no `MediaRecortada` ni `media.recortada`. El punto tiene otro significado en R y conviene no usarlo.
3. **Nunca uses un nombre que R ya usa.** Si defines tu propia `mean()`, `sum()`, `c()`, `data()` o `t()`, R usará la tuya a partir de ese momento y el resto del análisis se romperá de maneras difíciles de rastrear.

Para saber si un nombre está ocupado, escríbelo en la consola y presiona *Tab*, o prueba `exists("mi_nombre")`. Si devuelve `TRUE`, busca otro. Es el mismo problema del object of type 'closure' del capítulo de errores comunes, visto desde el otro lado.

```
df <- data.frame(
 a = c(2, 5, 8, 1),
 b = c(10, 20, 30, 40),
 c = c(-5, 0, 5, 10)
)

df$a <- reescalar01(df$a)
df$b <- reescalar01(df$b)
df$c <- reescalar01(df$c)
df
```

```
a b c
1 0.1428571 0.0000000 0.0000000
2 0.5714286 0.3333333 0.3333333
3 1.0000000 0.6666667 0.6666667
4 0.0000000 1.0000000 1.0000000
```

Una función siempre se debe **probar con valores cuya respuesta ya conoces** antes de confiar en ella.

---

## 26.4 Ejecución condicional: if y else

Dentro de una función puedes tomar decisiones con `if` y `else`.

```
signo <- function(x) {
 if (x > 0) {
 "positivo"
 } else if (x < 0) {
 "negativo"
 } else {
 "cero"
 }
}
```

```
signo(7)
```

```
[1] "positivo"
```

```
signo(-3)
```

```
[1] "negativo"
```

```
signo(0)
```

```
[1] "cero"
```

---

## 26.5 Argumentos y valores por defecto

Los argumentos pueden tener un **valor por defecto**, que se usa cuando quien llama la función no especifica ese argumento.

```
Calcular la media recortada; por defecto recorta un 10%
media_recortada <- function(x, recorte = 0.1) {
 mean(x, trim = recorte, na.rm = TRUE)
}
```

```
valores <- c(1, 2, 3, 4, 5, 100) # el 100 es un valor atípico
media_recortada(valores) # usa recorte = 0.1

[1] 19.16667

media_recortada(valores, recorte = 0.2)

[1] 3.5

mean(valores) # comparación: la media normal

[1] 19.16667
```

## 26.6 Comprobar los argumentos

Una función escrita por ti la va a usar alguien que no la escribió, y ese alguien casi siempre eres tú dentro de seis meses. Conviene que falle **rápido y claro** cuando le pasan algo que no esperaba, en vez de devolver un resultado raro.

**stopifnot()**: comprobar que los argumentos son lo que deben ser.

- **stopifnot(is.numeric(x))**: si la condición no se cumple, detiene la función con un mensaje que nombra la condición que falló.
- **Varias a la vez**: `stopifnot(is.numeric(x), length(x) > 0)`.
- **Con mensaje propio**: `stopifnot("x tiene que ser numérico" = is.numeric(x))`.

```
reescalar_seguro <- function(x) {
 stopifnot("x tiene que ser numérico" = is.numeric(x),
 "x no puede estar vacío" = length(x) > 0)
 rango <- range(x, na.rm = TRUE)
 (x - rango[1]) / (rango[2] - rango[1])
}
```

```
reescalar_seguro(c(0, 5, 10))
```

```
[1] 0.0 0.5 1.0
```

```
reescalar_seguro(c("a", "b"))
```

```
Error in `reescalar_seguro()`:
! x tiene que ser numérico
```

No hace falta comprobarlo todo. Comprueba lo que, si viene mal, produciría un resultado **equivocado en silencio**. Si viniera mal y diera un error claro por su cuenta, el `stopifnot()` sobra.

## 26.7 Los puntos suspensivos ...

... significa “cualquier otro argumento que me pasen, se lo entrego tal cual a la función de adentro”. Es lo que permite que tu función acepte los argumentos de otra sin tener que copiarlos todos.

```
promedio_limpio <- function(x, ...) {
 mean(x, na.rm = TRUE, ...)
}
```

```
valores <- c(1, 2, 3, 4, 5, 100, NA)
promedio_limpio(valores)
```

```
[1] 19.16667
```

```
promedio_limpio(valores, trim = 0.2) # trim viaja hasta mean()
```

```
[1] 3.5
```

... tiene un precio: si escribes mal el nombre de un argumento, **no hay error**. `promedio_limpio(valores, trimm = 0.2)` se traga el `trimm` sin decir nada y devuelve el promedio sin recortar. Es cómodo y silencioso, en el mal sentido.

## 26.8 Valores de retorno

Una función en R devuelve el valor de la **última línea** de su cuerpo. También puedes usar `return()` para devolver un valor antes del final, por ejemplo para salir temprano en un caso especial.

```
coef_variacion <- function(x) {
 if (mean(x, na.rm = TRUE) == 0) {
 return(NA) # evitar división por cero
 }
 sd(x, na.rm = TRUE) / mean(x, na.rm = TRUE)
}
```

```
coef_variacion(c(2, 4, 4, 4, 5, 5, 7, 9))
```

```
[1] 0.427618
```

```
coef_variacion(c(-5, 0, 5)) # la media es 0 -> NA
```

```
[1] NA
```



## 26.9 Lo que pasa dentro se queda dentro

Las variables que creas dentro de una función viven **solo dentro de ella**. Cuando la función termina, desaparecen, y nada de lo que pasó ahí dentro afecta a tu sesión.

```
rango <- "yo vivo fuera"

reescalar01(c(0, 5, 10)) # por dentro crea su propio 'rango'

[1] 0.0 0.5 1.0
rango # el de afuera no se tocó

[1] "yo vivo fuera"
```

Eso es una ventaja, no una limitación: significa que puedes usar una función escrita por otra persona sin miedo a que te pise tus objetos. Y significa también que una función **solo se comunica por dos puertas**: los argumentos que entran y el valor que devuelve.

Al revés no es simétrico: una función **sí** puede **leer** los objetos de fuera si no los encuentra dentro. Eso hace que una función pueda parecer que funciona en tu sesión y fallar en la de otra persona, porque dependía en secreto de un objeto que tú tenías creado. La regla: **todo lo que la función necesite tiene que entrar por un argumento**.

## 26.10 Un vistazo a los vectores

Las funciones trabajan sobre **vectores**. Conocer los tipos básicos de vectores atómicos explica muchos “comportamientos raros” de R.

```
lg1 <- c(TRUE, FALSE, NA) # lógico
int <- c(1L, 5L, 10L) # entero
dbl <- c(1.5, 2.7, 3.14) # doble (números reales)
chr <- c("a", "b", "c") # cadena de caracteres

typeof(lg1)

[1] "logical"
typeof(int)

[1] "integer"
typeof(dbl)

[1] "double"
```

```
typeof(chr)
```

```
[1] "character"
```

**Coerción:** cuando mezclas tipos, R convierte todo al tipo “más flexible”.

```
c(1, 2, "tres") # todo se convierte en texto
```

```
[1] "1" "2" "tres"
```

```
c(TRUE, 1L, 2.5) # lógico -> entero -> doble
```

```
[1] 1.0 1.0 2.5
```

**Valores faltantes (NA):** casi cualquier operación con NA da NA. Por eso muchas funciones tienen el argumento `na.rm = TRUE`.

Si olvidas `na.rm = TRUE`, basta con un solo NA en los datos para que `mean()`, `sd()` y muchas otras funciones devuelvan NA sin avisarte. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
x <- c(1, 2, NA, 4)
```

```
mean(x) # NA
```

```
[1] NA
```

```
mean(x, na.rm = TRUE) # 2.333...
```

```
[1] 2.333333
```

```
is.na(x) # ¿cuáles son NA?
```

```
[1] FALSE FALSE TRUE FALSE
```

## 26.11 ¿Cuándo no escribir una función?

La regla de tres es buena, pero tiene excepciones. No conviene envolver en una función:

- **una sola línea que ya se lee bien.** `function(x) mean(x)` no aporta nada;
- **algo que ya existe.** Antes de escribirla, busca: casi siempre alguien la escribió antes y mejor;
- **un cálculo que solo vas a hacer una vez**, aunque sea largo. Una función que se usa una sola vez añade un nivel de indirección sin ahorrar nada;
- **un paso de una cadena de dplyr.** Para eso están `mutate()` y `summarise()`, que ya trabajan sobre columnas enteras.

## 26.12 Ejercicios del libro R4DS

Hacer los ejercicios de las secciones 19.2 y 19.4 del libro en español.

---

## 26.13 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Cada ejercicio se entrega con tres cosas: la **función**, una **prueba** con valores cuya respuesta ya conoces, y una **explicación en palabras**. La prueba no es opcional: una función sin probar no sirve de nada, y en este capítulo vale puntos. Los ejercicios usan `millas` y `países` del paquete `datos`, no el `df` de tres columnas del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_26_funciones.Rmd`.

### 26.13.1 Ejercicio 1. Reescalar de verdad

Usa `reescalar01()` del capítulo sobre las columnas `ciudad` y `autopista` de `millas`. Después pruébala con una columna **constante**, por ejemplo `rep(5, 10)`.

**Verificación:** en las dos columnas de `millas`, el mínimo del resultado tiene que ser 0 y el máximo 1. Con la columna constante sale `NaN` en todas las posiciones.

**En palabras:** ¿de dónde sale ese `NaN`? Escribe la operación que lo produce. ¿Debería la función avisar en ese caso, o está bien que devuelva `NaN`? Justifica tu respuesta.

```
Escribe tu código aquí
```

### 26.13.2 Ejercicio 2. Una función que devuelve una tabla

Escribe `resumen_numerico(x)`, que reciba un vector numérico y devuelva un tibble de una fila con cinco columnas: `n`, `media`, `sd`, `mediana` e `IQR`, todas calculadas ignorando los `NA`.

Úsala con `millas$autopista` y después, dentro de un `group_by()` por `clase`, con `reframe()` o con `summarise()`.

**Verificación:** para `millas` completa, `n` debe dar 234. Por `clase`, la suma de los `n` también.

**En palabras:** ¿por qué conviene que devuelva un **tibble** y no un vector con nombres? Piensa en qué puedes hacer después con cada uno.

```
Escribe tu código aquí
```

### 26.13.3 Ejercicio 3. Valores por defecto

Escribe `es_atipico(x, k = 1.5)`, que devuelva `TRUE` para los valores que quedan a más de `k` rangos intercuartiles fuera de la caja, que es la misma regla del diagrama de caja.

Aplicala a `millas$autopista` con `k = 1.5` y con `k = 3`.

**Verificación:** con `k = 3` tiene que marcar **menos** valores que con `k = 1.5`, o los mismos, nunca más.

**En palabras:** ¿por qué el valor por defecto es 1.5 y no otro número? Y una pregunta más importante: ¿un valor marcado como atípico es un error del dato? Mira cuáles vehículos son.

*# Escribe tu código aquí*

### 26.13.4 Ejercicio 4. Que falle rápido y claro

Añádele a `resumen_numerico()` un `stopifnot()` que compruebe dos cosas: que `x` es numérico y que tiene al menos un valor no faltante. Ponle a cada comprobación un mensaje en español.

Pruébala con `millas$clase` (que es texto) y con `c(NA, NA)`, las dos en un bloque con `error=TRUE`.

**Verificación:** las dos llamadas malas se detienen con **tu** mensaje, no con un mensaje genérico de R.

**En palabras:** sin el `stopifnot()`, ¿qué habría devuelto la función con `millas$clase`? Pruébalo. ¿Un error o un resultado sin sentido? Explica por qué eso justifica las tres líneas de más.

*# Escribe tu código aquí*

### 26.13.5 Ejercicio 5. Lo que pasa dentro

Escribe una función que cree dentro una variable llamada **resultado**. Antes de llamarla, crea en tu sesión otra variable con **el mismo nombre** y un valor distinto. Llama a la función y vuelve a mirar la de fuera.

Después escribe una segunda función que use una variable que **no** recibe como argumento y que solo existe fuera. Compruébala, bórrala con `rm()` y vuelve a llamarla.

**Verificación:** la primera función no toca la variable de fuera. La segunda funciona hasta que borras la variable, y entonces da `object ... not found`.

**En palabras:** la segunda función parecía correcta y no lo era. Explica por qué es un problema si le pasas tu script a otra persona.

```
Escribe tu código aquí
```

### 26.13.6 Ejercicio 6. Los puntos suspensivos

Escribe `promedio_por_grupo(datos, columna, ...)` que calcule el promedio de una columna por `clase` en millas, pasando los `...` a `mean()`.

Llámalas una vez normal y otra con `trim = 0.1`. Después llámala con `trimm = 0.1`, con dos eses.

**Verificación:** las dos primeras dan resultados distintos entre sí; la tercera da **exactamente lo mismo que la primera**, y sin ningún aviso.

**En palabras:** eso último es el precio de `...`. Explica qué pasó con el `trimm`. ¿Qué tipo de error del capítulo de errores comunes se parece más a este?

```
Escribe tu código aquí
```

### 26.13.7 Ejercicio 7 (BONO). Una función que devuelve una gráfica

Escribe `grafica_consumo(datos, clase_elegida)` que filtre millas por la clase que le pasen y devuelva un `ggplot` de `cilindrada` contra `autopista`, con el título puesto automáticamente con el nombre de la clase.

**Verificación:** la función tiene que **devolver** el objeto gráfico, no dibujarlo por dentro. Compruébalo guardándolo en una variable: la gráfica no debe aparecer hasta que escribas el nombre de la variable.

**En palabras:** ¿qué ganas devolviendo el objeto en vez de dibujarlo dentro? Piensa en poder añadirle una capa después con `+`.

```
Escribe tu código aquí
```



## Chapter 27

# Iteración: purrr

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/iteration.html>

Español: <https://es.r4ds.hadley.nz/iteración.html>

---

Fecha de la última revisión

## [1] "2026-09-06"

### 27.1 Temas: El paquete purrr

- La idea de iterar (repetir una acción)
- El bucle `for` de base R
- La familia `map()` de `purrr`
- Fórmulas anónimas con `~` y `.x`
- Iterar sobre dos listas con `map2()`
- `across()`: iterar sobre columnas dentro de `dplyr`
- `walk()`: iterar para hacer algo, no para calcular
- `safely()`: cuando alguna vuelta puede fallar
- Un modelo por grupo
- Cuándo un `for` sigue siendo mejor

`purrr` es parte de `tidyverse`, así que se activa con `library(tidyverse)`.

```
library(tidyverse)
library(datos)
```

## 27.2 ¿Qué es iterar?

**Iterar** es repetir la misma acción sobre cada elemento de una lista, vector o columna. Igual que con las funciones, la meta es **no repetir código**. Si las funciones evitan copiar y pegar código *dentro* de una tarea, la iteración evita copiar y pegar la *misma tarea* muchas veces.

Supón que queremos la mediana de cada columna de este data frame:

```
df <- tibble(
 a = rnorm(10),
 b = rnorm(10),
 c = rnorm(10),
 d = rnorm(10)
)

Repetir a mano es tedioso y propenso a errores:
median(df$a)

[1] -0.2364174

median(df$b)

[1] 0.2332814

median(df$c)

[1] 1.136037

median(df$d)

[1] 0.4082237
```

---

## 27.3 El bucle for de base R

La forma clásica de iterar es con un bucle `for`:

```
resultado <- vector("double", ncol(df)) # espacio para guardar la salida
for (i in seq_along(df)) { # para cada columna
 resultado[[i]] <- median(df[[i]]) # calcular y guardar
}
resultado

[1] -0.2364174 0.2332814 1.1360375 0.4082237
```

Funciona, pero hay que escribir bastante “andamiaje” (crear el vector de salida, manejar el índice `i`). `purrr` simplifica esto.

Crea el vector de salida con su tamaño final *antes* del bucle. Hacerlo crecer



dentro del `for` (por ejemplo con `c()` en cada vuelta) obliga a R a copiarlo una y otra vez y vuelve el código muy lento.

## 27.4 La familia `map()`

La idea de **map** (aplicar una función a cada elemento) viene de la programación funcional y del lenguaje **Lisp** (años 1960). El paquete **purrr**, de **Hadley Wickham** y **Lionel Henry**, la trajo al tidyverse con una familia de funciones de tipos predecibles.

**La familia `map()` (`purrr`):** aplicar una función a cada elemento.

- **`map(x, f)`:** aplica `f` a cada elemento y devuelve una **lista**.
- **Variantes por tipo:** `map_dbl()`, `map_int()`, `map_chr()`, `map_lgl()` devuelven un vector del tipo indicado.
- **Fórmula anónima:** `~ f(.x)` define una función corta donde `.x` es cada elemento; `map2()` itera sobre dos listas a la vez.

Las funciones `map()` toman un vector o lista, aplican una función a **cada elemento**, y devuelven un resultado. Hay una versión por tipo de salida:

- `map()` devuelve una **lista**
- `map_dbl()` devuelve un vector de **dobles**
- `map_int()` devuelve un vector de **enteros**
- `map_chr()` devuelve un vector de **texto**
- `map_lgl()` devuelve un vector **lógico**

Esa lista de variantes no es un capricho. La diferencia importante es que `map_dbl()` **comprueba el tipo**: si alguna vuelta devuelve algo que no es un número, se detiene con un error. `map()` no comprueba nada y te devuelve una lista con cosas de tipos distintos, que descubres tres pasos después.

Usa siempre la variante **más específica** que puedas: `map_dbl()` si esperas números, `map_chr()` si esperas texto. El error inmediato vale mucho más que una lista rara. Deja `map()` a secas para cuando cada vuelta devuelve algo complejo, como un modelo o un data frame.

```
map_dbl(df, median) # la mediana de cada columna, en una sola línea
```

```
a b c d
-0.2364174 0.2332814 1.1360375 0.4082237
```

```
map_dbl(df, mean)
```

```
a b c d
-0.3534311 0.1467468 1.0998756 0.2888905
```

```
map_dbl(df, sd)
```

```
a b c d
0.9753734 0.8503055 0.5548967 1.0877976
```

Compara: el bucle `for` completo de arriba se reduce a `map_dbl(df, median)`.

---

## 27.5 Fórmulas anónimas con `~` y `.x`

A veces necesitas una pequeña operación que no tiene nombre de función. Puedes escribirla como una **fórmula** empezando con `~`, y usar `.x` para representar “el elemento actual”.

```
El rango (máximo - mínimo) de cada columna
map_dbl(df, ~ max(.x) - min(.x))
```

```
a b c d
3.042304 2.203092 1.832097 3.338961
```

```
¿Cuántos valores positivos hay en cada columna?
map_int(df, ~ sum(.x > 0))
```

```
a b c d
3 5 10 6
```

Esto es muy útil con datos reales. Por ejemplo, el promedio de atraso de salida por mes en los vuelos:

```
vuelos %>%
 split(.$mes) %>% # una lista: un data frame por me
 map_dbl(~ mean(.x$atraso_salida, na.rm = TRUE)) # promedio de atraso por mes
```

```
1 2 3 4 5 6 7 8
10.036665 10.816843 13.227076 13.938038 12.986859 20.846332 21.727787 12.611040
9 10 11 12
6.722476 6.243988 5.435362 16.576688
```

---

## 27.6 Iterar sobre dos listas con `map2()`

Cuando necesitas recorrer **dos** listas en paralelo, usa `map2()`. Los elementos se representan con `.x` (de la primera) y `.y` (de la segunda).

```
medias <- c(0, 10, 100)
desv <- c(1, 2, 3)
```

```
Generar 5 números al azar de cada combinación de media y desviación
map2(medias, desv, ~ rnorm(5, mean = .x, sd = .y))

[[1]]
[1] 1.01720062 -0.45401384 1.11807753 0.02410736 -0.30014527
##
[[2]]
[1] 13.09959 10.12764 10.11323 9.00243 10.06829
##
[[3]]
[1] 99.31260 99.88415 98.67224 100.17103 103.52231
```

---

## 27.7 across(): iterar sobre columnas dentro de dplyr

Para el caso concreto de “hacerle lo mismo a varias columnas de una tabla”, `dplyr` tiene su propia herramienta y suele ser la mejor opción, porque el resultado sale como tabla y encaja con `group_by()`.

**across()** (`dplyr`): aplicar una función a varias columnas dentro de `mutate()` o `summarise()`.

- **summarise(across(where(is.numeric), mean))**: el promedio de todas las columnas numéricas.
- **across(c(a, b), ~ .x / 100)**: columnas escogidas a mano, con una fórmula anónima.
- **Varias funciones a la vez**: `across(where(is.numeric), list(media = mean, sd = sd))`.
- **Con group\_by()**: funciona por grupo, que es lo que casi nunca es cómodo hacer con `map()`.

```
millas |>
 summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE)))

A tibble: 1 x 5
cilindrada anio cilindros ciudad autopista
<dbl> <dbl> <dbl> <dbl> <dbl>
1 3.47 2004. 5.89 16.9 23.4

millas |>
 group_by(clase) |>
 summarise(across(c(ciudad, autopista), \(x) mean(x, na.rm = TRUE)))

A tibble: 7 x 3
clase ciudad autopista
```

```
<chr> <dbl> <dbl>
1 2asientos 15.4 24.8
2 compacto 20.1 28.3
3 mediano 18.8 27.3
4 minivan 15.8 22.4
5 pickup 13 16.9
6 subcompacto 20.4 28.1
7 suv 13.5 18.1
```

`\(x) mean(x)` es la manera moderna de escribir una función anónima en R, desde la versión 4.1. Es lo mismo que `function(x) mean(x)`, solo que más corto. En `purrr` también se puede usar `~ mean(.x)`; las tres formas conviven y vas a encontrarte las tres en código ajeno.

La regla para escoger: **`across()` cuando trabajas dentro de una tabla**, y **`map()` cuando trabajas con una lista de cosas** que no son columnas, como una lista de modelos, de archivos o de gráficas.

## 27.8 walk(): iterar para hacer algo

A veces no quieres un resultado: quieres que pase algo. Guardar diez archivos, imprimir diez gráficas, escribir diez mensajes. Para eso está `walk()`, que hace exactamente lo mismo que `map()` pero **descarta el resultado** y devuelve la entrada sin cambios.

```
nombres <- c("compact", "suv", "pickup")

walk(nombres, \(cl) {
 millas |>
 filter(clase == cl) |>
 write_csv(paste0("Datos/millas_", cl, ".csv"))
})
```

Si usaras `map()` para eso, funcionaría igual pero además te imprimiría en la consola una lista con los tres data frames enteros, que no le interesan a nadie. `walk()` existe para decir “esto es por el efecto, no por el valor”.

## 27.9 safely(): cuando alguna vuelta puede fallar

Un `map()` sobre cuarenta grupos se detiene entero si **uno solo** falla, y no te dice cuál. `safely()` envuelve una función para que en vez de fallar devuelva una lista con dos partes: **result** y **error**. Una de las dos siempre es `NULL`.

**safely()**, **possibly()** y **quietly()** (purrr): iterar sin que un fallo lo pare todo.

- **safely(f)**: devuelve `list(result = ..., error = ...)` en cada vuelta. La que falló trae el error y `result = NULL`.
- **possibly(f, otherwise = NA)**: más simple, devuelve el valor que le digas cuando falla.
- **quietly(f)**: captura también los mensajes y las advertencias, en vez de dejarlos salir por la consola.

```
promedio_exigente <- function(x) {
 if (nrow(x) < 10) stop("muy pocos datos")
 mean(x$autopista)
}

resultados <- millas |>
 split(millas$clase) |>
 purrr::map(safely(promedio_exigente))

¿cuáles fallaron?
resultados |> purrr::map("error") |> compact() |> names()

[1] "2asientos"

los que sí salieron
resultados |> purrr::map("result") |> compact() |> unlist()
```

##	compacto	mediano	minivan	pickup	subcompacto	suv
##	28.29787	27.29268	22.36364	16.87879	28.14286	18.12903

Fíjate en que aquí se escribe `purrr::map()` y no `map()` a secas. En el capítulo de datos relacionados se activó el paquete `maps` (el de los mapas del mundo), y ese paquete **también** tiene una función llamada `map()`. Como se activó después, la suya tapa a la de `purrr` en todo lo que viene a continuación del libro, y una llamada a `map()` sin apellido acaba en un error rarísimo sobre bases de datos geográficas.

Es exactamente el enmascaramiento del que avisa `library()` con su mensaje “The following object is masked from...”, el mismo del capítulo de opciones de knitr. La solución es la de siempre: poner el apellido del paquete con `::`.

`compact()` quita los `NULL` de una lista, así que las dos líneas de arriba separan lo que funcionó de lo que no. Es el patrón estándar para procesar muchos archivos o muchos grupos sabiendo de antemano que alguno va a dar problemas.

## 27.10 Un modelo por grupo

Un uso poderoso es ajustar **un modelo por grupo**. Aquí ajustamos una regresión del atraso de llegada contra el de salida, por separado para cada aerolínea.

Como el paquete `maps` también se carga en este libro, escribimos `purrr::map()` de forma explícita para no confundirla con `maps::map()`.

```
modelos <- vuelos %>%
 split(.$aerolinea) %>%
 purrr::map(~ lm(atraso_llegada ~ atraso_salida, data = .x))

Ver el R2 de cada modelo
modelos %>%
 purrr::map(summary) %>%
 map_dbl("r.squared") %>%
 head()
```

```
9E AA AS B6 DL EV
0.8622935 0.7952061 0.7012039 0.8369837 0.8192728 0.9080101
```

Este patrón (dividir, aplicar un modelo a cada parte, y resumir) es la base del tema de *muchos modelos* con `purrr` y `broom`.

`map_dbl("r.squared")` funciona porque, cuando en vez de una función le pasas un **texto**, `map()` lo entiende como “sácame ese elemento de cada uno”. Es un atajo muy usado y algo críptico la primera vez que se ve. Lo mismo con un número: `map_dbl(lista, 1)` saca el primer elemento de cada uno.

---

## 27.11 Cuándo un for sigue siendo mejor

`map()` no sustituye al `for` en todos los casos. El bucle sigue siendo la herramienta correcta cuando:

- **cada vuelta depende de la anterior**, como en una simulación de crecimiento poblacional donde el año siguiente se calcula a partir del actual. `map()` supone que las vueltas son independientes;
- **no hay nada que devolver y el código es largo**, aunque para eso normalmente `walk()` queda más claro;
- **estás depurando**: dentro de un `for` puedes parar en la vuelta 17 y mirar el estado de todo. Dentro de un `map()` eso es bastante más incómodo.

Y al revés: en cuanto haya que **guardar un resultado por vuelta**, `map()` gana, porque se ocupa solo de crear el vector de salida del tamaño y el tipo correctos, que es justo la parte del `for` donde se cometen los errores.

---

## 27.12 Ejercicios del libro R4DS

Hacer los ejercicios de las secciones 21.5 y 21.9 del libro en español.

---

## 27.13 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan `millas` y `países` del paquete `datos`, no el `df` de números al azar, `vuelos` ni `diamantes`, que son los del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_27_iteracion.Rmd`.

### 27.13.1 Ejercicio 1. Del `for` al `map()`

Escribe un bucle `for` que calcule el promedio de cada columna **numérica** de `millas`, creando el vector de salida con su tamaño final antes del bucle. Después escribe la misma cosa con `map_dbl()`, en una línea.

**Verificación:** `millas` tiene 5 columnas numéricas, y los dos caminos tienen que dar exactamente los mismos cinco números.

**En palabras:** cuenta cuántas líneas ocupa cada versión. ¿Cuál de las partes del `for` desapareció al usar `map_dbl()`, y por qué esa parte era justamente la que más errores causa?

```
Escribe tu código aquí
```

### 27.13.2 Ejercicio 2. El tipo importa

Cuenta los valores distintos de **todas** las columnas de `millas` con `n_distinct()`, de tres maneras: con `map_dbl()`, con `map_chr()` y con `map()` a secas. Pon el que falle en un bloque con `error=TRUE`.

**Verificación:** una de las tres da error y las otras dos funcionan, pero devuelven cosas de **clase distinta**. Compruébalo con `class()`.

**En palabras:** ¿por qué falla la que falla? ¿Cuál de las tres usarías y por qué? Relaciónalo con la idea de que un error inmediato vale más que un resultado raro.

```
Escribe tu código aquí
```

### 27.13.3 Ejercicio 3. Fórmulas anónimas

Con `millas`, usa `map_int()` y una fórmula anónima con `~` y `.x` para contar, en cada columna, cuántos valores distintos hay. Después usa `map_lgl()` para

saber cuáles columnas son numéricas.

**Verificación:** `map_lgl(millas, is.numeric)` tiene que dar **5** TRUE, y `clase` tiene que salir con **7** valores distintos.

**En palabras:** escribe la misma fórmula anónima de las tres maneras que existen: con `~ .x`, con `\(x)` y con `function(x)`. ¿Cuál prefieres y por qué?

*# Escribe tu código aquí*

#### 27.13.4 Ejercicio 4. Un modelo por grupo

Divide `millas` por `clase` con `split()` y ajusta, para cada clase, una regresión de autopista contra `cilindrada`. Saca después el  $R^2$  de cada una.

**Verificación:** tienen que salir **7** modelos y 7 valores de  $R^2$ , todos entre 0 y 1.

**En palabras:** ¿en cuál clase explica mejor la cilindrada el consumo, y en cuál peor? Mira cuántos vehículos tiene cada clase antes de sacar conclusiones: ¿te fías igual del  $R^2$  de una clase con 5 vehículos que del de una con 62?

*# Escribe tu código aquí*

#### 27.13.5 Ejercicio 5. `across()`, la alternativa dentro de la tabla

Consigue el resultado del Ejercicio 1 otra vez, pero ahora con `across()` dentro de `summarise()`. Después hazlo **por clase**, con `group_by()`.

**Verificación:** la primera versión da una tabla de 1 fila y 5 columnas; la segunda, una de 7 filas.

**En palabras:** compara la salida de `map_dbl()` con la de `across()`. ¿Qué te da una que no te da la otra? ¿Cuál de las dos podrías haber usado para hacer la versión por clase, y cuál no sin complicarte mucho?

*# Escribe tu código aquí*

#### 27.13.6 Ejercicio 6. Cuando una vuelta falla

Escribe una función que calcule el promedio de `autopista` de un grupo pero que **dé error** si el grupo tiene menos de 10 filas. Aplícala a los grupos de `clase` de dos maneras: con `map()` a secas, en un bloque con `error=TRUE`, y con `safely()`.

**Verificación:** con `map()` a secas se detiene todo y no obtienes ningún resultado. Con `safely()` obtienes los resultados de las clases que sí tenían suficientes filas, y una lista con los nombres de las que fallaron.

**En palabras:** ¿cuál clase fue la culpable? ¿Por qué `safely()` es imprescindible cuando procesas cien archivos y no sabes si alguno está corrupto?



```
Escribe tu código aquí
```

### 27.13.7 Ejercicio 7 (BONO). `walk()` y `map2()`

Con `walk()`, guarda un archivo `.csv` por cada `clase` de `millas` dentro de una carpeta `Datos/por_clase/`, con el nombre de la clase en el nombre del archivo.

Después usa `map2()` para hacer lo mismo pasando dos listas en paralelo: la de tablas y la de nombres de archivo.

**Verificación:** tienen que quedar **7** archivos en la carpeta. Compruébalo con `list.files()`.

**En palabras:** ¿qué habría pasado si hubieras usado `map()` en vez de `walk()`? Pruébalo. ¿Por qué existe `walk()` si `map()` hace lo mismo?

```
Escribe tu código aquí
```



## Chapter 28

# Modelos: modelr

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/model-basics.html>

Español: <https://es.r4ds.hadley.nz/conceptos-b%C3%A1sicos-de-modelos.html>

---

Fecha de la última revisión

## [1] "2026-09-06"

### 28.1 Temas: Construir modelos con modelr

- Qué es un modelo y para qué sirve
- Visualizar la relación entre dos variables
- Ajustar un modelo lineal con `lm()`
- Cómo se lee un coeficiente
- Predicciones con `data_grid()` y `add_predictions()`
- Residuales con `add_residuals()`: lo que el modelo no explicó
- Predictoras categóricas y modelos con más de una variable
- Predecir no es explicar

Este capítulo usa el enfoque de `modelr` (el del libro *R for Data Science*). Para un flujo más moderno con `tidymodels`, y para dar formato matemático a los ejes con LaTeX, mira los capítulos de *tidyverse avanzado* y de *ecuaciones matemáticas*.

```
library(tidyverse)
library(modelr)
library(datos)
options(na.action = na.warn)
```

## 28.2 Qué es un modelo

Un **modelo** es un resumen: una descripción sencilla que captura el patrón general de los datos y deja fuera el detalle. Y como todo resumen, es deliberadamente **falso**. La utilidad no está en que sea verdad, está en que sea útil.

Construirlo tiene siempre dos pasos:

1. **Escoger una familia de modelos.** “Una línea recta”,  $y = a + b \cdot x$ , es una familia: contiene todas las rectas posibles.
2. **Ajustar el modelo**, es decir, encontrar dentro de esa familia el miembro concreto que mejor se acerca a los datos. Para una recta, eso significa encontrar los valores de **a** y de **b**.

Lo que hace `lm()` es el segundo paso. La familia la escoges tú, y esa decisión es tuya y no de la computadora.

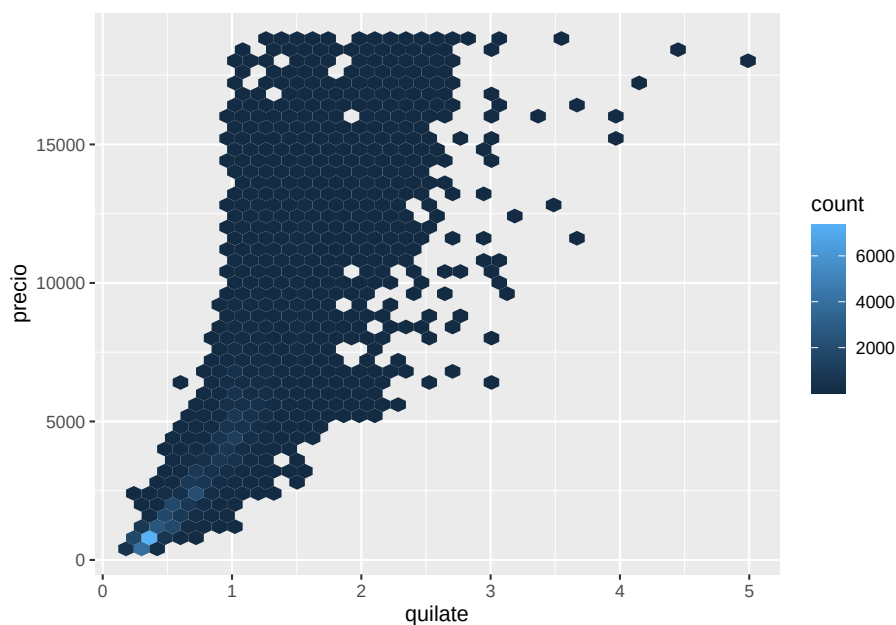
El objetivo de un modelo no es siempre predecir. Muchas veces es **quitar de en medio** un patrón que ya conoces, para poder ver el que hay debajo. Eso es lo que hace este capítulo con los diamantes, y es la respuesta a la pregunta que quedó abierta en el capítulo de análisis exploratorio.

---

## 28.3 Visualizar la relación

Usaremos los **diamantes**. A primera vista, los diamantes de peor calidad parecen costar más, porque el peso (**quilate**) influye fuertemente en el precio. Primero miramos la relación entre peso y precio:

```
ggplot(diamantes, aes(quilate, precio)) +
 geom_hex(bins = 40)
```

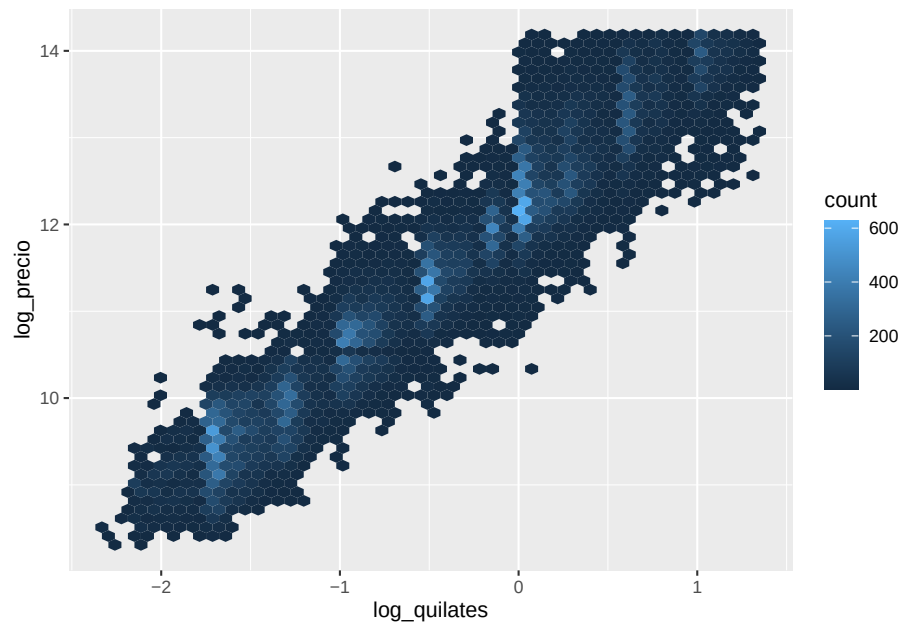


La relación se ve curva. Nos enfocamos en los diamantes de menos de 2.5 quilates (el 99.7% de los datos) y aplicamos una **transformación logarítmica**, que convierte la relación en una línea recta:

Cuando una relación se ve curva o los datos están muy sesgados, una transformación logarítmica a menudo la endereza y facilita ajustar un modelo lineal.

```
diamantes2 <- diamantes %>%
 filter(quilate <= 2.5) %>%
 mutate(log_precio = log2(precio),
 log_quilates = log2(quilate))

ggplot(diamantes2, aes(log_quilates, log_precio)) +
 geom_hex(bins = 50)
```



## 28.4 Ajustar un modelo lineal con `lm()`

`lm()`, `data_grid()`, `add_predictions()`, `add_residuals()` (`modelr`): modelar y explorar.

- `lm(y ~ x, data)`: ajusta un modelo lineal.
- `data_grid()`: crea una rejilla de valores de las predictoras.
- `add_predictions()` / `add_residuals()`: añaden columnas con las predicciones y los residuales del modelo, cómodas para graficar.

La función `lm()` ajusta un modelo lineal. La fórmula `y ~ x` se lee “y explicada por x”.

```
mod_diamantes <- lm(log_precio ~ log_quilates, data = diamantes2)
mod_diamantes
```

```
##
Call:
lm(formula = log_precio ~ log_quilates, data = diamantes2)
##
Coefficients:
(Intercept) log_quilates
12.194 1.681
```

### 28.4.1 Cómo se lee un coeficiente

Un modelo lineal devuelve dos números por cada predictora, y hay que saber leerlos:

- El **intercepto** es el valor predicho cuando la predictora vale cero. Muchas veces no significa nada por sí solo (un diamante de cero quilates), y no pasa nada: está ahí para que la recta quede a la altura correcta.
- La **pendiente** es lo que cambia la respuesta cuando la predictora sube **una unidad**. Y aquí está lo importante: la unidad depende de la escala en que estén las variables.

En este modelo las dos variables están en logaritmo de base 2, así que “una unidad más de `log_quilates`” significa **el doble de quilates**. Una pendiente cercana a 1.7 quiere decir entonces que **al duplicar el peso, el precio se multiplica por algo más de 3** (por 2 elevado a 1.7). Esa es la clase de lectura que hay que hacer siempre, y es la razón por la que conviene saber en qué escala está cada variable antes de interpretar nada.

**summary() de un modelo:** los números que se reportan.

- **coef(modelo):** solo los coeficientes, que es lo que casi siempre se quiere.
- **summary(modelo)\$r.squared:** la proporción de la variación de la respuesta que el modelo explica, entre 0 y 1.
- **Los valores p:** contestan “¿podría este coeficiente ser cero?”. Con muchos datos casi todo sale significativo, así que un valor p pequeño no quiere decir que el efecto sea grande ni importante.

Un  $R^2$  alto no significa que el modelo sea correcto, y uno bajo no significa que sea inútil. Un modelo puede tener un  $R^2$  de 0.9 y una gráfica de residuales con una curva evidente, lo que quiere decir que la familia escogida estaba equivocada. **Mira siempre los residuales antes de mirar el  $R^2$ .**

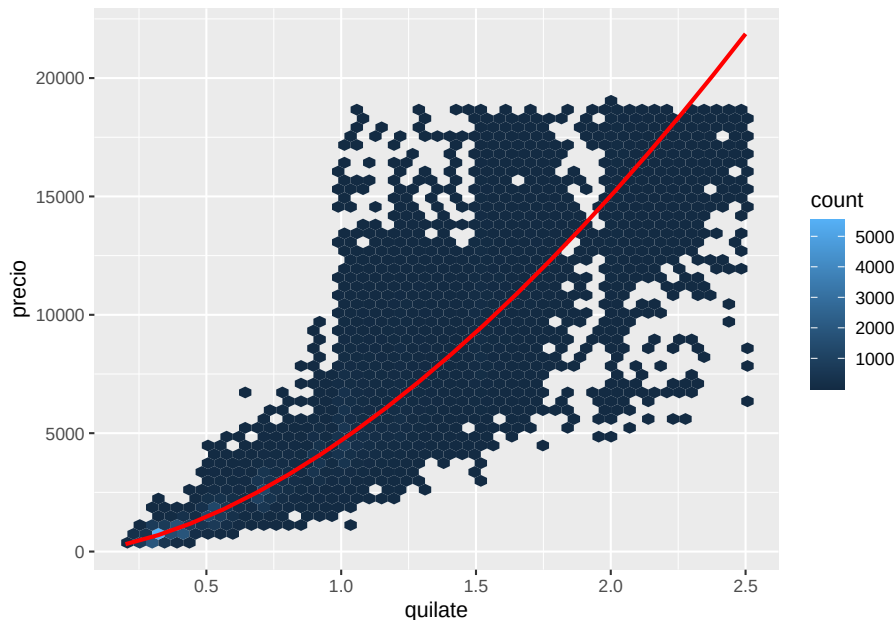
## 28.5 Predicciones con `data_grid()` y `add_predictions()`

Para ver lo que el modelo “aprendió”, generamos una cuadrícula de valores de quilate con `data_grid()`, y le añadimos la predicción del modelo con `add_predictions()`. Deshacemos la transformación logarítmica para volver a la escala original de precio.

```
cuadrícula <- diamantes2 %>%
 data_grid(quilate = seq_range(quilate, 20)) %>%
 mutate(log_quilates = log2(quilate)) %>%
 add_predictions(mod_diamantes, "log_precio") %>%
 mutate(precio = 2 ^ log_precio)

ggplot(diamantes2, aes(quilate, precio)) +
```

```
geom_hex(bins = 50) +
geom_line(data = cuadrícula, colour = "red", linewidth = 1)
```



La línea roja es la predicción del modelo.

La cuadrícula se construyó con `seq_range(quilate, 20)`, que solo cubre el rango de los datos **observados**. Eso es a propósito. Un modelo no sabe nada de lo que pasa fuera del rango en que se ajustó, y extrapolar es la manera más rápida de decir una tontería con aspecto de resultado: este modelo predecirá alegremente el precio de un diamante de 40 quilates, y ese número no tiene ningún valor.

## 28.6 Residuales con `add_residuals()`

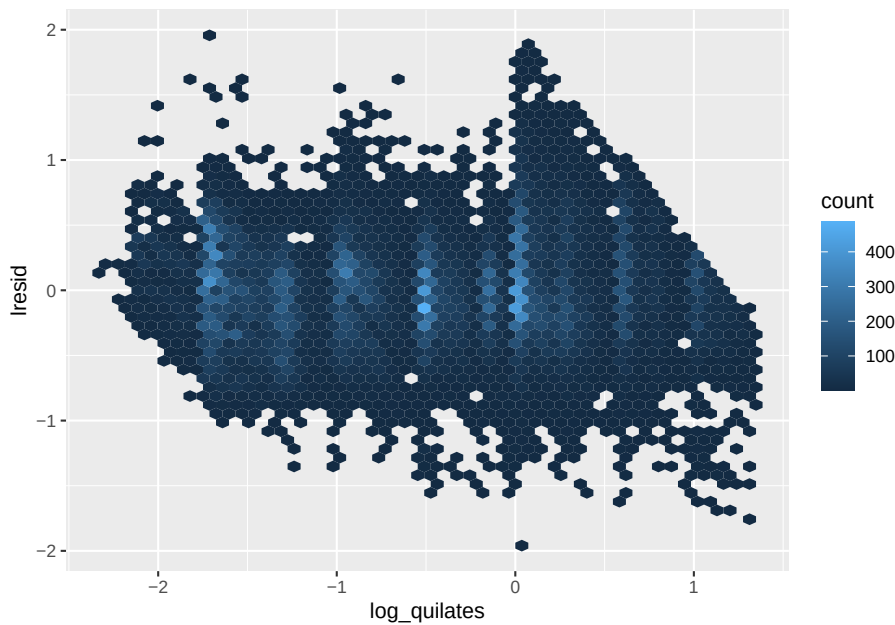
Los **residuales** son lo que el modelo *no* explica: la diferencia entre el valor observado y el predicho. Si el modelo es bueno, al graficar los residuales no debe quedar ningún patrón evidente.

Si en la gráfica de residuales todavía ves una tendencia o una forma clara, el modelo se está perdiendo algo: considera transformar las variables o añadir más predictores.

```
diamantes2 <- diamantes2 %>%
 add_residuals(mod_diamantes, "lresid")
```



```
ggplot(diamantes2, aes(log_quilates, lresid)) +
 geom_hex(bins = 50)
```

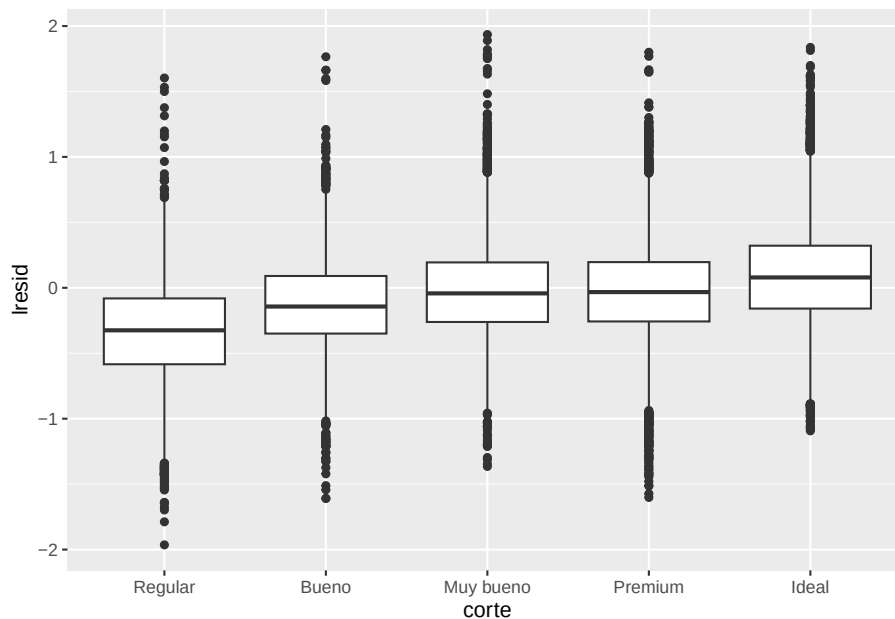


### 28.6.1 La pregunta que quedó abierta

Ahora sí se puede cerrar el asunto que dejamos pendiente en el capítulo de análisis exploratorio. Allí salió, y sorprendió a todo el mundo, que los diamantes de **mejor corte parecían los más baratos**. La explicación era que el corte *Ideal* se aplica sobre todo a piedras pequeñas, así que el quilate estaba escondido detrás.

Los residuales quitan de en medio exactamente eso: son el precio **ya limpio del efecto del peso**. Si ahora miramos el corte contra los residuales, en vez del corte contra el precio, la relación se endereza:

```
ggplot(diamantes2, aes(corte, lresid)) +
 geom_boxplot()
```



Un residual positivo quiere decir “más caro de lo que le tocaría por su peso”. Y ahí el corte *Ideal* sale por encima, que es lo que la intuición decía desde el principio.

Esta es la respuesta a la pregunta de para qué sirve un modelo cuando no quieres predecir nada: **para quitar de en medio un patrón conocido y poder ver el que estaba debajo**. Es la herramienta que convierte una paradoja en un resultado.

## 28.7 Predictoras categóricas y varias variables

La fórmula de `lm()` admite mucho más que una variable numérica:

- $y \sim x$  una predictora numérica: una recta.
- $y \sim \text{categoria}$  una predictora categórica: R crea variables indicadoras por detrás, y los coeficientes son **diferencias respecto al primer nivel**, que queda absorbido en el intercepto. Con siete clases salen siete coeficientes: el intercepto más seis diferencias.
- $y \sim x1 + x2$  dos predictoras a la vez: el efecto de cada una **manteniendo la otra constante**, que casi nunca es lo mismo que su efecto por separado.
- $y \sim x1 * x2$  además de las dos, su **interacción**: permite que el efecto de una dependa del valor de la otra.

El orden de los niveles de un factor decide cuál queda como referencia en el

intercepto, y por lo tanto contra cuál se comparan todos los demás coeficientes. Si quieres que la referencia sea una categoría concreta (el control, por ejemplo), muévela al principio con `fct_relevel()` del capítulo de factores.

---

## 28.8 Predecir no es explicar

Conviene terminar con una distinción que se olvida a menudo. Un modelo puede usarse para dos cosas muy distintas:

- **Predecir:** dado un diamante nuevo, ¿cuánto costará? Aquí lo único que importa es acertar, y da igual por qué.
- **Explicar:** ¿el corte influye en el precio, y cuánto? Aquí importan los coeficientes, su signo y su tamaño, y hace falta mucho más cuidado.

Que una variable prediga bien **no** quiere decir que sea la causa. El número de cigüeñas predice bien el número de nacimientos en algunos países, y no por eso las cigüeñas traen bebés: hay una tercera variable detrás, igual que el quilate estaba detrás del corte.

---

## 28.9 Ejercicios del libro R4DS

Hacer los ejercicios de la sección 23.2 del libro en español.

---

## 28.10 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que la gráfica o la tabla se vean) y una **explicación en palabras**. Aquí la explicación pesa más que en ningún otro capítulo: un modelo sin interpretar no es un resultado, es una lista de números. Los ejercicios usan `millas` y `países` del paquete `datos`, no `diamantes` ni `vuelos`, que son los del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_28_modelos.Rmd`.

### 28.10.1 Ejercicio 1. El modelo más sencillo

Datos: `millas`. Ajusta `lm(autopista ~ cilindrada)` y mira los coeficientes. Grafica los puntos con la recta encima.

**Verificación:** el coeficiente de `cilindrada` tiene que salir **negativo**, y el intercepto por encima de 30.

**En palabras:** traduce los dos coeficientes a una oración que entienda alguien que no sabe estadística. Empieza por “por cada litro más de cilindrada...”. ¿Qué significa el intercepto aquí, y tiene sentido físico?

*# Escribe tu código aquí*

### 28.10.2 Ejercicio 2. Los residuales delatan

Añade los residuales del modelo anterior con `add_residuals()` y gráfilalos contra `cilindrada`.

**Verificación:** la nube de residuales **no** está repartida al azar: se ve una curva.

**En palabras:** describe la forma que ves. ¿Qué te está diciendo sobre la familia de modelos que escogiste? Escribe qué harías a continuación.

*# Escribe tu código aquí*

### 28.10.3 Ejercicio 3. Enderezar con logaritmos

Datos: `países`, solo el año 2007. Grafica `esperanza_de_vida` contra `pib_per_capita` y ajusta un modelo lineal. Después repite las dos cosas usando el **logaritmo** del PIB.

**Verificación:** el  $R^2$  del segundo modelo tiene que ser claramente mayor que el del primero, y la nube del segundo se ve mucho más recta.

**En palabras:** con el modelo en logaritmo, ¿qué significa la pendiente? Escríbelo en la forma “cada vez que el PIB per cápita se **duplica**, la esperanza de vida sube en ...”. Usa `log2()` para que la interpretación sea directa.

*# Escribe tu código aquí*

### 28.10.4 Ejercicio 4. Una predictora categórica

Con `millas`, ajusta `lm(autopista ~ clase)` y mira los coeficientes.

**Verificación:** salen **7** coeficientes: el intercepto más **6** diferencias. Una de las siete clases no aparece por su nombre.

**En palabras:** ¿cuál clase falta y dónde se metió? Comprueba que el intercepto es igual al promedio de `autopista` de esa clase. Después usa `fct_relevel()` para que la referencia sea `pickup` y explica qué cambió en los coeficientes y qué no.

*# Escribe tu código aquí*

### 28.10.5 Ejercicio 5. Dos predictor

Ajusta ahora `lm(autopista ~ cilindrada + clase)` y compara sus residuales con los del Ejercicio 2.

**Verificación:** el  $R^2$  sube y los residuales quedan más planos que antes.

**En palabras:** el coeficiente de `cilindrada` cambió respecto al Ejercicio 1. Explica por qué. ¿Qué contesta exactamente ese coeficiente nuevo, y en qué se diferencia de lo que contestaba el viejo?

*# Escribe tu código aquí*

### 28.10.6 Ejercicio 6. Predecir, y hasta dónde

Con el modelo del Ejercicio 1, construye una cuadrícula con `data_grid()` y `seq_range()` y añade las predicciones con `add_predictions()`. Grafícalas sobre los datos.

Después construye a mano una segunda cuadrícula que llegue hasta **15 litros** de cilindrada y predice también ahí.

**Verificación:** la primera cuadrícula cubre el rango real de `cilindrada`; la segunda predice consumos claramente absurdos.

**En palabras:** ¿qué consumo predice el modelo para un motor de 15 litros? Explica por qué el modelo no se queja, y qué precaución hay que tomar siempre antes de usar una predicción.

*# Escribe tu código aquí*

### 28.10.7 Ejercicio 7 (BONO). Cerrar el círculo

En el capítulo de análisis exploratorio quedó una anomalía: los `2seater` rendían mucho más de lo que les tocaba por su cilindrada. Compruébalo ahora con residuales: usa el modelo `autopista ~ cilindrada` y haz un diagrama de caja de los residuales por `clase`.

**Verificación:** la caja de `2seater` queda claramente por encima de cero.

**En palabras:** ¿qué significa un residual positivo aquí? Este es el mismo razonamiento que resolvió el asunto del corte `Ideal` en `diamantes`. Explica el paralelo entre los dos casos, y di cuál es la variable escondida en cada uno.

*# Escribe tu código aquí*



## Chapter 29

# Hex Stickers

```
[1] "2026-09-06"
library(hexSticker)
library(magick)
library(sysfonts)
library(tidyverse)
```

### 29.1 Temas

- Qué es un hex sticker y por qué son hexagonales
  - `sticker()` y sus tres grupos de argumentos
  - El sistema de coordenadas: dónde va cada cosa
  - Fuentes: `sysfonts` y `showtext`
  - Guardar: png, svg y resolución
  - Consejos de diseño para que se lea a dos centímetros
- 

### 29.2 Qué es un hex sticker

Un **hex sticker** es una calcomanía hexagonal que identifica un paquete, un proyecto o un laboratorio. La forma no es un capricho: los hexágonos **teselan**, es decir, encajan unos con otros sin dejar huecos, así que se pueden pegar en una laptop formando un panal. Los círculos y los cuadrados no dan ese resultado.

La comunidad de R acordó un **estándar de tamaño y proporciones** para que las calcomanías de distintas personas encajen entre sí. Antes de mandar a imprimir conviene consultarlo, porque un hexágono con proporciones propias no encaja con los demás y se nota.

El estándar y las medidas exactas están documentados en el repositorio `terinjokes/StickersStandard` de GitHub, y hay una galería de calcomanías de paquetes en <http://hexb.in>. Comprueba ahí las medidas antes de mandar a imprimir; `hexSticker` genera las proporciones correctas por defecto, pero la imprenta necesita el tamaño físico y el margen de corte.

Este capítulo es distinto a los demás del libro: no hay datos que analizar, hay un objeto que diseñar. Aun así encaja aquí por dos razones. La primera es que el `subplot` de una calcomanía puede ser un `ggplot`, así que todo lo aprendido sobre gráficas se aplica. La segunda es que hacer una calcomanía obliga a pensar en algo que casi nunca se piensa: **qué se ve y qué no cuando la figura mide dos centímetros**.

## 29.3 La función y los valores “default”

Los **hex stickers** (calcomanías hexagonales) se popularizaron en la comunidad de R alrededor de 2015, impulsados por desarrolladores de **RStudio/Posit** y el proyecto **rOpenSci**. El formato hexagonal permite unir las calcomanías sin dejar huecos (como un panal), por lo que se convirtió en la forma estándar de identificar paquetes de R en conferencias y computadoras portátiles. El paquete `hexSticker`, de **Guangchuang Yu**, automatiza su creación.

`sticker()`: crea una calcomanía hexagonal a partir de una imagen o gráfico.

- **subplot**: la imagen o gráfico (`ggplot`) que va en el centro; `s_x`, `s_y`, `s_width`, `s_height` controlan su posición y tamaño.
- **package**: el texto (normalmente el nombre del paquete); `p_size`, `p_color`, `p_family` ajustan su tipografía.
- **h\_fill**, **h\_color**, **h\_size**: color de relleno, color y grosor del borde del hexágono.
- **filename**: dónde se guarda; la extensión (`.png`, `.svg`) decide el formato de salida.

La lista de argumentos asusta al principio y en realidad son **tres grupos**, cada uno con su prefijo:

- **s\_** es el *subplot*, la imagen o gráfica del centro.
- **p\_** es el *package*, el texto principal.
- **u\_** es la *url*, el texto pequeño del borde de abajo.
- **h\_** es el *hexágono* en sí: relleno, borde y grosor.
- **l\_** es el *spotlight*, un brillo opcional en diagonal.

En cuanto reconoces los prefijos, la lista deja de dar miedo: son cinco cosas con sus propias posiciones, tamaños y colores.



### 29.3.1 El sistema de coordenadas

Este es el detalle que más tiempo ahorra y que no está en ninguna parte de la documentación de forma clara: **el hexágono ocupa un cuadrado que va de 0 a 2 en los dos ejes, con el centro en (1, 1).**

Por eso:

- `s_x = 1`, `s_y = 1` deja la imagen justo en el centro;
- `p_y = 1.5` sube el texto hacia la parte de arriba;
- `u_y = 0.08` deja la url pegada al borde inferior;
- `s_width` y `s_height` son fracciones, así que 0.5 es media anchura.

No intentes calcular las posiciones: **prueba y mira**. Cambia un número, vuelve a generar la calcomanía, y repite. Es más rápido que razonarlo, y en cinco intentos ya le agarras el truco a la escala.

```
sticker(
 subplot, # position and size of subplot
 s_x = 0.8,
 s_y = 0.75,
 s_width = 0.4,
 s_height = 0.5,
 package, # package name and characteristics
 p_x = 1,
 p_y = 1.4,
 p_color = "#FFFFFF",
 p_family = "Aller_Rg",
 p_fontface = "plain",
 p_size = 8,
 h_size = 1.2, # hexagon border size and colors
 h_fill = "#1881C2",
 h_color = "#87B13F",
 spotlight = FALSE,
 l_x = 1, # spotlight x position
 l_y = 0.5,
 l_width = 3,
 l_height = 3,
 l_alpha = 0.4,
 url = "", # url at lower border
 u_x = 1,
 u_y = 0.08,
 u_color = "black",
 u_family = "Aller_Rg",
 u_size = 1.5,
 u_angle = 30,
 white_around_sticker = FALSE, # if TRUE, puts white triangles in the corners
 ...,
```

```
filename = paste0(package, ".png"), # filename to save sticker
asp = 1, # aspect ratio, only works if subplot is an image file
dpi = 300 # plot resolution
)
```

---

### Arguments

```
library(gt)
gt(argumentos)
```

### 29.3.2 Las fuentes

Aquí está el tropiezo clásico del capítulo: la fuente que escoges en `p_family` tiene que estar **disponible para R**, y las fuentes de tu computadora no lo están automáticamente. El proceso son siempre tres pasos:

1. `sysfonts::font_add_google("Nombre de la fuente")` baja la fuente de Google Fonts.
2. `showtext::showtext_auto()` le dice a R que use `showtext` para dibujar texto.
3. `showtext_opts(dpi = 300)` hace que el tamaño del texto cuadre con la resolución de salida.

Sin el paso 2, la fuente se baja y no se usa, y el texto sale en la tipografía por defecto sin que nada avise.

```
sysfonts::font_add_google("Rubik")
sysfonts::font_add_google("Sixtyfour Convergence")
sysfonts::font_add_google("Young Serif") # esta no siempre baja
```

No todas las fuentes de Google Fonts bajan bien, y las que fallan lo hacen en silencio o con un error poco claro. Si el texto no sale como esperabas, comprueba primero que la fuente esté cargada, con `sysfonts::font_families()`. Y ten un plan B: una fuente muy elaborada se ve preciosa en la pantalla y se vuelve ilegible impresa a dos centímetros.

### 29.3.3 Guardar la calcomanía

La **extensión** del nombre del archivo decide el formato: `"sticker.png"` produce un png y `"sticker.svg"` produce un svg. `hexSticker` usa por dentro `ggplot2::ggsave()`, así que sirve cualquier formato que esa función acepte.

**Formatos de salida:** cuál usar según para qué.

- **.png con dpi = 300 o más:** para imprimir. Es un mapa de puntos, así que el tamaño físico y la resolución hay que decidirlos al generarlo.

- **.svg**: vectorial, se puede agrandar sin perder calidad y la imprenta suele preferirlo. La pega es que el `subplot`, si era una foto, sigue siendo una foto por dentro.
- **save\_sticker("archivo.png")**: guarda la última calcomanía dibujada, sin tener que repetir toda la llamada a `sticker()`.

### 29.3.4 Consejos de diseño

Una calcomanía se ve **impresa y pequeña**, no en la pantalla y grande, y eso cambia todas las decisiones:

- **Contraste antes que color bonito**. Texto claro sobre fondo oscuro o al revés. Dos colores de intensidad parecida se ven bien en el monitor y se convierten en una mancha al imprimirse.
- **Una sola idea en el centro**. Una figura con detalle fino desaparece. Si el `subplot` es un `ggplot`, quítale los ejes, la leyenda y la rejilla con `theme_void()`.
- **El nombre corto y grande**. Si no se lee de un vistazo, no sirve.
- **La url es opcional y casi siempre estorba**. Solo ponla si de verdad quieres que alguien la teclee.

Prueba de fuego: genera la calcomanía, ábrela, y **aléjala hasta que ocupe el tamaño de una moneda**. Si a esa distancia no se distingue qué es y qué dice, hay que simplificarla.

---

## 29.4 Ejemplo de Crear un Hex con un gatito

El ejemplo siguiente junta todo lo anterior: carga una fuente de Google, activa `showtext`, lee una imagen con `magick::image_read()` y llama a `sticker()` con las posiciones y los colores puestos a mano. Fíjate en el `spotlight = T` del final, que añade el brillo diagonal.

Para usar una fuente de Google, cárgala con `font_add_google()` y activa `showtext_auto()` antes de llamar a `sticker()`. Si no, el texto no se dibujará con la fuente elegida.

```
#font_files() #a list of all fonts available

Loading Google fonts (https://fonts.google.com/)

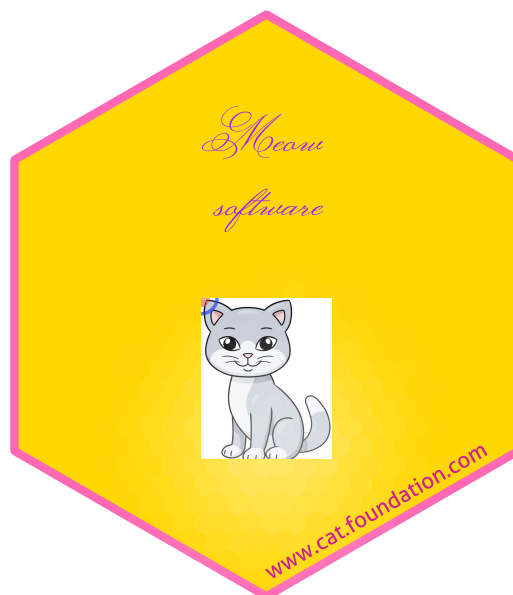
#sysfonts::font_add_google("Dancing Script") # selección de fonts de google
#sysfonts::font_add_google("Bebas Neue")
#sysfonts::font_add_google("Fuggles")
#sysfonts::font_add_google("Roboto")
#sysfonts::font_add_google("Montserrat")
#sysfonts::font_add_google("Lobster Two")
```

```
sysfonts::font_add_google("Monsieur La Doulaise")
#sysfonts::font_add_google("monsieur")

cat_img=image_read('img/cat.jpg')
analitica_img=image_read('img/Analitica3.png')

library(showtext)
font_add_google("Monsieur La Doulaise")
showtext_opts(dpi = 300)
showtext_auto()

sticker(
 subplot= cat_img,
 package="Meow \n software",
 filename = "img/hex_meow_software.png",
 s_width =.55,
 s_height = .55,
 s_x=1,
 s_y=.75,
 p_size=8, # tamaño del texto
 p_x = 1.0,
 p_y = 1.5,
 p_color= 'purple',
 p_family= "Monsieur La Doulaise",
 h_fill = 'gold',
 h_color='hotpink',
 h_size = 2,
 url="www.cat.foundation.com",
 u_size=4,
 u_color = "violetred",
 spotlight=T
) %>% print
```



```
save_sticker("img/hex_sticker_cat.jpg")
```

## 29.5 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Este capítulo no produce números sino un objeto de diseño, así que la entrega es distinta: cada ejercicio necesita **el código**, **la imagen resultante** (insértala en el documento con `knitr::include_graphics()`) y **una explicación en palabras** de las decisiones que tomaste. Las decisiones son lo que se evalúa: por qué ese color, ese tamaño, esa fuente. Guarda todas tus calcomanías en una carpeta `img/ejercicios/`. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_29_hex.Rmd`.

### 29.5.1 Ejercicio 1. Una calcomanía a partir de una gráfica

Haz un `ggplot` sencillo con millas (por ejemplo cilindrada contra autopista), quítale todo lo que sobra con `theme_void()`, y úsalo como `subplot` de una calcomanía llamada `millas`.

**Verificación:** el archivo `.png` tiene que existir y verse. Compruébalo con `file.exists()`.

**En palabras:** compara tu calcomanía con la misma gráfica **sin** `theme_void()`. ¿Qué desapareció y por qué mejora a este tamaño?

```
Escribe tu código aquí
```

### 29.5.2 Ejercicio 2. El sistema de coordenadas

Partiendo de la calcomanía del ejercicio anterior, genera **cuatro** versiones cambiando solo las posiciones: la gráfica arriba y el texto abajo, las dos centradas y superpuestas, la gráfica muy pequeña en una esquina, y la gráfica ocupando casi todo el hexágono.

**Verificación:** las cuatro tienen que salir distintas y todas legibles dentro del hexágono, sin nada cortado por el borde.

**En palabras:** anota qué valores de `s_x`, `s_y`, `p_x` y `p_y` usaste en cada una. ¿Cuál es el centro exacto del hexágono, y cómo lo dedujiste?

```
Escribe tu código aquí
```

### 29.5.3 Ejercicio 3. Fuentes

Baja **dos** fuentes distintas de Google Fonts con `font_add_google()`, una de palo seco y otra decorativa, y haz la misma calcomanía con cada una.

**Verificación:** las dos versiones tienen que verse con tipografías claramente distintas. Si salen iguales, se te olvidó `showtext_auto()`.

**En palabras:** reduce las dos imágenes al tamaño de una moneda y míralas. ¿Cuál sigue siendo legible? ¿Cambia tu opinión sobre cuál es más bonita?

```
Escribe tu código aquí
```

### 29.5.4 Ejercicio 4. Contraste

Haz dos calcomanías con exactamente el mismo contenido: una con **buen** contraste entre el texto y el fondo, y otra con dos colores de intensidad parecida (por ejemplo verde medio sobre azul medio).

**Verificación:** las dos tienen que existir como archivos. Insértalas una al lado de la otra en el documento.

**En palabras:** describe qué le pasa a la mala cuando la miras de lejos. Busca qué son la **luminosidad** y el **contraste** de un color y explica en dos oraciones por qué dos colores distintos pueden verse igual de oscuros.

```
Escribe tu código aquí
```

### 29.5.5 Ejercicio 5. Los adornos

Añádele a tu calcomanía la url de un sitio (real o inventado) con `url` y sus argumentos `u_`, y prueba también `spotlight = TRUE` con distintos valores de

`l_alpha`.

**Verificación:** la url tiene que quedar pegada al borde de abajo, sin salirse del hexágono y sin tocar la gráfica.

**En palabras:** ¿aporta algo la url a esta calcomanía en concreto? ¿Y el spotlight? Defiende quitarlos o dejarlos.

*# Escribe tu código aquí*

### 29.5.6 Ejercicio 6. Formatos

Guarda la misma calcomanía en `.png` con `dpi = 300` y en `.svg`. Compara el tamaño de los dos archivos con `file.size()` y ábrelos los dos, agrandándolos mucho.

**Verificación:** al agrandar mucho, uno de los dos se ve pixelado y el otro no.

**En palabras:** ¿cuál se pixela y por qué? ¿Cuál mandarías a una imprenta? Y una pregunta más fina: si el `subplot` fuera una **foto** en vez de un `ggplot`, ¿el `.svg` seguiría siendo perfectamente escalable? Justifícalo.

*# Escribe tu código aquí*

### 29.5.7 Ejercicio 7 (BONO). La tuya

Diseña la calcomanía de **tu** proyecto: tu tesis, tu laboratorio, tu curso o un paquete que te gustaría escribir. Aplica todo lo del capítulo.

**Verificación:** tiene que pasar la prueba de la moneda: se entiende qué es y qué dice al tamaño de una moneda.

**En palabras:** escribe un párrafo justificando cada decisión: la imagen, el color, la fuente y el tamaño del texto. Pídele a un compañero que la mire dos segundos y te diga qué entendió, y anota lo que te dijo.

*# Escribe tu código aquí*

Función	Descripción
subplot	subplot
s_x	x position for subplot
s_y	y position for subplot
s_width	width for subplot
s_height	height for subplot
package	package name
p_x	x position for package name
p_y	y position for package name
p_color	color for package name
p_family	font family for package name
p_fontface	fontface for package name
p_size	font size for package name
h_size	size for hexagon border
h_fill	color to fill hexagon
h_color	color for hexagon border
spotlight	whether add spotlight
l_x	x position for spotlight
l_y	y position for spotlight
l_width	width for spotlight
l_height	height for spotlight
l_alpha	maximum alpha for spotlight
url	url at lower border
u_x	x position for url
u_y	y position for url
u_color	color for url
u_family	font family for url
u_size	text size for url
u_angle	angle for url
white_around_sticker	default to FALSE. If set to TRUE, it puts white triangles in the
...	additional parameter to geom_pkgname
filename	filename to save sticker
asp	aspect ratio, only works if subplot is an image file
dpi	plot resolution



## Chapter 30

# Nubes de palabra en R

## [1] "2026-09-06"

### 30.1 Temas

- Los cinco pasos de una nube de palabras
  - El corpus y el paquete `tm`
  - Limpieza: minúsculas, números, puntuación, espacios
  - Las palabras vacías (*stopwords*), y las tuyas propias
  - La matriz de términos y el conteo de frecuencias
  - `wordcloud()` y `wordcloud2()`
  - Cuándo una nube de palabras **no** es la gráfica correcta
- 

### 30.2 Los cinco pasos

Una nube de palabras parece un truco de una línea y en realidad es un proceso de cinco pasos, siempre los mismos. Vale la pena tenerlos en la cabeza, porque cuando una nube sale mal, casi siempre es que falló uno de los tres del medio:

1. **Leer el texto** desde un archivo o desde la web.
2. **Convertirlo en corpus**, que es la estructura con la que trabaja el paquete `tm`.
3. **Limpiarlo**: pasar todo a minúsculas, quitar números, puntuación, espacios sobrantes y las palabras vacías.
4. **Contar**: construir una matriz de términos y sacar la frecuencia de cada palabra.
5. **Dibujar** la nube.

El paso 5 es una línea. El paso 3 es donde está el trabajo de verdad, y donde se decide si la nube dice algo o no dice nada.

Sin el paso 3, la nube enseña **de**, **la**, **que** y **en** en letras gigantes, que es información cero. Ese es el resultado por defecto de cualquier texto en cualquier idioma, y es la razón por la que las **palabras vacías** son el concepto central del capítulo.

### 30.3 Ejemplo #1

Para generar nubes de palabras con R. Se necesita los paquetes **wordcloud**, **RColorBrewer** y **wordcloud2**

La **minería de texto** (*text mining*), analizar palabras con la computadora para descubrir patrones, tiene raíces en la lingüística computacional de los años 1950 y 60. Las **nubes de palabras** se popularizaron en la web alrededor de 2004 con sitios como *Flickr* y *Wordle*, mostrando el tamaño de cada palabra según su frecuencia. En R, el paquete **wordcloud** de **Ian Fellows** (2011) y el paquete **tm** (*text mining*) de **Ingo Feinerer** llevaron estas ideas al análisis de datos reproducible.

**wordcloud()** y **wordcloud2()**: dibujan una nube donde el tamaño de cada palabra es proporcional a su frecuencia.

- **words**, **freq**: el vector de palabras y su vector de frecuencias (normalmente calculados con el paquete **tm**).
- **min.freq**: frecuencia mínima para que una palabra aparezca; **max.words**: número máximo de palabras a mostrar.
- **colors**: la paleta de colores, a menudo de **brewer.pal()** del paquete **RColorBrewer**.
- **wordcloud2()** es una versión interactiva (HTML) que produce nubes más vistosas; recibe un **data.frame** con columnas de palabra y frecuencia.

Ese ejemplo es una copia de la siguiente página de web

<http://www.sthda.com/english/wiki/word-cloud-generator-in-r-one-killer-function-to-do-everything-you-need>

```
if (!require("pacman")) install.packages("pacman")
pacman::p_load(tm, SnowballC, wordcloud, RColorBrewer, wordcloud2)

library(wordcloud) # Un paquete para hacer word cloud
library(wordcloud2) # paquete más sencillo para hacer word cloud2
library(RColorBrewer) # paquete para cambiar los colores
library(tm) # paquete de text mining
library(SnowballC) # paquete para trabajar en otro idioma aparte de inglés
```

## 30.4 Usando los datos en el paquete wordcloud2 que se llama demoFreq

[https://raymondltremblay.github.io/Ciencia\\_Datos\\_R/appendix-a.html](https://raymondltremblay.github.io/Ciencia_Datos_R/appendix-a.html)

```
#ejercicio-de-transformaci%C3%B3n
library(wordcloud2)
head(demoFreq, n=10)
wordcloud2(data = demoFreq)
```

## 30.5 Paso 1

Importar los datos de la web

Este bloque **descarga un texto desde internet**, así que necesita conexión. Si el servidor no responde verás `cannot open the connection` o `cannot open URL`: no es un error de tu código, sino de la red o del servidor. Por eso lo envolvemos en `tryCatch()`, de modo que si la descarga falla se usa una copia local y el análisis continúa. Encontrarás más sobre estos errores en el capítulo *Errores comunes* (14).

```
filePath <- "http://www.sthda.com/sthda/RDoc/example-files/martin-luther-king-i-have-a-dream-speech.txt"

Intenta leer el discurso desde la web; si no hay conexión o el servidor no
responde, usa un texto local de ejemplo para que el análisis no se detenga.
text <- tryCatch(
 suppressWarnings(readLines(filePath)),
 error = function(e) readLines("Datos/alice_original.txt")
)
head(text)
```

```
[1] ""
[2] "And so even though we face the difficulties of today and tomorrow, I still have a dream."
[3] " "
[4] "I have a dream that one day this nation will rise up and live out the true meaning of its promise."
[5] " "
[6] "We hold these truths to be self-evident, that all men are created equal."
```

Importar un texto de su computadora en formato `.txt` No va a funcionar el formato `.doc` de MSWord.

Guarda tu documento como texto plano (`.txt`) antes de leerlo con `readLines()`. Un archivo `.doc` o `.docx` guarda el formato en binario y obtendrás caracteres ilegibles en lugar del texto.

```
#mi_texto <- readLines(file.choose())
```

## 30.6 Subir el texto en formato Corpus

```
mi_texto=iconv(text,"WINDOWS-1252","UTF-8") # Use this for removing accents and non -

Load the data as a corpus
docs <- Corpus(VectorSource(mi_texto))

head(docs)

<<SimpleCorpus>>
Metadata: corpus specific: 1, document level (indexed): 0
Content: documents: 6
```

## 30.7 Mirar el documento, para evaluar su contenido

```
#inspect(docs) # quita el hashtag para ver el documento
```

## 30.8 Transformar el texto para reemplazar algunos caracteres especiales, y remplazarlos por espacio en blanco

```
toSpace <- content_transformer(function (x , pattern) gsub(pattern, " ", x))
docs <- tm_map(docs, toSpace, "-")
docs <- tm_map(docs, toSpace, "@")
docs <- tm_map(docs, toSpace, "\\|")
```

## 30.9 el paquete tm es para text mining

El bloque siguiente es el paso 3 completo, y conviene leer línea por línea qué quita cada `tm_map()`.

`tm_map()` (tm): aplicar una transformación a todo el corpus.

- **content\_transformer(tolower):** todo a minúsculas, para que Casa y casa cuenten como la misma palabra.
- **removeNumbers:** quita los dígitos.
- **removeWords, stopwords("english"):** quita las palabras vacías del idioma que le digas.
- **removePunctuation:** quita los signos. Ojo: pega las palabras que estaban unidas por guion.
- **stripWhitespace:** deja un solo espacio donde había varios.

- **stemDocument**: reduce cada palabra a su raíz (*stemming*), de modo que *corría*, *corriendo* y *correr* cuenten juntas. Está comentado en el bloque, y a menudo conviene dejarlo así: en español produce raíces feas y difíciles de leer en una nube.

**El orden importa.** Quita las palabras vacías **antes** de quitar la puntuación, no después. Si quitas primero la puntuación, *don't* se convierte en *dont*, que ya no coincide con la palabra vacía *don't* de la lista, y se queda dentro. Lo mismo pasa en español con las palabras acentuadas si el texto trae los acentos en un formato raro.

```
Convert the text to lower case
docs <- tm_map(docs, content_transformer(tolower))

Remove numbers
docs <- tm_map(docs, removeNumbers)

Remove english common stopwords
stopwords("english") # list of common english stopwords that are often removed
```

```
[1] "i" "me" "my" "myself" "we"
[6] "our" "ours" "ourselves" "you" "your"
[11] "yours" "yourself" "yourselves" "he" "him"
[16] "his" "himself" "she" "her" "hers"
[21] "herself" "it" "its" "itself" "they"
[26] "them" "their" "theirs" "themselves" "what"
[31] "which" "who" "whom" "this" "that"
[36] "these" "those" "am" "is" "are"
[41] "was" "were" "be" "been" "being"
[46] "have" "has" "had" "having" "do"
[51] "does" "did" "doing" "would" "should"
[56] "could" "ought" "i'm" "you're" "he's"
[61] "she's" "it's" "we're" "they're" "i've"
[66] "you've" "we've" "they've" "i'd" "you'd"
[71] "he'd" "she'd" "we'd" "they'd" "i'll"
[76] "you'll" "he'll" "she'll" "we'll" "they'll"
[81] "isn't" "aren't" "wasn't" "weren't" "hasn't"
[86] "haven't" "hadn't" "doesn't" "don't" "didn't"
[91] "won't" "wouldn't" "shan't" "shouldn't" "can't"
[96] "cannot" "couldn't" "mustn't" "let's" "that's"
[101] "who's" "what's" "here's" "there's" "when's"
[106] "where's" "why's" "how's" "a" "an"
[111] "the" "and" "but" "if" "or"
[116] "because" "as" "until" "while" "of"
[121] "at" "by" "for" "with" "about"
[126] "against" "between" "into" "through" "during"
[131] "before" "after" "above" "below" "to"
```

```
[136] "from" "up" "down" "in" "out"
[141] "on" "off" "over" "under" "again"
[146] "further" "then" "once" "here" "there"
[151] "when" "where" "why" "how" "all"
[156] "any" "both" "each" "few" "more"
[161] "most" "other" "some" "such" "no"
[166] "nor" "not" "only" "own" "same"
[171] "so" "than" "too" "very"
```

```
docs <- tm_map(docs, removeWords, stopwords("english"))

Remove your own stop word
specify your stopwords as a character vector
docs <- tm_map(docs, removeWords, c("blabla1", "blabla2"))

Remove punctuations
docs <- tm_map(docs, removePunctuation)

Eliminate extra white spaces
docs <- tm_map(docs, stripWhitespace)
Text stemming
docs <- tm_map(docs, stemDocument)

#stopwords() # Here are all the stopwords in the function **stopwords**
```

### 30.10 Crear una matriz de las palabras de mi documento

```
dtm <- TermDocumentMatrix(docs) # convertir el texto en una lista de palabras
m <- as.matrix(dtm) # convertir en una matriz
v <- sort(rowSums(m),decreasing=TRUE) # ordenar las palabras por frecuencia
d <- data.frame(word = names(v),freq=v) # crea un nuevo data frame de las palabras y s
head(d, n=10) # las primeras 10 palabras más comunes
```

```
word freq
will will 17
freedom freedom 13
ring ring 12
dream dream 11
day day 11
let let 11
every every 9
one one 8
able able 8
```

```
together together 7
```

### 30.10.1 Revisar la lista y quitar palabras a mano

Después de la limpieza automática **siempre** quedan cosas raras, y hay que mirarlas. Los tres casos típicos, con ejemplos de textos reales:

- **Basura de codificación**, como "òó" o òorchid: son acentos que se leyeron mal, y aparecen como palabras propias. Se arreglan en la importación, no aquí.
- **Palabras del dominio que no aportan**: en un texto sobre orquídeas, orchid va a ser la más frecuente y no dice nada, porque de eso trata todo el documento. Lo interesante es lo que la acompaña.
- **Restos de formato**: números de página, figura, tabla, et, al.

La solución es la misma en los tres casos: añadir las a tu propia lista de palabras vacías y volver a correr la limpieza.

```
mis_vacias <- c("orchid", "pollinators", "figura", "tabla")
docs <- tm_map(docs, removeWords, mis_vacias)
```

### 30.10.2 Cómo quitar del data frame las palabras con 3 apariciones o menos

Una vez que tienes el data frame `d` con las columnas `word` y `freq`, esto ya es `dplyr` del capítulo de transformación, nada nuevo:

```
d_filtrado <- d |> filter(freq > 3)
nrow(d); nrow(d_filtrado)
```

Filtrar por frecuencia mínima y usar `min.freq` en `wordcloud()` hacen cosas parecidas pero no iguales: `filter()` cambia **los datos** y `min.freq` cambia solo **lo que se dibuja**. Si vas a reportar cuántas palabras distintas hay, la cifra tiene que salir de los datos, no de la gráfica.

---

Del paquete **wordcloud**

```
#head(d)
wordcloud(words = d$word, # las palabras
 freq = d$freq, # la frecuencia
 min.freq = 1, # la frecuencia mínima
 max.words=50, # el número máximo de palabras
 random.order=TRUE, # orden aleatorio
 rot.per=0.35, # rotación de las palabras
 colors=brewer.pal(8, "Dark2")) # colores
```



### 30.11 Ejemplo 2

Del paquete **wordcloud2**

```
wordcloud2(data = d)
```

`wordcloud2()` produce una nube **interactiva** en HTML: al pasar el ratón por encima de una palabra aparece su frecuencia. Esa es su ventaja de verdad sobre `wordcloud()`, más allá de que se vea más vistosa. La desventaja es que solo funciona en salida HTML, y por eso el bloque lleva `eval=knitr::is_html_output()`.

### 30.12 Cuándo una nube de palabras no es la gráfica correcta

Conviene decirlo con claridad, porque las nubes de palabras son bonitas y por eso se usan mucho más de lo que se deberían.

El problema es que el ojo humano **compara mal las áreas**. Si una palabra aparece el doble de veces que otra, en la nube se dibuja con letra del doble de alto, pero ocupa aproximadamente **cuatro veces** más superficie, y eso exagera



la diferencia. Además, una palabra larga ocupa más sitio que una corta con la misma frecuencia, y la posición es aleatoria, así que la que quede en el centro parecerá más importante sin serlo.

Traducido: **una nube de palabras no permite ordenar ni comparar cantidades**. Sirve para dar una impresión general, para una portada o para abrir una presentación. No sirve para sostener un resultado.

Si lo que quieres es que se vea **cuáles** palabras son más frecuentes y **cuánto** más, la gráfica correcta es una de barras horizontales ordenada por frecuencia, como la del capítulo de factores:

```
d |>
 slice_max(freq, n = 20) |>
 mutate(word = fct_reorder(word, freq)) |>
 ggplot(aes(freq, word)) +
 geom_col()
```

Es menos vistosa y se lee infinitamente mejor. Un buen término medio para un informe: la nube como figura de apertura y las barras como figura de resultados.

## 30.13 Como remover palabras de otra idioma

### 30.13.1 Vea este enlace para los “stopwords” de muchos idiomas

<https://cran.r-project.org/web/packages/stopwords/readme/README.html>

## 30.14 En español

```
from CRAN
install.packages("stopwords")

Or get the development version from GitHub:
install.packages("devtools")
devtools::install_github("quanteda/stopwords")
```

### 30.14.1 Las 30 primeras palabras en la lista de stopwords del paquete “stopwords” en español

```
head(stopwords::stopwords("es", source = "snowball"), 30)
```

```
[1] "de" "la" "que" "el" "en" "y" "a" "los"
[9] "del" "se" "las" "por" "un" "para" "con" "no"
[17] "una" "su" "al" "lo" "como" "más" "pero" "sus"
```

```
[25] "le" "ya" "o" "este" "sí" "porque"
```

---

Ejemplos #3

Aquí un tercer ejemplo

```
install.packages("pacman") # Si no tiene instalada la Biblioteca Pacman ejecutar esto
library("pacman")

p_load("tm") # Biblioteca para realizar el preprocesado del texto,
p_load("tidyverse") # Biblioteca con funciones para manipular datos.
p_load("wordcloud") # Biblioteca para graficar nuestra nube de palabras.
p_load("RColorBrewer") # Biblioteca para seleccionar una paleta de colores de nuestra s
```

### 30.15 Un documento para hacer un word cloud “La inteligencia artificial”

```
library(readr)
articulo_IA <- "https://gist.github.com/EverVino/7bdbbe7ebdff5987970036f52f0e384f/raw/"
texto <- read_file(articulo_IA)
texto

[1] "La inteligencia artificial (IA) es, en informática, la inteligencia expresada p
Tratan de emular de forma racional el comportamiento humano; por ejemplo los agentes i
337 y el conocimiento sobre el conocimiento (lo que sabemos sobre lo que saben otras p
604\n\nEn los problemas clásicos de planificación, el agente puede asumir que es el ún.
449 La planificación de múltiples agentes utiliza la cooperación y la competencia de m
455\nAprendizaje\n\nEl aprendizaje automático es un concepto fundamental de la investi
892 La visión artificial es la capacidad de analizar la información visual, que suele s
```

### 30.16 Convertir su documento en Corpus y identificar que es en español

```
texto2 <- VCorpus(VectorSource(texto),
 readerControl = list(reader = readPlain, language = "es", load = TRUE))
texto2

<<VCorpus>>
Metadata: corpus specific: 0, document level (indexed): 0
Content: documents: 1
```

```
#inspect(texto2)
```

## 30.17 Limpieza del documento

- remover los números
- remover las puntuaciones
- cambiar a letras minúsculas
- remover las palabras comunes en español
- usar solamente la base de las palabras (“stem”) por ejemplo remover las conjugaciones **espero**, **esperas**, **espera**, **esperamos**... se convierte en “esper”
- remover espacios blancos

Quita siempre los “stopwords” del idioma correcto. Si tu texto está en español usa `stopwords("es")` y no la lista en inglés, o palabras como “de”, “la” y “que” dominarán la nube.

```
texto2 <- tm_map(texto2, removeNumbers)
texto2 <- tm_map(texto2, removePunctuation)
texto2 <- tm_map(texto2, tolower)
texto2 <- tm_map(texto2, removeWords, stopwords::stopwords("es", source = "snowball"))
#texto2 <- tm_map(texto2, stemDocument, language="spanish")
texto2 <- tm_map(texto2, stripWhitespace)
```

## 30.18 Transformar el texto en un documento de texto sencillo (Plain Text)

```
texto2 <- tm_map(texto2, PlainTextDocument)
```

## 30.19 Transformar el texto para reemplazar algunos caracteres especiales

```
toSpace <- content_transformer(function (x , pattern) gsub(pattern, " ", x))
#texto2 <- tm_map(texto2, toSpace, "-")
texto2 <- tm_map(texto2, toSpace, "@")
texto2 <- tm_map(texto2, toSpace, "\\|")
```

## 30.20 Crear una matriz de las palabras

```
tabla_frecuencia <- DocumentTermMatrix(texto2)
```

```
Ordenamos nuestra tabla de frecuencias de acuerdo a sus valores numéricos.
tabla_frecuencia<-tabla_frecuencia[order(tabla_frecuencia$frecuencia, decreasing=TRUE)
```

```
head(tabla_frecuencia) # aquí vemos las 6 palabras más comunes en el texto
```

## 30.21 Calcular la frecuencia de cada palabra

```
palabras frecuencia
809 inteligencia 79
126 artificial 67
1378 sistemas 32
1347 ser 22
737 humano 19
1172 problemas 18
```

```
wordcloud(words = tabla_frecuencia$palabras,
 freq = tabla_frecuencia$frecuencia,
 min.freq = 5,
 max.words = 100,
 random.order = FALSE,
 colors = brewer.pal(8, "Paired"))
```




---

Como ver la figuras en la pestaña “Plots”

De esa forma puede bajar los WordClouds como .pdf o otro formato.

<<https://stackoverflow.com/questions/40570621/rstudio-how-to-show-plot-output-in-bottom-right-pane>>

---

## 30.22 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que la nube o la tabla se vean) y una **explicación en palabras**. Los ejercicios usan `Datos/el_quijote.txt`, que está en el repositorio del curso, no el discurso ni el documento sobre inteligencia artificial del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_30_wordclouds.Rmd`.

### 30.22.1 Ejercicio 1. La nube inútil

Lee `Datos/el_quijote.txt`, conviértelo en corpus y haz una nube **sin quitar ninguna palabra vacía**. Saca también la tabla de las diez palabras más frecuentes.

**Verificación:** entre las diez primeras tienen que estar `que`, `de` y `y`. `que` tiene que pasar de 10 000 apariciones.

**En palabras:** ¿qué te dice esta nube sobre el Quijote? Contesta con honestidad.

```
Escribe tu código aquí
```

### 30.22.2 Ejercicio 2. Las palabras vacías

Repite todo el proceso quitando `stopwords("spanish")` y vuelve a sacar las diez primeras.

**Verificación:** ahora `quijote` y `sancho` tienen que estar entre las diez primeras, y `quijote` tiene que pasar de 700 apariciones.

**En palabras:** compara las dos listas. ¿Cuántas de las diez primeras cambiaron? Explica por qué el paso de las palabras vacías es el más importante de los cinco.

```
Escribe tu código aquí
```

### 30.22.3 Ejercicio 3. Tus propias palabras vacías

Mira las treinta primeras palabras del resultado anterior. Encontrarás algunas que tampoco dicen nada sobre el contenido: fórmulas de tratamiento como `vuestra` y `merced`, o verbos de diálogo como `dijo` y `respondió`.

Decide cuáles quitar, ponlas en tu propia lista, y vuelve a generar la nube.

**Verificación:** la lista de las diez primeras tiene que cambiar respecto al Ejercicio 2, y tu lista propia tiene que tener al menos cinco palabras.

**En palabras:** justifica cada palabra que quitaste. ¿Podría alguien defender que `dijo` sí dice algo sobre el Quijote? Explica en qué situación quitarla sería un error.

```
Escribe tu código aquí
```

### 30.22.4 Ejercicio 4. Los parámetros de la nube

Con la tabla de frecuencias del Ejercicio 3, genera cuatro nubes cambiando solo los parámetros: `max.words = 20`, `max.words = 200`, `min.freq = 100`, y `rot.per = 0`.

**Verificación:** las cuatro tienen que salir distintas, y la de `max.words = 200` tiene que ser claramente más difícil de leer.

**En palabras:** ¿cuál escogerías para la portada de un informe y cuál para la sección de resultados? ¿Qué hace `rot.per` y por qué ponerlo en cero ayuda a leer?

```
Escribe tu código aquí
```

### 30.22.5 Ejercicio 5. La nube contra las barras

Con las **veinte** palabras más frecuentes del Ejercicio 3, haz dos gráficas: la nube y un diagrama de barras horizontales ordenado con `fct_reorder()`.

**Verificación:** las dos usan exactamente los mismos veinte datos.

**En palabras:** mirando solo la nube, ordena de memoria las cinco primeras palabras. Ahora comprueba con las barras si acertaste. ¿Cuántas veces más frecuente es la primera que la quinta? ¿Se puede contestar eso con la nube?

```
Escribe tu código aquí
```

### 30.22.6 Ejercicio 6. Dos idiomas

Repite el proceso con `Datos/Alice_Adventures_in_Wonderland.txt`, que está en inglés, usando `stopwords("english")`.

Después, a propósito, hazlo **al revés**: al Quijote quítale las palabras vacías inglesas y a Alicia las españolas.

**Verificación:** las dos versiones cruzadas quedan prácticamente igual que sin limpiar.

**En palabras:** explica por qué. ¿Qué pasaría con un texto que mezcla los dos idiomas, como un artículo científico en español con citas en inglés?

```
Escribe tu código aquí
```

### 30.22.7 Ejercicio 7 (BONO). Los acentos que se parten

En la lista de palabras del Quijote aparecen cosas raras como **habi** en vez de **había**. No es un error del paquete `tm`: el archivo guarda los acentos en forma **descompuesta**, es decir, la letra y la tilde como dos caracteres separados.

Compruébalo: cuenta cuántos caracteres tiene la palabra **había** tal como sale del archivo, con `nchar()`, y compáralo con `nchar("había")` escrito por ti. Después arrégalo con `stringi::stri_trans_nfc()` antes de limpiar el texto, y vuelve a contar las palabras.

**Verificación:** después de normalizar, las palabras con acento dejan de salir partidas en la lista de frecuencias.

**En palabras:** este problema es primo del de la codificación del capítulo de importar datos. Explica en qué se parecen y en qué se diferencian. ¿En cuál de los cinco pasos habría que arreglarlo?

```
Escribe tu código aquí
```



## Chapter 31

# Temas Avanzados en Data Science con Tidyverse

Fecha de la última revisión

```
[1] "2026-09-06"
```

### 31.1 Temas

- Por qué una función normal no puede recibir un nombre de columna
- `{{ }}`: la solución, y `:=` para nombrar la columna de salida
- Modelado reproducible con `tidymodels`: receta, modelo y flujo
- Datos anidados: una tabla dentro de una tabla
- Un modelo por grupo con `nest()`, `map()` y `broom`

Este capítulo supone que ya hiciste los de **funciones** e **iteración**. Todo lo que hay aquí es la combinación de esos dos: funciones que reciben columnas, y modelos aplicados a muchos grupos a la vez.

### 31.2 Introducción

Este documento presenta temas avanzados en el ecosistema **tidyverse** con ejemplos prácticos en R.

#### 31.2.1 1. Manipulación avanzada con dplyr

Empecemos por el problema, porque si no se ve el problema la solución parece arbitraria.

En el capítulo de funciones aprendimos a escribir funciones propias. Pero si intentas escribir una que reciba **el nombre de una columna**, no funciona:

```
promedio_por <- function(datos, grupo, valor) {
 datos |>
 group_by(grupo) |> # <- esto está mal
 summarise(media = mean(valor)) # <- y esto también
}

promedio_por(millas, clase, autopista)
Error: object 'clase' not found
```

El motivo es que `dplyr` hace algo poco común: te deja escribir `clase` sin comillas, buscando ese nombre **dentro de la tabla** en vez de en tu sesión. A eso se le llama **evaluación tidy**. Es comodísimo cuando escribes código a mano y se vuelve un problema en cuanto quieres envolverlo en una función, porque `group_by(grupo)` busca una columna que se llame literalmente `grupo`, no el contenido del argumento.

**Evaluación tidy:** `{{ }}` y `:=`: funciones que reciben nombres de columnas.

- `{{ var }}` (“embrace”): permite pasar el nombre de una columna como argumento a una función que usa `dplyr` por dentro.
- `:=`: se usa en `mutate/summarise` cuando el nombre de la nueva columna también es una variable.
- **Para qué:** escribir tus propias funciones reutilizables sobre pipelines de `dplyr`.

La solución son las **llaves dobles**, `{{ }}`, que en inglés se llaman *embrace* (“abrazar”). Le dicen a `dplyr`: “no busques una columna con este nombre; mira qué me pasaron a mí y usa **eso**”.

Explicación: Uso de *tidy evaluation* para funciones personalizadas.

```
library(dplyr)

Función personalizada con tidy evaluation
mean_by_group <- function(data, group_var, value_var) {
 data %>%
 group_by({{ group_var }}) %>%
 summarise(mean_value = mean({{ value_var }}, na.rm = TRUE))
}

mean_by_group

function (data, group_var, value_var)
{
data %>% group_by({
{
group_var
```

```
}
}) %>% summarise(mean_value = mean({
{
value_var
}
}, na.rm = TRUE))
}

df <- tibble(grupo = c("A", "A", "B", "B"),
 valor = c(10, 20, 30, 40)) # los datos

mean_by_group(df, grupo, valor) # El uso de la función nueva
```

```
A tibble: 2 x 2
grupo mean_value
<chr> <dbl>
1 A 15
2 B 35
```

Otro ejemplo

```
dfnew= tibble(especie = c("setosa", "setosa", "versicolor", "versicolor"),
 longitud_sepalo = c(5.1, 4.9, 7.0, 6.4))

mean_by_group(dfnew, especie, longitud_sepalo)

A tibble: 2 x 2
especie mean_value
<chr> <dbl>
1 setosa 5
2 versicolor 6.7

mean_sd_by_group <- function(data, group_var, value_var) {
 data %>%
 group_by({{ group_var }}) %>%
 summarise(sd_value = sd({{ value_var }}), na.rm = TRUE),
 mean_value = mean({{ value_var }}), na.rm = TRUE))
}

mean_sd_by_group(dfnew, especie, longitud_sepalo)

A tibble: 2 x 3
especie sd_value mean_value
<chr> <dbl> <dbl>
1 setosa 0.141 5
2 versicolor 0.424 6.7
```

### 31.2.2 Componentes clave:

1. Definición de la función

```
mean_by_group <- function(data, group_var, value_var) { ... }
```

- data: El data frame o tibble que contiene los datos.
- group\_var: La variable por la que se agruparán los datos (por ejemplo, categoría).
- value\_var: La variable numérica de la que se calculará la media.

2. `{{ }}` (Curly-curly operator):

- Es parte del tidy evaluation en dplyr.
- Permite que los argumentos group\_var y value\_var se pasen como nombres de columna sin comillas.
- Sin esto, la función no podría interpretar correctamente los nombres de columna.

3. `group_by({{ group_var }})`:

- Agrupa el conjunto de datos por la variable indicada.
- Ejemplo: Si group\_var = especie, agrupa por especie.

4. `summarise(mean_value = mean({{ value_var }}, na.rm = TRUE))`:

- Calcula la media de la columna indicada (value\_var) dentro de cada grupo.
- na.rm = TRUE elimina valores faltantes (NA) para evitar errores.

### 31.2.3 Nombrar la columna de salida: `:=`

Hay una limitación en la función de arriba: la columna nueva siempre se llama `mean_value`, diga lo que diga el usuario. Para que el nombre dependa del argumento hace falta `:=` (dos puntos e igual) en vez del `=` normal:

```
promedio_con_nombre <- function(datos, grupo, valor, nombre = "media") {
 datos |>
 group_by({{ grupo }}) |>
 summarise({{ nombre }} := mean({{ valor }}, na.rm = TRUE))
}
```

Regla corta para no perderse: `{{ }}` a la derecha del igual, `:=` en lugar del igual. Todo lo que sea “una columna que me pasaron” va abrazado; el `:=` solo hace falta cuando el **nombre** de la columna nueva también es un argumento.

Si olvidas `na.rm = TRUE`, funciones como `mean()` o `sd()` devuelven NA en cuanto la columna tiene un solo valor faltante, en vez de calcular el promedio.

```
datos=c(1, NA, 3:8, NA, 10:12)
mean(datos, na.rm=TRUE)
```

```
[1] 6.7
```

## 31.3 2. Modelado con tidymodels

Explicación: Flujo reproducible para regresión lineal.

En el capítulo de modelos se ajustaba un `lm()` de una sola línea. `tidymodels` es otra manera de hacerlo, más larga, y conviene entender qué se gana a cambio.

**Las tres piezas de tidymodels:** separar la preparación, el método y el ajuste.

- **La receta (`recipe()`):** describe **qué** se va a modelar y qué hay que hacerle a los datos antes (centrar, escalar, convertir categorías, imputar faltantes). Es una descripción, no un cálculo: no toca los datos hasta que se aplica.
- **El modelo (`linear_reg()`, `set_engine()`):** describe **con qué** método, separado del paquete concreto que lo calcula. El mismo modelo puede correr con `lm`, con `glmnet` o con `stan` cambiando una palabra.
- **El flujo (`workflow()`):** junta receta y modelo en un solo objeto, que es el que se ajusta con `fit()` y el que se guarda.

¿Para qué tanta ceremonia si `lm()` cabía en una línea? Por tres razones, y todas aparecen en cuanto el análisis crece:

1. **La preparación viaja con el modelo.** Si centraste una variable para ajustar, hay que centrarla igual al predecir. Con `lm()` eso lo tienes que recordar tú; con una receta, va incluido.
2. **Cambiar de método cuesta una palabra.** Pasar de regresión lineal a bosque aleatorio no obliga a reescribir el resto.
3. **Evita la fuga de información.** Al validar, la receta se aplica solo a la parte de entrenamiento, que es lo correcto y lo que casi nadie hace a mano.

Para un modelo suelto de un artículo, `lm()` sigue siendo la respuesta correcta y más clara. `tidymodels` se justifica cuando hay preparación de datos, varios modelos que comparar, o validación cruzada de por medio.

```
library(tidymodels)
data(mtcars)

Convertir variables a factores
mtcars <- mtcars %>%
 mutate(cyl = factor(cyl),
 gear = factor(gear))

receta <- recipe(mpg ~ wt + hp + cyl + gear, data = mtcars)
modelo <- linear_reg() %>%
```

```

 set_engine("lm")

workflow() %>%
 add_recipe(receta) %>%
 add_model(modelo) %>%
 fit(data = mtcars)

== Workflow [trained] =====
Preprocessor: Recipe
Model: linear_reg()
##
-- Preprocessor -----
0 Recipe Steps
##
-- Model -----
##
Call:
stats::lm(formula = ..y ~ ., data = data)
##
Coefficients:
(Intercept) wt hp cyl6 cyl8 gear4
34.46173 -2.79186 -0.03424 -2.93920 -1.33080 1.42125
gear5
2.08577

```

Ver los resultados del modelo y hacer predicciones.

```

Ajustar el modelo
ajuste <- workflow() %>%
 add_recipe(receta) %>%
 add_model(modelo) %>%
 fit(data = mtcars)

Extraer el modelo ajustado
modelo_ajustado <- ajuste %>% extract_fit_parsnip()

Ver coeficientes

modelo_ajustado %>% tidy() # Coeficientes en formato tibble

A tibble: 7 x 5
term estimate std.error statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 34.5 2.46 14.0 2.52e-13
2 wt -2.79 0.856 -3.26 3.18e- 3

```

```
3 hp -0.0342 0.0177 -1.93 6.44e- 2
4 cyl6 -2.94 1.47 -1.99 5.71e- 2
5 cyl8 -1.33 2.78 -0.479 6.36e- 1
6 gear4 1.42 1.51 0.941 3.56e- 1
7 gear5 2.09 2.14 0.972 3.40e- 1

modelo_ajustado %>% glance() # Métricas globales (R², AIC, etc.)

A tibble: 1 x 12
r.squared adj.r.squared sigma statistic p.value df logLik AIC BIC
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 0.864 0.831 2.48 26.4 0.00000000113 6 -70.5 157. 169.
i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>

summary(modelo_ajustado$fit) # Resumen clásico del modelo lm

##
Call:
stats::lm(formula = ..y ~ ., data = data)
##
Residuals:
Min 1Q Median 3Q Max
-3.4215 -1.2428 -0.3401 0.8961 5.3657
##
Coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept) 34.46173 2.46379 13.987 2.52e-13 ***
wt -2.79186 0.85567 -3.263 0.00318 **
hp -0.03424 0.01770 -1.935 0.06444 .
cyl6 -2.93920 1.47362 -1.995 0.05710 .
cyl8 -1.33080 2.77885 -0.479 0.63617
gear4 1.42125 1.51065 0.941 0.35580
gear5 2.08577 2.14490 0.972 0.34015

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
Residual standard error: 2.477 on 25 degrees of freedom
Multiple R-squared: 0.8638, Adjusted R-squared: 0.8311
F-statistic: 26.42 on 6 and 25 DF, p-value: 1.128e-09

Predicciones y métricas
predicciones <- predict(modelo_ajustado, mtcars)

bind_cols(mtcars, predicciones) %>%
 metrics(truth = mpg, estimate = .pred)

A tibble: 3 x 3
.metric .estimator .estimate
```

```
<chr> <chr> <dbl>
1 rmse standard 2.19
2 rsq standard 0.864
3 mae standard 1.68
```

- `extract_fit_parsnip()` obtiene el modelo ajustado.
- `tidy()` muestra coeficientes y errores estándar.
- `metrics()` calcula métricas como RMSE,  $R^2$ , MAE.

**broom:** convertir la salida de un modelo en tablas ordenadas.

- **`tidy(modelo)`:** una fila por **coeficiente**, con estimado, error estándar, estadístico y valor p.
- **`glance(modelo)`:** una sola fila con las métricas del **modelo entero**:  $R^2$ , AIC, sigma, número de observaciones.
- **`augment(modelo, datos)`:** los datos originales más las predicciones y los residuales, que es lo que hacía `add_predictions()` y `add_residuals()` del capítulo de modelos.

Esto es lo que cierra el círculo con los capítulos anteriores: `summary()` imprime un texto bonito que **no se puede usar**, y `tidy()` y `glance()` devuelven **tibbles**, que sí se pueden filtrar, unir, graficar y guardar. Es la misma lección del Ejercicio 2 del capítulo de funciones: devuelve una tabla, no un adorno.

`metrics()` calculado sobre **los mismos datos con los que se ajustó** siempre sale demasiado optimista: el modelo ya vio esas observaciones. Para saber de verdad qué tan bien predice hace falta separar los datos en entrenamiento y prueba, con `initial_split()`. En el ejemplo de arriba no se hace, y por eso las métricas hay que leerlas como descripción del ajuste, no como capacidad de predicción.

---

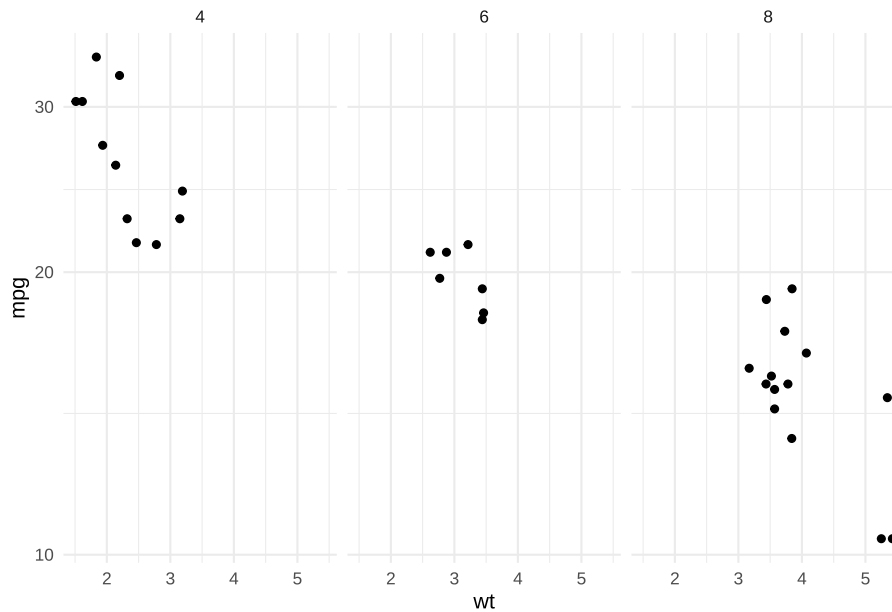
## 31.4 3. Visualización avanzada con ggplot2

Explicación: Facetas y escalas personalizadas.

```
library(ggplot2)

ggplot(mtcars, aes(x = wt, y = mpg)) +
 geom_point() +
 facet_wrap(~ cyl) +
 scale_y_log10() +
 theme_minimal()
```





## 31.5 4. Datos anidados y listas con purrr

Explicación: Ajustar modelos por grupo y extraer coeficientes.

Esta sección es la continuación directa del capítulo de iteración, con una idea nueva: una columna de un tibble puede contener **cosas que no son números**. Puede contener tablas enteras, o modelos.

**nest()** y **unnest()** (tidyr): meter tablas dentro de una tabla.

- **group\_by(g) |> nest()**: deja una fila por grupo y una columna **data** donde cada celda es **el tibble completo de ese grupo**.
- Con **mutate()** y **map()** se le puede añadir una columna **modelo**, donde cada celda es un objeto **lm**, y otra con sus coeficientes.
- **unnest(columna)**: deshace el anidamiento y devuelve una tabla plana.

La ventaja sobre el **split()** del capítulo de iteración es que **todo se queda en una sola tabla**: los datos, el modelo y los resultados de cada grupo viajan juntos en la misma fila, y no hay que ir emparejando listas por su nombre.

El patrón completo, que vale la pena aprenderse de memoria porque resuelve una enorme cantidad de análisis reales:

```
datos |>
 group_by(grupo) |>
```

```

nest() |>
mutate(
 modelo = map(data, \(d) lm(y ~ x, data = d)),
 resumen = map(modelo, broom::glance),
 coefs = map(modelo, broom::tidy)
) |>
unnest(resumen)

library(tidyr)
library(purrr)
library(broom)

nested <- mtcars %>%
 group_by(cyl) %>%
 nest()

nested <- nested %>%
 mutate(modelo = purrr::map(data, ~ lm(mpg ~ wt, data = .x)),
 coeficientes = purrr::map(modelo, tidy))

nested$data

[[1]]
A tibble: 7 x 10
mpg disp hp drat wt qsec vs am gear carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct> <dbl>
1 21 160 110 3.9 2.62 16.5 0 1 4 4
2 21 160 110 3.9 2.88 17.0 0 1 4 4
3 21.4 258 110 3.08 3.22 19.4 1 0 3 1
4 18.1 225 105 2.76 3.46 20.2 1 0 3 1
5 19.2 168. 123 3.92 3.44 18.3 1 0 4 4
6 17.8 168. 123 3.92 3.44 18.9 1 0 4 4
7 19.7 145 175 3.62 2.77 15.5 0 1 5 6
##
[[2]]
A tibble: 11 x 10
mpg disp hp drat wt qsec vs am gear carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct> <dbl>
1 22.8 108 93 3.85 2.32 18.6 1 1 4 1
2 24.4 147. 62 3.69 3.19 20 1 0 4 2
3 22.8 141. 95 3.92 3.15 22.9 1 0 4 2
4 32.4 78.7 66 4.08 2.2 19.5 1 1 4 1
5 30.4 75.7 52 4.93 1.62 18.5 1 1 4 2
6 33.9 71.1 65 4.22 1.84 19.9 1 1 4 1
7 21.5 120. 97 3.7 2.46 20.0 1 0 3 1
8 27.3 79 66 4.08 1.94 18.9 1 1 4 1

```

```
9 26 120. 91 4.43 2.14 16.7 0 1 5 2
10 30.4 95.1 113 3.77 1.51 16.9 1 1 5 2
11 21.4 121 109 4.11 2.78 18.6 1 1 4 2
##
[[3]]
A tibble: 14 x 10
mpg disp hp drat wt qsec vs am gear carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct> <dbl>
1 18.7 360 175 3.15 3.44 17.0 0 0 3 2
2 14.3 360 245 3.21 3.57 15.8 0 0 3 4
3 16.4 276. 180 3.07 4.07 17.4 0 0 3 3
4 17.3 276. 180 3.07 3.73 17.6 0 0 3 3
5 15.2 276. 180 3.07 3.78 18 0 0 3 3
6 10.4 472 205 2.93 5.25 18.0 0 0 3 4
7 10.4 460 215 3 5.42 17.8 0 0 3 4
8 14.7 440 230 3.23 5.34 17.4 0 0 3 4
9 15.5 318 150 2.76 3.52 16.9 0 0 3 2
10 15.2 304 150 3.15 3.44 17.3 0 0 3 2
11 13.3 350 245 3.73 3.84 15.4 0 0 3 4
12 19.2 400 175 3.08 3.84 17.0 0 0 3 2
13 15.8 351 264 4.22 3.17 14.5 0 1 5 4
14 15 301 335 3.54 3.57 14.6 0 1 5 8

nested$modelo

[[1]]
##
Call:
lm(formula = mpg ~ wt, data = .x)
##
Coefficients:
(Intercept) wt
28.41 -2.78
##
##
[[2]]
##
Call:
lm(formula = mpg ~ wt, data = .x)
##
Coefficients:
(Intercept) wt
39.571 -5.647
##
##
[[3]]
```

```
##
Call:
lm(formula = mpg ~ wt, data = .x)
##
Coefficients:
(Intercept) wt
23.868 -2.192

nested$coeficientes

[[1]]
A tibble: 2 x 5
term estimate std.error statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 28.4 4.18 6.79 0.00105
2 wt -2.78 1.33 -2.08 0.0918
##
[[2]]
A tibble: 2 x 5
term estimate std.error statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 39.6 4.35 9.10 0.00000777
2 wt -5.65 1.85 -3.05 0.0137
##
[[3]]
A tibble: 2 x 5
term estimate std.error statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 23.9 3.01 7.94 0.00000405
2 wt -2.19 0.739 -2.97 0.0118
```

La alternativa es hacer cada factor individualmente

- Hacemos un modelo `lm(mpg ~ wt)` para cada subconjunto de datos agrupados por `cyl == 6`

```
modelo_cyl6 <- lm(mpg ~ wt, data = filter(mtcars, cyl == 6))
summary(modelo_cyl6)
```

```
##
Call:
lm(formula = mpg ~ wt, data = filter(mtcars, cyl == 6))
##
Residuals:
Mazda RX4 Mazda RX4 Wag Hornet 4 Drive Valiant Merc 280
-0.1250 0.5840 1.9292 -0.6897 0.3547
Merc 280C Ferrari Dino
```

```
-1.0453 -1.0080
##
Coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept) 28.409 4.184 6.789 0.00105 **
wt -2.780 1.335 -2.083 0.09176 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
Residual standard error: 1.165 on 5 degrees of freedom
Multiple R-squared: 0.4645, Adjusted R-squared: 0.3574
F-statistic: 4.337 on 1 and 5 DF, p-value: 0.09176
```

---



---

## 31.6 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Cada ejercicio se entrega con tres cosas: el **script**, el **resultado** (que el bloque corra) y una **explicación en palabras**. Los ejercicios usan `millas` y `países` del paquete `datos`, no `mtcars` ni los tibbles pequeños del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_31_avanzado.Rmd`.

### 31.6.1 Ejercicio 1. El problema, antes de la solución

Escribe la función `promedio_por(datos, grupo, valor)` sin las llaves dobles, tal como saldría de lo aprendido en el capítulo de funciones, y llámala con `millas`, `clase` y `autopista`. Ponla en un bloque con `error=TRUE`.

**Verificación:** tiene que dar object 'clase' not found.

**En palabras:** la columna `clase` sí existe en `millas`. Explica entonces dónde la fue a buscar R y por qué no la encontró.

```
Escribe tu código aquí
```

### 31.6.2 Ejercicio 2. Las llaves dobles

Arregla la función anterior con `{{ }}`. Pruébala con `millas` (por `clase`, el promedio de `autopista`) y después con `países` (por `continente`, el promedio de `esperanza_de_vida`).

**Verificación:** la primera llamada devuelve 7 filas y la segunda 5. La **misma** función sirve para las dos tablas sin cambiar nada.

**En palabras:** ¿por qué `datos` no necesita llaves dobles y `grupo` sí? Fíjate en qué tipo de cosa es cada argumento.

```
Escribe tu código aquí
```

### 31.6.3 Ejercicio 3. El nombre de la columna de salida

Añádele a tu función un cuarto argumento, `nombre`, que decida cómo se llama la columna del resultado. Usa `:=`.

**Verificación:** con `nombre = "consumo_medio"`, la tabla que sale tiene que tener una columna llamada exactamente así. Compruébalo con `names()`.

**En palabras:** prueba primero con `=` normal en vez de `:=` y copia el error. ¿Qué intentaba hacer R con ese `=`?

```
Escribe tu código aquí
```

### 31.6.4 Ejercicio 4. Una tabla dentro de una tabla

Con `millas`, agrupa por `clase` y usa `nest()`. Mira el resultado, y después mira el contenido de una sola celda de la columna `data`.

**Verificación:** la tabla anidada tiene **7 filas** y **2 columnas**. La celda de `data` de la clase `suv` contiene un tibble de **62 filas**.

**En palabras:** ¿cuántas filas tiene la tabla anidada y cuántas tenía `millas`? ¿Dónde se fueron las demás? Explica qué es una “columna lista”.

```
Escribe tu código aquí
```

### 31.6.5 Ejercicio 5. Un modelo por grupo, todo en una tabla

Sobre la tabla anidada, añade con `mutate()` y `map()` una columna con el modelo `lm(autopista ~ cilindrada)` de cada clase, y otra con `broom::glance()` de cada modelo. Después haz `unnest()` de esa última y ordena por  $R^2$ .

**Verificación:** la tabla final tiene 7 filas y una columna `r.squared`.

**En palabras:** compara esto con lo que hiciste en el capítulo de iteración con `split()` y `map()`. ¿Qué ventaja tiene tener los datos, el modelo y las métricas en la misma fila?

```
Escribe tu código aquí
```

### 31.6.6 Ejercicio 6. `tidy()`, `glance()` y `augment()`

Ajusta `lm(autopista ~ cilindrada + clase, data = millas)` y aplícale las tres funciones de `broom`. Compara cada salida con la de `summary()`.

**Verificación:** `tidy()` devuelve una fila por coeficiente, `glance()` devuelve **una sola fila**, y `augment()` devuelve 234 filas.

**En palabras:** `summary()` enseña la misma información. ¿Por qué preferir `broom`? Contesta con algo que puedas **hacer** con la salida de `tidy()` y no con la de `summary()`.

*# Escribe tu código aquí*

### 31.6.7 Ejercicio 7 (BONO). Un flujo de `tidymodels`

Monta con `tidymodels` el mismo modelo del ejercicio anterior: una receta, un modelo `linear_reg()` con motor `lm`, y un `workflow()`. Ajústalo y saca los coeficientes con `extract_fit_parsnip()` y `tidy()`.

**Verificación:** los coeficientes tienen que ser **idénticos** a los del `lm()` del Ejercicio 6.

**En palabras:** si dan exactamente lo mismo y ocupa cinco veces más código, ¿en qué situación vale la pena `tidymodels`? Da un ejemplo concreto de tu propio trabajo.

*# Escribe tu código aquí*





## Chapter 32

# Leaflet: Mapas interactivos

## [1] "2026-09-06"

### 32.1 Temas

- Qué hace distinto a un mapa interactivo
  - Las tres piezas: `leaflet()`, `addTiles()` y una capa de marcadores
  - Latitud y longitud: cuál va primero y qué pasa si te equivocas
  - Marcadores: círculos, iconos y agrupamiento
  - Color por categoría y por valor, y la leyenda
  - Ventanas emergentes y etiquetas
  - Capas que se encienden y se apagan
- 

### 32.2 Por qué un mapa interactivo

Una gráfica de puntos con la longitud en un eje y la latitud en el otro ya es un mapa, y para muchas cosas basta. Lo que añade `leaflet` es lo que **no cabe en una imagen fija**:

- el lector puede **acercarse** a una zona concreta sin que tú decidas por él cuál;
- cada punto puede llevar su propia **ventana emergente** con los datos de esa observación;
- el fondo es un **mapa de verdad**, con calles y nombres, así que los puntos se sitúan solos sin que haya que dibujar la costa;
- se pueden **encender y apagar capas**, y comparar dos conjuntos en el mismo mapa.

Un mapa de `leaflet` es HTML: solo funciona en la salida **HTML**. Si compilas el documento a PDF o a Word, el mapa no aparece. Por eso varios bloques de este capítulo llevan `eval=knitr::is_html_output()`. Para un artículo impreso hace falta un mapa estático, con `ggplot2` y `sf`, o una captura de pantalla.

---

```
library(tidyverse)
```

**Las tres piezas de un mapa:** siempre las mismas, en este orden.

- **leaflet(datos):** crea el mapa vacío y le dice de dónde salen los datos.
- **addTiles():** pone el fondo. Por defecto usa OpenStreetMap; con `addProviderTiles()` se puede escoger otro (satélite, relieve, blanco y negro).
- **Una capa de marcadores:** `addMarkers()` (el alfiler clásico), `addCircleMarkers()` (círculos, que se pueden colorear y dimensionar), `addAwesomeMarkers()` (iconos).
- **Se encadenan con %>%**, igual que las capas de `ggplot` se suman con `+`.

**El orden de latitud y longitud es la fuente número uno de mapas mal hechos.** La latitud va del sur al norte (Puerto Rico está alrededor de 18) y la longitud de oeste a este (Puerto Rico está alrededor de menos 66). En `leaflet` los argumentos se llaman `lat` y `lng`, así que escríbelos siempre **con su nombre** y nunca por posición. Si tus puntos aparecen en el océano Índico o en la Antártida, casi seguro los cambiaste de sitio.

## 32.3 Bajar los datos de la web del USGS

Website: <https://earthquake.usgs.gov/earthquakes/feed/v1.0/csv.php>

Este capítulo **descarga datos en vivo desde internet**. Si al ejecutar ves un error como `Error in download.file(...): cannot open URL` o un HTTP error, casi siempre significa que no hay conexión a internet o que el servidor no está respondiendo en ese momento. Verifica tu conexión y vuelve a intentar. En este libro el código usa una copia local guardada en `Datos/` cuando la descarga falla, para que el capítulo siempre se pueda generar. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
library(data.table)
Intenta bajar los datos en vivo del USGS; si no hay conexión o el servidor
no responde, usa la copia local guardada en Datos/.
earthquakes <- tryCatch(
 fread("https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_month.csv"),
 error = function(e) readr::read_csv("Datos/Earthquake_all_month.csv")
)

head(earthquakes, n=10)
```

```

time latitude longitude depth mag magType nst gap
<POSc> <num> <num> <num> <num> <char> <int> <num>
1: 2026-09-06 23:37:15 68.64400 -148.6150 0.000 2.70 ml 47 55
2: 2026-09-06 23:35:06 38.79016 -122.7617 1.180 1.04 md 17 63
3: 2026-09-06 23:32:15 19.03150 -155.3648 31.160 1.71 md 15 255
4: 2026-09-06 23:12:47 19.28767 -155.4302 28.510 1.74 md 27 57
5: 2026-09-06 22:58:52 19.42700 -155.5860 39.830 2.09 md 30 66
6: 2026-09-06 22:35:20 38.82367 -122.8108 1.310 0.95 md 16 41
7: 2026-09-06 22:02:07 61.60400 -146.3570 13.900 2.60 ml 83 16
8: 2026-09-06 21:55:04 19.04100 -155.3708 31.070 2.52 md 52 215
9: 2026-09-06 21:38:07 32.72400 -115.8905 11.340 1.61 ml 35 39
10: 2026-09-06 21:08:44 -17.92050 -178.4632 608.631 4.50 mb 65 59
dmin rms net id updated
<num> <num> <char> <char> <POSc>
1: 0.4000000 0.70 ak aka2026rrwrsv 2026-09-06 23:40:36
2: 0.0105600 0.03 nc nc75430887 2026-09-06 23:36:41
3: 0.1538000 0.18 hv hv75030277 2026-09-06 23:35:50
4: 0.0586700 0.15 hv hv75030267 2026-09-06 23:16:18
5: 0.0396400 0.21 hv hv75030257 2026-09-06 23:02:31
6: 0.0006543 0.02 nc nc75430872 2026-09-06 23:07:22
7: 0.5000000 0.90 ak aka2026rrtnhu 2026-09-06 23:38:36
8: 0.1428000 0.17 hv hv75030237 2026-09-06 23:29:48
9: 0.0808300 0.21 ci ci41542200 2026-09-06 21:41:46
10: 2.5190000 0.54 us us7000tert 2026-09-06 23:21:50
place type horizontalError depthError
<char> <char> <num> <num>
1: 139 km WNW of Arctic Village, Alaska earthquake 3.10 2.8016
2: 2 km NNW of The Geysers, CA earthquake 0.23 0.4300
3: 22 km SSE of Pāhala, Hawaii earthquake 1.29 2.4100
4: 10 km NNE of Pāhala, Hawaii earthquake 0.63 0.8700
5: 27 km NNW of Pāhala, Hawaii earthquake 0.69 1.2700
6: 7 km NW of The Geysers, CA earthquake 0.22 0.4300
7: 48 km SSE of Nelchina, Alaska earthquake 1.90 1.6537
8: 21 km SSE of Pāhala, Hawaii earthquake 0.55 0.7400
9: 10 km E of Ocotillo, CA earthquake 0.24 0.4000
10: 235 km E of Levuka, Fiji earthquake 12.74 7.1520
magError magNst status locationSource magSource
<num> <int> <char> <char> <char>
1: 0.10000000 25 automatic ak ak
2: 0.17000000 17 automatic nc nc
3: 0.12606141 13 automatic hv hv
4: 0.13136835 10 automatic hv hv
5: 0.16406129 11 automatic hv hv
6: 0.24000000 17 automatic nc nc
7: 0.10000000 55 automatic ak ak
8: 0.07684615 8 automatic hv hv

```

```
9: 0.19587047 30 automatic ci ci
10: 0.06000000 82 reviewed us us

unique(earthquakes$type)

[1] "earthquake" "explosion" "quarry blast" "landslide" "ice quake"

earthquakes %>%
 filter(type=="sonic boom")

Empty data.table (0 rows and 22 cols): time,latitude,longitude,depth,mag,magType...
quarry_blasts=earthquakes %>%
 filter(type=="quarry blast")
```

## 32.4 Si bajo los datos a mi computadora

Use esta alternativa si bajo los datos a su computadora

```
#library(readr)
#Earthquake_all_month <- read_csv("Datos/Earthquake_all_month.csv")

#EQ=Earthquake_all_month
#head(EQ, n=10)

names(earthquakes)

[1] "time" "latitude" "longitude" "depth"
[5] "mag" "magType" "nst" "gap"
[9] "dmin" "rms" "net" "id"
[13] "updated" "place" "type" "horizontalError"
[17] "depthError" "magError" "magNst" "status"
[21] "locationSource" "magSource"

str(earthquakes)

Classes 'data.table' and 'data.frame': 11032 obs. of 22 variables:
$ time : POSIXct, format: "2026-09-06 23:37:15" "2026-09-06 23:35:06" ..
$ latitude : num 68.6 38.8 19 19.3 19.4 ...
$ longitude : num -149 -123 -155 -155 -156 ...
$ depth : num 0 1.18 31.16 28.51 39.83 ...
$ mag : num 2.7 1.04 1.71 1.74 2.09 0.95 2.6 2.52 1.61 4.5 ...
$ magType : chr "ml" "md" "md" "md" ...
$ nst : int 47 17 15 27 30 16 83 52 35 65 ...
$ gap : num 55 63 255 57 66 41 16 215 39 59 ...
$ dmin : num 0.4 0.0106 0.1538 0.0587 0.0396 ...
$ rms : num 0.7 0.03 0.18 0.15 0.21 ...
$ net : chr "ak" "nc" "hv" "hv" ...
```

```
$ id : chr "aka2026rrwrsv" "nc75430887" "hv75030277" "hv75030267" ...
$ updated : POSIXct, format: "2026-09-06 23:40:36" "2026-09-06 23:36:41" ...
$ place : chr "139 km WNW of Arctic Village, Alaska" "2 km NNW of The Geysers, CA"
$ type : chr "earthquake" "earthquake" "earthquake" "earthquake" ...
$ horizontalError: num 3.1 0.23 1.29 0.63 0.69 ...
$ depthError : num 2.8 0.43 2.41 0.87 1.27 ...
$ magError : num 0.1 0.17 0.126 0.131 0.164 ...
$ magNst : int 25 17 13 10 11 17 55 8 30 82 ...
$ status : chr "automatic" "automatic" "automatic" "automatic" ...
$ locationSource : chr "ak" "nc" "hv" "hv" ...
$ magSource : chr "ak" "nc" "hv" "hv" ...
- attr(*, ".internal.selfref")=<pointer: 0x1038a3f80>
```

## 32.5 Importación de Paquetes y Datos

```
library(tidyverse)
library(leaflet)
#library(rinat)
library(RColorBrewer)
library(flextable)
```

Seleccionar un subgrupo de los datos, solamente los primeros 100 datos

```
nrow(earthquakes)
```

```
[1] 11032
```

```
EQ_100 = earthquakes |> slice(1:100)
head(EQ_100)
```

```
time latitude longitude depth mag magType nst gap
<POSc> <num> <num> <num> <num> <char> <int> <num>
1: 2026-09-06 23:37:15 68.64400 -148.6150 0.00 2.70 ml 47 55
2: 2026-09-06 23:35:06 38.79016 -122.7617 1.18 1.04 md 17 63
3: 2026-09-06 23:32:15 19.03150 -155.3648 31.16 1.71 md 15 255
4: 2026-09-06 23:12:47 19.28767 -155.4302 28.51 1.74 md 27 57
5: 2026-09-06 22:58:52 19.42700 -155.5860 39.83 2.09 md 30 66
6: 2026-09-06 22:35:20 38.82367 -122.8108 1.31 0.95 md 16 41
dmin rms net id updated
<num> <num> <char> <char> <POSc>
1: 0.4000000 0.70 ak aka2026rrwrsv 2026-09-06 23:40:36
2: 0.0105600 0.03 nc nc75430887 2026-09-06 23:36:41
3: 0.1538000 0.18 hv hv75030277 2026-09-06 23:35:50
4: 0.0586700 0.15 hv hv75030267 2026-09-06 23:16:18
5: 0.0396400 0.21 hv hv75030257 2026-09-06 23:02:31
6: 0.0006543 0.02 nc nc75430872 2026-09-06 23:07:22
```

```
place type horizontalError depthError
<char> <char> <num> <num>
1: 139 km WNW of Arctic Village, Alaska earthquake 3.10 2.8016
2: 2 km NNW of The Geysers, CA earthquake 0.23 0.4300
3: 22 km SSE of Pāhala, Hawaii earthquake 1.29 2.4100
4: 10 km NNE of Pāhala, Hawaii earthquake 0.63 0.8700
5: 27 km NNW of Pāhala, Hawaii earthquake 0.69 1.2700
6: 7 km NW of The Geysers, CA earthquake 0.22 0.4300
magError magNst status locationSource magSource
<num> <int> <char> <char> <char>
1: 0.1000000 25 automatic ak ak
2: 0.1700000 17 automatic nc nc
3: 0.1260614 13 automatic hv hv
4: 0.1313683 10 automatic hv hv
5: 0.1640613 11 automatic hv hv
6: 0.2400000 17 automatic nc nc
```

## 32.6 Mapa básico con Leaflet

Leaflet es una biblioteca de **JavaScript** para mapas interactivos, creada por **Volodymyr Agafonkin** en 2011. Su ventaja es que es ligera y funciona en cualquier navegador. **RStudio/Posit** creó el paquete **leaflet** para R en 2015, permitiendo construir estos mapas sin escribir JavaScript. Los mapas base (“tiles”) provienen de **OpenStreetMap**, un proyecto colaborativo y abierto de cartografía mundial.

**leaflet()**: construye un mapa interactivo por capas, de forma parecida a **ggplot()**, pero uniendo las capas con el *pipe* (**%>%**).

- **leaflet(data)**: inicia el mapa con un conjunto de datos.
- **addTiles()**: añade el mapa base (por defecto, de OpenStreetMap).
- **addMarkers()** / **addCircleMarkers()**: colocan puntos; **~longitude** y **~latitude** (la tilde ~) indican qué columnas usar para las coordenadas, y **popup** el texto que aparece al hacer clic.

```
names(earthquakes)
```

```
[1] "time" "latitude" "longitude" "depth"
[5] "mag" "magType" "nst" "gap"
[9] "dmin" "rms" "net" "id"
[13] "updated" "place" "type" "horizontalError"
[17] "depthError" "magError" "magNst" "status"
[21] "locationSource" "magSource"
```

```
leaflet(EQ_100) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~longitude,
```

```
lat = ~latitude,
popup = ~mag,
color= ~(type))
```

```
#unique(earthquakes$type)
```

```
#color= ~(type),
```

## 32.7 Usando la paletas de color de viridis

- <https://cran.r-project.org/web/packages/viridis/vignettes/intro-to-viridis.html>
- Evaluar las siguientes paletas de colores:
  - viridis
  - inferno
  - plasma
  - magma
  - cividis
  - rocket
  - turbo

### Pasos

1. Seleccionar los datos en las áreas de PR, latitud y longitud
2. Crear una paleta de colores/ o usar viridis
3. Crear un mapa con los datos de los terremotos

```
names(EQ_100)
unique(EQ_100$type)
pal = colorFactor(palette = "magma", domain = NULL)

leaflet(EQ_100) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~longitude,
 lat = ~latitude,
 color= ~pal(mag),
 popup = ~paste("Magnitud", mag)
)
```

## 32.8 Trabajando con datos de iNaturalist

### 32.8.1 Ahora crea un mapa de los terremotos alrededor de Puerto Rico

**Obtener los datos de observaciones de una especie en iNaturalist desde R:**

Los datos a utilizar se obtendrán desde iNaturalist, una aplicación con base en la ciencia ciudadana que mantiene un registro de ocurrencias de especies.

La función `get_inat_obs()` permite acceder a los datos disponibles en iNaturalist, especificando los siguientes parámetros:

`get_inat_obs()` consulta la API de iNaturalist en tiempo real: requiere conexión a internet y el número de observaciones puede variar entre ejecuciones a medida que la comunidad añade registros.

- **taxon\_name:** La especie de interés, siempre escrita entre comillas (“). Puede utilizar nombre común o científico.
- **quality:** Siendo una aplicación basada en ciencia ciudadana, la calidad de algunos datos podría ser baja, debido a que les falta información - como nombre científico o ubicación - o que contienen información errónea - especies mal identificadas-. Colocando **'research'** en este parámetro se especifica que se desea obtener sólo aquellas observaciones con alta calidad - que tengan información completa y cuya identificación haya sido confirmada por varias personas de la comunidad -.
- **geo:** Al igual que el parámetro **quality**, este parámetro cuyo valor es lógico (**TRUE** o **FALSE**), es para especificar que se quiere obtener sólo los datos que están georeferenciados (cuando se coloca como **TRUE**).
- **maxresults:** A través del API (Interfaz de Programación de Aplicaciones) de iNaturalist se puede acceder a un máximo de 10,000 observaciones en una sola búsqueda. Con este parámetro, cuyo valor debe ser numérico, se especifica cuántas observaciones se desea obtener, y no se debe exceder de 10,000 por búsqueda.
- **bounds:** Un vector que contiene los límites geográficos de la búsqueda. Se puede insertar un cuadro delimitador ya especificado en un objeto, concatenar los valores del vector, o insertar un objeto de características simples (Simple Feature, sf). El argumento **bounds** es importante porque si obtenemos datos de iNaturalist, las únicas columnas que obtenemos con información acerca de la región o área donde se hicieron las observaciones son:
- **place\_guess:** Contiene el nombre de la región, pero su precisión y los nombres utilizados para cada región varía mucho.



- *latitude*: Posición en el eje vertical del sistema de coordenadas.
- *longitude*: Posición en el eje horizontal del sistema de coordenadas.  
Esto crea un problema a la hora de filtrar los datos por el nombre de una región, pues es difícil encontrar un solo valor que encompase a todas las variantes del nombre utilizado para esa región. iNaturalist se encuentra a nivel mundial, y usted puede acceder a todos esos datos. Sin embargo, hoy accederemos solamente a datos de Puerto Rico. Entonces, lo primero antes de acceder a los datos de iNaturalist, es crear un objeto que contenga los límites geográficos de interés crea. Para esto se crea un objeto que contenga un cuadro delimitador (*bounding box*):

```
cajapr <- c(
 17.75, #Latitud Sur
 -67.4, #Longitud Oeste
 18.75, #Latitud Norte
 -65.15 #Longitud Este
)

cajapr

[1] 17.75 -67.40 18.75 -65.15

library(rinat)
```

Ahora podemos acceder a datos de Puerto Rico utilizando el objeto `cajapr`. Para este ejercicio utilizaremos datos de orquídeas pertenecientes al género *Eulophia*. En Puerto Rico se han reportado 3 especies de este género *Eulophia alta*, *Eulophia maculata* y *Eulophia graminea*. De estos, sólo *E. alta* es considerada nativa de la isla. Entonces, utilizando la función `get_inat_obs` podemos acceder a los datos de *Eulophia* en Puerto Rico

```
Intenta consultar iNaturalist en vivo y guarda una copia local; si la
consulta falla (sin conexión o API no disponible), usa esa copia.
eulophia <- tryCatch(
 {
 d <- get_inat_obs(taxon_name = "Eulophia",
 quality = 'research',
 geo = TRUE,
 maxresults = 1000)
 readr::write_csv(d, "Datos/eulophia_inat.csv")
 d
 },
 error = function(e) readr::read_csv("Datos/eulophia_inat.csv")
)

eulophia$scientific_name <- as.factor(eulophia$scientific_name)
```

```
flectable(head(eulophia)) # Para ver las primeras 6 lineas en una tabla bonita
```

scientific_name	datetime	description	place_guess	latitude	longitude	tag_list	common_name
Eulophia graminea	2026-09-06 11:26:54 -0400		Florida, US	27.94570	-80.49273		Chinese Crown Orchid
Eulophia odon- toglossa	2016-01-23 15:57:00 +0200		Hhohho, SZ	-26.30483	31.09934		Toothtong Harlequin
Eulophia picta	2026-09-05 17:23:01 +0800		So Kwun Wat Tsuen Rd, So Kwun Wat, New Territories, HK	22.37764	114.00846		Shepherd Crook Orchid
Eulophia graminea	2026-09-05 18:07:02 -0400		Morrison, FL 32668, USA	29.22360	-82.62255		Chinese Crown Orchid
Eulophia maculata	2026-09-05 14:23:03 -0400		Loblolly Bay Rd, Labelle, FL, US	26.82412	-81.51964		Monk Orchid
Eulophia hians	2026-09-04 11:02:33 +0200	Conservancy - Suiker- bossie farm	Eccleston Crescent, Sandton, Gauteng, ZA	-26.05335	28.03251		Yawning Harlequin

bounds = cajapr Con el siguiente código podemos ver de forma rápida cuántas observaciones de cada especie hemos obtenido de iNaturalist:

```
summary(as.factor(eulophia$scientific_name))
```

##	Eulophia aculeata	Eulophia alta	Eulophia andamanensis
##	1	19	2
##	Eulophia angolensis	Eulophia bolusii	Eulophia callichroma
##	4	1	2
##	Eulophia caricifolia	Eulophia citrina	Eulophia clitellifera

##	5	4	17
##	Eulophia cochlearis	Eulophia coeloglossa	Eulophia cucullata
##	8	3	2
##	Eulophia diffusiflora	Eulophia ensata	Eulophia eulophioides
##	5	5	1
##	Eulophia exaltata	Eulophia flava	Eulophia galeoloides
##	1	1	1
##	Eulophia gracilis	Eulophia graminea	Eulophia guineensis
##	1	286	5
##	Eulophia herbacea	Eulophia hians	Eulophia hians hians
##	1	17	11
##	Eulophia hians inaequalis	Eulophia hians nutans	Eulophia horsfallii
##	4	2	16
##	Eulophia maculata	Eulophia micrantha	Eulophia nuda
##	267	3	16
##	Eulophia obtusa	Eulophia odontoglossa	Eulophia ovalis
##	1	2	9
##	Eulophia ovalis bainesii	Eulophia parviflora	Eulophia parvilabris
##	1	15	2
##	Eulophia parvula	Eulophia pauciflora	Eulophia petersii
##	2	2	12
##	Eulophia petiolata	Eulophia picta	Eulophia pratensis
##	2	190	1
##	Eulophia pulchra	Eulophia recurva	Eulophia roseovariegata
##	2	1	1
##	Eulophia siamensis	Eulophia speciosa	Eulophia streptopetala
##	3	10	6
##	Eulophia welwitschii	Eulophia zeyheriana	Eulophia zollingeri
##	2	1	24

Ahora que hemos obtenido los datos, podemos observar su distribución en el mapa de Puerto Rico utilizando `leaflet`.

## 32.9 Utilizando Leaflet

La función base del paquete `leaflet` es `leaflet()`, con esta se activa un artilugio (*widget* en inglés) sobre el cuál se construye el mapa. Esta función debe estar unida a otra función como `addTiles()` que añade el mapa.

```
leaflet() %>%
 addTiles()
```

Una vez tenemos el artilugio y el mapa, añadimos una capa que incluye los datos que se desean visualizar con marcadores sobre el mapa con la función `addMarkers()` dentro del cual se deben especificar los parámetros `lng` (longitud) y `lat` (latitud). Note que también se especifica el `data.frame` que contiene

los datos bajo la función `leaflet()`. Cuando se trabaja en Leaflet, al especificar las variables a utilizar, es necesario colocar una virgulilla (`~`) antes de la variable. Esto le deja saber a R que lo que se especifica es una variable dentro del `data.frame` mencionado en la primera capa bajo la función `leaflet()`.

```
names(eulophia)
leaflet(eulophia) %>%
 addTiles() %>%
 addMarkers(lng = ~longitude,
 lat = ~latitude)
```

Una vez los marcadores están establecidos, es posible añadir etiquetas que contengan información acerca de las observaciones con el parámetro `popup` de la función `addMarkers()`. Por ejemplo, se pueden colocar etiquetas que contengan información del lugar de la observación especificando que la etiqueta contenga la información de la variable `place_guess`. Para ver la etiqueta debe hacer clic sobre el marcador de interés.

```
names(eulophia)
leaflet(eulophia) %>%
 addTiles() %>%
 addMarkers(lng = ~ longitude,
 lat = ~ latitude,
 popup = ~ place_guess) # cambiar a la variable de interés
```

La función `paste0` es una función base de R que concatena todos los elementos en una línea de caracteres. Los caracteres introducidos serán interpretados como UTF-8 o codificación de caracteres. Estos caracteres pueden ser mezclados con valores de una variable de caracteres en un `data.frame`. En el siguiente ejemplo se crea una etiqueta que contiene la ubicación y la especie observada. Se utiliza la función `paste0()` para combinar caracteres y las variables `scientific_name` y `place_guess` del `data.frame`.

```
leaflet(eulophia) %>%
 addTiles() %>%
 addMarkers(lng = ~ longitude,
 lat = ~ latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado donde: ", eulophia$place_guess)
)
```

### 32.9.1 addAwesomeMarkers: Cambiando el diseño de los marcadores

La función `addAwesomeMarkers()` puede ser utilizada en lugar de `addMarkers()` y permite utilizar marcadores con diferentes diseños que se pueden obtener de

fontawesome. Es posible cambiar los colores de los marcadores utilizando la codificación de colores RGB en formato (#RRGGBB). La siguiente guía de colores en R creada por Melissa Clarkson (2010) explica como utilizar colores en R: A guide to using color in R. Adicional, esta guía creada por Derek H. Ogle (2022) es muy útil.

En `schemecolor` podemos encontrar las codificaciones de los colores y muchos funcionan para `leaflet`. Para hacer estos cambios utilizamos la función `awesomeIcons()` bajo el parámetro `icon` dentro de `addAwesomeMarkers()`.

Dentro de la función `awesomeIcons()` podemos especificar varios parámetros como:

- **icon:** El icono que desea utilizar para su marcador. Debe poner el nombre que aparece en la página web de iconos que esté utilizando (*i.e.*, fontawesome).
- **library:** Cuando se especifica un icono, es necesario especificar la biblioteca de donde se saca el icono. Si es de fontawesome sería 'fa'.
- **markerColor:** Para cambiar el color del marcador (parte que rodea el icono), utilizando la codificación de colores de `schemecolor`.
- **iconColor:** Para cambiar el color del icono del marcador, también con la codificación de `schemecolor`.

```
leaflet() %>%
 addTiles() %>%
 addAwesomeMarkers(lng = eulophia$longitude,
 lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 icon = awesomeIcons(icon = "leaf", library = "fa",
 markerColor = "blue",
 iconColor = "#FFFFFF")
)
```

Cuando se visualiza una gran cantidad de datos, esto puede resultar en el solapamiento de los marcadores, lo cual no es muy agradable a la vista. Es posible crear agregados que ayuden a visualizar mejor y faciliten el movimiento a través del mapa con el parámetro `clusterOptions` bajo la función `addAwesomeMarkers()`. El valor de este parámetro debe ser la función `markerClusterOptions()`.

```
leaflet() %>%
 addTiles() %>%
 addAwesomeMarkers(lng = eulophia$longitude, lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
```

```

 "Colectado por: ", eulophia$place_guess),
 icon = awesomeIcons(icon = "leaf", library = "fa",
 markerColor = "darkgreen",
 iconColor = "#FFFFFF"),
 clusterOptions = markerClusterOptions()
)

```

Y para culminar, es posible cambiar el diseño del mapa utilizando la función `addProviderTiles()` en lugar de `addTiles()`. Puede echar un vistazo a los diferentes diseños disponibles para Leaflet la página de proveedores de Leaflet en GitHub. Actualmente Leaflet en R admite diseños de los siguientes proveedores: OpenStreetMap, MapQuestOpen, Stamen, Esri and OpenWeatherMap.

```

icon1 <- awesomeIcons(icon = "smile-o",
 iconColor = "#FFFFFF",
 library = "fa",
 markerColor = "darkgreen")

leaflet() %>%
 addProviderTiles("OpenStreetMap.Mapnik") %>%
 addAwesomeMarkers(lng = eulophia$longitude, lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 icon = icon1
)

```

## 32.10 Trabajando Variables Categóricas con Leaflet:

Es posible distinguir entre especies utilizando diferentes colores, pero debe utilizar la función `addCircleMarkers()` en lugar de `addAwesomeMarkers()`. Para colorear grupos en Leaflet es necesario generar una paleta de colores para las variables de interés de acuerdo al tipo de variable (*i.e.*, numérica, factorial). Para esto se utilizan las funciones `colorFactor()`, `colorNumeric()`, `colorBin()`, o `colorQuantile()`. `colorFactor()` y `colorNumeric()` generan paletas de colores para variables factoriales y numéricas, respectivamente; mientras que `colorBin()` y `colorQuantile()` generan paletas de colores para valores numéricos resumidos en grupos.

En este caso se está trabajando con una variable factorial que divide las observaciones entre 3 especies de orquídeas. Primero se crea una paleta de colores para una variable factorial con la función `colorFactor()`. Bajo esta función se especifican los parámetros `palette` y `levels`. `palette`es para especificar la paleta de colores a utilizar y `levels`especifica los niveles de la variable factorial.

La paleta de colores que se utiliza ("Dark2") proviene del paquete RColorBrewer. Esta paleta de colores para los niveles de la variable factorial *scientific\_name* del data.frame *eulophia* se guardará en un objeto llamado *pal*.

```
mypal <- colorFactor(palette = 'Set3',
 levels = levels(eulophia$scientific_name))

eulophia$scientific_name=as.factor(eulophia$scientific_name)

mypal <- colorFactor(palette = 'Set3',
 levels = levels(eulophia$scientific_name))
```

Ahora que se ha especificado los colores para los valores factoriales, es posible utilizar el objeto *pal* para colorear los marcadores circulares del mapa de Leaflet de acuerdo a la especie observada especificando el parámetro *color*

```
leaflet(eulophia) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~ longitude, lat = ~ latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 color = ~ mypal(scientific_name))
```

Para ayudar con la visualización de datos pertenecientes a diferentes grupos, en ocasiones es bueno presentar una leyenda que indique los colores que representan cada grupo. Es posible añadir una leyenda al mapa de Leaflet utilizando la función `addLegend()` y especificando los siguientes parámetros:

- **position:** Para determinar la posición de la leyenda, en este caso se colocará en la parte superior derecha, "topright", del mapa, pero podría colocarla en la parte inferior derecha ("bottomright"), la parte inferior izquierda ("bottomleft"), o la parte superior izquierda ("topleft").
- **pal:** La paleta de colores a utilizar generada a través de la función `colorFactor()` o sus equivalentes (*i.e.*, `colorNumeric()`)
- **values:** Los valores o la variable u objeto que contiene los valores utilizados.
- **title:** El título de la leyenda especificado como una línea de caracteres.

```
leaflet(eulophia) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~ longitude, lat = ~ latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 color = ~ pal(scientific_name)) %>%
```

```

addLegend(position = "topright",
 pal = pal,
 values = ~ scientific_name,
 title = "Species"
)

iconcolor <- function(scientific_name)

leaflet() %>%
 addProviderTiles("Esri.WorldImagery") %>%
 addAwesomeMarkers(lng = eulophia$longitude, lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 icon = icon1
)

```

L.circleMarker([place.lon, place.lat], { color: getColor(place.constructdate), //  
 you can call the getColor function fillColor: getColor(place.constructdate), fil-  
 lOpacity: 0.5 })

## 32.11 Ejercicios del capítulo

**Sobre los ejercicios de este capítulo.** Cada ejercicio se entrega con tres cosas: el **script**, el **mapa** (que se vea en el `.html`) y una **explicación en palabras**. Como los mapas solo salen en HTML, entrega siempre el `.html`, no solo el `.Rmd`. Los ejercicios usan `Datos/crimen_pr.csv`, que está en el repositorio del curso y trae coordenadas de Puerto Rico, no los terremotos del USGS ni los datos de iNaturalist del capítulo. La hoja completa para entregar está en `Ejercicios/Ejercicios_Capitulo_32_leaflet.Rmd`.

### 32.11.1 Ejercicio 1. El mapa más sencillo

Haz un mapa vacío centrado en Puerto Rico, con `leaflet()`, `addTiles()` y `setView()`. Añade un marcador en tu pueblo.

**Verificación:** al abrir el `.html`, la isla tiene que verse completa y el marcador tiene que caer donde debe.

**En palabras:** anota las coordenadas que usaste. ¿Cuál de los dos números es la latitud y cuál la longitud? ¿Cómo lo sabes sin buscarlo?

*# Escribe tu código aquí*



### 32.11.2 Ejercicio 2. Los puntos que acabaron en el océano

Lee Datos/crimen\_pr.csv. Fíjate en que las columnas se llaman POINT\_X y POINT\_Y, y haz el mapa de la manera intuitiva: lng = ~POINT\_X y lat = ~POINT\_Y.

**Verificación:** los puntos **no** caen en Puerto Rico. Mira dónde caen.

Después arréglalo y vuelve a hacer el mapa.

**En palabras:** ¿en qué parte del mundo aparecieron los puntos? Mira el rango de las dos columnas con `range(..., na.rm = TRUE)` y explica cómo esos dos rangos te dicen cuál columna es cuál, sin necesidad de abrir el mapa.

*# Escribe tu código aquí*

### 32.11.3 Ejercicio 3. Los puntos que no existen

El archivo tiene coordenadas faltantes, y además unas pocas guardadas como cero. Cuenta las dos cosas y quítalas antes de mapear.

**Verificación:** hay **733** filas con coordenada faltante y **2** con un cero exacto. Después de quitarlas quedan **8135** filas.

**En palabras:** dibuja en un mapa **solo** las filas con coordenada cero. ¿Dónde caen? Busca en internet qué es “Null Island”. ¿Por qué un cero es más peligroso que un NA?

*# Escribe tu código aquí*

### 32.11.4 Ejercicio 4. Color por categoría

Colorea los puntos según el tipo de delito, con `colorFactor()`, y añade la leyenda con `addLegend()`.

**Verificación:** la leyenda tiene que mostrar **9** categorías.

**En palabras:** con nueve colores el mapa se vuelve difícil de leer. Reduce las categorías a cinco con `fct_lump_n()` del capítulo de factores y compara los dos mapas. ¿Cuál usarías y qué perdiste al agrupar?

*# Escribe tu código aquí*

### 32.11.5 Ejercicio 5. Las ventanas emergentes

Añádele a cada punto un **popup** que diga el tipo de delito, la fecha, la hora y el área policiaca. Prueba también `label` en vez de `popup`.

**Verificación:** al hacer clic (o al pasar el ratón, según cuál uses) tiene que salir la información de **esa** observación, no la de todas.

**En palabras:** ¿cuál es la diferencia entre `popup` y `label`? ¿Cuál usarías para un mapa que va a mirar mucha gente sin instrucciones?

*# Escribe tu código aquí*

### 32.11.6 Ejercicio 6. Ocho mil puntos

Dibuja los 8135 puntos de golpe y mira cuánto tarda en abrirse el `.html` y cuánto pesa el archivo. Después repítelo con `clusterOptions = markerClusterOptions()`.

**Verificación:** con agrupamiento, el mapa abre mucho más rápido y muestra círculos con números en vez de puntos sueltos.

**En palabras:** compara el tamaño de los dos `.html` con `file.size()`. ¿Qué pierdes al agrupar? ¿En qué nivel de acercamiento vuelven a verse los puntos individuales?

*# Escribe tu código aquí*

### 32.11.7 Ejercicio 7 (BONO). Capas que se encienden

Escoge dos tipos de delito y ponlos en **dos capas** distintas, con `group =`, y añade `addLayersControl()` para poder encender y apagar cada una.

**Verificación:** en el mapa tiene que aparecer arriba a la derecha un control con las dos capas, y al desmarcar una, sus puntos tienen que desaparecer.

**En palabras:** ¿qué pregunta puedes contestar con las dos capas encendidas y apagadas que no podrías contestar con un mapa fijo? Este es el argumento de fondo del capítulo: escríbelo con tus palabras.

*# Escribe tu código aquí*

# Apéndice A: COVID-19 acumulativo

```
[1] "2026-09-06"
```

---

## .1 COVID 19 Cumulativo

Usa los datos de COVID-19 en Puerto Rico (PR) del Departamento de Salud de PR.

```
url_COVID_PR <- read_csv("Datos/url_COVID_PR.csv")
head(url_COVID_PR)
```

```
A tibble: 6 x 18
...1 Fecha Muertes IncrementoMuertes CasosPCR_Salud IncCasosPCR_Salud
<dbl> <chr> <dbl> <dbl> <dbl> <dbl>
1 1 3/12/20 0 0 NA NA
2 2 3/13/20 0 0 NA NA
3 3 3/14/20 0 0 NA NA
4 4 3/15/20 0 0 NA NA
5 5 3/16/20 0 0 NA NA
6 6 3/17/20 0 0 NA NA
i 12 more variables: CasosSaludNuevo <dbl>, IncCasosSaludNuevo <dbl>,
HospitCOV19 <dbl>, CamasICU <dbl>, CamasICU_disp <dbl>, Ventiladores <dbl>,
MuertesSalud <dbl>, IncMueSalud <dbl>, VacDoses <dbl>, VacAdm <dbl>,
N1MoreDoses <dbl>, N2Doses <dbl>
```

---

1. Calcula los promedios en periodos de 7 (días; promedio corriendo) y añádele a tu dataframe

Un promedio *cumulative* (`cummean`) usa todos los datos desde el inicio hasta cada fecha, mientras que un promedio *corriendo* o móvil (`rollmean` con ventana de 7 días) solo promedia los días más cercanos.

`cummean()` (paquete `dplyr`) y `rollmean()` (paquete `zoo`) — resumen una serie de valores a lo largo del tiempo.

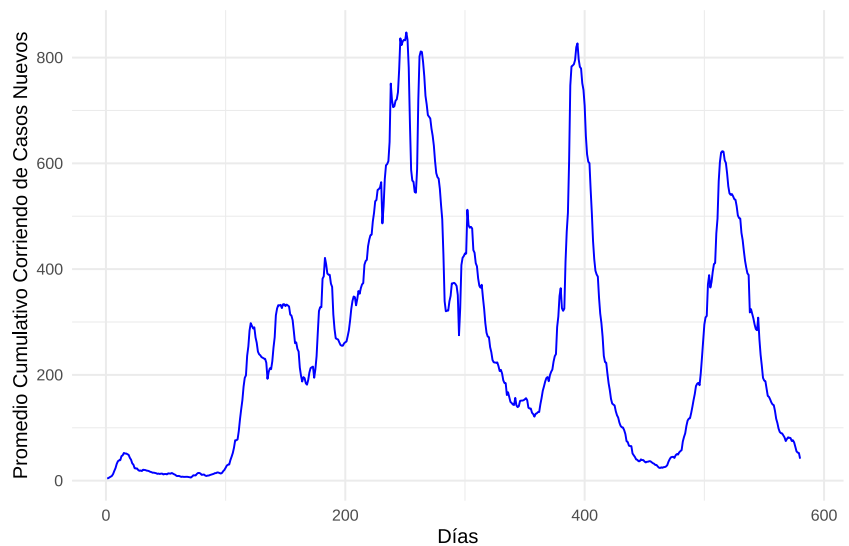
- `cummean(x)`: devuelve el promedio *acumulado* — cada valor es el promedio de todos los datos hasta esa fila.
- `rollmean(x, k = 7, fill = NA, align = "left")`: el promedio *móvil* de una ventana de `k` días; `fill = NA` rellena los extremos donde no hay ventana completa y `align` fija dónde se ancla la ventana.
- `rollapply(x, width, FUN)`: versión general que aplica *cualquier* función (aquí `var`) sobre una ventana móvil.

— Usa la columna de `IncCasosSaludNuevo` para todos los cálculos

- Enseñe las primeras 20 filas de tu dataframe con las nuevas columnas añadidas

2. Haz una figura que muestra el cambio en el promedio cumulativo corriendo

Promedio Cumulativo Corriendo de Casos Nuevos de COVID-19 en PR



de casos nuevos.